Amazon Rekognition

開發人員指南



Amazon Rekognition: 開發人員指南

Copyright © 2020 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

什麼是 Amazon Rekognition？ 1
Amazon Rekognition 與 HIPAA 標準 2
您是第一次使用 Amazon Rekognition 嗎？ 2
運作方式 4
分析類型 4
標籤 4
自訂標籤 4
臉部 5
臉部搜尋 5
人物路徑 5
名人 5
文字偵測 5
不安全的內容 5
影像和影片操作 6
Amazon Rekognition Image 操作 6
Amazon Rekognition Video 操作 6
非儲存與以儲存為基礎的操作 6
使用 AWS 開發套件或 HTTP 來呼叫 Amazon Rekognition API 操作 6
非儲存與以儲存為基礎的 API 操作 7
非儲存操作 7
以儲存為基礎的 API 操作 8
模型版本控制 9
入門 10
步驟 1. 設定帳戶 10
註冊 AWS 帳號 10
建立 IAM 使用者 10
後續步驟 11
步驟 2：設定 AWS CLI 和 AWS 開發套件 11
下一個步驟 12
步驟 3：開始使用 AWS CLI 和 AWS 開發套件 API 12
將 AWS CLI 範例格式化 13
後續步驟 13
步驟 4：開始使用主控台 13
練習 1：偵測物件和場景 (主控台) 13
練習 2：分析臉部 (主控台) 18
練習 3：比對臉部 (主控台) 20
練習 4：查看彙總指標 (主控台) 22
對 Amazon Rekognition API 進行程式設計 23
使用影像 23
影像 23
使用 Amazon S3 儲存貯體 24
使用本機檔案系統 29
顯示週框方塊 37
取得影像方向與週框方塊座標 42
使用儲存的影片來操作 49
分析類型 49
Amazon Rekognition VideoAPI 概觀 50
呼叫 Amazon Rekognition Video 操作 51
設定 Amazon Rekognition Video 55
分析已儲存的影片 (開發套件) 56
分析影片 (AWS CLI) 66
教學：建立 Amazon Rekognition Lambda 函數 68
參考：影片分析結果通知 74
故障診斷 Amazon Rekognition Video 75

處理串流影片 76
辨識串流影片中的臉部 77
提供對 Kinesis 串流的存取 77
開始串流影片分析 78
讀取分析結果 85
參考：Kinesis 臉部辨識記錄 87
串流影片故障診斷 91
錯誤處理 95
錯誤元件 95
錯誤訊息和錯誤碼 96
應用程式中的錯誤處理 99
搭配使用 Amazon Rekognition 與 FedRAMP 100
感應器、輸入影像和影片的最佳實務 103
Amazon Rekognition Image 操作延遲 103
臉部比較輸入影像的建議 103
相機設定的建議 (影像和影片) 104
相機設定的建議 (已存放和串流影片) 104
相機設定的建議 (串流影片) 105
偵測物件和場景 106
在影像中偵測標籤 107
DetectLabels 操作要求 112
DetectLabels 回應 112
偵測影片中的標籤 116
GetLabelDetection 操作回應 116
偵測自訂標籤 118
偵測與分析臉部 119
臉部偵測和臉部辨比較的概觀 120
臉部屬性的準則 120
偵測影像中的臉部 120
DetectFaces 操作要求 126
DetectFaces 操作回應 126
比較影像中的臉部 133
CompareFaces 操作要求 138
CompareFaces 操作回應 139
偵測已存放影片中的臉部 141
GetFaceDetection 操作回應 143
在集合中搜尋臉部 147
管理集合 147
管理集合中的臉孔 147
使用 IndexFaces 的指導 148
關鍵或公共安全應用 148
照片共享和社交媒體應用程式 148
一般用法 148
在集合中搜尋臉部 148
使用相似性臨界值來比對臉部 148
涉及公共安全的使用案例 149
使用Amazon Rekognition 以協助公共安全 149
建立集合 150
CreateCollection 操作要求 152
CreateCollection 操作回應 152
列出集合 153
ListCollections 操作要求 155
ListCollections 操作回應 155
描述集合 156
DescribeCollection 操作要求 159
DescribeCollection 操作回應 159
刪除集合 159

DeleteCollection 操作要求 161
DeleteCollection 操作回應 162
新增臉部到集合 162
篩選臉孔 163
IndexFaces 操作要求 167
IndexFaces 操作回應 167
在集合中列出臉部 172
ListFaces 操作要求 175
ListFaces 操作回應 176
刪除集合中的臉部 176
DeleteFaces 操作要求 179
DeleteFaces 操作回應 179
搜尋臉孔 (臉部 ID) 179
SearchFaces 操作要求 182
SearchFaces 操作回應 182
搜尋臉孔 (影像) 183
SearchFacesByImage 操作要求 186
SearchFacesByImage 操作回應 186
在儲存的視訊中搜尋臉部 187
GetFaceSearch 操作回應 191
人物路徑 195
GetPersonTracking 操作回應 197
辨識名人 201
相較於臉部搜尋的名人辨識 201
辨識影像中的名人 201
呼叫 RecognizeCelebrities 202
RecognizeCelebrities 操作要求 205
RecognizeCelebrities 操作回應 205
辨識已存放影片中的名人 207
GetCelebrityRecognition 操作回應 210
取得名人資訊 211
呼叫 GetCelebrityInfo 212
GetCelebrityInfo 操作要求 214
..... 214
偵測不安全的內容 215
使用影像和影片管制 API 215
偵測不安全的影像 216
偵測影像中不安全的內容 216
..... 216
DetectModerationLabels 操作要求 219
DetectModerationLabels 操作回應 220
偵測不安全的已存放影片 220
GetContentModeration 操作回應 223
使用 Amazon A2I 檢閱不安全的內容 224
偵測文字 227
在影像中偵測文字 228
DetectText 操作要求 231
DetectText 操作回應 231
偵測已儲存影片中的文字 238
篩選條件 241
GetTextDetection 回應 241
安全性 245
Identity and Access Management 245
對象 245
使用身分來驗證 246
使用政策管理存取權 247
Amazon Rekognition 如何搭配 IAM 運作 249

身分型政策範例 251
故障診斷 254
資料保護 256
資料加密 257
網際網路流量隱私權 257
監控 258
使用 Rekognition 的 CloudWatch 指標 258
存取 Rekognition 指標 259
建立警示 259
Rekognition 的 CloudWatch 指標 260
使用 AWS CloudTrail 記錄 Amazon Rekognition API 呼叫 261
Amazon RekognitionCloudTrail 中的資訊 262
範例：Amazon Rekognition 日誌檔案項目 262
使用 Amazon Rekognition 搭配 Amazon VPC 端點 267
建立適用於 Amazon Rekognition 的 Amazon VPC 端點 267
建立 Amazon Rekognition 的 VPC 端點政策 267
合規驗證 268
彈性 269
組態與漏洞分析 269
基礎設施安全 269
API 參考 270
Amazon Rekognition Image 270
Amazon Rekognition 自訂標籤 270
Amazon Rekognition Video 已存放影片 270
Amazon Rekognition Video 串流視訊 271
Actions 271
CompareFaces 273
CreateCollection 279
CreateProject 282
CreateProjectVersion 285
CreateStreamProcessor 289
DeleteCollection 292
DeleteFaces 294
DeleteProject 296
DeleteProjectVersion 298
DeleteStreamProcessor 300
DescribeCollection 302
DescribeProjects 305
DescribeProjectVersions 308
DescribeStreamProcessor 312
DetectCustomLabels 316
DetectFaces 320
DetectLabels 324
DetectModerationLabels 329
DetectText 333
GetCelebrityInfo 337
GetCelebrityRecognition 339
GetContentModeration 344
GetFaceDetection 348
GetFaceSearch 353
GetLabelDetection 358
GetPersonTracking 362
GetTextDetection 367
IndexFaces 371
ListCollections 379
ListFaces 382
ListStreamProcessors 385

RecognizeCelebrities ... 388
SearchFaces ... 392
SearchFacesByImage ... 395
StartCelebrityRecognition ... 400
StartContentModeration ... 403
StartFaceDetection ... 407
StartFaceSearch ... 411
StartLabelDetection ... 415
StartPersonTracking ... 419
StartProjectVersion ... 422
StartStreamProcessor ... 425
StartTextDetection ... 427
StopProjectVersion ... 431
StopStreamProcessor ... 433
Data Types ... 434
AgeRange ... 437
Asset ... 438
Beard ... 439
BoundingBox ... 440
Celebrity ... 442
CelebrityDetail ... 443
CelebrityRecognition ... 445
ComparedFace ... 446
ComparedSourceImageFace ... 447
CompareFacesMatch ... 448
ContentModerationDetection ... 449
CustomLabel ... 450
DetectionFilter ... 451
DetectTextFilters ... 452
Emotion ... 453
EvaluationResult ... 454
Eyeglasses ... 455
EyeOpen ... 456
Face ... 457
FaceDetail ... 459
FaceDetection ... 462
FaceMatch ... 463
FaceRecord ... 464
FaceSearchSettings ... 465
Gender ... 466
Geometry ... 467
GroundTruthManifest ... 468
HumanLoopActivationOutput ... 469
HumanLoopConfig ... 470
HumanLoopDataAttributes ... 471
Image ... 472
ImageQuality ... 473
Instance ... 474
KinesisDataStream ... 475
KinesisVideoStream ... 476
Label ... 477
LabelDetection ... 478
Landmark ... 479
ModerationLabel ... 480
MouthOpen ... 481
Mustache ... 482
NotificationChannel ... 483

OutputConfig 484
Parent 485
PersonDetail 486
PersonDetection 487
PersonMatch 488
Point 489
Pose 490
ProjectDescription 491
ProjectVersionDescription 492
RegionOfInterest 494
S3Object 495
Smile 496
StartTextDetectionFilters 497
StreamProcessor 498
StreamProcessorInput 499
StreamProcessorOutput 500
StreamProcessorSettings 501
Summary 502
Sunglasses 503
TestingData 504
TestingDataResult 505
TextDetection 506
TextDetectionResult 508
TrainingData 509
TrainingDataResult 510
UnindexedFace 511
Video 512
VideoMetadata 513
限制 515
Amazon Rekognition Image 515
Amazon Rekognition Video 已存放影片 515
Amazon Rekognition Video 串流視訊 515
文件歷史記錄 516
AWS Glossary 519

什麼是 Amazon Rekognition？

Amazon Rekognition 能協助您輕鬆將影像和影片分析新增至應用程式。只需提供影像或影片至 Amazon Rekognition API，服務即可辨識物件、人物、文字、場景與活動。該服務還可以偵測任何不當內容。此外，Amazon Rekognition 亦提供最精準的臉部分析、臉部比較及臉部搜尋功能。您可以偵測、分析及比對各種使用案例的臉部，包括驗證使用者、建立目錄、計算人數和公共安全。

Amazon Rekognition 以 Amazon 電腦視覺科學家所開發的深度學習技術為基礎，不但經過驗證且可高度擴展，每天可分析數十億張影像。就算沒有任何機器學習專業知識，也能輕鬆使用。Amazon Rekognition 內含簡單、易於使用的 API，可快速分析任何存放在 Amazon S3 中的影像或影片檔案。Amazon Rekognition 能不斷從新資料中學習，而該服務也會持續增加新標籤和臉部比較功能。如需詳細資訊，請參閱 Amazon Rekognition 常見問答集。

Amazon Rekognition 的常用案例包括：

- 可搜尋的影像和影片程式庫 – Amazon Rekognition 會將影像與儲存的影片轉為可搜尋項目，讓您能尋找其中出現的物件和場景。

- 以臉部為基礎的使用者驗證 – Amazon Rekognition 讓應用程式可以比較即時影像與參考影像，藉此確認使用者的身分。

- 情緒分析與人口統計分析 – Amazon Rekognition 不僅能解譯快樂、哀傷或驚訝等情緒表達，也能從臉部影像來分析性別等人口統計資訊。Amazon Rekognition 可以分析影像，並將情感和人口統計屬性傳送至 Amazon Redshift，以便定期回報當前趨勢 (如存放位置與相似情境等資訊)。請注意，情緒表達的預測僅根據一個人臉部的實體外觀。它不表示個人的內部情緒狀態，也不應使用 Rekognition 來進行這種判斷。

- 臉部搜尋 – 透過 Amazon Rekognition，即可搜尋影像、儲存的影片和串流影片，進而找出與容器 (稱之為人臉集合) 內存放資料相符的臉孔。人臉集合為您持有與管理的臉部資料索引。若要根據臉部來搜尋對象，則需在 Amazon Rekognition 中執行兩個主要步驟：

  1. 編制臉部索引。
  2. 搜尋臉部。

- 不安全的內容偵測 – Amazon Rekognition 可以偵測影像和已儲存影片中的成人和暴力內容。開發人員可使用傳回的中繼資料來根據企業需求篩選出不適當的內容。除了標記出現不安全內容的影像外，API 還會傳回附有可信度分數的階層式標籤清單。這些標籤可指出特定類別的不安全內容，並可對大量的使用者產生內容 (UGC) 執行精確篩選與管理。範例包括社群與交友網站、相片分享平台、部落格與論壇、兒童應用程式、電子商務網站、娛樂與線上廣告服務。

- 名人辨識 – Amazon Rekognition 能夠在提供的影像和影片中辨識出名人。Amazon Rekognition 可以識別數千位名人，且涵蓋範圍橫跨多個領域，例如政治、運動、商業、娛樂及媒體。

- 文字偵測 – Amazon Rekognition Text in Image (影像文字) 功能可讓您辨識出影像中的文字內容，並加以擷取。Text in Image 支援大部分字體，包括高度風格化的字體。它會以不同方向來偵測文字和數字，例如常在橫幅廣告與海報中看到的樣式。在影像分享與社交媒體應用程式中，您可以使用此功能來依據包含相同關鍵字的影像索引啟用視覺搜尋。在媒體與娛樂應用程式中，您可以依據畫面上的相關文字建立影片的

目錄，例如廣告、新聞、運動比分及標題。最後，在公共安全應用程式中，您可以依據街道攝影機所拍攝影像中的車牌號碼識別車輛。

- 自訂標籤 – 藉由 Amazon Rekognition 自訂標籤，您可以識別出影像中專屬於您業務需求的物體和場景。例如，您可以在社交群媒張貼上找到標誌、識別商店貨架上的產品、對裝配線中的機器零件進行分類，區分健康植物和受感染植物，或偵側影片中的動畫人物。如需詳細資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南中的什麼是 Amazon Rekognition 自訂標籤？。

使用 Amazon Rekognition 的若干優點包括：

- 將強大的影像和視訊分析整合至您的應用程式 – 無需具備電腦視覺或深度學習的專業知識，也能善用 Amazon Rekognition 可靠的影像和影片分析功能。您可以運用 API，藉此在任何 Web、行動裝置或連接至裝置的應用程式中，輕鬆且快速地建置影像和影片分析功能。

- 以深度學習為基礎的影像和影片分析 – Amazon Rekognition 是採用深度學習技術，因此能精確分析影像、尋找並比對影像中的臉孔，還可以偵測影像和影片內的物件及場景。

- 可擴展的影像分析 – Amazon Rekognition 能讓您分析數百萬筆影像，進而編排並整理大量的視覺化資料。

- 與其他 AWS 服務整合 – Amazon Rekognition 是為與 Amazon S3 及 AWS Lambda 等其它 AWS 服務無縫整合而設計之功能。您可以直接從 Lambda 呼叫 Amazon Rekognition API，以回應 Amazon S3 事件。Amazon S3 與 Lambda 會因應應用程式的需求而自動擴展，讓您能自行建置可擴展、經濟實惠且可靠的影像分析應用程式。舉例來說，每次有人到您的住所拜訪時，門口的攝影機就會將訪客的照片上傳至 Amazon S3。這會觸發採用 Amazon Rekognition API 操作的 Lambda 函數，藉此辨識訪客身分。您可以直接在存放於 Amazon S3 的影像上執行分析，無需上傳或移動資料。該服務支援 AWS Identity and Access Management (IAM)，讓您能輕鬆、安全地控制 Amazon Rekognition API 操作的存取權。您可以善用 IAM 來建立與管理 AWS 使用者及群組，並提供適當的存取權給開發人員和最終使用者。

- 成本低廉 – 有了 Amazon Rekognition，就只需支付影像和影片的分析費用，以及臉部中繼資料的儲存成本。沒有最低費用或者預付款項。您可立即免費試用，並隨著使用需求增長，搭配 Amazon Rekognition 的分層定價模式，省下更多成本。

Amazon Rekognition 與 HIPAA 標準

This is a HIPAA Eligible Service. For more information about AWS, U.S. Health Insurance Portability and Accountability Act of 1996 (HIPAA), and using AWS services to process, store, and transmit protected health information (PHI), see HIPAA Overview.

您是第一次使用 Amazon Rekognition 嗎？

若是第一次使用 Amazon Rekognition，建議您依序閱讀以下章節：

1. Amazon Rekognition 運作方式 (p. 4) – 本節會介紹各種 Amazon Rekognition 元件，這些元件可供您建立端對端的體驗。
2. Amazon Rekognition 入門 (p. 10) – 本節會說明如何設定帳戶，並測試 Amazon Rekognition API。
3. 使用影像 (p. 23) – 本節會提供相關資訊，讓您了解如何運用 Amazon Rekognition，並結合使用存放在 Amazon S3 儲存貯體的影像，以及從本機檔案系統載入的影像。

4. 使用儲存的影片來操作 (p. 49) – 本節會提供相關資訊，藉此說明如何將 Amazon Rekognition 與存放於 Amazon S3 儲存貯體中的影片搭配使用。
5. 處理串流影片 (p. 76) – 本節會提供相關資訊，藉此說明如何將 Amazon Rekognition 與串流影片搭配使用。

Amazon Rekognition 運作方式

Amazon Rekognition 提供兩種 API 集。您可將 Amazon Rekognition Image 用於分析影像，以及將 Amazon Rekognition Video 用於分析影片。

兩種 API 皆可分析影像與視訊，以提供可套用於應用程式中的資訊。例如，您可以使用 Amazon Rekognition Image 來強化相片管理應用程式的客戶使用體驗。當客戶上傳照片時，您的應用程式可以使用 Amazon Rekognition Image 來偵測影像中的真實世界物件或人物臉孔。您的應用程式存放 Amazon Rekognition Image 傳回的資訊後，使用者可以根據特定物件或臉孔來查詢相片集合中符合的相片。也可執行更進階的查詢。例如，使用者可以查詢微笑的臉孔、或者查詢特定年齡的臉孔。

您可以使用 Amazon Rekognition Video 追蹤已儲存的影片中的人物路徑。或者，您可以使用 Amazon Rekognition Video 來搜尋串流視訊中符合 Amazon Rekognition 所儲存的臉部描述之人物臉孔。

Amazon Rekognition API 讓深入學習影像分析更易於操作。例如，RecognizeCelebrities (p. 388) 可傳回最多 100 個在影像中偵測到的名人資訊。資訊包含名人臉孔在影像中被偵測到的位置、以及哪裡可以取得該名人的其他相關資訊。

以下資訊涵蓋 Amazon Rekognition 提供的分析類型，以及 Amazon Rekognition Image 與 Amazon Rekognition Video 的操作概觀。同時也將說明非儲存體與儲存體操作間的差別。

主題

- 分析類型 (p. 4)
- 影像和影片操作 (p. 6)
- 非儲存與以儲存為基礎的 API 操作 (p. 7)
- 模型版本控制 (p. 9)

分析類型

以下是 Amazon Rekognition Image API 與 Amazon Rekognition Video API 可執行的分析類型。如需 API 的相關資訊，請參閱 影像和影片操作 (p. 6)。

標籤

標籤 是指下列任何項目：物件 (例如，花朵、樹或桌子)、事件 (例如，結婚、畢業典禮或生日派對)、概念 (例如，景色、黃昏和自然) 或活動 (例如，下車)。Amazon Rekognition 可以偵測影像與視訊中的標籤。不過影像中不會偵測到活動。如需詳細資訊，請參閱 偵測物件和場景 (p. 106)。

若要偵測影像中的標籤，請使用 DetectLabels (p. 324)。若要偵測儲存的影片中的標籤，請使用 StartLabelDetection (p. 415)。

自訂標籤

Amazon Rekognition 自訂標籤可以藉由培訓機器學習模型，識別出影像中專屬於您業務需求的物體和場景。例如，您可以培訓模型以偵測標誌或偵測裝配線上的工程機具零件。

Note

有關 Amazon Rekognition 自訂標籤的資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南。

Amazon Rekognition 提供您用來建立、培訓、評估和執行機器學習模型的主控台。如需詳細資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南中的 Amazon Rekognition 自訂標籤入門。您也可以使用 Amazon Rekognition 自訂標籤 API 來培訓和執行模型。如需詳細資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南中的 Amazon Rekognition 自訂標籤開發套件入門。

若要使用經培訓模型來分析影像，請使用 DetectCustomLabels。

臉部

Amazon Rekognition 可以偵測影像與儲存的影片中的臉部。使用 Amazon Rekognition 可獲得臉部在影像或視訊中被偵測到的位置、臉部特徵點 (例如眼睛位置)、以及偵測到的情緒 (例如快樂或悲傷) 等相關資訊。您也可以將影像中的臉部與另一張影像中偵測到的臉孔進行比對。關於臉部的資訊也可儲存提供日後使用。如需詳細資訊，請參閱 偵測與分析臉部 (p. 119)。

若要偵測影像中的臉部，請使用 DetectFaces (p. 320)。若要偵測儲存的影片中的臉部，請使用 StartFaceDetection (p. 407)。

臉部搜尋

Amazon Rekognition 可以搜尋臉部。臉部資訊以索引編入容器中，也就是集合。集合中的臉部資訊可與影像、儲存的影片以及串流視訊中偵測到的臉孔進行比對。如需詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

要搜尋影像中已知的臉部，請使用 DetectFaces (p. 320)。要搜尋儲存的影片中已知的臉部，請使用 StartFaceDetection (p. 407)。要搜尋串流視訊中已知的臉部，請使用 CreateStreamProcessor (p. 289)。

人物路徑

Amazon Rekognition 可在儲存的影片中追蹤偵測到的人物路徑。Amazon Rekognition Video 提供追蹤、臉部詳細資訊、以及在視訊中偵測到人物所在的影格位置資訊。如需詳細資訊，請參閱 人物路徑 (p. 195)。

若要偵測儲存的影片中的人物，請使用 StartPersonTracking (p. 419)。

名人

Amazon Rekognition 可以在影像與已儲存視訊中辨識數千位名人。您可以取得名人臉孔在影像中的位置、臉部特徵點以及名人的臉部姿態等相關資訊。您可以獲得名人在儲存的影片中出現的追蹤資訊。您也可以取得成功辨識的名人之其他相關資訊。如需詳細資訊，請參閱 辨識名人 (p. 201)。

若要辨識影像中的名人，請使用 RecognizeCelebrities (p. 388)。若要辨識儲存的影片中的名人，請使用 StartCelebrityRecognition (p. 400)。

文字偵測

Amazon Rekognition Text in Image 可以偵測影像中的文字並將其轉換為機器可讀取的文字。如需詳細資訊，請參閱 偵測文字 (p. 227)。

若要偵測影像中的文字，請使用 DetectText (p. 333)。

不安全的內容

Amazon Rekognition 可以分析影像和已儲存的影片中是否含有成人和暴力的內容。如需詳細資訊，請參閱 偵測不安全的內容 (p. 215)。

如要偵測不安全的影像，請使用 DetectModerationLabels (p. 329)。若要偵測不安全的已儲存影片，請使用 StartContentModeration (p. 403)。

影像和影片操作

Amazon Rekognition 提供兩種 API 集。您可將 Amazon Rekognition Image 用於分析影像，以及將 Amazon Rekognition Video 用於分析已儲存及串流的影片。以下主題提供每個 API 集的簡要概觀。

Amazon Rekognition Image 與 Amazon Rekognition Video API 可偵測多種不同實體，例如臉部或物件。如需支援的比較與偵測類型相關資訊，請參閱 分析類型 (p. 4)。

Amazon Rekognition Image 操作

Amazon Rekognition 影像操作為同步。輸入與回應為 JSON 格式。Amazon Rekognition 影像操作用於分析 .jpg 或 .png 影像格式的輸入影像檔。傳遞到 Amazon Rekognition Image 操作的影像可儲存於 Amazon S3 儲存貯體中。如果您不是使用 AWS CLI，也可直接傳遞以 Base64 編碼的影像位元組到 Amazon Rekognition 操作。如需詳細資訊，請參閱 使用影像 (p. 23)。

Amazon Rekognition Video 操作

Amazon Rekognition Video 可分析儲存於 Amazon S3 儲存貯體中的影片以及透過 Amazon Kinesis Video Streams 串流的影片。

Amazon Rekognition Video 影片操作為非同步。使用 Amazon Rekognition Video 儲存影片操作，需先對您想使用的分析類型呼叫開始操作才可開始分析。例如，若要在儲存的影片中偵測臉部，請呼叫 StartFaceDetection (p. 407)。完成分析後，Amazon Rekognition 將發佈完成狀態至 Amazon SNS 主題。若要取得分析操作的結果，需對您要求的分析類型呼叫取得操作—例如，GetFaceDetection (p. 348)。如需詳細資訊，請參閱 使用儲存的影片來操作 (p. 49)。

使用 Amazon Rekognition Video 串流視訊操作，可搜尋儲存於 Amazon Rekognition Video 集合中的臉部。Amazon Rekognition Video 將分析 Kinesis video stream 並輸出搜尋結果至 Kinesis data stream。您可以建立並使用 Amazon Rekognition Video 串流處理器來管理影片分析。例如，您可以透過呼叫 CreateStreamProcessor (p. 289) 來建立串流處理器。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

非儲存與以儲存為基礎的操作

Amazon Rekognition 操作將依組分為以下類別。

- 非儲存 API 操作 – 在這些操作中，Amazon Rekognition 將不會保留任何資訊。您提供輸入影像與影片，操作將執行分析並傳回結果，但 Amazon Rekognition 不會儲存任何內容。如需詳細資訊，請參閱 非儲存操作 (p. 7)。
- 以儲存為基礎的 API 操作 – Amazon Rekognition 伺服器可將偵測到的臉部資訊儲存在容器中，也就是集合。Amazon Rekognition 提供額外的 API 操作，您可使用這些操作來搜尋保留的臉部資訊並尋找臉部配對。如需詳細資訊，請參閱 以儲存為基礎的 API 操作 (p. 8)。

使用 AWS 開發套件或 HTTP 來呼叫 Amazon Rekognition API 操作

您可以使用 AWS 開發套件或直接使用 HTTP 來呼叫 Amazon Rekognition API 操作。除非有充分理由不使用 AWS 開發套件，否則應一律使用 AWS 開發套件。本節中的 Java 範例使用 AWS 開發套件。未提供 Java 專案檔案，但您可以使用 AWS Toolkit for Eclipse 來使用 Java 開發 AWS 應用程式。

本節中的 .NET 範例使用 AWS SDK for .NET。您可以使用 AWS Toolkit for Visual Studio 使用 .NET 開發 AWS 應用程式。它包含用於部署應用程式和管理服務的有用範本和 AWS Explorer。

本指南中的 API 參考 (p. 270) 涵蓋使用 HTTP 呼叫 Amazon Rekognition 操作的方法。如需 Java 參考資訊，請參閱 AWS SDK for Java。

您可以使用的 Amazon Rekognition 服務端點記錄於 AWS 區域與端點。

以 HTTP 呼叫 Amazon Rekognition 時，請使用 POST HTTP 操作。

非儲存與以儲存為基礎的 API 操作

Amazon Rekognition 提供兩種類型的 API 操作。其中一種是非儲存操作，Amazon Rekognition 不會儲存資訊；另一種是儲存操作，Amazon Rekognition 會儲存特定臉部資訊。

非儲存操作

Amazon Rekognition 提供以下用於影像的非儲存 API 操作：

- DetectLabels (p. 324)
- DetectFaces (p. 320)
- CompareFaces (p. 273)
- DetectModerationLabels (p. 329)
- RecognizeCelebrities (p. 388)
- DetectText (p. 333)

Amazon Rekognition 提供以下用於影片的非儲存 API 操作：

- StartLabelDetection (p. 415)
- StartFaceDetection (p. 407)
- StartPersonTracking (p. 419)
- StartCelebrityRecognition (p. 400)
- StartContentModeration (p. 403)

這些稱為 非儲存 API 操作，因為當您執行操作呼叫時，Amazon Rekognition 不會保留任何從輸入影像中找到的任何相關資訊。與所有其他 Amazon Rekognition API 操作相同，非儲存 API 操作不會保留輸入影像位元組。

以下範例說明您在應用程式中可能採用非儲存 API 操作的情況。這些情況假設您有一個存放影像的本機儲存庫。

Example 1：用於在包含特定標籤的本機儲存庫中尋找影像的應用程式

首先，在儲存庫內的每個影像中使用 Amazon Rekognition `DetectLabels` 操作來偵測標籤，並建置客戶端索引，如下所示：

```
Label         ImageID

tree           image-1
flower         image-1
mountain       image-1
tulip          image-2
flower         image-2
```

```
apple          image-3
```

接著，您的應用程式可以搜尋這個索引來尋找在本機儲存庫中包含特定標籤的影像。例如，顯示出現樹的影像。

每個 Amazon Rekognition 偵測的標籤都有相關的信賴度數值。此分數顯示輸入影像內包含該標籤的可信度等級。您可以使用此可信度分數來選擇是否根據應用程式要求的偵測可信度等級來執行額外的客戶端標籤篩選。例如，如果您要求精確標籤，可篩選並選擇有較高可信度分數的標籤 (例如，95% 或更高分數)。如果您的應用程式未要求較高的可信度分數，您可以選擇篩選出有較低可信度分數的標籤 (接近於 50%)。

Example 2：顯示強化臉部影像的應用程式

首先，可使用 Amazon Rekognition `DetectFaces` 操作來偵測本機儲存庫內每個影像中的臉部，並建置客戶端索引，如下所示：操作將傳回每個臉部的中繼資料，包含週框方塊、臉部特徵點 (例如嘴巴與耳朵的位置)、以及臉部屬性 (例如，性別)。您可以將此中繼資料儲存於客戶端本機索引，如下所示：

```
ImageID     FaceID     FaceMetaData

image-1     face-1     <boundingbox>, etc.
image-1     face-2     <boundingbox>, etc.
image-1     face-3     <boundingbox>, etc.
...
```

在此索引中，主索引鑰為 `ImageID` 和 `FaceID` 的組合。

接著，您可以使用索引中的資訊，在應用程式顯示來自本機儲存庫的影像時用以強化影像。例如，您可以加入臉部周圍的週框方塊或者強調臉部特徵。

以儲存為基礎的 API 操作

Amazon Rekognition Image 支援 IndexFaces (p. 371) 操作，您可以使用此操作來偵測影像中臉孔，並將偵測到的臉部特徵資訊保留於 Amazon Rekognition 集合中。此為以儲存體為基礎的 API 操作範例，因服務將在伺服器中保留資訊。

Amazon Rekognition Image 影像提供以下儲存 API 操作：

- IndexFaces (p. 371)
- ListFaces (p. 382)
- SearchFacesByImage (p. 395)
- SearchFaces (p. 392)
- DeleteFaces (p. 294)
- DescribeCollection (p. 302)

Amazon Rekognition Video 影像提供以下儲存 API 操作：

- StartFaceSearch (p. 411)
- CreateStreamProcessor (p. 289)

若要儲存臉部資訊，您必須先在帳戶中其中一個 AWS 區域中建立臉部集合。當您呼叫 `IndexFaces` 操作時可指定此臉部集合。在建立臉部集合並儲存所有臉孔的臉部特徵資訊後，即可搜尋集合來比對臉部。例如，您可以呼叫 `searchFacesByImage.` 來偵測影像中最大的臉孔並在集合中搜尋相符的臉部

Amazon Rekognition Video 操作可存取由 `IndexFaces` 儲存於集合中的臉部資訊。例如，您可以呼叫 StartFaceSearch (p. 411) 來搜尋影片中符合現有集合中之臉孔的人物。

如需建立和管理集合的詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

> Note
>
> 此服務不會保留實際的影像位元組。反之，底層偵測演算法將先偵測輸入影像中的臉部，將臉部特徵擷取至每個臉孔的特徵向量中，接著存放於資料庫。Amazon Rekognition 將在執行臉部比對時使用這些特徵向量。

Example 1：建築物門禁驗證應用程式

首先需建立一個臉部集合，使用 `IndexFaces` 操作將掃描的名牌影像儲存於該集合，而此操作可擷取臉部並儲存為可搜尋的影像向量。接著，當員工進入大樓時，將拍攝員工臉部的影像並傳送至 `SearchFacesByImage` 操作。如果臉部比對的相似性分數達到設定的目標 (例如 99%)，即可驗證該員工身分。

模型版本控制

Amazon Rekognition 使用深入學習模型來執行臉部偵測並在集合中搜尋臉孔。持續根據客戶的意見回饋來改善模型的精確度，並推動深入學習研究領域。這些改善功能將隨模型更新提供。例如，在模型版本 1.0 中，IndexFaces (p. 371) 可將影像中前 15 個最大的臉部編入索引。模型的較新版本則可讓 `IndexFaces` 將影像中前 100 個最大的臉部編入索引。

當您建立新的集合時，將採用最新版的模型版本。為提高精確度，模型將不定期更新。

當有新版本的模型發行時，會發生下列情況：

- 您建立的新集合將採用最新模型。您使用 IndexFaces (p. 371) 加入至新集合中的臉孔將以最新模型進行偵測。
- 您現有的集合繼續使用建立時採用的模型版本。儲存在這些集合中的臉部向量將不會自動更新為最新模型版本。
- 新增到現有集合中的新臉孔將使用該集合原先使用的模型進行偵測。

不同的模型版本彼此無法相容。特別是當影像編入多個集合的索引，而這些集合皆採用不同版本的模型時，同一個偵測到的臉部會有不同的臉部識別碼。若影像編入多個集合的索引，而這些集合使用相同模型時，臉部識別碼將會相同。

若您的集合管理不負責模型的更新，應用程式可能遇到相容性問題。您可以使用為了回應集合操作所傳回的 `FaceModelVersion` 欄位 (例如 `CreateCollection`) 來判斷目前採用的模型版本。您可以透過呼叫，取得現有集合的模型版本 DescribeCollection (p. 302)。如需詳細資訊，請參閱 描述集合 (p. 156)。

集合中現有的臉部向量無法更新為較新的模型版本。由於Amazon Rekognition 不會存放原始影像位元組，因此將無法自動使用較新的模型版本來重新編制影像索引。

若要在現有集合中儲存的臉部使用最新模型，請建立新的集合 (CreateCollection (p. 279)) 並重新將原始影像編入新集合索引 (`Indexfaces`)。您需要更新任何由應用程式儲存的臉部識別碼，因為新集合中的臉部識別碼與舊集合中的臉部識別碼不同。若您不再需要因為舊集合，可使用 DeleteCollection (p. 292) 來刪除。

無狀態操作 (例如 DetectFaces (p. 320)) 會使用最新版本的模型。

Amazon Rekognition 入門

本節主題可協助您開始使用 Amazon Rekognition。如果您是 Amazon Rekognition 新手，建議您先回顧「Amazon Rekognition 運作方式 (p. 4)」中的概念與術語。

主題

- 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)
- 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)
- 步驟 3：開始使用 AWS CLI 和 AWS 開發套件 API (p. 12)
- 步驟 4：開始使用 Amazon Rekognition 主控台 (p. 13)

步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者

初次使用 Amazon Rekognition 之前，請先完成以下作業：

1. 註冊 AWS 帳號 (p. 10)
2. 建立 IAM 使用者 (p. 10)

註冊 AWS 帳號

註冊 Amazon Web Services (AWS) 時，您的 AWS 帳戶會自動註冊所有 AWS 服務，包括 Amazon Rekognition。您只需針對所使用的服務付費。

使用 Amazon Rekognition 時，您僅需為您使用的資源付費。如果您是 AWS 新客戶，可免費開始使用 Amazon Rekognition。如需詳細資訊，請參閱 AWS 免費用量方案。

若您已有 AWS 帳戶，請跳至下一個任務。如果您沒有 AWS 帳戶，請執行下列程序中的步驟建立一個帳戶。

建立 AWS 帳戶

1. Open https://portal.aws.amazon.com/billing/signup.
2. Follow the online instructions.

   Part of the sign-up procedure involves receiving a phone call and entering a verification code on the phone keypad.

請記下您的 AWS 帳戶 ID，因為您下一個任務會需要此 ID。

建立 IAM 使用者

AWS 服務，例如 Amazon Rekognition，會在您進行存取時，要求您提供登入資料。這樣服務可以確定您是否有權存取該服務所擁有的資源。主控台需要您的密碼。您可以建立 AWS 帳戶的存取金鑰，用以存取 AWS CLI 或 API。不過，不建議您透過使用 AWS 帳戶的登入資料來存取 AWS。我們建議您改進行下列動作：

- 使用 AWS Identity and Access Management (IAM) 建立 IAM 使用者。
- 新增使用者到具備管理許可的 IAM 群組。

您可以透過使用特殊 URL 與該 IAM 使用者的登入資料來存取 AWS。

如果您已註冊 AWS，但是尚未為自己建立 IAM 使用者，可以透過使用 IAM 主控台加以建立。請遵循程序在您的帳戶中建立 IAM 使用者。

建立 IAM 使用者並登入主控台

1. 在您的 AWS 帳戶中建立一個擁有管理員權限的 IAM 使用者。如需說明，請參閱 IAM User Guide 中的建立您的第一個 IAM 使用者和管理員群組。
2. 做為 IAM 使用者，請使用特殊 URL 登入 AWS Management Console。如需詳細資訊，請參閱《IAM User Guide》中的使用者如何登入您的帳戶。

   Note

   具有管理員許可的 IAM 使用者可以不受限制地存取您帳戶中的 AWS 服務。如需限制存取 Amazon Rekognition 操作的詳細資訊，請參閱Amazon Rekognition 身分型政策 (p. 249)。本指南中的程式碼範例假設您有一個擁有 `AmazonRekognitionFullAccess` 許可的使用者。`AmazonS3ReadOnlyAccess` 是存取存放在 Amazon S3 儲存貯體中的影像或影片的範例所必須的。Amazon Rekognition Video 存放影片程式碼範例還需要 `AmazonSQSFullAccess` 許可。根據您的安全要求，您可能希望使用限於這些許可的 IAM 群組。如需詳細資訊，請參閱建立 IAM 群組。

For more information about IAM, see the following:

- AWS Identity and Access Management (IAM)
- Getting Started
- IAM User Guide

後續步驟

步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)

步驟 2：設定 AWS CLI 和 AWS 開發套件

以下步驟說明如何安裝本文件中使用的範例的 AWS Command Line Interface (AWS CLI) 和 AWS 開發套件。有幾種不同的方式來驗證 AWS 開發套件呼叫。本指南中的範例假設您使用預設的登入資料設定檔來呼叫 AWS CLI 命令和 AWS 開發套件 API 操作。

如需可用 AWS 區域的清單，請參閱 Amazon Web Services General Reference 中的 區域與端點。

請遵循下列步驟來下載及設定 AWS 開發套件

設定 AWS CLI 和 AWS 開發套件

1. 下載並安裝您要使用的 AWS CLI 和 AWS 開發套件。本指南提供了 AWS CLI、Java、Python、Ruby、Node.js、PHP、.NET 和 JavaScript 的範例。如需其他 AWS 開發套件的相關資訊，請參閱適用於 Amazon Web Services 的工具。
2. 為您在 建立 IAM 使用者 (p. 10) 中建立的使用者建立存取金鑰。

   a. Sign in to the AWS Management Console and open the IAM console at https://console.aws.amazon.com/iam/.
   b. 在導覽窗格中，選擇 Users (使用者)。

c. 選擇您在 建立 IAM 使用者 (p. 10) 中建立之使用者選擇名稱。
d. 選擇 Security credentials (安全登入資料) 標籤。
e. 選擇 Create access key (建立新的存取金鑰)。然後選擇 Download .csv file (下載 .csv 檔案)，將存取金鑰 ID 和私密存取金鑰儲存至電腦上的 CSV 檔案。將檔案存放在安全位置。在關閉此對話方塊後，您將無法再次存取該私密存取金鑰。在您下載該 CSV 檔案後，選擇 Close (關閉)。

3. 若已安裝 AWS CLI，則您可以在命令提示字元中輸入 `aws configure`，為大多數的 AWS 設定登入資料和區域。否則，請使用下列說明。
4. 在您的電腦上，瀏覽至主目錄並建立 `.aws` 目錄。在 Unix 系統上 (例如 Linux 或 macOS)，其會是下列位置：

```
~/.aws
```

在 Windows 上，其會是下列位置：

```
%HOMEPATH%\.aws
```

5. 在 `.aws` 目錄中，建立稱為 `credentials` 的新檔案。
6. 開啟您在步驟 2 下載的登入資料 CSV 檔案，並使用下列格式，將其內容複製到 `credentials` 檔案：

```
[default]
aws_access_key_id = your_access_key_id
aws_secret_access_key = your_secret_access_key
```

以 your_access_key_id 與 your_secret_access_key 替換您的存取金鑰 ID 和私密存取金鑰。

7. 儲存 `Credentials` 檔案，並刪除該 CSV 檔案。
8. 在 `.aws` 目錄中，建立稱為 `config` 的新檔案。
9. 開啟 `config` 檔案，並使用下列格式，輸入您的區域。

```
[default]
region = your_aws_region
```

將您所需要的 AWS 區域 (例如，「us-west-2」) 替換為 your_aws_region。

Note

如果您不選擇區域，則預設使用 us-east-1。

10. 儲存 `config` 檔案。

下一個步驟

步驟 3：開始使用 AWS CLI 和 AWS 開發套件 API (p. 12)

步驟 3：開始使用 AWS CLI 和 AWS 開發套件 API

在設定好要使用的 AWS CLI 和 AWS 開發套件後，您可以建置使用 Amazon Rekognition 的應用程式。下列主題說明如何開始使用 Amazon Rekognition Image 和 Amazon Rekognition Video。

- 使用影像 (p. 23)
- 使用儲存的影片來操作 (p. 49)
- 處理串流影片 (p. 76)

將 AWS CLI 範例格式化

本指南中的 AWS CLI 範例已格式化為 Linux 作業系統。若要搭配 Microsoft Windows 使用範例，您需要變更 `--image` 參數的 JSON 格式，並將換行符號從反斜線 (\) 變更為插入號 (^)。如需 JSON 格式的詳細資訊，請參閱指定 AWS 命令列界面的參數值。以下是專為 Microsoft Windows 格式化的 AWS CLI 命令範例。

```
aws rekognition detect-labels ^
  --image "{\"S3Object\":{\"Bucket\":\"photo-collection\",\"Name\":\"photo.jpg\"}}" ^
  --region us-west-2
```

您也可以提供同時適用於 Microsoft Windows 與 Linux 的 JSON 速記版本。

```
aws rekognition detect-labels --image "S3Object={Bucket=photo-collection,Name=photo.jpg}"
 --region us-west-2
```

如需詳細資訊，請參閱搭配 AWS 命令列界面使用速記語法。

後續步驟

步驟 4：開始使用 Amazon Rekognition 主控台 (p. 13)

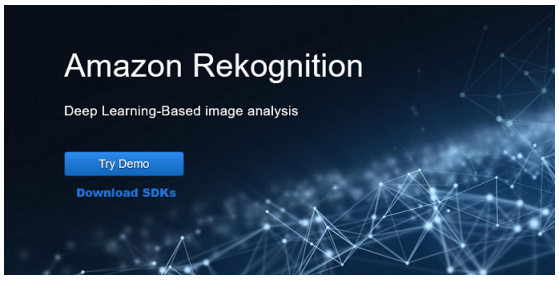
步驟 4：開始使用 Amazon Rekognition 主控台

本節說明如何使用部分 Amazon Rekognition 的功能，例如物件與場景偵測、臉部分析以及在一組影像中執行臉部比對。如需詳細資訊，請參閱 Amazon Rekognition 運作方式 (p. 4)。您也可以使用 Amazon Rekognition API 或 AWS CLI 來偵測物件和場景、偵測臉部，以及比對與搜尋臉部。如需詳細資訊，請參閱步驟 3：開始使用 AWS CLI 和 AWS 開發套件 API (p. 12)。

本節也將說明如何使用 Rekognition 主控台來查看 Rekognition 的 Amazon CloudWatch 彙總指標。

主題

- 練習 1：在影像中偵測物件和場景 (主控台) (p. 13)
- 練習 2：分析影像中的臉部 (主控台) (p. 18)
- 練習 3：比對影像中的臉部 (主控台) (p. 20)
- 練習 4：查看彙總指標 (主控台) (p. 22)



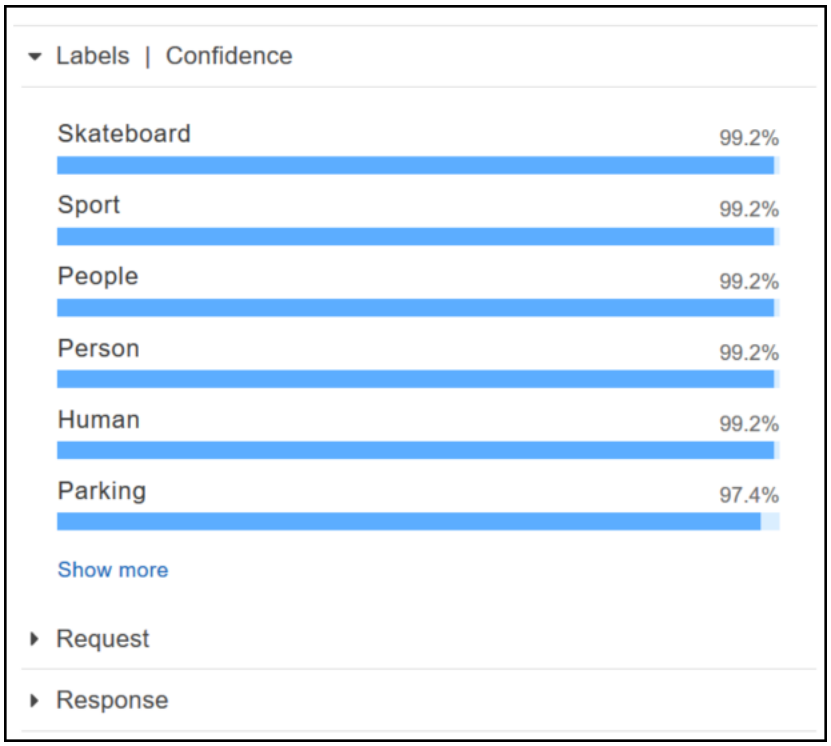
練習 1：在影像中偵測物件和場景 (主控台)

此節將具體說明 Amazon Rekognition 的物件與場景偵測功能的運作方式。當您指定影像做為輸入檔時，服務會偵測影像中的物件和場景，並加上每個物件與場景的可信度分數百分比後傳回結果。

例如，Amazon Rekognition 將偵測範例影像中的下列物件與場景：滑板、運動、人物、自動、汽車與其他車輛。



Amazon Rekognition 也將對範例影像中偵測到的物件傳回可信度分數，如下列範例回應所示。



若要查看所有顯示於回應中的可信度分數，請選擇 Labels | Confidence (標籤 | 可信度) 窗格中的 Show more (顯示其他)。

您也可以查看對 API 的請求以及來自 API 的回應做為參考。

要求

```
{
   "contentString":{
      "Attributes":[
         "ALL"
      ],
      "Image":{
         "S3Object":{
            "Bucket":"console-sample-images",
            "Name":"skateboard.jpg"
         }
      }
   }
}
```

回應

```
{
   "Labels":[
      {
         "Confidence":99.25359344482422,
         "Name":"Skateboard"
      },
      {
         "Confidence":99.25359344482422,
         "Name":"Sport"
      },
      {
         "Confidence":99.24723052978516,
         "Name":"People"
      },
      {
         "Confidence":99.24723052978516,
         "Name":"Person"
      },
      {
         "Confidence":99.23908233642578,
         "Name":"Human"
      },
      {
         "Confidence":97.42484283447266,
         "Name":"Parking"
      },
      {
         "Confidence":97.42484283447266,
         "Name":"Parking Lot"
      },
      {
         "Confidence":91.53300476074219,
         "Name":"Automobile"
      },
      {
         "Confidence":91.53300476074219,
         "Name":"Car"
      },
      {
         "Confidence":91.53300476074219,
         "Name":"Vehicle"
      },
      {
         "Confidence":76.85114288330078,
```

```
      "Name":"Intersection"
   },
   {
      "Confidence":76.85114288330078,
      "Name":"Road"
   },
   {
      "Confidence":76.21503448486328,
      "Name":"Boardwalk"
   },
   {
      "Confidence":76.21503448486328,
      "Name":"Path"
   },
   {
      "Confidence":76.21503448486328,
      "Name":"Pavement"
   },
   {
      "Confidence":76.21503448486328,
      "Name":"Sidewalk"
   },
   {
      "Confidence":76.21503448486328,
      "Name":"Walkway"
   },
   {
      "Confidence":66.71541595458984,
      "Name":"Building"
   },
   {
      "Confidence":62.04711151123047,
      "Name":"Coupe"
   },
   {
      "Confidence":62.04711151123047,
      "Name":"Sports Car"
   },
   {
      "Confidence":61.98909378051758,
      "Name":"City"
   },
   {
      "Confidence":61.98909378051758,
      "Name":"Downtown"
   },
   {
      "Confidence":61.98909378051758,
      "Name":"Urban"
   },
   {
      "Confidence":60.978023529052734,
      "Name":"Neighborhood"
   },
   {
      "Confidence":60.978023529052734,
      "Name":"Town"
   },
   {
      "Confidence":59.22066116333008,
      "Name":"Sedan"
   },
   {
      "Confidence":56.48063278198242,
      "Name":"Street"
   },
```

```
        {
            "Confidence":54.235477447509766,
            "Name":"Housing"
        },
        {
            "Confidence":53.85226058959961,
            "Name":"Metropolis"
        },
        {
            "Confidence":52.001792907714844,
            "Name":"Office Building"
        },
        {
            "Confidence":51.325313568115234,
            "Name":"Suv"
        },
        {
            "Confidence":51.26075744628906,
            "Name":"Apartment Building"
        },
        {
            "Confidence":51.26075744628906,
            "Name":"High Rise"
        },
        {
            "Confidence":50.68067932128906,
            "Name":"Pedestrian"
        },
        {
            "Confidence":50.59548568725586,
            "Name":"Freeway"
        },
        {
            "Confidence":50.568580627441406,
            "Name":"Bumper"
        }
    ]
}
```


如需詳細資訊，請參閱 Amazon Rekognition 運作方式 (p. 4)。

在您提供的影像中偵測物件和場景

您可以上傳您擁有的影像或提供影像的 URL 做為用於 Amazon Rekognition 主控台的輸入檔。Amazon Rekognition 將傳回物件和場景、每個物件的可信度分數以及在您所提供的影像中偵測到的場景。

> Note
>
> 影像大小必須少於 5MB，且須為 JPEG 或 PNG 格式。

若要在您提供的影像中偵測物件和場景

1. Open the Amazon Rekognition console at https://console.aws.amazon.com/rekognition/.
2. 選擇 Object and scene detection (物件和場景偵測)。
3. 請執行下列其中一項：
   - 上傳影像 – 選擇 Upload (上傳)，前往您儲存影像的位置，然後選擇影像。
   - 使用 URL – 在文字方塊中輸入 URL，然後選擇 Go (前往)。
4. 在 Labels | Confidence (標籤 | 可信度) 窗格中檢視每個偵測到的標籤之可信度分數。

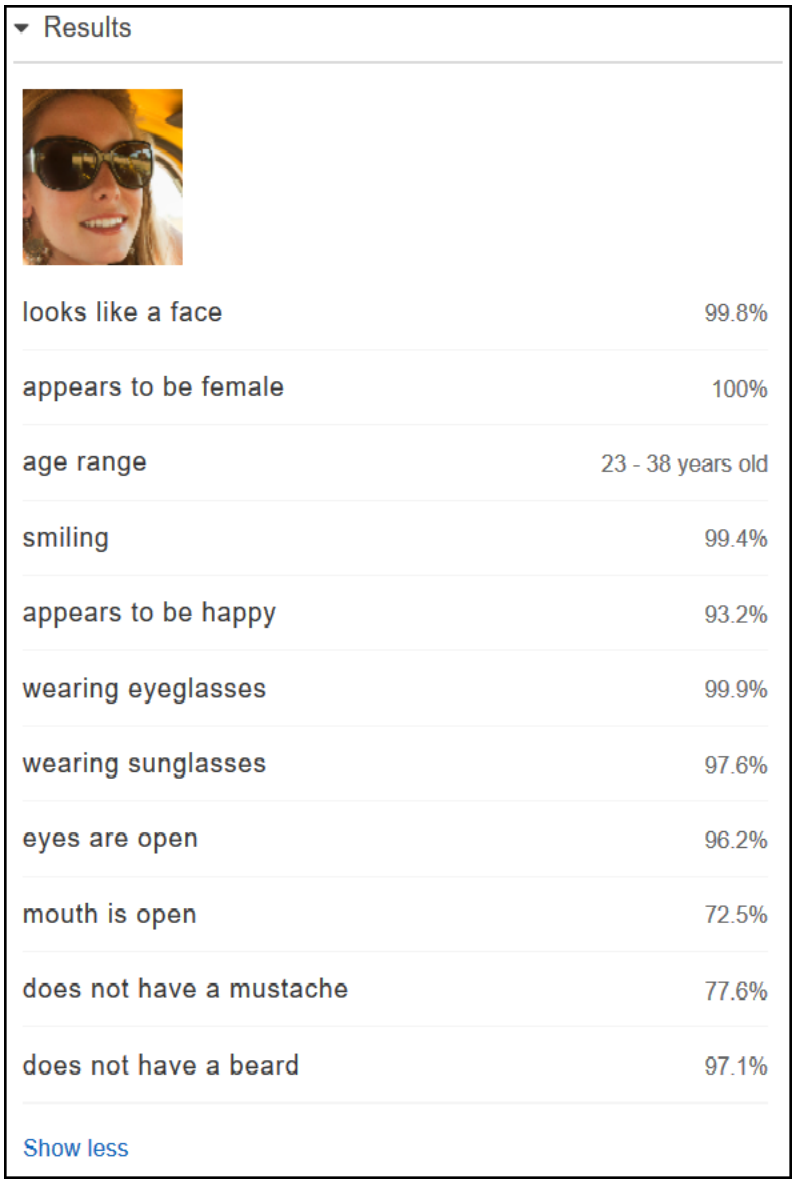
練習 2：分析影像中的臉部 (主控台)

本節說明如何使用 Amazon Rekognition 主控台來偵測臉部與分析影像中的臉部屬性。當您提供含有臉部的影像做為輸入檔時，服務會偵測影像中的臉部、分析臉部的臉部屬性，然後傳回影像中偵測到之臉部與臉部屬性的可信度分數百分比。如需詳細資訊，請參閱 Amazon Rekognition 運作方式 (p. 4)。

例如，如果您選擇以下範例影像做為輸入檔，Amazon Rekognition 將該視該影像為臉部進行偵測，並傳回偵測到的臉部與臉部屬性之可信度分數。



下列顯示回應範例。



如果輸入影像中有多個臉部，Rekognition 最多可在影像中偵測 100 個臉部。每一個偵測到的臉部皆以方框標註。當您按一下標註了臉部的方框時，Rekognition 將在 Faces | Confidence (臉部 | 可信度) 窗格中顯示偵測到的臉部與其屬性之可信度分數。

分析您提供的影像中的臉部

可在 Amazon Rekognition 主控台中上傳您自己的影像或提供影像的 URL。

> **Note**
>
> 影像大小必須少於 5MB，且須為 JPEG 或 PNG 格式。

若要分析您提供的影像中的臉部

1. Open the Amazon Rekognition console at https://console.aws.amazon.com/rekognition/.
2. 選擇 Facial analysis (臉部分析)。

3. 請執行下列其中一項：
   - 上傳影像 – 選擇 Upload (上傳)，前往您儲存影像的位置，然後選擇影像。
   - 使用 URL – 在文字方塊中輸入 URL，然後選擇 Go (前往)。
4. 在 Faces | Confidence (臉部 | 可信度) 窗格中檢視其中一個偵測到的臉部可信度分數與其屬性。
5. 如果影像中有多個臉部，請選擇其它臉部來查看屬性與分數。

練習 3：比對影像中的臉部 (主控台)

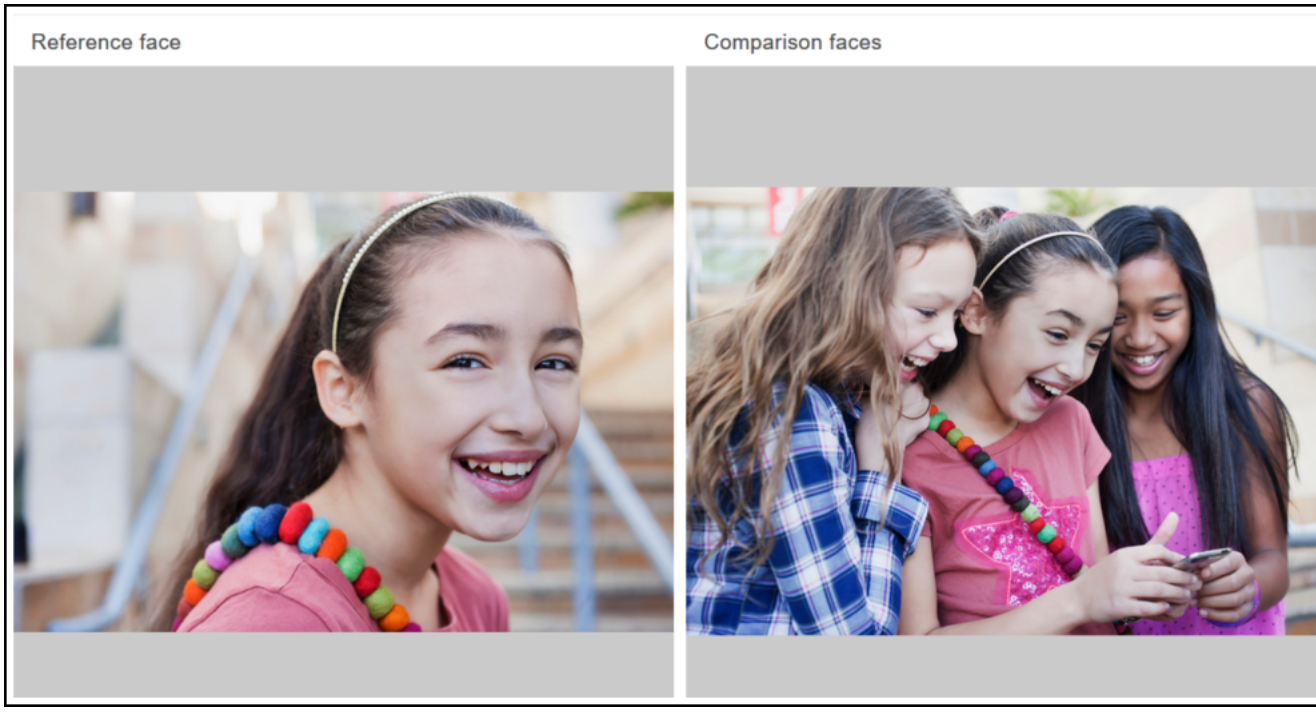
本節說明如何使用 Amazon Rekognition 主控台在一組影像的多個臉孔中進行臉部比對。當您指定 Reference face (參考臉部) (原始) 和 Comparison faces (對比臉部) (目標) 影像時，Rekognition 將以原始影像 (也就是參考臉部) 中最大的臉孔為基礎來比對目標影像 (也就是對比臉部) 中的臉孔，最多可辨識 100 張臉孔，接著便可判定來源影像中的臉部與目標影像中的臉部之間的相似度。每次比對的相似度分數會顯示在 Results (結果) 窗格中。

如果目標影像內包含多張臉孔，Rekognition 將對原始影像中的臉孔與目標影像中所偵測到的最多 100 個臉部進行比對，然後指定相似度分數給每個比對結果。

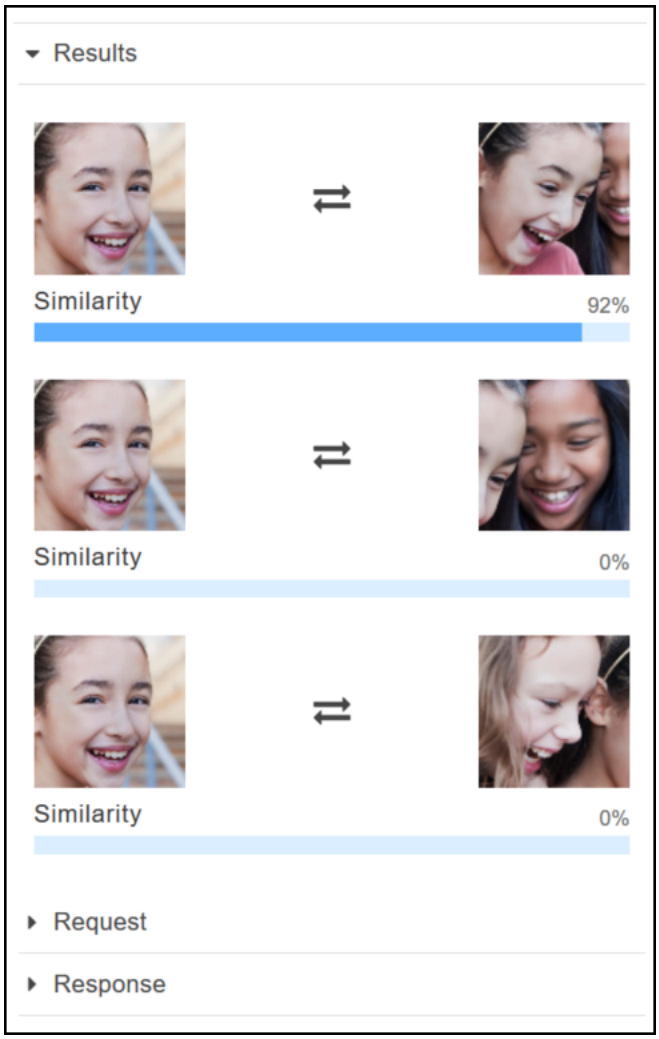
如果原始影像含有多張臉孔，服務將偵測原始影像中最大的臉孔，使用它與目標影像中偵測到的每個臉孔進行比對。

如需更多詳細資訊，請參閱 比較影像中的臉部 (p. 133)。

例如，使用顯示在左側的範例影像做為原始影像、右側的範例影像則做為目標影像，Rekognition 將偵測原始影像中的臉孔並與在目標影像中偵測到的每個臉孔進行比較，最後為每組顯示相似度分數。



以下顯示在目標影像偵測到的臉孔以及每個臉孔的相似度分數。



比較您提供的影像中的臉部

您可以上傳自己的原始和目標影像，提供 Rekognition 比對影像中的臉孔，或者您可以指定影像所在的 URL 位置。

Note

影像大小必須少於 5MB，且須為 JPEG 或 PNG 格式。

若要比較影像中的臉部

1. Open the Amazon Rekognition console at https://console.aws.amazon.com/rekognition/.
2. 選擇 Face comparison (臉部比較)。
3. 指定原始影像可使用以下其中一個方法：
   - 上傳影像 – 選擇左側的 Upload (上傳)，前往您儲存原始影像的位置，然後選擇影像。
   - 使用 URL – 在文字方塊中輸入原始影像的 URL，然後選擇 Go (前往)。
4. 指定目標影像可使用以下其中一個方法：
   - 上傳影像 – 選擇右側的 Upload (上傳)，前往您儲存目標影像的位置，然後選擇影像。
   - 使用 URL – 在文字方塊中輸入原始影像的 URL，然後選擇 Go (前往)。

5. Rekognition 將偵測原始影像中最大的臉孔並與目標影像中最多 100 張臉孔進行比對，接著在 Results (結果) 窗格中顯示每個臉部配對的相似度分數。

練習 4：查看彙總指標 (主控台)

Amazon Rekognition 指標窗格將顯示在一段期間內的個別 Rekognition 指標彙總活動圖形。例如，`SuccessfulRequestCount` 彙總指標會顯示過去七天內對所有 Rekognition API 操作的成功請求總數。

下表列出顯示在 Rekognition 指標窗格中的圖表以及和對應的 Rekognition 指標。如需詳細資訊，請參閱 Rekognition 的 CloudWatch 指標 (p. 260)。

圖表	彙總指標
成功呼叫	SuccessfulRequestCount
客戶端錯誤	UserErrorCount
伺服器錯誤	ServerErrorCount
受節制	ThrottledCount
偵測到的標籤	DetectedLabelCount
偵測到的臉部	DetectedFaceCount

每個圖表顯示指定期間內收集到的彙總指標資料。也會顯示時間期間內彙總指標資料的總數。若要查看個別 API 呼叫的指標，請選擇各圖表下方的連結。

若要允許使用者存取 Rekognition 指標窗格，請確認使用者擁有適用的 CloudWatch 和 Rekognition 許可。例如，使用者若擁有 `AmazonRekognitionReadOnlyAccess` 和 `CloudWatchReadOnlyAccess` 受管政策的使用權限，便可查看指標窗格。如果使用者沒有所需的權限，開啟指標窗格時便不會顯示圖表。如需詳細資訊，請參閱適用於 Amazon Rekognition 的 Identity and Access Management (p. 245)。

如需以 CloudWatch 監控 Rekognition 的詳細資訊，請參閱 監控 Rekognition (p. 258)。

若要查看彙總指標 (主控台)

1. Open the Amazon Rekognition console at https://console.aws.amazon.com/rekognition/.
2. 在導覽窗格中，選擇指標。
3. 在下拉式清單中，選取您需要的指標所屬期間。
4. 若要更新圖表，請選擇 Refresh (重新整理) 按鈕。
5. 要查看特定彙總指標的詳細 CloudWatch 指標，請選擇指標圖形下的 See details on CloudWatch (在 CloudWatch 上查看詳細資訊)。

對 Amazon Rekognition API 進行程式設計

您可以將 Amazon Rekognition API 操作用於影像、已儲存影片和串流視訊。本節提供有關編寫程式碼以用於存取 Amazon Rekognition 的一般資訊。本指南中的其他章節提供特定影像和影片類型 (例如臉部偵測) 的分析相關資訊。

主題

- 使用影像 (p. 23)
- 使用儲存的影片來操作 (p. 49)
- 處理串流影片 (p. 76)
- 錯誤處理 (p. 95)
- 使用 Amazon Rekognition 作為 FedRAMP 授權服務 (p. 100)

使用影像

本節說明 Amazon Rekognition Image 可對影像執行的分析類型。

- 標籤偵測
- 臉部偵測與比較
- 名人辨識
- 影像管制
- 影像偵測中的文字

這些分析皆由非儲存 API 操作執行，在這項操作中，Amazon Rekognition Image 不會保存該操作探索到的任何資訊。非儲存 API 操作不會保存任何輸入影像位元組。如需詳細資訊，請參閱 非儲存與以儲存為基礎的 API 操作 (p. 7)。

Amazon Rekognition Image 也可將臉部中繼資料存放在集合中，以供稍後擷取。如需詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

在本節中，您會使用 Amazon Rekognition Image API 操作來分析存放在 Amazon S3 儲存貯體中的影像，以及從本機檔案系統載入的影像位元組。本節也說明如何從 .jpg 影像取得影像方向資訊。

主題

- 影像 (p. 23)
- 分析存放在 Amazon S3 儲存貯體中的影像 (p. 24)
- 分析從本機檔案系統載入的影像 (p. 29)
- 顯示週框方塊 (p. 37)
- 取得影像方向與週框方塊座標 (p. 42)

影像

Amazon Rekognition Image 操作可以分析 .jpg 或 .png 格式的影像。

您可以將影像位元組做為呼叫的一部分傳遞至 Amazon Rekognition Image 操作，或參考現有的 Amazon S3 物件。如需分析存放在 Amazon S3 儲存貯體中影像的範例，請參閱 分析存放在 Amazon S3 儲存貯體中的

影像 (p. 24)。如需傳遞影像位元組到 Amazon Rekognition Image API 操作的範例，請參閱 分析從本機檔案系統載入的影像 (p. 29)。

如果您使用 HTTP 並將影像位元組做為 Amazon Rekognition Image 操作的一部分來傳遞，影像位元組必須是 base64 編碼字串。如果您使用 AWS 開發套件並將影像位元組做為 API 操作呼叫的一部分來傳遞，則需視您使用的語言，對影像位元組進行 base64 編碼。

下列常見的 AWS 開發套件會自動對影像進行 base64 編碼，因此您不需要在呼叫 Amazon Rekognition Image API 操作前，先對影像位元組進行編碼。

- Java
- JavaScript
- Python
- PHP

如果您使用其他 AWS 開發套件，並在呼叫 Rekognition API 操作時收到影像格式錯誤，請嘗試先對影像位元組進行 base64 編碼，再傳遞至 Rekognition API 操作。

如果您使用 AWS CLI 呼叫 Amazon Rekognition Image 操作，則不支援將影像位元組做為呼叫的一部分來傳遞。您必須先將影像上傳至 Amazon S3 儲存貯體，再呼叫參考已上傳影像的操作。

> Note
>
> 如果您傳遞存放在 `S3Object` 中的影像而不是影像位元組，影像就不需經過 Base64 編碼。

如需確保 Amazon Rekognition Image 操作延遲盡可能降到最低的資訊，請參閱 Amazon Rekognition Image 操作延遲 (p. 103)。

修正影像方向

在幾個 Rekognition API 操作中，會傳回已分析影像的方向。知道影像方向很重要，因為這可讓您改變影像的顯示方向。Rekognition 分析臉部的 API 操作也會傳回影像中臉部位置的週框方塊。您可以使用週框方塊，在影像上的臉部周圍顯示方塊。傳回的週框方塊座標受影像方向所影響，所以您可能需要移動週框方塊座標，才能在臉部周圍正確地顯示方塊。如需詳細資訊，請參閱 取得影像方向與週框方塊座標 (p. 42)。

分析存放在 Amazon S3 儲存貯體中的影像

Amazon Rekognition Image 可以分析存放在 Amazon S3 儲存貯體中的影像，或做為影像位元組提供的影像。

在本主題中，您會使用 DetectLabels (p. 324) API 操作，來偵測存放在 Amazon S3 儲存貯體之影像 (JPEG 或 PNG) 中的物件、概念與場景。您可以使用 the section called “Image” (p. 472) 輸入參數，將影像傳遞至 Amazon Rekognition Image API 操作。在 `Image` 內，您指定 S3Object (p. 495) 物件屬性以參考存放在 S3 儲存貯體中的影像。存放在 Amazon S3 儲存貯體中的影像位元組，不需要 base64 編碼。如需詳細資訊，請參閱 影像 (p. 23)。

含有 S3 物件的 S3 儲存貯體區域必須符合您用於 Amazon Rekognition Image 操作的區域。

在此範例中，JSON 要求 `DetectLabels`，而來源影像 (`input.jpg`) 是從名為 `MyBucket` 儲存貯體 Amazon S3 載入。

```
{
    "Image": {
        "S3Object": {
            "Bucket": "MyBucket",
            "Name": "input.jpg"
        }
    },
    "MaxLabels": 10,
    "MinConfidence": 75
```

```
}
```

下列範例使用各種 AWS 開發套件和 AWS CLI 呼叫 `DetectLabels` 操作。如需有關 `DetectLabels` 操作回應的資訊，請參閱 DetectLabels 回應 (p. 112)。

偵測影像中的標籤

1. 如果您尚未：
   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 上傳包含一個或多個物件的影像—例如樹、房子和船—至您的 S3 儲存貯體。影像的格式必須是 .jpg 或 .png 格式。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
3. 使用下列範例來呼叫 `DetectLabels` 操作。

   Java

   此範例顯示一份在輸入影像中偵測到的標籤清單。將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package com.amazonaws.samples;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.DetectLabelsRequest;
import com.amazonaws.services.rekognition.model.DetectLabelsResult;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.Label;
import com.amazonaws.services.rekognition.model.S3Object;
import java.util.List;

public class DetectLabels {

   public static void main(String[] args) throws Exception {

      String photo = "input.jpg";
      String bucket = "bucket";


      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      DetectLabelsRequest request = new DetectLabelsRequest()
           .withImage(new Image()
           .withS3Object(new S3Object()
           .withName(photo).withBucket(bucket)))
           .withMaxLabels(10)
           .withMinConfidence(75F);

      try {
         DetectLabelsResult result = rekognitionClient.detectLabels(request);
         List <Label> labels = result.getLabels();
```

```
         System.out.println("Detected labels for " + photo);
         for (Label label: labels) {
            System.out.println(label.getName() + ": " +
 label.getConfidence().toString());
         }
      } catch(AmazonRekognitionException e) {
         e.printStackTrace();
      }
   }
}
```

AWS CLI

此範例顯示 `detect-labels` CLI 操作的 JSON 輸出。將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
aws rekognition detect-labels \
--image '{"S3Object":{"Bucket":"bucket","Name":"file"}}'
```

Python

此範例顯示在輸入影像中偵測到的標籤。將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def detect_labels(photo, bucket):

    client=boto3.client('rekognition')

    response = client.detect_labels(Image={'S3Object':
{'Bucket':bucket,'Name':photo}},
        MaxLabels=10)

    print('Detected labels for ' + photo)
    print()
    for label in response['Labels']:
        print ("Label: " + label['Name'])
        print ("Confidence: " + str(label['Confidence']))
        print ("Instances:")
        for instance in label['Instances']:
            print ("  Bounding box")
            print ("    Top: " + str(instance['BoundingBox']['Top']))
            print ("    Left: " + str(instance['BoundingBox']['Left']))
            print ("    Width: " +  str(instance['BoundingBox']['Width']))
            print ("    Height: " +  str(instance['BoundingBox']['Height']))
            print ("  Confidence: " + str(instance['Confidence']))
            print()

        print ("Parents:")
        for parent in label['Parents']:
            print ("   " + parent['Name'])
        print ("----------")
        print ()
    return len(response['Labels'])


def main():
    photo=''
```

```
    bucket=''
    label_count=detect_labels(photo, bucket)
    print("Labels detected: " + str(label_count))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示一份在輸入影像中偵測到的標籤清單。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectLabels
{
    public static void Example()
    {
        String photo = "input.jpg";
        String bucket = "bucket";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectLabelsRequest detectlabelsRequest = new DetectLabelsRequest()
        {
            Image = new Image()
            {
                S3Object = new S3Object()
                {
                    Name = photo,
                    Bucket = bucket
                },
            },
            MaxLabels = 10,
            MinConfidence = 75F
        };

        try
        {
            DetectLabelsResponse detectLabelsResponse =
 rekognitionClient.DetectLabels(detectlabelsRequest);
            Console.WriteLine("Detected labels for " + photo);
            foreach (Label label in detectLabelsResponse.Labels)
                Console.WriteLine("{0}: {1}", label.Name, label.Confidence);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

Ruby

此範例顯示一份在輸入影像中偵測到的標籤清單。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

   # Add to your Gemfile
   # gem 'aws-sdk-rekognition'
   require 'aws-sdk-rekognition'
   credentials = Aws::Credentials.new(
      ENV['AWS_ACCESS_KEY_ID'],
      ENV['AWS_SECRET_ACCESS_KEY']
   )
   bucket = 'bucket' # the bucketname without s3://
   photo  = 'photo'# the name of file
   client   = Aws::Rekognition::Client.new credentials: credentials
   attrs = {
     image: {
       s3_object: {
         bucket: bucket,
         name: photo
       },
     },
     attributes: ['ALL']
   }
   response = client.detect_faces attrs
   puts "Detected faces for: #{photo}"
   response.face_details.each do |face_detail|
     low  = face_detail.age_range.low
     high = face_detail.age_range.high
     puts "The detected face is between: #{low} and #{high} years old"
     puts "All other attributes:"
     puts "  bounding_box.width:     #{face_detail.bounding_box.width}"
     puts "  bounding_box.height:    #{face_detail.bounding_box.height}"
     puts "  bounding_box.left:      #{face_detail.bounding_box.left}"
     puts "  bounding_box.top:       #{face_detail.bounding_box.top}"
     puts "  age.range.low:          #{face_detail.age_range.low}"
     puts "  age.range.high:         #{face_detail.age_range.high}"
     puts "  smile.value:            #{face_detail.smile.value}"
     puts "  smile.confidence:       #{face_detail.smile.confidence}"
     puts "  eyeglasses.value:       #{face_detail.eyeglasses.value}"
     puts "  eyeglasses.confidence:  #{face_detail.eyeglasses.confidence}"
     puts "  sunglasses.value:       #{face_detail.sunglasses.value}"
     puts "  sunglasses.confidence:  #{face_detail.sunglasses.confidence}"
     puts "  gender.value:           #{face_detail.gender.value}"
     puts "  gender.confidence:      #{face_detail.gender.confidence}"
     puts "  beard.value:            #{face_detail.beard.value}"
     puts "  beard.confidence:       #{face_detail.beard.confidence}"
     puts "  mustache.value:         #{face_detail.mustache.value}"
     puts "  mustache.confidence:    #{face_detail.mustache.confidence}"
     puts "  eyes_open.value:        #{face_detail.eyes_open.value}"
     puts "  eyes_open.confidence:   #{face_detail.eyes_open.confidence}"
     puts "  mout_open.value:        #{face_detail.mouth_open.value}"
     puts "  mout_open.confidence:   #{face_detail.mouth_open.confidence}"
     puts "  emotions[0].type:       #{face_detail.emotions[0].type}"
     puts "  emotions[0].confidence: #{face_detail.emotions[0].confidence}"
     puts "  landmarks[0].type:      #{face_detail.landmarks[0].type}"
     puts "  landmarks[0].x:         #{face_detail.landmarks[0].x}"
     puts "  landmarks[0].y:         #{face_detail.landmarks[0].y}"
     puts "  pose.roll:              #{face_detail.pose.roll}"
     puts "  pose.yaw:               #{face_detail.pose.yaw}"
```

```
    puts "  pose.pitch:             #{face_detail.pose.pitch}"
    puts "  quality.brightness:     #{face_detail.quality.brightness}"
    puts "  quality.sharpness:      #{face_detail.quality.sharpness}"
    puts "  confidence:             #{face_detail.confidence}"
    puts "------------"
    puts ""
  end
```

分析從本機檔案系統載入的影像

Amazon Rekognition Image 操作可以分析做為影像位元組提供的影像，或存放在 Amazon S3 儲存貯體中的影像。

這些主題提供範例，其示範透過使用從本機檔案系統載入的檔案，將影像位元組提供給 Amazon Rekognition Image API 操作。您可以使用 Amazon Rekognition 輸入參數，將影像位元組傳遞至 Image (p. 472) API 操作。在 `Image` 內，指定 `Bytes` 屬性，以傳遞 base64 編碼的影像位元組。

透過使用 `Bytes` 輸入參數傳遞至 Amazon Rekognition API 操作的影像位元組，必須經過 base64 編碼。這些範例使用的 AWS 開發套件，會自動以 base64 編碼影像。在呼叫 Amazon Rekognition API 操作前，您不需要再編碼影像位元組。如需詳細資訊，請參閱 影像 (p. 23)。

在這個範例中，JSON 請求 `DetectLabels`，來源影像位元組以 `Bytes` 輸入參數傳遞。

```
{
    "Image": {
        "Bytes": "/9j/4AAQSk....."
    },
    "MaxLabels": 10,
    "MinConfidence": 77
}
```

下列範例使用各種 AWS 開發套件和 AWS CLI 呼叫 `DetectLabels` 操作。如需有關 `DetectLabels` 操作回應的資訊，請參閱 DetectLabels 回應 (p. 112)。

對於用戶端 JavaScript 範例，請參閱 使用於 JavaScript (p. 34)。

偵測本機映像中的標籤

1. 如果您尚未：

   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。

2. 使用下列範例來呼叫 `DetectLabels` 操作。

   Java

   下列 Java 範例示範如何從本機檔案系統載入影像，並運用 detectLabels AWS 開發套件操作來偵測標籤。將 `photo` 的值變更為某個影像檔案的路徑和檔案名稱 (.jpg 或 .png 格式)。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
```

```
import java.nio.ByteBuffer;
import java.util.List;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.AmazonClientException;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.DetectLabelsRequest;
import com.amazonaws.services.rekognition.model.DetectLabelsResult;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.Label;
import com.amazonaws.util.IOUtils;

public class DetectLabelsLocalFile {
    public static void main(String[] args) throws Exception {
     String photo="input.jpg";


        ByteBuffer imageBytes;
        try (InputStream inputStream = new FileInputStream(new File(photo))) {
            imageBytes = ByteBuffer.wrap(IOUtils.toByteArray(inputStream));
        }


        AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

        DetectLabelsRequest request = new DetectLabelsRequest()
                .withImage(new Image()
                        .withBytes(imageBytes))
                .withMaxLabels(10)
                .withMinConfidence(77F);

        try {

            DetectLabelsResult result = rekognitionClient.detectLabels(request);
            List <Label> labels = result.getLabels();

            System.out.println("Detected labels for " + photo);
            for (Label label: labels) {
               System.out.println(label.getName() + ": " +
 label.getConfidence().toString());
            }

        } catch (AmazonRekognitionException e) {
            e.printStackTrace();
        }

    }
}
```

Python

下列適用於 Python 的 AWS 開發套件範例示範如何從本機檔案系統載入影像，並呼叫 detect_labels 操作。將 `imageFile` 的值變更為某個影像檔案的路徑和檔案名稱 (.jpg 或 .png 格式)。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def detect_labels_local_file(photo):
```

```
    client=boto3.client('rekognition')

    with open(photo, 'rb') as image:
        response = client.detect_labels(Image={'Bytes': image.read()})

    print('Detected labels in ' + photo)
    for label in response['Labels']:
        print (label['Name'] + ' : ' + str(label['Confidence']))

    return len(response['Labels'])

def main():
    photo='photo'

    label_count=detect_labels_local_file(photo)
    print("Labels detected: " + str(label_count))


if __name__ == "__main__":
    main()
```

.NET

下列 Java 範例示範如何從本機檔案系統載入影像，並運用 DetectLabels 操作來偵測標籤。將 photo 的值變更為某個影像檔案的路徑和檔案名稱 (.jpg 或 .png 格式)。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.IO;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectLabelsLocalfile
{
    public static void Example()
    {
        String photo = "input.jpg";

        Amazon.Rekognition.Model.Image image = new
 Amazon.Rekognition.Model.Image();
        try
        {
            using (FileStream fs = new FileStream(photo, FileMode.Open,
 FileAccess.Read))
            {
                byte[] data = null;
                data = new byte[fs.Length];
                fs.Read(data, 0, (int)fs.Length);
                image.Bytes = new MemoryStream(data);
            }
        }
        catch (Exception)
        {
            Console.WriteLine("Failed to load file " + photo);
            return;
        }
```

```
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectLabelsRequest detectlabelsRequest = new DetectLabelsRequest()
        {
            Image = image,
            MaxLabels = 10,
            MinConfidence = 77F
        };

        try
        {
            DetectLabelsResponse detectLabelsResponse =
 rekognitionClient.DetectLabels(detectlabelsRequest);
            Console.WriteLine("Detected labels for " + photo);
            foreach (Label label in detectLabelsResponse.Labels)
                Console.WriteLine("{0}: {1}", label.Name, label.Confidence);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

PHP

下列適用於 PHP 的 AWS 開發套件範例示範如何從本機檔案系統載入影像，並呼叫 DetectFaces API 操作。將 photo 的值變更為某個影像檔案的路徑和檔案名稱 (.jpg 或 .png 格式)。

```
<?php
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    require 'vendor/autoload.php';

    use Aws\Rekognition\RekognitionClient;

    $options = [
       'region'            => 'us-west-2',
        'version'           => 'latest'
    ];

    $rekognition = new RekognitionClient($options);

    // Get local image
    $photo = 'input.jpg';
    $fp_image = fopen($photo, 'r');
    $image = fread($fp_image, filesize($photo));
    fclose($fp_image);


    // Call DetectFaces
    $result = $rekognition->DetectFaces(array(
       'Image' => array(
          'Bytes' => $image,
       ),
       'Attributes' => array('ALL')
       )
    );

    // Display info for each detected person
    print 'People: Image position and estimated age' . PHP_EOL;
```

```
    for ($n=0;$n<sizeof($result['FaceDetails']); $n++){

      print 'Position: ' . $result['FaceDetails'][$n]['BoundingBox']['Left'] . " "
      . $result['FaceDetails'][$n]['BoundingBox']['Top']
      . PHP_EOL
      . 'Age (low): '.$result['FaceDetails'][$n]['AgeRange']['Low']
      .  PHP_EOL
      . 'Age (high): ' . $result['FaceDetails'][$n]['AgeRange']['High']
      .  PHP_EOL . PHP_EOL;
    }
?>
```

Ruby

此範例顯示一份在輸入影像中偵測到的標籤清單。將 photo 的值變更為某個影像檔案的路徑和檔案名稱 (.jpg 或 .png 格式)。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # gem 'aws-sdk-rekognition'
    require 'aws-sdk-rekognition'
    credentials = Aws::Credentials.new(
       ENV['AWS_ACCESS_KEY_ID'],
       ENV['AWS_SECRET_ACCESS_KEY']
    )
    client   = Aws::Rekognition::Client.new credentials: credentials
    photo = 'photo.jpg'
    path = File.expand_path(photo) # expand path relative to the current directory
    file = File.read(path)
    attrs = {
      image: {
        bytes: file
      },
      max_labels: 10
    }
    response = client.detect_labels attrs
    puts "Detected labels for: #{photo}"
    response.labels.each do |label|
      puts "Label:      #{label.name}"
      puts "Confidence: #{label.confidence}"
      puts "Instances:"
      label['instances'].each do |instance|
        box = instance['bounding_box']
        puts "  Bounding box:"
        puts "    Top:        #{box.top}"
        puts "    Left:       #{box.left}"
        puts "    Width:      #{box.width}"
        puts "    Height:     #{box.height}"
        puts "  Confidence: #{instance.confidence}"
      end
      puts "Parents:"
      label.parents.each do |parent|
        puts "  #{parent.name}"
      end
      puts "------------"
      puts ""
    end
```

使用於 JavaScript

以下 JavaScript 網頁範例可讓使用者選擇影像並檢視在影像中偵測到之臉部的估測年齡。估測年齡將由對 the section called “DetectFaces” (p. 320) 的呼叫傳回。

所選影像已使用 JavaScript `FileReader.readAsDataURL` 函數載入，並以 base64 編碼影像。此步驟對於顯示 HTML 畫布上的影像有幫助。但是，這表示影像位元組需要在傳遞到 Amazon Rekognition Image 操作前解碼。此範例說明如何解碼已載入的影像位元組。如果已編碼的影像位元組對您來說沒有幫助，請改用 `FileReader.readAsArrayBuffer`，因為載入的影像未經過編碼。這表示可以在未先解碼影像位元組的情況下呼叫 Amazon Rekognition Image 操作。如需範例，請參閱 使用 readAsArrayBuffer (p. 36)。

若要執行 JavaScript 範本

1. 載入範例原始程式碼到編輯器。
2. 取得 Amazon Cognito 身分集區識別碼。如需詳細資訊，請參閱 取得 Amazon Cognito 身分集區識別碼 (p. 37)。
3. 在範本程式碼的 `AnonLog` 函數中，變更 `IdentityPoolIdToUse` 與 `RegionToUse` 為您在 取得 Amazon Cognito 身分集區識別碼 (p. 37) 步驟 9 中記下的值。
4. 在 `DetectFaces` 函數中，變更 `RegionToUse` 為您在之前的步驟中使用的值。
5. 將範例原始程式碼儲存為 `.html` 檔案。
6. 載入檔案到您的瀏覽器。
7. 選擇 Browse... 按鈕，然後選擇其中包含一或多個臉部的影像。將顯示一個表格，其中包含每個於影像中偵測到的臉部之估測年齡。

> **Note**
>
> 以下程式碼範例使用不再是 Amazon Cognito 一部分的兩個指令碼。若要取得這些檔案，請按 aws-cognito-sdk.min.js 和 amazon-cognito-identity.min.js 的連結，將各連結的文字另存為個別的 `.js` 檔案。

JavaScript 範本程式碼

```
<!--
Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)
-->
<!DOCTYPE html>
<html>
<head>
  <script src="aws-cognito-sdk.min.js"></script>
  <script src="amazon-cognito-identity.min.js"></script>
  <script src="https://sdk.amazonaws.com/js/aws-sdk-2.16.0.min.js"></script>
  <script src="./app.js"></script>
  <meta charset="UTF-8">
  <title>Rekognition</title>
</head>

<body>
  <H1>Age Estimator</H1>
  <input type="file" name="fileToUpload" id="fileToUpload" accept="image/*">
  <p id="opResult"></p>
</body>
<script>

  document.getElementById("fileToUpload").addEventListener("change", function (event) {
    ProcessImage();
  }, false);
```

```
//Calls DetectFaces API and shows estimated ages of detected faces
function DetectFaces(imageData) {
  AWS.region = "RegionToUse";
  var rekognition = new AWS.Rekognition();
  var params = {
    Image: {
      Bytes: imageData
    },
    Attributes: [
      'ALL',
    ]
  };
  rekognition.detectFaces(params, function (err, data) {
    if (err) console.log(err, err.stack); // an error occurred
    else {
     var table = "<table><tr><th>Low</th><th>High</th></tr>";
      // show each face and build out estimated age table
      for (var i = 0; i < data.FaceDetails.length; i++) {
        table += '<tr><td>' + data.FaceDetails[i].AgeRange.Low +
          '</td><td>' + data.FaceDetails[i].AgeRange.High + '</td></tr>';
      }
      table += "</table>";
      document.getElementById("opResult").innerHTML = table;
    }
  });
}
//Loads selected image and unencodes image bytes for Rekognition DetectFaces API
function ProcessImage() {
  AnonLog();
  var control = document.getElementById("fileToUpload");
  var file = control.files[0];

  // Load base64 encoded image
  var reader = new FileReader();
  reader.onload = (function (theFile) {
    return function (e) {
      var img = document.createElement('img');
      var image = null;
      img.src = e.target.result;
      var jpg = true;
      try {
        image = atob(e.target.result.split("")[1]);

      } catch (e) {
        jpg = false;
      }
      if (jpg == false) {
        try {
          image = atob(e.target.result.split("")[1]);
        } catch (e) {
          alert("Not an image file Rekognition can process");
          return;
        }
      }
      //unencode image bytes for Rekognition DetectFaces API
      var length = image.length;
      imageBytes = new ArrayBuffer(length);
      var ua = new Uint8Array(imageBytes);
      for (var i = 0; i < length; i++) {
        ua[i] = image.charCodeAt(i);
      }
      //Call Rekognition
      DetectFaces(imageBytes);
    };
  })(file);
  reader.readAsDataURL(file);
```

```
  }
  //Provides anonymous log on to AWS services
  function AnonLog() {

    // Configure the credentials provider to use your identity pool
    AWS.config.region = 'RegionToUse'; // Region
    AWS.config.credentials = new AWS.CognitoIdentityCredentials({
      IdentityPoolId: 'IdentityPoolIdToUse',
    });
    // Make the call to obtain credentials
    AWS.config.credentials.get(function () {
      // Credentials will be available when this function is called.
      var accessKeyId = AWS.config.credentials.accessKeyId;
      var secretAccessKey = AWS.config.credentials.secretAccessKey;
      var sessionToken = AWS.config.credentials.sessionToken;
    });
  }
</script>
</html>
```

使用 readAsArrayBuffer

以下程式碼片段是在範本程式碼中的另一種 `ProcessImage` 函數實作選擇。它使用 `readAsArrayBuffer` 來載入影像並呼叫 `DetectFaces`。由於 `readAsArrayBuffer` 不會將載入的檔案以 base64 編碼，在呼叫 Amazon Rekognition Image 操作前無需解碼影像位元組。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

function ProcessImage() {
    AnonLog();
    var control = document.getElementById("fileToUpload");
    var file = control.files[0];

    // Load base64 encoded image for display
    var reader = new FileReader();
    reader.onload = (function (theFile) {
      return function (e) {
        //Call Rekognition
        AWS.region = "RegionToUse";
        var rekognition = new AWS.Rekognition();
        var params = {
          Image: {
          Bytes: e.target.result
        },
        Attributes: [
        'ALL',
      ]
    };
    rekognition.detectFaces(params, function (err, data) {
      if (err) console.log(err, err.stack); // an error occurred
      else {
       var table = "<table><tr><th>Low</th><th>High</th></tr>";
        // show each face and build out estimated age table
        for (var i = 0; i < data.FaceDetails.length; i++) {
          table += '<tr><td>' + data.FaceDetails[i].AgeRange.Low +
            '</td><td>' + data.FaceDetails[i].AgeRange.High + '</td></tr>';
        }
        table += "</table>";
        document.getElementById("opResult").innerHTML = table;
      }
    });
```

```
      };
    })(file);
    reader.readAsArrayBuffer(file);
  }
```

取得 Amazon Cognito 身分集區識別碼

為簡化程序，範例將使用匿名 Amazon Cognito 身分集區來提供 Amazon Rekognition Image API 未經授權的存取。這可能符合您的需求。例如，您可以使用未經驗證的存取來提供使用者在註冊前的免費存取或試用存取您的網站。為了提供經驗證的存取，使用 Amazon Cognito 使用者集區。如需詳細資訊，請參閱 Amazon Cognito 使用者集區。

下列程序說明如何建立身分集區，以允許存取未經驗證的身分，以及如何取得在範本程式碼中所需的身分集區識別碼。

若要取得身分集區識別碼

1. 開啟 Amazon Cognito 主控台。
2. 選擇 Create new identity pool (建立新的身分集區)。
3. 在 Identity pool name* (身分集區名稱*) 中，輸入您的身分集區名稱。
4. 在 Unauthenticated identites (未經驗證的身分) 中，選擇 Enable access to unauthenticated identities (允許存取未經驗證的身分) 。
5. 選擇 Create Pool (建立集區)。
6. 選擇 View Details (檢視詳細資訊)，並記下未經驗證的身分之角色名稱。
7. 選擇 Allow (允許)。
8. 在 Platform (平台) 中，選擇 JavaScript。
9. 在 Get AWS Credentials (取得 AWS 登入資料) 中，請記下程式碼片段中顯示的 `AWS.config.region` 和 `IdentityPooldId` 值。
10. Open the IAM console at https://console.aws.amazon.com/iam/.
11. 在導覽窗格中，選擇 Roles (角色)。
12. 選擇您在步驟 6 記下的角色名稱。
13. 在 Permissions (許可) 索引標籤中，選擇 Attach policies (連接政策)。
14. 選擇 AmazonRekognitionReadOnlyAccess。
15. 選擇 Attach Policy (連結政策)。

顯示週框方塊

Amazon Rekognition Image 操作可以針對影像中偵測到的項目傳回週框方塊。例如，the section called “DetectFaces” (p. 320) 操作會針對影像中偵測到的每個臉部傳回一個週框方塊 (the section called “BoundingBox” (p. 440))。您可以使用週框方塊座標，來顯示所偵測到之項目的周圍方塊。例如，以下影像會顯示圍繞臉部的週框方塊。



`BoundingBox` 具有下列屬性：

- 高度 – 週框方塊的高度，以整體影像高度的比例表示。
- 左邊 – 週框方塊的左邊座標，以整體影像寬度的比例表示。
- 上方 – 週框方塊的上方座標，以整體影像高度的比例表示。
- 寬度 – 週框方塊的寬度，以整體影像寬度的比例表示。

每個 BoundingBox 屬性都有一個介於 0 和 1 的值。每個屬性值是整體影像寬度 (`Left` 和 `Width`) 或高度 (`Height` 和 `Top`) 的比例。例如，如果輸入影像為 700 x 200 像素，而週框方塊的左上方座標為 350 x 50 像素，則 API 會傳回 `Left` 值 0.5 (350/700) 和 `Top` 值 0.25 (50/200)。

下圖顯示每個週框方塊屬性涵蓋的影像範圍。

若要顯示正確位置和大小的週框方塊，您必須將 BoundingBox 值乘以影像寬度或高度 (取決於您想要的值) 來取得像素值。您可以使用像素值來顯示週框方塊。例如，前一個影像的像素維度為 608 寬度 x 588 高度。臉部的週框方塊值如下：

```
BoundingBox.Left: 0.3922065
Bounding.Top: 0.15567766
BoundingBox.Width: 0.284666
BoundingBox.Height: 0.2930403
```

臉部週框方塊的位置 (以像素表示) 的計算方式如下：

```
Left coordinate = BoundingBox.Left (0.3922065) * image width (608) = 238
```

```
Top coordinate = BoundingBox.Top (0.15567766) * image height (588) = 91

Face width = BoundingBox.Width (0.284666) * image width (608) = 173

Face height = BoundingBox.Height (0.2930403) * image height (588) = 172
```

您可以使用這些值來顯示臉部的週框方塊。

Note

您可以透過多種方式導向影像。您的應用程式可能需要旋轉影像，才能以正確方向顯示它。週框方塊座標會受到影像方向的影響。您可能需要轉換座標，然後才能在正確位置顯示週框方塊。如需詳細資訊，請參閱 取得影像方向與週框方塊座標 (p. 42)。

以下範例說明如何顯示透過呼叫DetectFaces (p. 320)偵測到之臉部的週框方塊。這些範例假設影像導向至 0 度。這些範例也會顯示如何從 Amazon S3 儲存貯體下載影像。

顯示週框方塊

1. 如果您尚未：
   a. 建立或更新具有 AmazonRekognitionFullAccess 和 AmazonS3ReadOnlyAccess 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 DetectFaces 操作。將 bucket 的值變更為包含影像檔案的 Amazon S3 儲存貯體。將 photo 的值變更為某個影像檔案的檔案名稱 (.jpg 或 .png 格式)。

   Java

```
//Loads images, detects faces and draws bounding boxes.Determines exif orientation,
 if necessary.
package com.amazonaws.samples;

//Import the basic graphics classes.
import java.awt.*;
import java.awt.image.BufferedImage;
import java.util.List;
import javax.imageio.ImageIO;
import javax.swing.*;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;

import com.amazonaws.services.rekognition.model.BoundingBox;
import com.amazonaws.services.rekognition.model.DetectFacesRequest;
import com.amazonaws.services.rekognition.model.DetectFacesResult;
import com.amazonaws.services.rekognition.model.FaceDetail;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.s3.AmazonS3;
import com.amazonaws.services.s3.AmazonS3ClientBuilder;
import com.amazonaws.services.s3.model.S3ObjectInputStream;

// Calls DetectFaces and displays a bounding box around each detected image.
public class DisplayFaces extends JPanel {

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    BufferedImage image;
```

```
   static int scale;
   DetectFacesResult result;

   public DisplayFaces(DetectFacesResult facesResult, BufferedImage bufImage)
throws Exception {
       super();
       scale = 1; // increase to shrink image size.

       result = facesResult;
       image = bufImage;


   }
   // Draws the bounding box around the detected faces.
   public void paintComponent(Graphics g) {
       float left = 0;
       float top = 0;
       int height = image.getHeight(this);
       int width = image.getWidth(this);

       Graphics2D g2d = (Graphics2D) g; // Create a Java2D version of g.

       // Draw the image.
       g2d.drawImage(image, 0, 0, width / scale, height / scale, this);
       g2d.setColor(new Color(0, 212, 0));

       // Iterate through faces and display bounding boxes.
       List<FaceDetail> faceDetails = result.getFaceDetails();
       for (FaceDetail face : faceDetails) {

           BoundingBox box = face.getBoundingBox();
           left = width * box.getLeft();
           top = height * box.getTop();
           g2d.drawRect(Math.round(left / scale), Math.round(top / scale),
                   Math.round((width * box.getWidth()) / scale),
Math.round((height * box.getHeight())) / scale);

       }
   }


   public static void main(String arg[]) throws Exception {

       String photo = "photo.png";
       String bucket = "bucket";
       int height = 0;
       int width = 0;

       // Get the image from an S3 Bucket
       AmazonS3 s3client = AmazonS3ClientBuilder.defaultClient();

       com.amazonaws.services.s3.model.S3Object s3object =
s3client.getObject(bucket, photo);
       S3ObjectInputStream inputStream = s3object.getObjectContent();
       BufferedImage image = ImageIO.read(inputStream);
       DetectFacesRequest request = new DetectFacesRequest()
               .withImage(new Image().withS3Object(new
S3Object().withName(photo).withBucket(bucket)));

       width = image.getWidth();
       height = image.getHeight();

       // Call DetectFaces
       AmazonRekognition amazonRekognition =
AmazonRekognitionClientBuilder.defaultClient();
       DetectFacesResult result = amazonRekognition.detectFaces(request);
```

```
        //Show the bounding box info for each face.
        List<FaceDetail> faceDetails = result.getFaceDetails();
        for (FaceDetail face : faceDetails) {

            BoundingBox box = face.getBoundingBox();
            float left = width * box.getLeft();
            float top = height * box.getTop();
            System.out.println("Face:");

            System.out.println("Left: " + String.valueOf((int) left));
            System.out.println("Top: " + String.valueOf((int) top));
            System.out.println("Face Width: " + String.valueOf((int) (width *
 box.getWidth())));
            System.out.println("Face Height: " + String.valueOf((int) (height *
 box.getHeight())));
            System.out.println();

        }

        // Create frame and panel.
        JFrame frame = new JFrame("RotateImage");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        DisplayFaces panel = new DisplayFaces(result, image);
        panel.setPreferredSize(new Dimension(image.getWidth() / scale,
 image.getHeight() / scale));
        frame.setContentPane(panel);
        frame.pack();
        frame.setVisible(true);

    }
}
```

Python

```
import boto3
import io
from PIL import Image, ImageDraw, ExifTags, ImageColor

def show_faces(photo,bucket):


    client=boto3.client('rekognition')

    # Load image from S3 bucket
    s3_connection = boto3.resource('s3')
    s3_object = s3_connection.Object(bucket,photo)
    s3_response = s3_object.get()

    stream = io.BytesIO(s3_response['Body'].read())
    image=Image.open(stream)

    #Call DetectFaces
    response = client.detect_faces(Image={'S3Object': {'Bucket': bucket, 'Name':
 photo}},
        Attributes=['ALL'])

    imgWidth, imgHeight = image.size
    draw = ImageDraw.Draw(image)


    # calculate and display bounding boxes for each detected face
    print('Detected faces for ' + photo)
```

```
    for faceDetail in response['FaceDetails']:
        print('The detected face is between ' + str(faceDetail['AgeRange']['Low'])
              + ' and ' + str(faceDetail['AgeRange']['High']) + ' years old')

        box = faceDetail['BoundingBox']
        left = imgWidth * box['Left']
        top = imgHeight * box['Top']
        width = imgWidth * box['Width']
        height = imgHeight * box['Height']


        print('Left: ' + '{0:.0f}'.format(left))
        print('Top: ' + '{0:.0f}'.format(top))
        print('Face Width: ' + "{0:.0f}".format(width))
        print('Face Height: ' + "{0:.0f}".format(height))

        points = (
            (left,top),
            (left + width, top),
            (left + width, top + height),
            (left , top + height),
            (left, top)

        )
        draw.line(points, fill='#00d400', width=2)

        # Alternatively can draw rectangle. However you can't set line width.
        #draw.rectangle([left,top, left + width, top + height], outline='#00d400')

    image.show()

    return len(response['FaceDetails'])

def main():
    bucket="bucket"
    photo="photo"

    faces_count=show_faces(photo,bucket)
    print("faces detected: " + str(faces_count))


if __name__ == "__main__":
    main()
```

取得影像方向與週框方塊座標

使用 Amazon Rekognition Image 的應用程式通常需要顯示 Amazon Rekognition Image 操作偵測到的影像，並在偵測到的臉部周圍顯示方塊。若要在您的應用程式中正確顯示影像，您需要了解影像的方向並盡可能地加以修正。對於某些 .jpg 檔案，影像的方向會包含在影像的可交換影像檔案格式 (Exif) 中繼資料內。至於其他 .jpg 檔案與所有 .png 檔案，Amazon Rekognition Image 操作都會傳回估計的方向。

若要在臉部周圍顯示方塊，您需要臉部週框方塊的座標。如果方塊的方向不正確，您可能需要調整這些座標。Amazon Rekognition Image 臉部偵測操作會傳回每個偵測到之臉部的週框方塊座標。

下列 Amazon Rekognition Image 操作會傳回用於修正影像方向與週框方塊座標的資訊：

- IndexFaces (p. 371) (與集合相關聯的臉部模型必須是第 3 版或更低版本)
- RecognizeCelebrities (p. 388)

此範例示範如何為您的程式碼取得下列資訊：

- 影像的估計方向 (如果 Exif 中繼資料內沒有方向資訊)
- 影像中偵測到之臉部的週框方塊座標
- 受估計的影像方向影響之週框方塊的移動後週框方塊座標

請使用此範例中的資訊來確保您的影像方向正確，以及確保週框方塊在您應用程式中顯示的位置正確。

由於用來旋轉及顯示影像與週框方塊的程式碼取決於您所使用的語言和環境，因此我們不會說明如何在您的程式碼中顯示影像與週框方塊，也不會說明如何從 Exif 中繼資料取得方向資訊。

找出影像的方向

若要在您的應用程式中正確顯示影像，您可能需要加以旋轉。下列影像旋轉為 0 度並正確顯示。



不過，下列影像卻逆時針旋轉 90 度。若要正確顯示，您需要找出影像的方向，並在您的程式碼中使用該資訊將影像旋轉為 0 度。



某些 .jpg 格式的影像在 Exif 中繼資料內含有方向資訊。如果操作回應中的 `OrientationCorrection` 欄位值為 `null`，影像的 Exif 中繼資料會含有方向。在 Exif 中繼資料內，您可以在 `orientation` 欄位中找到影像的方向。雖然 Amazon Rekognition Image 會識別 Exif 中繼資料內具有影像方向資訊，但並不提供該資訊的存取權。若要存取影像中的 Exif 中繼資料，請使用第三方程式庫或自行撰寫程式碼。如需詳細資訊，請參閱 Exif Version 2.32。

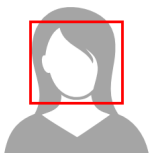
.png 格式的影像沒有 Exif 中繼資料。對於沒有 Exif 中繼資料的 .jpg 影像及對於所有 .png 影像，Amazon Rekognition Image 操作會在 `OrientationCorrection` 欄位中傳回估計的影像方向。估計的方向是以逆時針測量，並以 90 度為單位累加。例如，若影像旋轉為 0 度，Amazon Rekognition Image 會傳回 ROTATE_0，若影像逆時針旋轉了 90 度，則其會傳回 ROTATE_90。

當您知道影像的方向後，就可以撰寫程式碼加以旋轉並正確顯示。

顯示週框方塊

分析影像中臉部的 Amazon Rekognition Image 操作也會傳回臉部周圍的週框方塊座標。如需詳細資訊，請參閱 BoundingBox (p. 440)。

若要在您的應用程式中於臉部周圍顯示週框方塊 (類似下圖所示的方塊)，請在您的程式碼中使用週框方塊座標。操作傳回的週框方塊座標會反映影像的方向。如果您必須旋轉影像才能將其正確顯示，可能需要移動週框方塊座標。



當 Exif 中繼資料內沒有方向資訊時顯示週框方塊

如果影像沒有 Exif 中繼資料，或是如果 Exif 中繼資料中的 `orientation` 欄位未填入，則 Amazon Rekognition Image 操作會傳回下列內容：

- 影像的估計方向
- 旋轉為估計方向的週框方塊座標

如果您需要旋轉影像才能將其正確顯示，則您也需要旋轉週框方塊。

例如，下列影像逆時針旋轉至 90 度，並在臉部周圍顯示了週框方塊。此週框方塊使用 Amazon Rekognition Image 操作所傳回的估計方向座標來顯示。



當您將影像的方向旋轉為 0 度時，您也需要透過移動週框方塊座標來旋轉週框方塊。例如，下列影像已從逆時針 90 度旋轉為 0 度。由於尚未移動週框方塊座標，因此週框方塊會顯示在錯誤的位置。



在 Exif 中繼資料內沒有方向時旋轉並顯示週框方塊

1. 呼叫 Amazon Rekognition Image 操作，並提供至少含有一個臉部但不含 Exif 中繼資料方向的輸入影像。如需範例，請參閱 偵測影像中的臉部 (p. 120)。
2. 請注意，估計的方向會傳回於回應的 `OrientationCorrection` 欄位。
3. 在您的程式碼中使用於步驟 2 所記下的估計方向，將影像的方向旋轉為 0 度。
4. 將上方與左方週框方塊座標移至 0 度方向，並將其轉換為程式碼中影像上的像素點。使用下列清單中符合您在步驟 2 中記下之估計方向的公式。

   請注意下列定義：

   - `ROTATE_(n)` 是 Amazon Rekognition Image 操作所傳回的估計影像方向。
   - `<face>` 代表 Amazon Rekognition Image 操作所傳回之臉部的資訊。例如，ComparedFace (p. 446) 操作所傳回的 RecognizeCelebrities (p. 388) 資料類型包含來源影像中偵測到之臉部的週框方塊資訊。
   - `image.width` 與 `image.height` 是來源影像寬度與高度的像素值。
   - 週框方塊座標是影像大小的相對值，介於 0 到 1 之間。例如，若影像的方向為 0 度，0.9 的 `BoundingBox.left` 值會將左方座標放在接近影像右側。若要顯示方塊，請將週框方塊座標值移至影像上的像素點，並將其旋轉為 0 度，如下列每個公式所示。如需詳細資訊，請參閱 BoundingBox (p. 440)。

   ROTATE_0

   ```
   left = image.width*BoundingBox.Left

   top = image.height*BoundingBox.Top
   ```

   ROTATE_90

   ```
   left = image.height * (1 - (<face>.BoundingBox.Top +
   <face>.BoundingBox.Height))

   top = image.width * <face>.BoundingBox.Left
   ```

   ROTATE_180

   ```
   left = image.width - (image.width*(<face>.BoundingBox.Left
   +<face>.BoundingBox.Width))
   ```

```
top = image.height * (1 - (<face>.BoundingBox.Top +
<face>.BoundingBox.Height))
```

ROTATE_270

```
left = image.height * BoundingBox.top

top = image.width * (1 - BoundingBox.Left - BoundingBox.Width)
```

5. 使用下列公式，即可將週框方塊的寬度與高度計算成您程式碼中影像上的像素範圍。

   週框方塊的寬度與高度會傳回於 `BoundingBox.Width` 與 `BoundingBox.Height` 欄位。寬度與高度是影像大小的相對數值，介於 0 到 1 之間。`image.width` 與 `image.height` 是來源影像寬度與高度的像素值。

```
box width = image.width * (<face>.BoundingBox.Width)

box height = image.height * (<face>.BoundingBox.Height)
```

6. 透過使用步驟 4 與 5 中計算得到的值，在旋轉的影像上顯示週框方塊。

在 Exif 中繼資料內有方向資訊時顯示週框方塊

如果 Exif 中繼資料包含影像的方向，Amazon Rekognition Image 操作會執行下列動作：

- 在操作回應的方向修正欄位中傳回 null。若要旋轉影像，請在您的程式碼中使用 Exif 中繼資料提供的方向。
- 傳回已旋轉為 0 度的週框方塊座標。若要在正確位置顯示週框方塊，請使用傳回的座標。您不需要移動該座標。

範例：取得影像方向與影像的週框方塊座標

下列範例示範如何使用適用於 AWS 開發套件來取得影像的估計方向，並移動 `RecognizeCelebrities` 操作偵測到之名人的週框方塊座標。

此範例會從本機檔案系統載入影像、呼叫 `RecognizeCelebrities` 操作、判斷影像的高度與寬度，並計算旋轉影像的臉部週框方塊座標。此範例不會示範如何處理存放在 Exif 中繼資料內的方向資訊。

在 `main` 函數中，請將 `photo` 的值取代為以 .png 或 .jpg 格式存放於本機之影像的名稱與路徑。

Java

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package com.amazonaws.samples;
import java.awt.image.BufferedImage;
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.nio.ByteBuffer;
import java.util.List;
import javax.imageio.ImageIO;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.Image;
```

```
import com.amazonaws.services.rekognition.model.RecognizeCelebritiesRequest;
import com.amazonaws.services.rekognition.model.RecognizeCelebritiesResult;
import com.amazonaws.util.IOUtils;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.BoundingBox;
import com.amazonaws.services.rekognition.model.Celebrity;
import com.amazonaws.services.rekognition.model.ComparedFace;

public class RotateImage {

public static void main(String[] args) throws Exception {

  String photo = "photo.png";

  //Get Rekognition client
 AmazonRekognition amazonRekognition = AmazonRekognitionClientBuilder.defaultClient();


  // Load image
  ByteBuffer imageBytes=null;
  BufferedImage image = null;

  try (InputStream inputStream = new FileInputStream(new File(photo))) {
     imageBytes = ByteBuffer.wrap(IOUtils.toByteArray(inputStream));

  }
  catch(Exception e)
  {
      System.out.println("Failed to load file " + photo);
      System.exit(1);
  }

  //Get image width and height
  InputStream imageBytesStream;
  imageBytesStream = new ByteArrayInputStream(imageBytes.array());

  ByteArrayOutputStream baos = new ByteArrayOutputStream();
  image=ImageIO.read(imageBytesStream);
  ImageIO.write(image, "jpg", baos);

  int height = image.getHeight();
  int width = image.getWidth();

  System.out.println("Image Information:");
  System.out.println(photo);
  System.out.println("Image Height: " + Integer.toString(height));
  System.out.println("Image Width: " + Integer.toString(width));

  //Call GetCelebrities

  try{
    RecognizeCelebritiesRequest request = new RecognizeCelebritiesRequest()
            .withImage(new Image()
               .withBytes((imageBytes)));


      RecognizeCelebritiesResult result =
 amazonRekognition.recognizeCelebrities(request);
      System.out.println("Orientation: " + result.getOrientationCorrection() + "\n");
      List <Celebrity> celebs = result.getCelebrityFaces();

      for (Celebrity celebrity: celebs) {
          System.out.println("Celebrity recognized: " + celebrity.getName());
          System.out.println("Celebrity ID: " + celebrity.getId());
          ComparedFace  face = celebrity.getFace()
;             ShowBoundingBoxPositions(height,
```

```
                    width,
                    face.getBoundingBox(),
                    result.getOrientationCorrection());

            System.out.println();
        }

    } catch (AmazonRekognitionException e) {
       e.printStackTrace();
    }

}


public static void ShowBoundingBoxPositions(int imageHeight, int imageWidth,
 BoundingBox box, String rotation) {

  float left = 0;
  float top = 0;

  if(rotation==null){
      System.out.println("No estimated estimated orientation. Check Exif data.");
      return;
  }
  //Calculate face position based on image orientation.
  switch (rotation) {
     case "ROTATE_0":
        left = imageWidth * box.getLeft();
        top = imageHeight * box.getTop();
        break;
     case "ROTATE_90":
        left = imageHeight * (1 - (box.getTop() + box.getHeight()));
        top = imageWidth * box.getLeft();
        break;
     case "ROTATE_180":
        left = imageWidth - (imageWidth * (box.getLeft() + box.getWidth()));
        top = imageHeight * (1 - (box.getTop() + box.getHeight()));
        break;
     case "ROTATE_270":
        left = imageHeight * box.getTop();
        top = imageWidth * (1 - box.getLeft() - box.getWidth());
        break;
     default:
        System.out.println("No estimated orientation information. Check Exif data.");
        return;
  }

  //Display face location information.
  System.out.println("Left: " + String.valueOf((int) left));
  System.out.println("Top: " + String.valueOf((int) top));
  System.out.println("Face Width: " + String.valueOf((int)(imageWidth *
 box.getWidth())));
  System.out.println("Face Height: " + String.valueOf((int)(imageHeight *
 box.getHeight())));

  }
}
```

Python

此範例使用 PIL/枕狀影像程式庫來取得影像寬度和高度。如需詳細資訊，請參閱 枕狀。此範例會保留 exif 中繼資料，您可能需要應用程式中的其他位置。如果您選擇不儲存 exif 中繼資料，會從呼叫將估計方向傳回至 `RecognizeCelebrities`。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
import io
from PIL import Image

# Calculate positions from from estimated rotation
def show_bounding_box_positions(imageHeight, imageWidth, box, rotation):
    left = 0
    top = 0

    if rotation == 'ROTATE_0':
        left = imageWidth * box['Left']
        top = imageHeight * box['Top']

    if rotation == 'ROTATE_90':
        left = imageHeight * (1 - (box['Top'] + box['Height']))
        top = imageWidth * box['Left']

    if rotation == 'ROTATE_180':
        left = imageWidth - (imageWidth * (box['Left'] + box['Width']))
        top = imageHeight * (1 - (box['Top'] + box['Height']))

    if rotation == 'ROTATE_270':
        left = imageHeight * box['Top']
        top = imageWidth * (1- box['Left'] - box['Width'] )

    print('Left: ' + '{0:.0f}'.format(left))
    print('Top: ' + '{0:.0f}'.format(top))
    print('Face Width: ' + "{0:.0f}".format(imageWidth * box['Width']))
    print('Face Height: ' + "{0:.0f}".format(imageHeight * box['Height']))


def celebrity_image_information(photo):


    client=boto3.client('rekognition')


    #Get image width and height
    image = Image.open(open(photo,'rb'))
    width, height = image.size

    print ('Image information: ')
    print (photo)
    print ('Image Height: ' + str(height))
    print('Image Width: ' + str(width))


    # call detect faces and show face age and placement
    # if found, preserve exif info
    stream = io.BytesIO()
    if 'exif' in image.info:
        exif=image.info['exif']
        image.save(stream,format=image.format, exif=exif)
    else:
        image.save(stream, format=image.format)
    image_binary = stream.getvalue()

    response = client.recognize_celebrities(Image={'Bytes': image_binary})

    if 'OrientationCorrection'  in response:
        print('Orientation: ' + response['OrientationCorrection'])
```

```
    else:
        print('No estimated orientation. Check Exif')

    print()
    print('Detected celebrities for ' + photo)

    for celebrity in response['CelebrityFaces']:
        print ('Name: ' + celebrity['Name'])
        print ('Id: ' + celebrity['Id'])

        if 'OrientationCorrection'  in response:
            show_bounding_box_positions(height, width, celebrity['Face']
['BoundingBox'], response['OrientationCorrection'])

        print()
    return len(response['CelebrityFaces'])


def main():
    photo='photo.png'

    celebrity_count=celebrity_image_information(photo)
    print("celebrities detected: " + str(celebrity_count))


if __name__ == "__main__":
    main()
```

使用儲存的影片來操作

Amazon Rekognition Video 為可用以分析影片的 API。有了 Amazon Rekognition Video，您可以偵測儲存於 Amazon Simple Storage Service (Amazon S3) 儲存貯體的影片中之標籤、臉部、人物、名人以及成人內容 (包括暗示與露骨)。您可以在媒體/娛樂與公共安全等類別使用 Amazon Rekognition Video。在過去，掃描影片中的物件或人員可能必須耗費數小時的人工檢視，且容易發生錯誤。若項目貫穿影片各處，Amazon Rekognition Video 可對項目執行自動化偵測。

本節涵蓋 Amazon Rekognition Video 可執行的分析類型，API 概觀以及使用 Amazon Rekognition Video 的範例。

主題

- 分析類型 (p. 49)
- Amazon Rekognition VideoAPI 概觀 (p. 50)
- 呼叫 Amazon Rekognition Video 操作 (p. 51)
- 設定 Amazon Rekognition Video (p. 55)
- 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)
- 以 AWS Command Line Interface 分析影片 (p. 66)
- 教學：建立 Amazon Rekognition Lambda 函數 (p. 68)
- 參考：影片分析結果通知 (p. 74)
- 故障診斷 Amazon Rekognition Video (p. 75)

分析類型

您可以使用 Amazon Rekognition Video 來分析影片中的下列資訊：

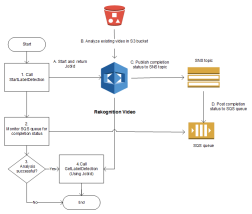
- 標籤 (p. 106)
- 臉部 (p. 119)
- 人物 (p. 195)
- 名人 (p. 201)
- 暗示與露骨的成人內容 (p. 215)
- Text (文字) (p. 227)

如需更多詳細資訊，請參閱 Amazon Rekognition 運作方式 (p. 4)。

Amazon Rekognition VideoAPI 概觀

Amazon Rekognition Video 可處理儲存於 Amazon S3 儲存貯體中的影片。設計模式為一組非同步的操作。呼叫如 StartLabelDetection (p. 415) 的 `Start` 操作來開始影片分析。請求的完成狀態將發佈到 Amazon Simple Notification Service (Amazon SNS) 主題。若要自 Amazon SNS 主題取得完成狀態，您可以使用 Amazon Simple Queue Service (Amazon SQS) 佇列或 AWS Lambda 函數。在獲得完成狀態後，可呼叫如 GetLabelDetection (p. 358) 的 `Get` 操作來取得請求結果。

下圖顯示在儲存於 Amazon S3 儲存貯體的影片中偵測標籤的程序。在圖表內，Amazon SQS 佇列將自 Amazon SNS 主題取得完成狀態。或者，您可以使用 AWS Lambda 函數。



偵測臉部與人物的程序相同。下表列出每個非儲存體 Amazon Rekognition 操作的 `Start` 與 `Get` 操作。

偵測	開始操作	取得操作
人員	StartPersonTracking (p. 419)	GetPersonTracking (p. 362)
臉部	StartFaceDetection (p. 407)	GetFaceDetection (p. 348)
標籤	StartLabelDetection (p. 415)	GetLabelDetection (p. 358)
名人	StartCelebrityRecognition (p. 400)	GetCelebrityRecognition (p. 339)
露骨或暗示性的成人內容	StartContentModeration (p. 403)	GetContentModeration (p. 344)

對於 `GetCelebrityRecognition` 以外的 `Get` 操作，Amazon Rekognition Video 將傳回追蹤資訊，說明將在影片輸入檔的哪個部分偵測到實體。

如需使用 Amazon Rekognition Video 的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。如需使用 Amazon SQS 執行影片分析的範例，請參閱 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。如需 AWS CLI 範例，請參閱 以 AWS Command Line Interface 分析影片 (p. 66)。

影片格式和儲存體

Amazon Rekognition 操作可分析儲存於 Amazon S3 儲存貯體的影片。影片需使用 H.264 編解碼器來編碼。支援的檔案格式為 MPEG-4 與 MOV。

編解碼器為壓縮資料讓傳遞更快速並將收到的資料解壓縮為原始格式的軟體或硬體工具。The H.264 編解碼器通常用於記錄、壓縮和發佈影片內容。影片檔案格式可包含一個或多個編解碼器。若您的 MOV 或 MPEG-4 格式影片檔案不適用於 Amazon Rekognition Video，請確認用於影片編碼的編解碼器為 H.264。

可儲存影片的檔案大小上限為 10 GB。

搜尋人物

您可以使用儲存在集合中的臉部中繼資料來搜尋影片中的人物。例如，您可以在封存的影片中搜尋特定人物或多個人物。使用 IndexFaces (p. 371) 操作從集合中的原始影像儲存臉部中繼資料。接著可以使用 StartFaceSearch (p. 411) 以於集合中啟動非同步臉部搜尋。可使用 GetFaceSearch (p. 353) 來取得搜尋結果。如需詳細資訊，請參閱 在儲存的視訊中搜尋臉部 (p. 187)。搜尋人物為以儲存體為基礎的 Amazon Rekognition 操作之範例。如需詳細資訊，請參閱 以儲存為基礎的 API 操作 (p. 8)。

您也可以在串流影片中搜尋人物。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

呼叫 Amazon Rekognition Video 操作

Amazon Rekognition Video 是一種非同步的 API，可用來分析儲存於 Amazon Simple Storage Service (Amazon S3) 儲存貯體中的影片。呼叫如 StartPersonTracking (p. 419) 的 Amazon Rekognition Video `Start` 操作來開始影片分析。Amazon Rekognition Video 將發佈分析請求結果至 Amazon Simple Notification Service (Amazon SNS) 主題。您可以使用 Amazon Simple Queue Service (Amazon SQS) 佇列或 AWS Lambda 函數來自 Amazon SNS 主題取得影片分析請求的完成狀態。最後，您可以透過如 GetPersonTracking (p. 362) 的 Amazon Rekognition `Get` 操作來取得影片分析請求結果。

下節中的資訊使用標籤偵測操作來顯示 Amazon Rekognition Video 在儲存於 Amazon S3 儲存貯體中的影片內偵測標籤 (物件、活動、概念與活動) 的方法。相同的方法也適用於其他 Amazon Rekognition Video 操作—例如 StartFaceDetection (p. 407) 以及 StartPersonTracking (p. 419)。範例 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 說明如何使用 Amazon SQS 佇列自 Amazon SNS 主題取得完成狀態分析以分析影片。同時也用做為其他 Amazon Rekognition Video 範例的基礎，例如人物路徑 (p. 195)。如需 AWS CLI 範例，請參閱 以 AWS Command Line Interface 分析影片 (p. 66)。

主題

- 開始影片分析 (p. 51)
- 取得 Amazon Rekognition Video 分析請求的完成狀態 (p. 52)
- 取得 Amazon Rekognition Video 分析結果 (p. 53)

開始影片分析

透過呼叫 StartLabelDetection (p. 415) 來開始 Amazon Rekognition Video 標籤偵測請求。以下是由 `StartLabelDetection` 傳遞的 JSON 請求範例。

```
{
    "Video": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "video.mp4"
        }
    },
    "ClientRequestToken": "LabelDetectionToken",
    "MinConfidence": 50,
    "NotificationChannel": {
        "SNSTopicArn": "arn:aws:sns:us-east-1:nnnnnnnnnn:topic",
        "RoleArn": "arn:aws:iam::nnnnnnnnnn:role/roleopic"
    },
    "JobTag": "DetectingLabels"
```

```
}
```

輸入參數 `Video` 提供影片檔案名稱以及擷取檔案的 Amazon S3 儲存貯體。`NotificationChannel` 包含在影片分析請求完成時由 Amazon Rekognition Video 所通知的 Amazon SNS 主題 Amazon Resource Name (ARN)。Amazon SNS 主題必須與您呼叫的 Amazon Rekognition Video 端點位於同一個 AWS 區域。`NotificationChannel` 也包含允許 Amazon Rekognition Video 發佈到 Amazon SNS 主題的角色 ARN。透過 IAM 服務角色的建立，您給予 Amazon Rekognition 發佈至 Amazon SNS 主題的許可。如需詳細資訊，請參閱 設定 Amazon Rekognition Video (p. 55)。

您也可以指定選用的輸入參數，`JobTag`，可讓您找出顯示完成狀態且已發佈至 Amazon SNS 主題的任務。

為避免分析任務發生意外的重複，您可以選擇性地提供等冪符記 `ClientRequestToken`。如果您提供用於 `ClientRequestToken` 的值，`Start` 操作將傳回相同的 `JobId` 以用於對開始操作執行多個相同的呼叫，例如 `StartLabelDetection`。`ClientRequestToken` 符記有 7 天的存留期。在 7 天後，您可以再次使用它。如果您在符記的存留期內重新使用符記，會發生下列情況：

- 如果您使用相同的 `Start` 操作與相同的輸入參數來重新使用字符，將傳回相同的 `JobId`。不會再次執行任務，而 Amazon Rekognition Video 也不會傳送完成狀態到已註冊的 Amazon SNS 主題。
- 如果您以相同的 `Start` 操作搭配些微變更的參數來重新使用符記，您會得到一個 `IdempotentParameterMismatchException` (HTTP 狀態碼：400) 例外狀況。
- 由於您會從 Amazon Rekognition 取得無法預測的結果，因此您不應使用具有不同 `Start` 操作的字符。

對於 `StartLabelDetection` 操作的回應為任務識別碼 (`JobId`)。使用 `JobId` 來追蹤請求並在 Amazon Rekognition Video 發佈完成狀態到 Amazon SNS 主題後取得分析結果。例如：

```
{"JobId":"270c1cc5e1d0ea2fbc59d97cb69a72a5495da75851976b14a1784ca90fc180e3"}
```

如果您同時開始太多任務，呼叫 `StartLabelDetection` 將引發 `LimitExceededException` (HTTP 狀態碼：400)，直到數量同時執行任務的數量低於 Amazon Rekognition 服務限制。

如果您發現 `LimitExceededException` 例外狀況發生且產生活動激增，請考慮使用 Amazon SQS 佇列來管理傳入的請求。如果您發現同時進行的請求之平均數量無法由 Amazon SQS 佇列控管，而您仍收到 `LimitExceededException` 例外狀況，請聯絡 AWS 支援。

取得 Amazon Rekognition Video 分析請求的完成狀態

Amazon Rekognition Video 將傳送分析完成通知到已註冊的 Amazon SNS 主題。通知包含任務識別碼和並以 JSON 字串顯示操作的完成狀態。成功的影片分析請求將包含 `SUCCEEDED` 狀態。例如，以下結果顯示偵測標籤任務已成功處理。

```
{
    "JobId": "270c1cc5e1d0ea2fbc59d97cb69a72a5495da75851976b14a1nnnnnnnnnnn",
    "Status": "SUCCEEDED",
    "API": "StartLabelDetection",
    "JobTag": "DetectingLabels",
    "Timestamp": 1510865364756,
    "Video": {
        "S3ObjectName": "video.mp4",
        "S3Bucket": "bucket"
    }
}
```

如需詳細資訊，請參閱 參考：影片分析結果通知 (p. 74)。

若要取得 Amazon Rekognition Video 發佈至 Amazon SNS 主題的狀態資訊，請使用以下其中一個選項：

- AWS Lambda– 您可以註冊 AWS Lambda 函數，以寫入 Amazon SNS 主題。當 Amazon Rekognition 通知 Amazon SNS 主題請求已完成時，將呼叫此功能。如果您需要伺服器端程式碼來處理影片分析請求的結果，請使用 Lambda 函數。例如，您可能想要使用伺服器端程式碼來標註視訊，或在傳回資訊到客戶端應用程式前針對影片內容建立報告。我們也建議使用伺服器端處理大型影片，因為 Amazon Rekognition API 可能會傳回大量資料。
- Amazon Simple Queue Service– 您可以訂閱 Amazon SQS 佇列到 Amazon SNS 主題。接著請輪詢 Amazon SQS 佇列以擷取由 Amazon Rekognition 在影片分析請求完成時發佈的完成狀態。如需詳細資訊，請參閱 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。如果您只想從客戶端應用程式呼叫 Amazon Rekognition Video 操作，請使用 Amazon SQS 佇列。

  Important

  我們不建議您透過重複呼叫 Amazon Rekognition Video `Get` 操作來取得請求完成狀態。這是因為如果執行太多請求，Amazon Rekognition Video 將對 `Get` 操作進行節制。如果您同時處理多個影片，監控一個 SQS 佇列的完成通知將會比輪詢 Amazon Rekognition Video 以獲得個別影片的狀態更容易且有效率。

取得 Amazon Rekognition Video 分析結果

若要取得影片分析請求結果，首先請確認自 Amazon SNS 主題擷取的完成狀態為 `SUCCEEDED`。然後呼叫 `GetLabelDetection`，將傳遞自 `StartLabelDetection` 傳回的 `JobId` 值。請求 JSON 格式類似於以下範例：

```
{
    "JobId": "270c1cc5e1d0ea2fbc59d97cb69a72a5495da75851976b14a1784ca90fc180e3",
    "MaxResults": 10,
    "SortBy": "TIMESTAMP"
}
```

JobId 為用於影片分析操作的識別碼。由於影片分析可以產生大量資料，請使用 `MaxResults` 來指定最大數量的結果，以在單次「取得」操作中傳回結果。`MaxResults` 的預設值為 1000。如果您指定的值大於 1000，最多只能傳回 1000 個結果。如果操作不會傳回整組結果，將在操作回應中傳回下一頁的分頁符記。如果您自前一個「取得」請求中獲得一個分頁符記，請搭配 `NextToken` 一起使用以取得下一頁結果。

  Note

  Amazon Rekognition 會保留影片分析操作的結果 7 天。在此時間後您將無法擷取分析結果。

`GetLabelDetection` 操作回應 JSON 格式類似於以下範例：

```
{
    "Labels": [
        {
            "Timestamp": 0,
            "Label": {
                "Instances": [],
                "Confidence": 60.51791763305664,
                "Parents": [],
                "Name": "Electronics"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Instances": [],
                "Confidence": 99.53411102294922,
                "Parents": [],
```

```
                "Name": "Human"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Instances": [
                    {
                        "BoundingBox": {
                            "Width": 0.11109819263219833,
                            "Top": 0.08098889887332916,
                            "Left": 0.8881205320358276,
                            "Height": 0.9073750972747803
                        },
                        "Confidence": 99.5831298828125
                    },
                    {
                        "BoundingBox": {
                            "Width": 0.1268676072359085,
                            "Top": 0.14018426835536957,
                            "Left": 0.0003282368124928324,
                            "Height": 0.7993982434272766
                        },
                        "Confidence": 99.46029663085938
                    }
                ],
                "Confidence": 99.53411102294922,
                "Parents": [],
                "Name": "Person"
            }
        },
        .
        .
        .

        {
            "Timestamp": 166,
            "Label": {
                "Instances": [],
                "Confidence": 73.6471176147461,
                "Parents": [
                    {
                        "Name": "Clothing"
                    }
                ],
                "Name": "Sleeve"
            }
        }

    ],
    "LabelModelVersion": "2.0",
    "JobStatus": "SUCCEEDED",
    "VideoMetadata": {
        "Format": "QuickTime / MOV",
        "FrameRate": 23.976024627685547,
        "Codec": "h264",
        "DurationMillis": 5005,
        "FrameHeight": 674,
        "FrameWidth": 1280
    }
}
```


您可以透過偵測時間 (依據影片開始的毫秒計算) 來排序結果，或依照偵測的實體 (物件、臉部、名人、內容管制標籤或人物) 字母排序。若要依時間排序，將 `SortBy` 輸入參數值設為 `TIMESTAMP`。如果未指定 `SortBy`，預設行為是依據時間排序。前述範例是依時間排序。若要依據實體排序，使用 `SortBy` 輸入參數

搭配適合您要執行的操作值。例如，在對 `GetLabelDetection` 的呼叫中依偵測的標籤排序時，需使用值 `NAME`。

設定 Amazon Rekognition Video

若要在儲存的影片中使用 Amazon Rekognition Video API，您必須設定 IAM 使用者以及 IAM 服務角色來存取您的 Amazon SNS 主題。您也必須讓 Amazon SQS 佇列訂閱您的 Amazon SNS 主題。

> Note
>
> 如果您正依照這些說明設定 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 範例，則無需進行步驟 3、4、5 和 6。此範例包括用於建立並設定 Amazon SNS 主題和 Amazon SQS 佇列的程式碼。

本節中的範例將依據說明來提供 Amazon Rekognition Video 存取多個主題的權限，並以此來建立新的 Amazon SNS 主題。如果您想要使用現有 Amazon SNS 主題，請在步驟 3 使用 提供存取目前 Amazon SNS 主題的權限 (p. 56)。

設定 Amazon Rekognition Video

1. 設定 AWS 帳戶以存取 Amazon Rekognition Video。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

   確保使用者至少有以下權限：

   - AmazonSQSFullAccess
   - AmazonRekognitionFullAccess
   - AmazonS3FullAccess
   - AmazonSNSFullAccess
2. 安裝並設定所需的 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
3. 使用 Amazon SNS 主控台建立 Amazon SNS 主題。將 AmazonRekognition 置於主題名稱的前面。請記下主題的 Amazon Resource Name (ARN)。請確認此主題與您使用的 AWS 端點位於同一個區域。
4. 使用 Amazon SQS 主控台建立 Amazon SQS 標準佇列。記下佇列 ARN.
5. 將佇列訂閱至您在步驟 3 中建立的主題。
6. 將許可提供給 Amazon SNS 主題，以將訊息傳送至 Amazon SQS 佇列。
7. 建立 IAM 服務角色來提供存取您 Amazon SNS 主題的 Amazon Rekognition Video 權限。記下服務角色的 Amazon Resource Name (ARN)。如需詳細資訊，請參閱 提供對多個 Amazon SNS 主題的存取權限 (p. 56)。
8. 將以下內嵌政策新增到您在步驟 1 所建立的 IAM 使用者：

   ```
   {
       "Version": "2012-10-17",
       "Statement": [
           {
               "Sid": "MySid",
               "Effect": "Allow",
               "Action": "iam:PassRole",
               "Resource": "arn:Service role ARN from step 7"
           }
       ]
   }
   ```

   將內嵌政策命名為您想要的名稱。
9. 您現在可執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 和 以 AWS Command Line Interface 分析影片 (p. 66) 中的範例。

提供對多個 Amazon SNS 主題的存取權限

使用 IAM 服務角色來提供存取您所建立的 Amazon SNS 主題的 Amazon Rekognition Video 權限。IAM 提供用於建立 Amazon Rekognition Video 服務角色的 Rekognition 使用案例。

您可以提供 Amazon Rekognition Video 存取多個 Amazon SNS 主題的許可，方法包括使用 `AmazonRekognitionServiceRole` 許可政策並將 AmazonRekognition— 置於主題名稱前方，例如 `AmazonRekognitionMyTopicName`。

提供 Amazon Rekognition Video 對多個 Amazon SNS 主題的存取權限

1. 建立 IAM 服務角色。請使用下列資訊來建立 IAM 服務角色：
    1. 針對服務名稱選擇 Rekognition。
    2. 為服務角色使用案例選擇 Rekognition。您應該會看到列出的 AmazonRekognitionServiceRole 權限政策。AmazonRekognitionServiceRole 將提供 Amazon Rekognition Video 存取名稱前方加上 AmazonRekognition 的 Amazon SNS 主題的權限。
    3. 將服務角色命名為您想要的名稱。
2. 請記下服務角色 ARN。您需要此資訊才可開始影片分析操作。

提供存取目前 Amazon SNS 主題的權限

您可以建立許可政策，以允許 Amazon Rekognition Video 存取現有的 Amazon SNS 主題。

提供 Amazon Rekognition Video 存取目前 Amazon SNS 主題的權限

1. 使用 IAM JSON 政策編輯器建立新的許可政策，並使用下列政策。以想要的 Amazon SNS 主題 Amazon Resource Name (ARN) 來取代 `topicarn`。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "sns:Publish"
            ],
            "Resource": "topicarn"
        }
    ]
}
```

2. 建立 IAM 服務角色，或更新現有的 IAM 服務角色。請使用下列資訊來建立 IAM 服務角色：
    1. 針對服務名稱選擇 Rekognition。
    2. 為服務角色使用案例選擇 Rekognition。
    3. 連接您在步驟 1 中建立的許可政策。
3. 請記下服務角色 ARN。您需要此資訊才可開始影片分析操作。

使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片

此程序說明如何使用 Amazon Rekognition Video 標籤偵測操作、儲存在 Amazon S3 儲存貯體中的影片、以及 Amazon SNS 主題等方法來偵測影片中的標籤。程序同時也將說明如何使用 Amazon SQS 佇列從 Amazon SNS 主題取得完成狀態。如需詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操

作 (p. 51)。不限於使用 Amazon SQS 佇列。例如，您可以使用 AWS Lambda 函數來取得完成狀態。如需詳細資訊，請參閱使用 Amazon SNS 通知來呼叫 Lambda 函數。

此程序中的範例程式碼說明如何執行下列動作：

1. 建立 Amazon SNS 主題。
2. 建立 Amazon SQS 佇列。
3. 給予 Amazon Rekognition Video 發佈影片分析操作完成狀態到 Amazon SNS 主題的許可。
4. 將 Amazon SQS 佇列訂閱至 Amazon SNS 主題。
5. 呼叫 StartLabelDetection (p. 415) 以開始影片分析請求。
6. 從 Amazon SQS 佇列取得完成狀態。此範例追蹤在 `StartLabelDetection` 傳回的任務識別碼 (`JobId`)，在自完成狀態中讀取後，只會顯示相符的工作識別碼之結果。如果其他應用程式使用相同的佇列和主題，這便是項重要的考量條件。為了方便起見，此範例刪除不符合的任務。考慮將它們新增到 Amazon SQS 無效字母佇列以供進一步調查。
7. 呼叫 GetLabelDetection (p. 358) 來取得並顯示影片分析結果。

先決條件

此程序的範例程式碼以 Java 和 Python 提供。您需要安裝適合的 AWS 開發套件。如需詳細資訊，請參閱 Amazon Rekognition 入門 (p. 10)。您使用的 AWS 帳戶必須有 Amazon Rekognition API 的存取許可。如需詳細資訊，請參閱 Amazon Rekognition 定義的動作。

若要偵測影片中的標籤

1. 設定使用者存取 Amazon Rekognition Video 的權限，並設定 Amazon Rekognition Video 存取 Amazon SNS 的權限。如需詳細資訊，請參閱 設定 Amazon Rekognition Video (p. 55)。由於範例程式碼會建立並設定 Amazon SNS 主題和 Amazon SQS 佇列，因此無須進行步驟 3、4、5 和 6。
2. 將 MOV 或 MPEG-4 格式的影片上傳到 Amazon S3 儲存貯體。為達測試目的，請上傳長度不超過 30 秒的影片。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
3. 使用以下程式碼來偵測影片中的標籤。

   在函數 `main` 中：

   - 將 `roleArn` 取代為您在設定 Amazon Rekognition Video (p. 55)步驟 7 建立的 IAM 服務角色 ARN。
   - 以您在步驟 2 中指定的儲存貯體與影片檔名稱來取代 `bucket` 與 `video` 的值。

   Java

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package com.amazonaws.samples;
import com.amazonaws.auth.policy.Policy;
import com.amazonaws.auth.policy.Condition;
import com.amazonaws.auth.policy.Principal;
import com.amazonaws.auth.policy.Resource;
import com.amazonaws.auth.policy.Statement;
import com.amazonaws.auth.policy.Statement.Effect;
import com.amazonaws.auth.policy.actions.SQSActions;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
```

```
import com.amazonaws.services.rekognition.model.CelebrityDetail;
import com.amazonaws.services.rekognition.model.CelebrityRecognition;
import com.amazonaws.services.rekognition.model.CelebrityRecognitionSortBy;
import com.amazonaws.services.rekognition.model.ContentModerationDetection;
import com.amazonaws.services.rekognition.model.ContentModerationSortBy;
import com.amazonaws.services.rekognition.model.Face;
import com.amazonaws.services.rekognition.model.FaceDetection;
import com.amazonaws.services.rekognition.model.FaceMatch;
import com.amazonaws.services.rekognition.model.FaceSearchSortBy;
import com.amazonaws.services.rekognition.model.GetCelebrityRecognitionRequest;
import com.amazonaws.services.rekognition.model.GetCelebrityRecognitionResult;
import com.amazonaws.services.rekognition.model.GetContentModerationRequest;
import com.amazonaws.services.rekognition.model.GetContentModerationResult;
import com.amazonaws.services.rekognition.model.GetFaceDetectionRequest;
import com.amazonaws.services.rekognition.model.GetFaceDetectionResult;
import com.amazonaws.services.rekognition.model.GetFaceSearchRequest;
import com.amazonaws.services.rekognition.model.GetFaceSearchResult;
import com.amazonaws.services.rekognition.model.GetLabelDetectionRequest;
import com.amazonaws.services.rekognition.model.GetLabelDetectionResult;
import com.amazonaws.services.rekognition.model.GetPersonTrackingRequest;
import com.amazonaws.services.rekognition.model.GetPersonTrackingResult;
import com.amazonaws.services.rekognition.model.Instance;
import com.amazonaws.services.rekognition.model.Label;
import com.amazonaws.services.rekognition.model.LabelDetection;
import com.amazonaws.services.rekognition.model.LabelDetectionSortBy;
import com.amazonaws.services.rekognition.model.NotificationChannel;
import com.amazonaws.services.rekognition.model.Parent;
import com.amazonaws.services.rekognition.model.PersonDetection;
import com.amazonaws.services.rekognition.model.PersonMatch;
import com.amazonaws.services.rekognition.model.PersonTrackingSortBy;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.model.StartCelebrityRecognitionRequest;
import com.amazonaws.services.rekognition.model.StartCelebrityRecognitionResult;
import com.amazonaws.services.rekognition.model.StartContentModerationRequest;
import com.amazonaws.services.rekognition.model.StartContentModerationResult;
import com.amazonaws.services.rekognition.model.StartFaceDetectionRequest;
import com.amazonaws.services.rekognition.model.StartFaceDetectionResult;
import com.amazonaws.services.rekognition.model.StartFaceSearchRequest;
import com.amazonaws.services.rekognition.model.StartFaceSearchResult;
import com.amazonaws.services.rekognition.model.StartLabelDetectionRequest;
import com.amazonaws.services.rekognition.model.StartLabelDetectionResult;
import com.amazonaws.services.rekognition.model.StartPersonTrackingRequest;
import com.amazonaws.services.rekognition.model.StartPersonTrackingResult;
import com.amazonaws.services.rekognition.model.Video;
import com.amazonaws.services.rekognition.model.VideoMetadata;
import com.amazonaws.services.sns.AmazonSNS;
import com.amazonaws.services.sns.AmazonSNSClientBuilder;
import com.amazonaws.services.sns.model.CreateTopicRequest;
import com.amazonaws.services.sns.model.CreateTopicResult;
import com.amazonaws.services.sqs.AmazonSQS;
import com.amazonaws.services.sqs.AmazonSQSClientBuilder;
import com.amazonaws.services.sqs.model.CreateQueueRequest;
import com.amazonaws.services.sqs.model.Message;
import com.amazonaws.services.sqs.model.QueueAttributeName;
import com.amazonaws.services.sqs.model.SetQueueAttributesRequest;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.*;

public class VideoDetect {


    private static String sqsQueueName=null;
    private static String snsTopicName=null;
    private static String snsTopicArn = null;
    private static String roleArn= null;
```

```
    private static String sqsQueueUrl = null;
    private static String sqsQueueArn = null;
    private static String startJobId = null;
    private static String bucket = null;
    private static String video = null;
    private static AmazonSQS sqs=null;
    private static AmazonSNS sns=null;
    private static AmazonRekognition rek = null;

    private static NotificationChannel channel= new NotificationChannel()
            .withSNSTopicArn(snsTopicArn)
            .withRoleArn(roleArn);


    public static void main(String[] args) throws Exception {

        video = "";
        bucket = "";
        roleArn= "";

        sns = AmazonSNSClientBuilder.defaultClient();
        sqs= AmazonSQSClientBuilder.defaultClient();
        rek = AmazonRekognitionClientBuilder.defaultClient();

        CreateTopicandQueue();

        //=================================================

        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();

       //=================================================


        DeleteTopicandQueue();
        System.out.println("Done!");

    }


    static boolean GetSQSMessageSuccess() throws Exception
    {
     boolean success=false;


        System.out.println("Waiting for job: " + startJobId);
        //Poll queue for messages
        List<Message> messages=null;
        int dotLine=0;
        boolean jobFound=false;

        //loop until the job status is published. Ignore other messages in queue.
        do{
            messages = sqs.receiveMessage(sqsQueueUrl).getMessages();
            if (dotLine++<40){
                System.out.print(".");
            }else{
                System.out.println();
                dotLine=0;
            }

            if (!messages.isEmpty()) {
                //Loop through messages received.
                for (Message message: messages) {
```

```
                    String notification = message.getBody();

                    // Get status and job id from notification.
                    ObjectMapper mapper = new ObjectMapper();
                    JsonNode jsonMessageTree = mapper.readTree(notification);
                    JsonNode messageBodyText = jsonMessageTree.get("Message");
                    ObjectMapper operationResultMapper = new ObjectMapper();
                    JsonNode jsonResultTree =
operationResultMapper.readTree(messageBodyText.textValue());
                    JsonNode operationJobId = jsonResultTree.get("JobId");
                    JsonNode operationStatus = jsonResultTree.get("Status");
                    System.out.println("Job found was " + operationJobId);
                    // Found job. Get the results and display.
                    if(operationJobId.asText().equals(startJobId)){
                        jobFound=true;
                        System.out.println("Job id: " + operationJobId );
                        System.out.println("Status : " +
operationStatus.toString());
                        if (operationStatus.asText().equals("SUCCEEDED")){
                         success=true;
                        }
                        else{
                            System.out.println("Video analysis failed");
                        }

                        sqs.deleteMessage(sqsQueueUrl,message.getReceiptHandle());
                    }

                    else{
                        System.out.println("Job received was not job " +
startJobId);
                        //Delete unknown message. Consider moving message to dead
letter queue
                        sqs.deleteMessage(sqsQueueUrl,message.getReceiptHandle());
                    }
                }
            }
            else {
                Thread.sleep(5000);
            }
        } while (!jobFound);

        System.out.println("Finished processing video");
        return success;
    }


    private static void StartLabelDetection(String bucket, String video) throws
Exception{

        NotificationChannel channel= new NotificationChannel()
                .withSNSTopicArn(snsTopicArn)
                .withRoleArn(roleArn);


        StartLabelDetectionRequest req = new StartLabelDetectionRequest()
                .withVideo(new Video()
                        .withS3Object(new S3Object()
                                .withBucket(bucket)
                                .withName(video)))
                .withMinConfidence(50F)
                .withJobTag("DetectingLabels")
                .withNotificationChannel(channel);

        StartLabelDetectionResult startLabelDetectionResult =
rek.startLabelDetection(req);
```

```
        startJobId=startLabelDetectionResult.getJobId();

    }

    private static void GetLabelDetectionResults() throws Exception{

        int maxResults=10;
        String paginationToken=null;
        GetLabelDetectionResult labelDetectionResult=null;

        do {
            if (labelDetectionResult !=null){
                paginationToken = labelDetectionResult.getNextToken();
            }

            GetLabelDetectionRequest labelDetectionRequest= new
GetLabelDetectionRequest()
                    .withJobId(startJobId)
                    .withSortBy(LabelDetectionSortBy.TIMESTAMP)
                    .withMaxResults(maxResults)
                    .withNextToken(paginationToken);


            labelDetectionResult = rek.getLabelDetection(labelDetectionRequest);

            VideoMetadata videoMetaData=labelDetectionResult.getVideoMetadata();

            System.out.println("Format: " + videoMetaData.getFormat());
            System.out.println("Codec: " + videoMetaData.getCodec());
            System.out.println("Duration: " + videoMetaData.getDurationMillis());
            System.out.println("FrameRate: " + videoMetaData.getFrameRate());


            //Show labels, confidence and detection times
            List<LabelDetection> detectedLabels= labelDetectionResult.getLabels();

            for (LabelDetection detectedLabel: detectedLabels) {
                long seconds=detectedLabel.getTimestamp();
                Label label=detectedLabel.getLabel();
                System.out.println("Millisecond: " + Long.toString(seconds) + " ");

                System.out.println("   Label:" + label.getName());
                System.out.println("   Confidence:" +
detectedLabel.getLabel().getConfidence().toString());

                List<Instance> instances = label.getInstances();
                System.out.println("   Instances of " + label.getName());
                if (instances.isEmpty()) {
                    System.out.println("        " + "None");
                } else {
                    for (Instance instance : instances) {
                        System.out.println("        Confidence: " +
instance.getConfidence().toString());
                        System.out.println("        Bounding box: " +
instance.getBoundingBox().toString());
                    }
                }
                System.out.println("   Parent labels for " + label.getName() +
":");
                List<Parent> parents = label.getParents();
                if (parents.isEmpty()) {
                    System.out.println("        None");
                } else {
                    for (Parent parent : parents) {
                        System.out.println("        " + parent.getName());
                    }
```

```
                }
                System.out.println();
            }
        } while (labelDetectionResult !=null &&
labelDetectionResult.getNextToken() != null);

    }

    // Creates an SNS topic and SQS queue. The queue is subscribed to the topic.
    static void CreateTopicandQueue()
    {
        //create a new SNS topic
        snsTopicName="AmazonRekognitionTopic" +
Long.toString(System.currentTimeMillis());
        CreateTopicRequest createTopicRequest = new
CreateTopicRequest(snsTopicName);
        CreateTopicResult createTopicResult = sns.createTopic(createTopicRequest);
        snsTopicArn=createTopicResult.getTopicArn();

        //Create a new SQS Queue
        sqsQueueName="AmazonRekognitionQueue" +
Long.toString(System.currentTimeMillis());
        final CreateQueueRequest createQueueRequest = new
CreateQueueRequest(sqsQueueName);
        sqsQueueUrl = sqs.createQueue(createQueueRequest).getQueueUrl();
        sqsQueueArn = sqs.getQueueAttributes(sqsQueueUrl,
Arrays.asList("QueueArn")).getAttributes().get("QueueArn");

        //Subscribe SQS queue to SNS topic
        String sqsSubscriptionArn = sns.subscribe(snsTopicArn, "sqs",
sqsQueueArn).getSubscriptionArn();

        // Authorize queue
          Policy policy = new Policy().withStatements(
                  new Statement(Effect.Allow)
                  .withPrincipals(Principal.AllUsers)
                  .withActions(SQSActions.SendMessage)
                  .withResources(new Resource(sqsQueueArn))
                  .withConditions(new
Condition().withType("ArnEquals").withConditionKey("aws:SourceArn").withValues(snsTopicArn))
                  );


          Map queueAttributes = new HashMap();
          queueAttributes.put(QueueAttributeName.Policy.toString(),
policy.toJson());
          sqs.setQueueAttributes(new SetQueueAttributesRequest(sqsQueueUrl,
queueAttributes));


         System.out.println("Topic arn: " + snsTopicArn);
         System.out.println("Queue arn: " + sqsQueueArn);
         System.out.println("Queue url: " + sqsQueueUrl);
         System.out.println("Queue sub arn: " + sqsSubscriptionArn );
     }
    static void DeleteTopicandQueue()
    {
        if (sqs !=null) {
            sqs.deleteQueue(sqsQueueUrl);
            System.out.println("SQS queue deleted");
        }

        if (sns!=null) {
            sns.deleteTopic(snsTopicArn);
            System.out.println("SNS topic deleted");
        }
```

```
    }
}
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
import json
import sys
import time



class VideoDetect:
    jobId = ''
    rek = boto3.client('rekognition')
    sqs = boto3.client('sqs')
    sns = boto3.client('sns')

    roleArn = ''
    bucket = ''
    video = ''
    startJobId = ''

    sqsQueueUrl = ''
    snsTopicArn = ''
    processType = ''

    def __init__(self, role, bucket, video):
        self.roleArn = role
        self.bucket = bucket
        self.video = video

    def GetSQSMessageSuccess(self):

        jobFound = False
        succeeded = False

        dotLine=0
        while jobFound == False:
            sqsResponse = self.sqs.receive_message(QueueUrl=self.sqsQueueUrl,
 MessageAttributeNames=['ALL'],
                                           MaxNumberOfMessages=10)

            if sqsResponse:

                if 'Messages' not in sqsResponse:
                    if dotLine<40:
                        print('.', end='')
                        dotLine=dotLine+1
                    else:
                        print()
                        dotLine=0
                    sys.stdout.flush()
                    time.sleep(5)
                    continue

                for message in sqsResponse['Messages']:
                    notification = json.loads(message['Body'])
                    rekMessage = json.loads(notification['Message'])
                    print(rekMessage['JobId'])
```

```
                    print(rekMessage['Status'])
                    if rekMessage['JobId'] == self.startJobId:
                        print('Matching Job Found:' + rekMessage['JobId'])
                        jobFound = True
                        if (rekMessage['Status']=='SUCCEEDED'):
                            succeeded=True

                        self.sqs.delete_message(QueueUrl=self.sqsQueueUrl,
                                      ReceiptHandle=message['ReceiptHandle'])
                    else:
                        print("Job didn't match:" +
                              str(rekMessage['JobId']) + ' : ' + self.startJobId)
                    # Delete the unknown message. Consider sending to dead letter
queue
                    self.sqs.delete_message(QueueUrl=self.sqsQueueUrl,
                                  ReceiptHandle=message['ReceiptHandle'])


        return succeeded

    def StartLabelDetection(self):
        response=self.rek.start_label_detection(Video={'S3Object': {'Bucket':
self.bucket, 'Name': self.video}},
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)


    def GetLabelDetectionResults(self):
        maxResults = 10
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_label_detection(JobId=self.startJobId,
                                        MaxResults=maxResults,
                                        NextToken=paginationToken,
                                        SortBy='TIMESTAMP')

            print('Codec: ' + response['VideoMetadata']['Codec'])
            print('Duration: ' + str(response['VideoMetadata']['DurationMillis']))
            print('Format: ' + response['VideoMetadata']['Format'])
            print('Frame rate: ' + str(response['VideoMetadata']['FrameRate']))
            print()

            for labelDetection in response['Labels']:
                label=labelDetection['Label']

                print("Timestamp: " + str(labelDetection['Timestamp']))
                print("   Label: " + label['Name'])
                print("   Confidence: " +  str(label['Confidence']))
                print("   Instances:")
                for instance in label['Instances']:
                    print ("      Confidence: " + str(instance['Confidence']))
                    print ("      Bounding box")
                    print ("        Top: " + str(instance['BoundingBox']['Top']))
                    print ("        Left: " + str(instance['BoundingBox']['Left']))
                    print ("        Width: " +  str(instance['BoundingBox']
['Width']))
                    print ("        Height: " +  str(instance['BoundingBox']
['Height']))
                    print()
                print()
                print ("   Parents:")
```

```
                for parent in label['Parents']:
                    print ("      " + parent['Name'])
                print ()

                if 'NextToken' in response:
                    paginationToken = response['NextToken']
                else:
                    finished = True


    def CreateTopicandQueue(self):

        millis = str(int(round(time.time() * 1000)))

        #Create SNS topic

        snsTopicName="AmazonRekognitionExample" + millis

        topicResponse=self.sns.create_topic(Name=snsTopicName)
        self.snsTopicArn = topicResponse['TopicArn']

        #create SQS queue
        sqsQueueName="AmazonRekognitionQueue" + millis
        self.sqs.create_queue(QueueName=sqsQueueName)
        self.sqsQueueUrl = self.sqs.get_queue_url(QueueName=sqsQueueName)
['QueueUrl']

        attribs = self.sqs.get_queue_attributes(QueueUrl=self.sqsQueueUrl,
                                                AttributeNames=['QueueArn'])
['Attributes']

        sqsQueueArn = attribs['QueueArn']

        # Subscribe SQS queue to SNS topic
        self.sns.subscribe(
            TopicArn=self.snsTopicArn,
            Protocol='sqs',
            Endpoint=sqsQueueArn)

        #Authorize SNS to write SQS queue
        policy = """{{
  "Version":"2012-10-17",
  "Statement":[
    {{
      "Sid":"MyPolicy",
      "Effect":"Allow",
      "Principal" : {{"AWS" : "*"}},
      "Action":"SQS:SendMessage",
      "Resource": "{}",
      "Condition":{{
        "ArnEquals":{{
          "aws:SourceArn": "{}"
        }}
      }}
    }}
  ]
}}""".format(sqsQueueArn, self.snsTopicArn)

        response = self.sqs.set_queue_attributes(
            QueueUrl = self.sqsQueueUrl,
            Attributes = {
                'Policy' : policy
            })

    def DeleteTopicandQueue(self):
        self.sqs.delete_queue(QueueUrl=self.sqsQueueUrl)
```

```
        self.sns.delete_topic(TopicArn=self.snsTopicArn)


def main():
    roleArn = ''
    bucket = ''
    video = ''

    analyzer=VideoDetect(roleArn, bucket,video)
    analyzer.CreateTopicandQueue()

    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()

    analyzer.DeleteTopicandQueue()


if __name__ == "__main__":
    main()
```

4. 建置並執行程式碼。此操作可能需要一些時間來完成。完成之後，將顯示在影片中偵測到的標籤清單。如需詳細資訊，請參閱 偵測影片中的標籤 (p. 116)。

以 AWS Command Line Interface 分析影片

您可以使用 AWS Command Line Interface (AWS CLI) 來呼叫 Amazon Rekognition Video 操作。設計模式與透過 AWS SDK for Java 或其他 AWS 開發套件使用 Amazon Rekognition Video API 相同。如需詳細資訊，請參閱 Amazon Rekognition VideoAPI 概觀 (p. 50)。以下程序示範如何使用 AWS CLI 來偵測影片中的標籤。

呼叫 `start-label-detection` 來開始偵側影片中的標籤。當 Amazon Rekognition 完成影片分析時，完成狀態會傳送到 `start-label-detection` 的 `--notification-channel` 參數中所指定的 Amazon SNS 主題。您可以透過訂閱 Amazon Simple Queue Service (Amazon SQS) 佇列到 Amazon SNS 主題來取得完成狀態。接著輪詢 receive-message 以自 Amazon SQS 佇列取得完成狀態。

完成狀態通知的為 `receive-message` 回應內的 JSON 結構。您需要自回應中擷取出 JSON。如需完成狀態 JSON 的相關資訊，請參閱 參考：影片分析結果通知 (p. 74)。如果已完整狀態 JSON 的 `Status` 欄位值為 `SUCCEEDED`，您可以呼叫 `get-label-detection` 來取得影片分析請求的結果。

以下程序不包含用以輪詢 Amazon SQS 佇列的程式碼。此外也不包含用於剖析自 Amazon SQS 佇列傳回的 JSON 之程式碼。如需 Java 範例，請參閱 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。

先決條件

若要執行此程序，您需要安裝 AWS CLI。如需詳細資訊，請參閱 Amazon Rekognition 入門 (p. 10)。您使用的 AWS 帳戶必須有 Amazon Rekognition API 的存取許可。如需詳細資訊，請參閱 Amazon Rekognition 定義的動作。

若要設定 Amazon Rekognition Video 並上傳影片

1. 設定使用者存取 Amazon Rekognition Video 的權限，並設定 Amazon Rekognition Video 存取 Amazon SNS 的權限。如需詳細資訊，請參閱 設定 Amazon Rekognition Video (p. 55)。
2. 將 MOV 或 MPEG-4 格式的影片上傳到 S3 儲存貯體。在開發和測試階段，我們建議使用長度不超過 30 秒的短片。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。

若要偵測影片中的標籤

1. 執行以下 AWS CLI 命令來開始偵測影片中的標籤。

```
aws rekognition start-label-detection --video
 "S3Object={Bucket=bucketname,Name=videofile}" \
--notification-channel "SNSTopicArn=TopicARN,RoleArn=RoleARN" \
--region us-east-1
```

更新下列的值：

- 將 bucketname 與 videofile 變更為您在步驟 2 中指定的 Amazon S3 儲存貯體與影片檔名稱。
- 將 us-east-1 變更為您正在使用的 AWS 區域。
- 將 TopicARN 變更為您在設定 Amazon Rekognition Video (p. 55)步驟 3 建立的 Amazon SNS 主題的 ARN。
- 將 RoleARN 變更為您在設定 Amazon Rekognition Video (p. 55)步驟 7 建立的 IAM 服務角色的 ARN。

2. 請記下回應中的 JobId 值。回應看起來類似以下 JSON 範例。

```
{
    "JobId": "547089ce5b9a8a0e7831afa655f42e5d7b5c838553f1a584bf350ennnnnnnnnn"
}
```

3. 編寫程式碼來輪詢 Amazon SQS 佇列以取得完成狀態 JSON (使用 receive-message)。
4. 編寫程式碼以自完成狀態 JSON 擷取 Status 欄位。
5. 如果 Status 的值是 SUCCESS，請執行以下 AWS CLI 命令來顯示標籤偵測結果。

```
aws rekognition get-label-detection  --job-id JobId \
--endpoint-url Endpoint \
--region us-east-1
```

更新下列的值：

- 變更 JobId 以符合您在步驟 2 中所記下的任務識別碼。
- 變更 Endpoint 和 us-east-1 為 AWS 端點與您正在使用的區域。

結果看起來類似以下 JSON 範例：

```
{
    "Labels": [
        {
            "Timestamp": 0,
            "Label": {
                "Confidence": 99.03720092773438,
                "Name": "Speech"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Confidence": 71.6698989868164,
                "Name": "Pumpkin"
            }
        },
        {
            "Timestamp": 0,
```

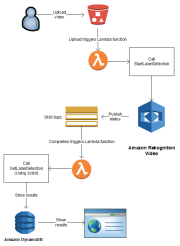
```
            "Label": {
                "Confidence": 71.6698989868164,
                "Name": "Squash"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Confidence": 71.6698989868164,
                "Name": "Vegetable"
            }
        }, .......
```

教學：建立 Amazon Rekognition Lambda 函數

本教學說明如何使用 Java Lambda 函數，取得標籤偵測影片分析操作的結果。

您可以在 Amazon Rekognition Video 操作中使用 Lambda 函數。例如，以下圖表顯示網站使用 Lambda 函數，在影片上傳到 Amazon S3 儲存貯體時自動開始分析影片。當 Lambda 函數觸發時，會呼叫 the section called “StartLabelDetection” (p. 415) 開始偵測上傳的影片中的標籤。如需使用 Lambda 處理來自 Amazon S3 儲存貯體事件通知的相關資訊，請參閱搭配 Amazon S3 事件使用 AWS Lambda 。

當分析完成狀態傳送到註冊的 Amazon SNS 主題時，第二個 Lambda 函數就會觸發。第二個 Lambda 函數呼叫 the section called “GetLabelDetection” (p. 358)，以取得分析結果。然後，結果會存放在資料庫以準備顯示於網頁。本教學的重點在於此第二個 lambda 函數。



在本教學中，當 Amazon Rekognition Video 將影片分析完成狀態傳送到註冊的 Amazon SNS 主題時，Lambda 函數就會觸發。然後，它會呼叫 the section called “GetLabelDetection” (p. 358) 以收集影片分析結果。為了示範，此教學將標籤偵測結果寫入至 CloudWatch 日誌。在您應用程式中的 Lambda 函數，您應該儲存分析結果以供日後使用。例如，您可以使用 Amazon DynamoDB 以儲存分析結果。如需詳細資訊，請參閱使用 DynamoDB。

下列程序將告訴您如何做到：

- 建立 Amazon SNS 主題和設定許可。
- 使用 AWS Management Console 建立 Lambda 函數並將它訂閱至 Amazon SNS 主題。
- 使用 AWS Management Console 設定 Lambda 函數。
- 將範本程式碼新增至 AWS Toolkit for Eclipse 專案，然後將它上傳至 Lambda 函數。
- 使用 AWS CLI 測試 Lambda 函數。

> **Note**
>
> 請在教學中使用相同的 AWS 區域。

先決條件

本教學假設您已熟悉 AWS Toolkit for Eclipse。如需詳細資訊，請參閱 AWS Toolkit for Eclipse。

建立 SNS 主題

Amazon Rekognition Video 影片分析操作完成狀態已傳送至 Amazon SNS 主題。此程序會建立 Amazon SNS 主題和 IAM 服務角色，讓 Amazon Rekognition Video 存取您的 Amazon SNS 主題。如需詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

建立 Amazon SNS 主題

1. 如果您尚未建立 IAM 服務角色，請先建立才可對該角色提供存取 Amazon SNS 主題的 Amazon Rekognition Video 權限。請記下 Amazon Resource Name (ARN)。如需詳細資訊，請參閱 提供對多個 Amazon SNS 主題的存取權限 (p. 56)。
2. 使用 Amazon SNS 主控台建立 Amazon SNS 主題。您只需要指定主題名稱。將 AmazonRekognition 置於主題名稱的前面。請記下主題 ARN。

建立 Lambda 函數

您使用 AWS Management Console 建立 Lambda 函數。然後，您使用 AWS Toolkit for Eclipse 專案將 Lambda 函數套件上傳至 AWS Lambda。您也可以使用 AWS Toolkit for Eclipse 建立 Lambda 函數。如需詳細資訊，請參閱教學：如何建立、上傳和叫用 AWS Lambda 函數。

建立 Lambda 函數

1. 登入至 AWS 管理主控台，並開啟位於 https://console.aws.amazon.com/lambda/ 的 AWS Lambda 主控台。
2. 選擇 Create function (建立函式)。
3. 選擇 Author from scratch (從頭開始撰寫)。
4. 在 Function Name (函數名稱) 中，為您的函數輸入名稱。
5. 在 Runtime* (執行時間*) 中，選擇 Java 8。
6. 選擇 Choose or create an execution role (選擇或建立執行角色)。
7. 在 Execution role (執行角色) 中，選擇 Create a new role with basic Lambda permissions (建立具備基本 Lambda 許可的新角色)。
8. 請注意，新角色名稱會顯示在 Basic information (基本資訊) 區段的底部。
9. 選擇 Create function (建立函式)。

設定 Lambda 函數

建立 Lambda 函數後，您可以設定它來讓您在 建立 SNS 主題 (p. 69) 中建立的 Amazon SNS 主題予以觸發。您也將調整 Lambda 函數的記憶體需求和逾時期間。

設定 Lambda 函數

1. 在 Function Code (函數程式碼) 的 Handler (處理常式) 中輸入 `com.amazonaws.lambda.demo.JobCompletionHandler`。
2. 在 Basic settings (基本設定) 中，選擇 Edit (編輯)。Edit basic settings (編輯基本設定) 對話方塊隨即顯示。
   a. 在 Memory (記憶體) 中選擇 1024。
   b. 在 Timeout (逾時) 中選擇 10 秒。
   c. 選擇 Save。
3. 在 Designer (設計工具) 中，選擇 +Add trigger (+新增觸發條件)。[Add trigger (新增觸發條件)] 對話方塊隨即顯示。

4. 在 Trigger configuration (觸發條件組態) 中選擇 SNS。

   在 SNS topic (SNS 主題) 中，選擇您在 建立 SNS 主題 (p. 69) 中建立的 Amazon SNS 主題。
5. 選擇 Enable trigger (啟用觸發條件)。
6. 若要新增觸發條件，請選擇 Add (新增)。
7. 選擇 Save 以儲存 Lambda 功能。

設定 IAM Lambda 角色

若要呼叫 Amazon Rekognition Video 操作，您可以將 AmazonRekognitionFullAccess AWS 受管政策新增至 IAM Lambda 角色。開始操作，例如 the section called “StartLabelDetection” (p. 415)，還需要傳遞 IAM 服務角色的角色許可，以供 Amazon Rekognition Video 用於存取 Amazon SNS 主題。

設定角色

1. Sign in to the AWS Management Console and open the IAM console at https://console.aws.amazon.com/iam/.
2. 在導覽窗格中，選擇 Roles (角色)。
3. 在清單中，選擇您在 建立 Lambda 函數 (p. 69) 中建立之執行角色的名稱。
4. 選擇 Permissions (許可) 索引標籤。
5. 選擇 Attach policies (連接政策)。
6. 從政策清單中選擇 AmazonRekognitionFullAccess。
7. 選擇 Attach policy (連接政策)。
8. 再次選擇執行角色。
9. 選擇 Add inline policy (新增內嵌政策)。
10. 請選擇 JSON 索引標籤。
11. 用以下政策取代現有政策。用您在 建立 SNS 主題 (p. 69) 中建立的 IAM 服務角色取代 `servicerole`。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "mysid",
            "Effect": "Allow",
            "Action": "iam:PassRole",
            "Resource": "arn:servicerole"
        }
    ]
}
```

12. 選擇 Review policy (檢閱政策)。
13. 在 Name* (名稱*) 中輸入政策的名稱。
14. 選擇 Create policy (建立政策)。

建立 AWS Toolkit for Eclipse Lambda 專案

當 Lambda 函數觸發時，以下程式碼會從 Amazon SNS 主題取得完成狀態，然後呼叫 the section called “GetLabelDetection” (p. 358) 以取得分析結果。偵測到標籤計數，偵測到的標籤清單寫入至 CloudWatch 日誌。您的 Lambda 函數應儲存影片分析結果以供日後使用。

建立 AWS Toolkit for Eclipse Lambda 專案

1. 建立 AWS Toolkit for Eclipse AWS Lambda 專案。
   - 在 Project name: (專案名稱：) 中，輸入您選擇的專案名稱。
   - 在 Class Name: (類別名稱：) 中，輸入 JobCompletionHandler。
   - 在 Input type: (輸入類型：) 中選擇 SNS Event (SNS 事件)。
   - 保留其他欄位不變。
2. 在 Eclipse Project (Eclipse 專案) 總管中，開啟產生的 Lambda 處理常式 (JobCompletionHandler.java)，並以下列內容取代：

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package com.amazonaws.lambda.demo;

import com.amazonaws.services.lambda.runtime.Context;
import com.amazonaws.services.lambda.runtime.LambdaLogger;
import com.amazonaws.services.lambda.runtime.RequestHandler;
import com.amazonaws.services.lambda.runtime.events.SNSEvent;
import java.util.List;
import com.amazonaws.regions.Regions;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.GetLabelDetectionRequest;
import com.amazonaws.services.rekognition.model.GetLabelDetectionResult;
import com.amazonaws.services.rekognition.model.LabelDetection;
import com.amazonaws.services.rekognition.model.LabelDetectionSortBy;
import com.amazonaws.services.rekognition.model.VideoMetadata;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;



public class JobCompletionHandler implements RequestHandler<SNSEvent, String> {

   @Override
   public String handleRequest(SNSEvent event, Context context) {

      String message = event.getRecords().get(0).getSNS().getMessage();
      LambdaLogger logger = context.getLogger();

      // Parse SNS event for analysis results. Log results
      try {
         ObjectMapper operationResultMapper = new ObjectMapper();
         JsonNode jsonResultTree = operationResultMapper.readTree(message);
         logger.log("Rekognition Video Operation:==========================");
         logger.log("Job id: " + jsonResultTree.get("JobId"));
         logger.log("Status : " + jsonResultTree.get("Status"));
         logger.log("Job tag : " + jsonResultTree.get("JobTag"));
         logger.log("Operation : " + jsonResultTree.get("API"));

         if (jsonResultTree.get("API").asText().equals("StartLabelDetection")) {

            if (jsonResultTree.get("Status").asText().equals("SUCCEEDED")){
               GetResultsLabels(jsonResultTree.get("JobId").asText(), context);
            }
            else{
               String errorMessage = "Video analysis failed for job "
                     + jsonResultTree.get("JobId")
                     + "State " + jsonResultTree.get("Status");
               throw new Exception(errorMessage);
```

```
            }

         } else
            logger.log("Operation not StartLabelDetection");

      } catch (Exception e) {
         logger.log("Error: " + e.getMessage());
         throw new RuntimeException (e);


      }

      return message;
   }

   void GetResultsLabels(String startJobId, Context context) throws Exception {

      LambdaLogger logger = context.getLogger();

      AmazonRekognition rek =
AmazonRekognitionClientBuilder.standard().withRegion(Regions.US_EAST_1).build();

      int maxResults = 1000;
      String paginationToken = null;
      GetLabelDetectionResult labelDetectionResult = null;
      String labels = "";
      Integer labelsCount = 0;
      String label = "";
      String currentLabel = "";

      //Get label detection results and log them.
      do {

         GetLabelDetectionRequest labelDetectionRequest = new
GetLabelDetectionRequest().withJobId(startJobId)

.withSortBy(LabelDetectionSortBy.NAME).withMaxResults(maxResults).withNextToken(paginationToken);

         labelDetectionResult = rek.getLabelDetection(labelDetectionRequest);

         paginationToken = labelDetectionResult.getNextToken();
         VideoMetadata videoMetaData = labelDetectionResult.getVideoMetadata();

         // Add labels to log
         List<LabelDetection> detectedLabels = labelDetectionResult.getLabels();

         for (LabelDetection detectedLabel : detectedLabels) {
            label = detectedLabel.getLabel().getName();
            if (label.equals(currentLabel)) {
               continue;
            }
            labels = labels + label + " / ";
            currentLabel = label;
            labelsCount++;

         }
      } while (labelDetectionResult != null && labelDetectionResult.getNextToken() !=
null);

      logger.log("Total number of labels : " + labelsCount);
      logger.log("labels : " + labels);

   }


}
```

```
```

3. Rekognition 命名空間未解析。若要修正此問題：
   - 將滑鼠暫停在 `import com.amazonaws.services.rekognition.AmazonRekognition;` 這行上有底線的部分。
   - 選擇 Fix project set up... (修復專案設定…)。
   - 選擇 Amazon Rekognition 存檔的最新版本。
   - 選擇 OK 以新增存檔到專案。
4. 儲存檔案。
5. 在 Eclipse 程式碼視窗中按一下滑鼠右鍵，選擇 AWS Lambda，然後選擇 Upload function to AWS Lambda (將函數上傳至 AWS Lambda)。
6. 在 Select Target Lambda Function (選擇目標 Lambda 函數) 頁面中，選擇要使用的 AWS 區域。
7. 選擇 Choose an existing lambda function (選擇現有的 Lambda 函數)，然後選取您在 建立 Lambda 函數 (p. 69) 中建立的 Lambda 函數。
8. 選擇 Next (下一步)。Function Configuration (功能組態) 對話方塊隨即顯示。
9. 在 IAM role (IAM 角色) 中選擇您在 建立 Lambda 函數 (p. 69) 建立的 IAM 角色。
10. 選擇 Finish (完成)，Lambda 函數就會上傳至 AWS。

測試 Lambda 函數

使用以下 AWS CLI 命令，啟動影片標籤偵測分析來測試 Lambda 函數。在分析完成後，就會觸發 Lambda 函數。檢查 CloudWatch Logs 日誌以確認分析已經成功。

若要測試 Lambda 函數

1. 將 MOV 或 MPEG-4 格式的影片上傳到 S3 儲存貯體。為達測試目的，請上傳長度不超過 30 秒的影片。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
2. 執行以下 AWS CLI 命令來開始偵測影片中的標籤。

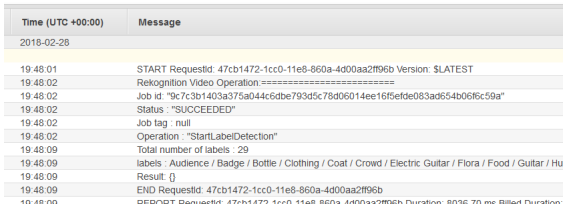
```
aws rekognition start-label-detection --video
 "S3Object={Bucket="bucketname",Name="videofile"}" \
--notification-channel "SNSTopicArn=TopicARN,RoleArn=RoleARN" \
--region Region
```

   更新下列的值：
   - 將 `bucketname` 和 `videofile` 變更為 Amazon S3 儲存貯體名稱和您想要偵測標籤的影片檔案名稱。
   - 將 `TopicARN` 變更為您在 建立 SNS 主題 (p. 69) 中建立的 Amazon SNS 主題的 ARN。
   - 將 `RoleARN` 變更為您在 建立 SNS 主題 (p. 69) 中建立的 IAM 角色的 ARN。
   - 將 `Region` 變更為您正在使用的 AWS 區域。
3. 請記下回應中的 `JobId` 值。回應看起來類似以下 JSON 範例。

```
{
    "JobId": "547089ce5b9a8a0e7831afa655f42e5d7b5c838553f1a584bf350ennnnnnnnnn"
```

```
}
```

4. 開啟 https://console.aws.amazon.com/cloudwatch/ 主控台。
5. 當分析完成後，Lambda 函數的日誌項目會顯示在 Log Group (日誌群組)。
6. 選擇 Lambda 函數以查看日誌串流。
7. 選擇最新的日誌串流，以查看 Lambda 函數建立的日誌項目。如果操作成功，將如下所示：



Time (UTC +00:00)	Message
2018-02-28	
19:48:01	START RequestId: 47cb1472-1cc0-11e8-860a-4d00aa2ff96b Version: $LATEST
19:48:02	Rekognition Video Operation:==========
19:48:02	Job id: "9c7c3b1403a375a044c6dbe793d5c78d06014ee16f5efde083ad654b06f6c59a"
19:48:02	Status : "SUCCEEDED"
19:48:02	Job tag : null
19:48:02	Operation : "StartLabelDetection"
19:48:09	Total number of labels : 29
19:48:09	labels : Audience / Badge / Bottle / Clothing / Coat / Crowd / Electric Guitar / Flora / Food / Guitar / Hu
19:48:09	Result: {}
19:48:09	END RequestId: 47cb1472-1cc0-11e8-860a-4d00aa2ff96b

Job ID 的值應符合您在步驟 3 記下的 `JobId` 值。

參考：影片分析結果通知

Amazon Rekognition 發佈 Amazon Rekognition Video 分析請求的結果至 Amazon Simple Notification Service (Amazon SNS) 主題，結果中包括完成狀態。要自 Amazon SNS 主題來取得通知，請使用 Amazon Simple Queue Service 佇列或 AWS Lambda 函數。如需詳細資訊，請參閱 the section called “呼叫 Amazon Rekognition Video 操作” (p. 51)。如需範例，請參閱 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。

承載內容以下方 JSON 格式顯示：

```
{
  "JobId": "String",
  "Status": "String",
  "API": "String",
  "JobTag": "String",
  "Timestamp": Number,
  "Video": {
    "S3ObjectName": "String",
    "S3Bucket": "String"
  }
}
```

名稱	描述
JobId	任務識別碼。符合自 `Start` 操作傳回的任務識別碼，例如 StartPersonTracking (p. 419)。
狀態	工作的狀態。有效值為「SUCCEEDED」(成功)、「FAILED」(失敗) 或「ERROR」(錯誤)。
API	用於分析輸入影片的 Amazon Rekognition Video 操作。
JobTag	任務的識別碼。在呼叫中指定 `JobTag` 以開始操作，例如 StartLabelDetection (p. 415)。
時間戳記	顯示任務完成的 Unix 時間戳記。
影片	已完成處理的視訊詳細資訊。包含檔案名稱與檔案所存放的 Amazon S3 儲存貯體。

以下為傳送到 Amazon SNS 主題的成功通知範例。

```
{
  "JobId": "6de014b0-2121-4bf0-9e31-856a18719e22",
  "Status": "SUCCEEDED",
  "API": "LABEL_DETECTION",
  "Message": "",
  "Timestamp": 1502230160926,
  "Video": {
    "S3ObjectName": "video.mpg",
    "S3Bucket": "videobucket"
  }
}
```

故障診斷 Amazon Rekognition Video

以下內容涵蓋使用 Amazon Rekognition Video 與已儲存影片時的故障診斷資訊。

我未曾收到傳送至 Amazon SNS 主題的完成狀態訊息

當影片分析完成時，Amazon Rekognition Video 會將狀態資訊發佈至 Amazon SNS 主題。一般而言，您可以使用 Amazon SQS 佇列或 Lambda 函數來訂閱主題，以取得完成狀態訊息。為順利進行調查，您可以透過電子郵件訂閱 Amazon SNS 主題。如此，電子郵件收件匣即會收到傳送至 Amazon SNS 主題的訊息。如需詳細資訊，請參閱訂閱主題。

如果您的應用程式沒有收到該訊息，請考慮下列情況：

- 請確認分析已完成。請檢查「取得」操作中的 `JobStatus` 值 (例如，`GetLabelDetection`)。若該值為 `IN_PROGRESS`，則代表系統尚未完成分析，且尚未將完成狀態發佈至 Amazon SNS 主題。
- 請確認您擁有 IAM 服務角色，以授予 Amazon Rekognition Video 發佈至 Amazon SNS 主題的許可。如需詳細資訊，請參閱 設定 Amazon Rekognition Video (p. 55)。
- 確認目前使用的 IAM 服務角色可利用角色登入資料來發佈至 Amazon SNS 主題。使用下列步驟：
  - 取得使用者的 Amazon Resource Name (ARN)：

    ```
    aws sts get-caller-identity --profile RekognitionUser
    ```

  - 使用 AWS 管理主控台來將使用者 ARN 新增至角色信任關係。例如：

    ```
    {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": "rekognition.amazonaws.com",
            "AWS": "arn:User ARN"
          },
          "Action": "sts:AssumeRole",
          "Condition": {}
        }
      ]
    }
    ```

  - 假設該角色：`aws sts assume-role --role-arn arn:Role ARN --role-session-name SessionName --profile RekognitionUser`
  - 發佈至 Amazon SNS 主題：`aws sns publish --topic-arn arn:Topic ARN --message "Hello World!" --region us-east-1 --profile RekognitionUser`

如果 AWS CLI 命令可正常運作，您就會收到訊息 (若您已透過電子郵件訂閱主題，即可在電子郵件收件匣中查看該訊息)。如果您沒有收到該訊息：

- 確認您已設定 Amazon Rekognition Video。如需詳細資訊，請參閱 設定 Amazon Rekognition Video (p. 55)。
- 請查看對此問題執行故障排除的其他秘訣。

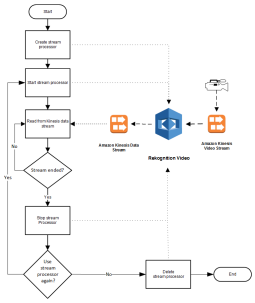
- 確認您所使用的 Amazon SNS 主題正確無誤。
  - 如果您使用 IAM 服務角色來授予 Amazon Rekognition Video 存取單一 Amazon SNS 主題的權限，請確認取得許可的 Amazon SNS 主題正確無誤。如需詳細資訊，請參閱 提供存取目前 Amazon SNS 主題的權限 (p. 56)。
  - 如果您使用 IAM 服務角色來授予 Amazon Rekognition Video 存取多個 SNS 主題的權限，則請確認您使用的是正確主題，且該主題名稱以 AmazonRekognition 為開頭。如需詳細資訊，請參閱 提供對多個 Amazon SNS 主題的存取權限 (p. 56)。
  - 如果您是使用 AWS Lambda 函數，請確認 Lambda 函數已訂閱正確的 Amazon SNS 主題。如需詳細資訊，請參閱使用 Amazon SNS 通知叫用 Lambda 函數。
- 如果 Amazon SQS 佇列成功訂閱 Amazon SNS 主題，請確定 Amazon SNS 主題擁有傳送訊息至 Amazon SQS 佇列的許可。如需詳細資訊，請參閱將許可提供給 Amazon SNS 主題，以將訊息傳送至 Amazon SQS 佇列。

處理串流影片

您可以使用 Amazon Rekognition Video 來偵測及辨識串流影片中的臉部。典型的使用案例為當您想要偵測視訊串流中的已知臉部時。Amazon Rekognition Video 使用 Amazon Kinesis Video Streams 接收和處理視訊串流。分析結果會從 Amazon Rekognition Video 輸出到 Kinesis data stream，然後供您的用戶端應用程式讀取。Amazon Rekognition Video 提供串流處理器 (CreateStreamProcessor (p. 289))，可用於啟動和管理串流視訊的分析。

> Note
>
> Amazon Rekognition Video 串流 API 僅在下列地區提供：美國東部 (維吉尼亞北部)、美國西部 (奧勒岡)、亞太區域 (東京)、歐洲 (法蘭克福) 及歐洲 (愛爾蘭)。

下圖說明 Amazon Rekognition Video 如何偵測及辨識串流影片中的臉部。



若要搭配串流影片使用 Amazon Rekognition Video，您的應用程式需要實作下列項目：

- 用於將串流影片傳送至 Amazon Rekognition Video 的 Kinesis video stream。如需詳細資訊，請參閱 Kinesis video stream。
- 用以管理串流影片分析的 Amazon Rekognition Video 串流處理器。如需詳細資訊，請參閱 開始串流影片分析 (p. 78)。
- 用以讀取 Amazon Rekognition Video 傳送至 Kinesis data stream 之分析結果的 Kinesis data stream 取用者。如需詳細資訊，請參閱 Amazon Kinesis Streams 的取用者。

本節包含有關撰寫應用程式的資訊，可供您撰寫出將 Kinesis video stream 和 Kinesis data stream 串流影片建立至 Amazon Rekognition Video，並取用分析結果的應用程式。如需詳細資訊，請參閱 辨識串流影片中的臉部 (p. 77)。

主題

- 辨識串流影片中的臉部 (p. 77)
- 提供 Amazon Rekognition Video 對 Kinesis 串流的存取 (p. 77)
- 開始串流影片分析 (p. 78)
- 讀取串流影片分析結果 (p. 85)
- 參考：Kinesis 臉部辨識記錄 (p. 87)
- 串流影片故障診斷 (p. 91)

辨識串流影片中的臉部

Amazon Rekognition Video 可在集合中搜尋與串流影片中偵測到之臉部相符的臉部。如需集合的詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。下列程序說明辨識串流影片中的臉部時要採取的步驟。

先決條件

若要執行此程序，您需要安裝 AWS SDK for Java。如需詳細資訊，請參閱 Amazon Rekognition 入門 (p. 10)。您使用的 AWS 帳戶必須有 Amazon Rekognition API 的存取許可。如需詳細資訊，請參閱 Amazon Rekognition 定義的動作。

辨識影片串流中的臉部 (AWS 開發套件)

1. 如果您尚未建立 IAM 服務角色，請先建立才可對該角色提供存取 Kinesis video stream 和 Kinesis data stream 的 Amazon Rekognition Video 權限。請記下 ARN。如需詳細資訊，請參閱 提供對 Kinesis 影片串流與 Kinesis 資料串流的存取 (p. 77)。
2. 建立集合 (p. 150)並記下您所使用的集合識別符。
3. 將您要搜尋之臉部的索引建立 (p. 162)至您於步驟 2 中建立的集合。
4. 建立 Kinesis video stream 並記下串流的 Amazon Resource Name (ARN)。
5. 建立 Kinesis data stream。在串流名稱前面加上 AmazonRekognition 並記下串流的 ARN。
6. 建立串流處理器 (p. 80)。將下列項目做為參數傳遞至 the section called “CreateStreamProcessor” (p. 289)：您選擇的名稱、Kinesis video streamARN (步驟 4)、Kinesis data stream ARN (步驟 5) 與集合識別符 (步驟 2)。
7. 使用您在步驟 6 中選擇的串流處理器名稱來啟動串流處理器 (p.)。
8. 使用 PutMedia 操作將來源影片串流至您在步驟 4 中建立的 Kinesis video stream。如需詳細資訊，請參閱 PutMedia API 範例。
9. 從 Amazon Rekognition Video 取用分析輸出 (p. 85)。

提供 Amazon Rekognition Video 對 Kinesis 串流的存取

您可以使用 AWS Identity and Access Management (IAM) 服務角色，提供 Amazon Rekognition Video 對 Kinesis video stream 的讀取存取，以及對 Kinesis data stream 的寫入存取。

提供對 Kinesis 影片串流與 Kinesis 資料串流的存取

IAM 提供 Rekognition 服務角色使用案例，配合 `AmazonRekognitionServiceRole` 許可政策使用時，可以寫入到多個 Kinesis data stream 和從所有 Kinesis video stream 讀取。若要提供 Amazon

Rekognition Video 對多個 Kinesis data stream 的寫入存取，您可以在 Kinesis data stream 的名稱之前加上 AmazonRekognition，—例如，`AmazonRekognitionMyDataStreamName`

提供 Amazon Rekognition Video 對 Kinesis video stream 與 Kinesis data stream 的存取

1. 建立 IAM 服務角色。請使用下列資訊來建立 IAM 服務角色：
    1. 針對服務名稱選擇 Rekognition。
    2. 為服務角色使用案例選擇 Rekognition。
    3. 選擇 AmazonRekognitionServiceRole 許可政策，該政策提供 Amazon Rekognition Video 對字首為 AmazonRekognition 之 Kinesis data stream 的寫入存取，以及對您所有 Kinesis video stream 的讀取存取。
2. 記下服務角色的 Amazon Resource Name (ARN)。您需要此資訊才可開始影片分析操作。

提供對個別 Kinesis 串流的存取

您可以建立許可政策來允許 Amazon Rekognition Video 存取個別 Kinesis video stream 與 Kinesis data stream。

提供 Amazon Rekognition Video 對個別 Kinesis video stream 與 Kinesis data stream 的存取

1. 使用 IAM JSON 政策編輯器建立新的許可政策，並使用下列政策。將 `data-arn` 取代為所需 Kinesis data stream 的 ARN，並將 `video-arn` 取代為所需 Kinesis video stream 的 ARN。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "kinesis:PutRecord",
                "kinesis:PutRecords"
            ],
            "Resource": "data-arn"
        },
        {
            "Effect": "Allow",
            "Action": [
                "kinesisvideo:GetDataEndpoint",
                "kinesisvideo:GetMedia"
            ],
            "Resource": "video-arn"
        }
    ]
}
```

2. 建立 IAM 服務角色，或更新現有的 IAM 服務角色。請使用下列資訊來建立 IAM 服務角色：
    1. 針對服務名稱選擇 Rekognition。
    2. 為服務角色使用案例選擇 Rekognition。
    3. 連接您在步驟 1 中建立的許可政策。
3. 請記下服務角色 ARN。您需要此資訊才可開始影片分析操作。

開始串流影片分析

若要開始分析串流影片，請啟動 Amazon Rekognition Video 串流處理器，並將影片串流至 Amazon Rekognition Video。Amazon Rekognition Video 串流處理器可讓您啟動、停止及管理串流處理器。您可以

透過呼叫 CreateStreamProcessor (p. 289) 來建立串流處理器。要求參數包含 Kinesis video stream 與 Kinesis data stream 的 Amazon Resource Name (ARN)，以及用來辨識串流影片中臉部的集合識別符。該參數也包含您為串流處理器指定的名稱。

您可以透過呼叫

Starts processing a stream processor. You create a stream processor by calling CreateStreamProcessor (p. 289). To tell `StartStreamProcessor` which stream processor to start, use the value of the `Name` field specified in the call to `CreateStreamProcessor`.

Request Syntax

```
{
   "Name": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Name (p. 425)

The name of the stream processor to start processing.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Elements

If the action is successful, the service sends back an HTTP 200 response with an empty HTTP body.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface

- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

(p. 425) 操作來開始處理影片。若要取得串流處理器的狀態資訊，請呼叫 DescribeStreamProcessor (p. 312)。您可以呼叫的其他操作包括 StopStreamProcessor (p. 433) (停止串流處理器) 與 DeleteStreamProcessor (p. 300) (刪除串流處理器)。若要取得您帳戶中的串流處理器清單，請呼叫 ListStreamProcessors (p. 385)。

在串流處理器開始執行之後，您可以透過在 `CreateStreamProcessor` 中指定的 Kinesis video stream，將影片串流至 Amazon Rekognition Video。使用 Kinesis Video Streams 開發套件的 PutMedia 操作，將影片傳遞至 Kinesis video stream。如需範例，請參閱 PutMedia API 範例。

如需您的應用程式如何取用 Amazon Rekognition Video 分析結果的資訊，請參閱 讀取串流影片分析結果 (p. 85)。

建立 Amazon Rekognition Video 串流處理器

您必須先建立 Amazon Rekognition Video 串流處理器 (CreateStreamProcessor (p. 289))，才能分析串流影片。串流處理器包含 Kinesis data stream 與 Kinesis video stream 的資訊。它也會包含集合的識別符，該集合含有您要在輸入串流影片中辨識的臉部。您也可以指定串流處理器的名稱。以下是 `CreateStreamProcessor` 要求的 JSON 範例。

```
{
	"Name": "streamProcessorForCam",
	"Input": {
		"KinesisVideoStream": {
			"Arn": "arn:aws:kinesisvideo:us-east-1:nnnnnnnnnnnn:stream/inputVideo"
		}
	},
	"Output": {
		"KinesisDataStream": {
			"Arn": "arn:aws:kinesis:us-east-1:nnnnnnnnnnnn:stream/outputData"
		}
	},
	"RoleArn": "arn:aws:iam::nnnnnnnnnnn:role/roleWithKinesisPermission",
	"Settings": {
		"FaceSearch": {
			"CollectionId": "collection-with-100-faces",
			"FaceMatchThreshold": 85.5
		}
	}
}
```

以下是 `CreateStreamProcessor` 的回應範例。

```
{
	“StreamProcessorArn”: “arn:aws:rekognition:us-east-1:nnnnnnnnnnnn:streamprocessor/
streamProcessorForCam”
}
```

啟動 Amazon Rekognition Video 串流處理器

您可以透過呼叫 StartStreamProcessor (p. 425) 並提供您在 `CreateStreamProcessor` 中指定的串流處理器名稱，來開始分析串流影片。以下是 `StartStreamProcessor` 要求的 JSON 範例。

```
{
	"Name": "streamProcessorForCam"
}
```

如果串流處理器成功啟動，就會傳回 HTTP 200 回應，以及空的 JSON 內文。

使用串流處理器

下列程式碼範例示範如何呼叫各種串流處理器操作，例如 CreateStreamProcessor (p. 289) 與 StartStreamProcessor (p. 425)。範例包含串流處理器管理員類別 (StreamManager)，可提供方法呼叫串流處理器操作。入門課程 (Starter) 建立 StreamManager 物件並呼叫多個操作。

設定範例：

1. 設定數值的 Starter 類別成員欄位至您所需的數值。
2. 在 Starter 類別函數 `main`，移除所需的函數呼叫。

Starter 類別

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

// Starter class. Use to create a StreamManager class
```

```
// and call stream processor operations.
package com.amazonaws.samples;
import com.amazonaws.samples.*;

public class Starter {

 public static void main(String[] args) {


     String streamProcessorName="Stream Processor Name";
     String kinesisVideoStreamArn="Kinesis Video Stream Arn";
     String kinesisDataStreamArn="Kinesis Data Stream Arn";
     String roleArn="Role Arn";
     String collectionId="Collection ID";
     Float matchThreshold=50F;

  try {
   StreamManager sm= new StreamManager(streamProcessorName,
     kinesisVideoStreamArn,
     kinesisDataStreamArn,
     roleArn,
     collectionId,
     matchThreshold);
   //sm.createStreamProcessor();
   //sm.startStreamProcessor();
   //sm.deleteStreamProcessor();
   //sm.deleteStreamProcessor();
   //sm.stopStreamProcessor();
   //sm.listStreamProcessors();
   //sm.describeStreamProcessor();
  }
  catch(Exception e){
   System.out.println(e.getMessage());
  }
 }
}
```

StreamManager 類別

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

// Stream manager class. Provides methods for calling
// Stream Processor operations.
package com.amazonaws.samples;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.CreateStreamProcessorRequest;
import com.amazonaws.services.rekognition.model.CreateStreamProcessorResult;
import com.amazonaws.services.rekognition.model.DeleteStreamProcessorRequest;
import com.amazonaws.services.rekognition.model.DeleteStreamProcessorResult;
import com.amazonaws.services.rekognition.model.DescribeStreamProcessorRequest;
import com.amazonaws.services.rekognition.model.DescribeStreamProcessorResult;
import com.amazonaws.services.rekognition.model.FaceSearchSettings;
import com.amazonaws.services.rekognition.model.KinesisDataStream;
import com.amazonaws.services.rekognition.model.KinesisVideoStream;
import com.amazonaws.services.rekognition.model.ListStreamProcessorsRequest;
import com.amazonaws.services.rekognition.model.ListStreamProcessorsResult;
import com.amazonaws.services.rekognition.model.StartStreamProcessorRequest;
import com.amazonaws.services.rekognition.model.StartStreamProcessorResult;
import com.amazonaws.services.rekognition.model.StopStreamProcessorRequest;
```

```
import com.amazonaws.services.rekognition.model.StopStreamProcessorResult;
import com.amazonaws.services.rekognition.model.StreamProcessor;
import com.amazonaws.services.rekognition.model.StreamProcessorInput;
import com.amazonaws.services.rekognition.model.StreamProcessorOutput;
import com.amazonaws.services.rekognition.model.StreamProcessorSettings;

public class StreamManager {

    private String streamProcessorName;
    private String kinesisVideoStreamArn;
    private String kinesisDataStreamArn;
    private String roleArn;
    private String collectionId;
    private float matchThreshold;

    private AmazonRekognition rekognitionClient;


    public StreamManager(String spName,
      String kvStreamArn,
      String kdStreamArn,
      String iamRoleArn,
      String collId,
      Float threshold){
     streamProcessorName=spName;
     kinesisVideoStreamArn=kvStreamArn;
     kinesisDataStreamArn=kdStreamArn;
     roleArn=iamRoleArn;
     collectionId=collId;
     matchThreshold=threshold;
     rekognitionClient=AmazonRekognitionClientBuilder.defaultClient();

    }

    public void createStreamProcessor() {
     //Setup input parameters
        KinesisVideoStream kinesisVideoStream = new
 KinesisVideoStream().withArn(kinesisVideoStreamArn);
        StreamProcessorInput streamProcessorInput =
                new StreamProcessorInput().withKinesisVideoStream(kinesisVideoStream);
        KinesisDataStream kinesisDataStream = new
 KinesisDataStream().withArn(kinesisDataStreamArn);
        StreamProcessorOutput streamProcessorOutput =
                new StreamProcessorOutput().withKinesisDataStream(kinesisDataStream);
        FaceSearchSettings faceSearchSettings =
                new
 FaceSearchSettings().withCollectionId(collectionId).withFaceMatchThreshold(matchThreshold);
        StreamProcessorSettings streamProcessorSettings =
                new StreamProcessorSettings().withFaceSearch(faceSearchSettings);

        //Create the stream processor
        CreateStreamProcessorResult createStreamProcessorResult =
 rekognitionClient.createStreamProcessor(
                new
 CreateStreamProcessorRequest().withInput(streamProcessorInput).withOutput(streamProcessorOutput)

 .withSettings(streamProcessorSettings).withRoleArn(roleArn).withName(streamProcessorName));

        //Display result
        System.out.println("Stream Processor " + streamProcessorName + " created.");
        System.out.println("StreamProcessorArn - " +
createStreamProcessorResult.getStreamProcessorArn());
    }

    public void startStreamProcessor() {
        StartStreamProcessorResult startStreamProcessorResult =
```

```
                rekognitionClient.startStreamProcessor(new
 StartStreamProcessorRequest().withName(streamProcessorName));
        System.out.println("Stream Processor " + streamProcessorName + " started.");
    }

    public void stopStreamProcessor() {
        StopStreamProcessorResult stopStreamProcessorResult =
                rekognitionClient.stopStreamProcessor(new
 StopStreamProcessorRequest().withName(streamProcessorName));
        System.out.println("Stream Processor " + streamProcessorName + " stopped.");
    }

    public void deleteStreamProcessor() {
        DeleteStreamProcessorResult deleteStreamProcessorResult = rekognitionClient
                .deleteStreamProcessor(new
 DeleteStreamProcessorRequest().withName(streamProcessorName));
        System.out.println("Stream Processor " + streamProcessorName + " deleted.");
    }

    public void describeStreamProcessor() {
        DescribeStreamProcessorResult describeStreamProcessorResult = rekognitionClient
                .describeStreamProcessor(new
 DescribeStreamProcessorRequest().withName(streamProcessorName));

        //Display various stream processor attributes.
        System.out.println("Arn - " +
 describeStreamProcessorResult.getStreamProcessorArn());
        System.out.println("Input kinesisVideo stream - "
                +
 describeStreamProcessorResult.getInput().getKinesisVideoStream().getArn());
        System.out.println("Output kinesisData stream - "
                +
 describeStreamProcessorResult.getOutput().getKinesisDataStream().getArn());
        System.out.println("RoleArn - " + describeStreamProcessorResult.getRoleArn());
        System.out.println(
                "CollectionId - " +
 describeStreamProcessorResult.getSettings().getFaceSearch().getCollectionId());
        System.out.println("Status - " + describeStreamProcessorResult.getStatus());
        System.out.println("Status message - " +
 describeStreamProcessorResult.getStatusMessage());
        System.out.println("Creation timestamp - " +
 describeStreamProcessorResult.getCreationTimestamp());
        System.out.println("Last update timestamp - " +
 describeStreamProcessorResult.getLastUpdateTimestamp());
    }

    public void listStreamProcessors() {
        ListStreamProcessorsResult listStreamProcessorsResult =
                rekognitionClient.listStreamProcessors(new
 ListStreamProcessorsRequest().withMaxResults(100));

        //List all stream processors (and state) returned from Rekognition
        for (StreamProcessor streamProcessor :
 listStreamProcessorsResult.getStreamProcessors()) {
            System.out.println("StreamProcessor name - " + streamProcessor.getName());
            System.out.println("Status - " + streamProcessor.getStatus());
        }
    }
}
```

將影片串流至 Amazon Rekognition Video

若要將影片串流至 Amazon Rekognition Video，您可以使用 Amazon Kinesis Video Streams 開發套件來建立及使用 Kinesis video stream。`PutMedia` 操作會將影片資料「片段」寫入 Amazon Rekognition

Video 所取用的 Kinesis 影片串流。每個影片資料片段的長度通常是 2–10 秒，並包含一連串獨立的影片影格。Amazon Rekognition Video 支援 H.264 編碼影片，包含三種影格類型 (I、B 與 P)。如需詳細資訊，請參閱 Inter Frame。片段中的第一個影格必須是 I 影格。I 影格可以與任何其他影格分開解碼。

當影片資料傳抵 Kinesis video stream 時，Kinesis Video Streams 會指派唯一號碼給片段。如需範例，請參閱 PutMedia API 範例。

讀取串流影片分析結果

您可以使用 Amazon Kinesis Data Streams Client Library 取用傳送至 Amazon Kinesis Data Streams 輸出串流的分析結果。如需詳細資訊，請參閱從 Kinesis 資料串流讀取資料。Amazon Rekognition Video 會將每個已分析影格的 JSON 影格記錄放入 Kinesis 輸出串流。Amazon Rekognition Video 不會分析透過 Kinesis video stream 傳遞給它的每個影格。

傳送至 Kinesis data stream 的影格記錄包含有關影格所在之 Kinesis video stream 片段、影格在片段中的位置，以及影格中已辨識之臉部的資訊。它也包含串流處理器的狀態資訊。如需詳細資訊，請參閱 參考：Kinesis 臉部辨識記錄 (p. 87)。

Amazon Rekognition Video 會將 Amazon Rekognition Video 分析資訊串流至 Kinesis data stream。以下是單一記錄的 JSON 範例。


```
{
  "InputInformation": {
    "KinesisVideo": {
      "StreamArn": "arn:aws:kinesisvideo:us-west-2:nnnnnnnnnnnn:stream/stream-name",
      "FragmentNumber": "91343852333289682796718532614445757584843717598",
      "ServerTimestamp": 1510552593.455,
      "ProducerTimestamp": 1510552593.193,
      "FrameOffsetInSeconds": 2
    }
  },
  "StreamProcessorInformation": {
    "Status": "RUNNING"
  },
  "FaceSearchResponse": [
    {
      "DetectedFace": {
        "BoundingBox": {
          "Height": 0.075,
          "Width": 0.05625,
          "Left": 0.428125,
          "Top": 0.40833333
        },
        "Confidence": 99.975174,
        "Landmarks": [
          {
            "X": 0.4452057,
            "Y": 0.4395594,
            "Type": "eyeLeft"
          },
          {
            "X": 0.46340984,
            "Y": 0.43744427,
            "Type": "eyeRight"
          },
          {
            "X": 0.45960626,
            "Y": 0.4526856,
            "Type": "nose"
          },
          {
            "X": 0.44958648,
```

```
            "Y": 0.4696949,
            "Type": "mouthLeft"
          },
          {
            "X": 0.46409217,
            "Y": 0.46704912,
            "Type": "mouthRight"
          }
        ],
        "Pose": {
          "Pitch": 2.9691637,
          "Roll": -6.8904796,
          "Yaw": 23.84388
        },
        "Quality": {
          "Brightness": 40.592964,
          "Sharpness": 96.09616
        }
      },
      "MatchedFaces": [
        {
          "Similarity": 88.863960,
          "Face": {
            "BoundingBox": {
              "Height": 0.557692,
              "Width": 0.749838,
              "Left": 0.103426,
              "Top": 0.206731
            },
            "FaceId": "ed1b560f-d6af-5158-989a-ff586c931545",
            "Confidence": 99.999201,
            "ImageId": "70e09693-2114-57e1-807c-50b6d61fa4dc",
            "ExternalImageId": "matchedImage.jpeg"
          }
        }
      ]
    }
  ]
}
```


在 JSON 範例中，請注意下列事項：

- InputInformation – 用來將影片串流至 Amazon Rekognition Video 之 Kinesis video stream 的資訊。如需詳細資訊，請參閱 InputInformation (p. 89)。
- StreamProcessorInformation–Amazon Rekognition Video 串流處理器的狀態資訊。唯一可用於 `Status` 欄位的值是「RUNNING」。如需詳細資訊，請參閱 StreamProcessorInformation (p. 89)。
- FaceSearchResponse – 包含串流影片中與輸入集合中臉部相符之臉部的資訊。FaceSearchResponse (p. 89) 包含 DetectedFace (p. 90) 物件，是在用於分析的視訊影格中偵測到的臉部。對於每個偵測到的臉部，陣列 `MatchedFaces` 皆包含在輸入集合中找到之相符臉部物件的陣列 (MatchedFace (p. 90))，以及相似度分數。

將 Kinesis 影片串流對應至 Kinesis 資料串流

您可能會想要將 Kinesis video stream 影格對應至傳送到 Kinesis data stream 的已分析影格。例如，在串流影片顯示期間，您可能想要在已辨識的人臉周圍顯示方塊。週框方塊座標會做為 Kinesis 臉部辨識記錄的一部分傳送至 Kinesis data stream。若要正確顯示週框方塊，您需要將隨 Kinesis 臉部辨識記錄傳送的時間資訊，與來源 Kinesis video stream 中相應的影格相對應。

您用來將 Kinesis video stream 對應至 Kinesis data stream 的技術取決於您要串流即時媒體 (例如即時串流影片)，還是要串流封存媒體 (例如已存放影片)。

串流即時媒體時的對應

將 Kinesis video stream 影格對應至 Kinesis data stream 影格

1. 將 PutMedia 操作的輸入參數 `FragmentTimeCodeType` 設定為 `RELATIVE`。
2. 呼叫 `PutMedia`，將即時媒體傳遞至 Kinesis video stream。
3. 當您從 Kinesis data stream 收到 Kinesis 臉部辨識記錄時，請存放 KinesisVideo (p. 89) 欄位中 `ProducerTimestamp` 與 `FrameOffsetInSeconds` 的值。
4. 將 `ProducerTimestamp` 與 `FrameOffsetInSeconds` 欄位值相加，藉此計算對應至 Kinesis video stream 影格的時間戳記。

串流封存媒體時的對應

將 Kinesis video stream 影格對應至 Kinesis data stream 影格

1. 呼叫 PutMedia，將封存媒體傳遞至 Kinesis video stream。
2. 當您從 `PutMedia` 操作回應收到 `Acknowledgement` 物件時，請存放 Payload 欄位中的 `FragmentNumber` 欄位值。`FragmentNumber` 是 MKV 叢集的片段號碼。
3. 當您從 Kinesis data stream 收到 Kinesis 臉部辨識記錄時，請存放 KinesisVideo (p. 89) 欄位中的 `FrameOffsetInSeconds` 欄位值。
4. 使用您在步驟 2 與 3 中存放的 `FrameOffsetInSeconds` 與 `FragmentNumber` 值來計算對應。`FrameOffsetInSeconds` 是具有特定 `FragmentNumber` 之片段的位移，其會傳送至 Amazon Kinesis data stream。如需取得指定片段號碼之影片影格的詳細資訊，請參閱 Amazon Kinesis Video Streams 的封存媒體。

參考：Kinesis 臉部辨識記錄

Amazon Rekognition Video 可辨識串流影片中的臉部。對於每個已分析的影格，Amazon Rekognition Video 都會將 JSON 影格記錄輸出至 Kinesis 資料串流。Amazon Rekognition Video 不會分析透過 Kinesis video stream 傳遞給它的每個影格。

JSON 影格記錄包含輸入與輸出串流的資訊、串流處理器的狀態，以及已分析影格中所辨識之臉部的資訊。本節包含 JSON 影格記錄的參考資訊。

以下是 Kinesis data stream 記錄的 JSON 語法。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

> **Note**
>
> Amazon Rekognition Video API 的運作原理是將輸入流中的臉部與一組臉部進行比較，並傳回最接近所找到的符合項目以及相似性分數。

```
{
    "InputInformation": {
        "KinesisVideo": {
            "StreamArn": "string",
            "FragmentNumber": "string",
            "ProducerTimestamp": number,
            "ServerTimestamp": number,
            "FrameOffsetInSeconds": number
        }
    },
    "StreamProcessorInformation": {
        "Status": "RUNNING"
    },
    "FaceSearchResponse": [
        {
```

```
            "DetectedFace": {
                "BoundingBox": {
                    "Width": number,
                    "Top": number,
                    "Height": number,
                    "Left": number
                },
                "Confidence": 23,
                "Landmarks": [
                    {
                        "Type": "string",
                        "X": number,
                        "Y": number
                    }
                ],
                "Pose": {
                    "Pitch": number,
                    "Roll": number,
                    "Yaw": number
                },
                "Quality": {
                    "Brightness": number,
                    "Sharpness": number
                }
            },
            "MatchedFaces": [
                {
                    "Similarity": number,
                    "Face": {
                        "BoundingBox": {
                            "Width": number,
                            "Top": number,
                            "Height": number,
                            "Left": number
                        },
                        "Confidence": number,
                        "ExternalImageId": "string",
                        "FaceId": "string",
                        "ImageId": "string"
                    }
                }
            ]
        }
    ]
}
```

JSON 記錄

JSON 記錄包含 Amazon Rekognition Video 所處理之影格的資訊。此記錄包括串流影片的資訊、已分析影格的狀態，以及影格中所辨識之臉部的資訊。

InputInformation

用來將影片串流至 Amazon Rekognition Video 之 Kinesis video stream 的資訊。

類型：InputInformation (p. 89) 物件

StreamProcessorInformation

Amazon Rekognition Video 串流處理器的資訊。這包括串流處理器目前狀態的狀態資訊。

類型：StreamProcessorInformation (p. 89) 物件

FaceSearchResponse

關於串流影片影格內偵測到的臉部與輸入集合中之相符臉部的資訊。

類型：FaceSearchResponse (p. 89) 物件陣列

InputInformation

Amazon Rekognition Video 使用之來源影片串流的資訊。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

KinesisVideo

類型：KinesisVideo (p. 89) 物件

KinesisVideo

將來源影片串流至 Amazon Rekognition Video 之 Kinesis video stream 的資訊。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

StreamArn

Kinesis video stream 的 Amazon Resource Name (ARN)。

類型：字串

FragmentNumber

包含此記錄代表之影格的串流影片片段。

類型：字串

ProducerTimestamp

片段的生產者端 Unix 時間戳記。如需詳細資訊，請參閱 PutMedia。

類型：數字

ServerTimestamp

片段的伺服器端 Unix 時間戳記。如需詳細資訊，請參閱 PutMedia。

類型：數字

FrameOffsetInSeconds

片段內影格的位移 (以秒為單位)。

類型：數字

StreamProcessorInformation

串流處理器的狀態資訊。

狀態

串流處理器目前的狀態。唯一可能的值為「RUNNING」。

類型：字串

FaceSearchResponse

關於串流影片影格內偵測到的臉部以及輸入集合中與偵測到的臉部相符之臉部的資訊。您可以在 CreateStreamProcessor (p. 289) 的呼叫中指定集合。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

DetectedFace

在用於分析的視訊影格中偵測到的臉部之臉部詳細資訊。

類型：DetectedFace (p. 90) 物件

MatchedFaces

集合中的臉部之臉部詳細資訊陣列，符合於 `DetectedFace` 中偵測到的臉部。

類型：MatchedFace (p. 90) 物件陣列

DetectedFace

串流影片影格中偵測到的臉部之資訊。輸入集合中相符的臉部可於 MatchedFace (p. 90) 物件欄位中找到。

BoundingBox

在已分析影片影格中偵測到之臉部的週框方塊座標。此 BoundingBox 物件與用於影像分析的 BoundingBox 物件具有同一個屬性。

類型：BoundingBox (p. 440) 物件

信賴度

Amazon Rekognition Video 對偵測到的臉部確實是臉部的可信度 (1-100)。1 是最低可信度，100 則是最高可信度。

類型：數字

特徵點

臉部特徵點的陣列。

類型：Landmark (p. 479) 物件陣列

姿態

表示由俯仰、側傾與偏轉所決定的臉部姿態。

類型：Pose (p. 490) 物件

品質

識別臉部影像的亮度與銳利度。

類型：ImageQuality (p. 473) 物件

MatchedFace

用於分析的視訊影格中偵測到的相符臉部之資訊。

臉部

輸入集合內與 DetectedFace (p. 90) 物件中臉部相符之臉部的臉部配對資訊。

類型：Face (p. 457) 物件

相似度

臉部匹配的可信度 (1-100)。1 是最低可信度，100 則是最高可信度。

類型：數字

串流影片故障診斷

此主題提供使用 Amazon Rekognition Video 搭配串流影片的故障診斷資訊。

主題

- 我不知道我的串流處理器是否已成功建立 (p. 91)
- 我不知道是否已正確設定我的串流處理器 (p. 91)
- 我的串流處理器未傳回結果 (p. 92)
- 我的串流處理器狀態是 FAILED (p. 93)
- 我的串流處理器未傳回預期的結果 (p. 95)

我不知道我的串流處理器是否已成功建立

使用以下 AWS CLI 命令取得串流處理器清單及其目前狀態。

```
aws rekognition list-stream-processors
```

您可以使用以下 AWS CLI 命令取得其他詳細資訊。將 `stream-processor-name` 取代為所需串流處理器的名稱。

```
aws rekognition describe-stream-processor --name stream-processor-name
```

我不知道是否已正確設定我的串流處理器

如果您的程式碼不會輸出 Amazon Rekognition Video 的分析結果，則您的串流處理器可能未正確設定。執行以下項目確認您的串流處理器已正確設定並能夠產生結果。

若要判斷您的解決方案是否正確設定

1. 執行以下命令確認您的串流處理器處於執行狀態。將 `stream-processor-name` 變更為您的串流處理器名稱。若 `Status` 的值為 `RUNNING`，則串流處理器正在執行。若狀態為 `RUNNING` 而您未取得結果，請參閱我的串流處理器未傳回結果 (p. 92)。若狀態為 `FAILED`，請參閱我的串流處理器狀態是 FAILED (p. 93)。

   ```
   aws rekognition describe-stream-processor --name stream-processor-name
   ```

2. 若您的串流處理器正在執行，請執行以下 Bash 或 PowerShell 命令從輸出 Kinesis 資料串流讀取資料。

   Bash

   ```
   SHARD_ITERATOR=$(aws kinesis get-shard-iterator --shard-id shardId-000000000000
    --shard-iterator-type TRIM_HORIZON --stream-name kinesis-data-stream-name --query
    'ShardIterator')
                           aws kinesis get-records --shard-iterator $SHARD_ITERATOR
   ```

   PowerShell

   ```
   aws kinesis get-records --shard-iterator ((aws kinesis get-shard-iterator --shard-id
    shardId-000000000000 --shard-iterator-type TRIM_HORIZON --stream-name kinesis-data-
   stream-name).split('"')[4])
   ```

3. 使用 Base64 Decode 網站上的解碼工具將輸出解碼為人類可讀的字串。如需詳細資訊，請參閱步驟 3：取得記錄。
4. 如果命令可作用且您在 Kinesis 資料串流中看到臉部偵測結果，則您的解決方案已正確設定。如果命令失敗，請查看其他故障診斷建議並參閱提供 Amazon Rekognition Video 對 Kinesis 串流的存取 (p. 77)。

或者，您可以使用「kinesis-process-record」AWS Lambda 藍圖將 Kinesis 資料串流的訊息記錄至 CloudWatch 以用於持續視覺化。這會為 AWS Lambda 和 CloudWatch 產生額外的成本。

我的串流處理器未傳回結果

可能有幾個原因讓您的串流處理器無法傳回結果。

原因 1：您的串流處理器未正確設定

您的串流處理器可能未正確設定。如需詳細資訊，請參閱 我不知道是否已正確設定我的串流處理器 (p. 91)。

原因 2：您的串流處理器未處於 RUNNING 狀態

故障診斷串流處理器的狀態

1. 使用以下 AWS CLI 命令檢查串流處理器的狀態。

```
aws rekognition describe-stream-processor --name stream-processor-name
```

2. 若 `Status` 的值為 `STOPPED`，請使用以下命令啟動串流處理器：

```
aws rekognition start-stream-processor --name stream-processor-name
```

3. 若 `Status` 的值為 `FAILED`，請參閱 我的串流處理器狀態是 FAILED (p. 93)。
4. 若 `Status` 的值為 `STARTING`，請等待 2 分鐘並重複步驟 1 以檢查狀態。如果狀態的值仍為 `STARTING`，請執行以下項目：
   a. 使用以下命令刪除串流處理器。

```
aws rekognition delete-stream-processor --name stream-processor-name
```

   b. 使用相同組態建立新的串流處理器。如需詳細資訊，請參閱 處理串流影片 (p. 76)。
   c. 如果您仍然無法解決，請聯絡 AWS 支援。
5. 若 `Status` 的值為 `RUNNING`，請參閱 原因 3：在 Kinesis video stream 中沒有作用中的資料 (p. 92)。

原因 3：在 Kinesis video stream 中沒有作用中的資料

檢查 Kinesis video stream 中是否有作用中的資料

1. Sign in to the AWS Management Console, and open the Amazon Kinesis Video Streams console at https://console.aws.amazon.com/kinesisvideo/.
2. 選取做為 Amazon Rekognition 串流處理器之輸入的 Kinesis video stream。
3. 如果預覽顯示為 No data on stream (無串流資料)，則在輸入串流中沒有可供 Amazon Rekognition Video 處理的資料。

如需使用 Kinesis Video Streams 產生影片的資訊，請參閱 Kinesis Video Streams Producer Libraries。

我的串流處理器狀態是 FAILED

您可以使用下列 AWS CLI 命令檢查串流處理器的狀態。

```
aws rekognition describe-stream-processor --name stream-processor-name
```

如果狀態的值為 FAILED，請查看以下錯誤訊息的故障診斷資訊。

錯誤：「角色存取遭拒」

由串流處理器使用的 IAM 角色不存在或 Amazon Rekognition Video 未具有擔任角色的許可。

故障診斷 IAM 角色的存取

1. Sign in to the AWS Management Console and open the IAM console at https://console.aws.amazon.com/iam/.
2. 在左側導覽窗格中，請選擇 Roles (角色) 並確認角色存在。
3. 若角色存在，請確認角色具有 AmazonRekognitionServiceRole 許可政策。
4. 若角色不存在或未具有正確的許可，請參閱提供 Amazon Rekognition Video 對 Kinesis 串流的存取 (p. 77)。
5. 使用以下 AWS CLI 命令啟動串流處理器。

```
aws rekognition start-stream-processor --name stream-processor-name
```

錯誤：「存取 Kinesis Video 遭拒或存取 Kinesis Data 遭拒。」

角色無法存取 Kinesis Video Streams API 操作 `GetMedia` 和 `GetDataEndpoint`。角色可能也無法存取 Kinesis Data Streams API 操作 `PutRecord` 和 `PutRecords`。

故障診斷 API 許可

1. Sign in to the AWS Management Console and open the IAM console at https://console.aws.amazon.com/iam/.
2. 開啟角色並確認角色已連接以下許可政策。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "kinesis:PutRecord",
                "kinesis:PutRecords"
            ],
            "Resource": "data-arn"
        },
        {
            "Effect": "Allow",
            "Action": [
                "kinesisvideo:GetDataEndpoint",
                "kinesisvideo:GetMedia"
            ],
            "Resource": "video-arn"
        }
    ]
}
```

3. 如果遺失任何許可，請更新政策。如需更多詳細資訊，請參閱 提供 Amazon Rekognition Video 對 Kinesis 串流的存取 (p. 77)。

錯誤：「串流 *input-video-stream-name* 不存在」

Kinesis video stream 對串流處理器的輸入不存在或未正確設定。

故障診斷 Kinesis video stream

1. 使用以下命令確認串流存在。

```
aws kinesisvideo list-streams
```

2. 如果串流存在，請檢查下列各項。

   - Amazon Resource Name (ARN) 與串流處理器的輸入串流 ARN 相同。
   - Kinesis video stream 與串流處理器位於相同區域。

   如果串流處理器未正確設定，請使用以下 AWS CLI 命令將其刪除。

```
aws rekognition delete-stream-processor --name stream-processor-name
```

3. 以預期的 Kinesis 視訊串流建立新的串流處理器。如需更多詳細資訊，請參閱 建立 Amazon Rekognition Video 串流處理器 (p. 80)。

錯誤：「找不到集合」

串流處理器用於比對臉部的 Amazon Rekognition 集合不存在或使用了錯誤的集合。

確認集合

1. 使用以下 AWS CLI 命令判斷所需集合是否存在。將 `region` 變更為您執行串流處理器的 AWS 區域。

```
aws rekognition list-collections --region region
```

   如果所需集合不存在，請建立新的集合並新增臉部資訊。如需詳細資訊，請參閱 ??? (p. 147)。
2. 如果您呼叫 the section called “CreateStreamProcessor” (p. 289)，請確認 `CollectionId` 輸入參數的值正確。
3. 使用以下 AWS CLI 命令啟動串流處理器。

```
aws rekognition start-stream-processor --name stream-processor-name
```

錯誤：「找不到帳戶 *account-id* 下的串流 *output-kinesis-data-stream-name*」

串流處理器使用的輸出 Kinesis 資料串流不存在於您的 AWS 帳戶，或與串流處理器位於不相同的 AWS 區域。

故障診斷 Kinesis 資料串流

1. 使用以下 AWS CLI 命令判斷 Kinesis 資料串流是否存在。將 `region` 變更為您使用串流處理器的 AWS 區域。

```
aws kinesis list-streams --region region
```

2. 若 Kinesis 資料串流存在，請確認 Kinesis 資料串流名稱與串流處理器使用的輸出串流名稱相同。
3. 若 Kinesis 資料串流不存在，可能是存在於另一個 AWS 區域。Kinesis 資料串流必須與串流處理器位於相同區域。
4. 如有必要，請建立新的 Kinesis 資料串流。

   a. 使用與串流處理器所用名稱相同的名稱建立 Kinesis 資料串流。如需詳細資訊，請參閱 步驟 1：建立資料串流。

   b. 使用以下 AWS CLI 命令啟動串流處理器。

```
aws rekognition start-stream-processor --name stream-processor-name
```

我的串流處理器未傳回預期的結果

如果您的串流處理器未傳回預期的臉部比對結果，請使用以下資訊。

- 在集合中搜尋臉部 (p. 147)
- 相機設定的建議 (串流影片) (p. 105)

錯誤處理

本節說明執行時間錯誤和其處理方式。它也會說明 Amazon Rekognition 特有的錯誤訊息和程式碼。

主題

- 錯誤元件 (p. 95)
- 錯誤訊息和錯誤碼 (p. 96)
- 應用程式中的錯誤處理 (p. 99)

錯誤元件

當您的程式傳送請求時，Amazon Rekognition 會嘗試進行處理。如果請求成功，則 Amazon Rekognition 會傳回 HTTP 成功狀態碼 (`200 OK`) 以及所請求操作的結果。

如果請求不成功，Amazon Rekognition 會傳回錯誤。每個錯誤都會有三個元件：

- HTTP 狀態碼 (例如 `400`)。
- 例外狀況名稱 (例如 `InvalidS3ObjectException`)。
- 錯誤訊息 (例如 `Unable to get object metadata from S3. Check object key, region and/or access permissions.`)。

AWS 開發套件會負責將錯誤傳播至應用程式，讓您可以採取適當的動作。例如，在 Java 程式中，您可以撰寫 `try-catch` 邏輯來處理 `ResourceNotFoundException`。

如果您不是使用 AWS 開發套件，則需要剖析來自 Amazon Rekognition 之下層回應的內容。以下是這類回應的範例：

```
HTTP/1.1 400 Bad Request
Content-Type: application/x-amz-json-1.1
```

```
Date: Sat, 25 May 2019 00:28:25 GMT
x-amzn-RequestId: 03507c9b-7e84-11e9-9ad1-854a4567eb71
Content-Length: 222
Connection: keep-alive

{"__type":"InvalidS3ObjectException","Code":"InvalidS3ObjectException","Logref":"5022229e-7e48-11e9-9ad
 to get object metadata from S3. Check object key, region and/or access permissions."}
```

錯誤訊息和錯誤碼

以下是 Amazon Rekognition 所傳回並依 HTTP 狀態碼進行群組的例外狀況清單。確定要重試嗎？為是，則可以重新提交相同的請求。確定要重試嗎？為否，則需要先修正用戶端上的問題，再提交新的請求。

HTTP 狀態碼 400

HTTP 400 狀態碼表示您的請求有問題。一些問題範例包括驗證失敗、必要參數遺失，或超過資料表的佈建輸送量。您必須先修正應用程式中的問題，再重新提交請求。

AccessDeniedException

訊息：呼叫 <Operation> 操作時發生錯誤 (AccessDeniedException)：使用者：<User ARN> 未獲授權於資源：<Resource ARN> 上執行：<Operation>。

您未獲授權執行動作。使用已獲授權之使用者或 IAM 角色的 Amazon Resource Name (ARN) 來執行操作。

OK to retry? (確定要重試嗎？) 否

GroupFacesInProgressException

訊息：無法排程 GroupFaces 任務。此集合有現有的群組表面任務。

請在現有任務完成後重試操作。

OK to retry? (確定要重試嗎？) 否

IdempotentParameterMismatchException

訊息：ClientRequestToken：您提供的 <Token> 已在使用中。

ClientRequestToken 輸入參數重複用於一個操作，但至少有一個其他輸入參數不同於先前對操作的呼叫。

OK to retry? (確定要重試嗎？) 否

ImageTooLargeException

訊息：影像大小太大。

輸入影像大小超過允許的限制。如需更多詳細資訊，請參閱 Amazon Rekognition 中的限制 (p. 515)。

OK to retry? (確定要重試嗎？) 否

InvalidImageFormatException

訊息：請求的影像格式無效。

不支援所提供的影像格式。請使用支援的影像格式 (.JPEG 和 .PNG)。如需更多詳細資訊，請參閱 Amazon Rekognition 中的限制 (p. 515)。

OK to retry? (確定要重試嗎？) 否

InvalidPaginationTokenException

訊息

- 無效的字符
- 無效的分頁符記

請求中的分頁符記無效。符記可能已過期。

OK to retry? (確定要重試嗎？) 否

InvalidParameterException

訊息：請求有無效的參數。

輸入參數違反限制。請驗證您的參數，然後再次呼叫 API 操作。

OK to retry? (確定要重試嗎？) 否

InvalidS3ObjectException

訊息：

- 請求有無效的 S3 物件。
- 無法從 S3 取得物件中繼資料。檢查物件金鑰、區域和/或存取許可。

Amazon Rekognition 無法存取請求中指定的 S3 物件。如需詳細資訊，請參閱設定對 S3 的存取：AWS S3 管理存取。如需故障診斷資訊，請參閱故障診斷 Amazon S3。

OK to retry? (確定要重試嗎？) 否

LimitExceededException

訊息：

- 超出帳戶的串流處理器限制，限制 - <目前限制>。
- <開啟任務數> 開啟任務，適用於使用者 <使用者 ARN> 最大限制：<最大限制>

超出 Amazon Rekognition 服務限制。例如，如果您同時開始太多 Amazon Rekognition Video 任務，啟動操作的呼叫 (例如 `StartLabelDetection`) 將引發 `LimitExceededException` 例外狀況 (HTTP 狀態碼：400)，直到同時執行任務的數量低於 Amazon Rekognition 服務限制。

OK to retry? (確定要重試嗎？) 否

ProvisionedThroughputExceededException

訊息：

- 超過佈建率。
- 超過 S3 下載限制。

請求數超過您的傳輸量限制。如需詳細資訊，請參閱 Amazon Rekognition 服務限制。

若要請求提高限制，請聯絡 AWS Support。

OK to retry? (確定要重試嗎？) 是

ResourceAlreadyExistsException

訊息：集合 ID：<集合 ID> 已存在。

已存在使用指定 ID 的集合。

OK to retry? (確定要重試嗎？) 否

ResourceInUseException

訊息：
- 串流處理器名稱已在使用中。
- 指定的資源已在使用中。
- 處理器無法用於停止串流。
- 無法刪除串流處理器。

當資源可用時重試。

OK to retry? (確定要重試嗎？) 否

ResourceNotFoundException

訊息：根據 API 呼叫的不同訊息。

指定的資源不存在。

OK to retry? (確定要重試嗎？) 否

ThrottlingException

訊息：減慢；請求率突然增加。

您的請求率增加過快。請減慢並逐漸增加您的請求率。我們建議您以指數方式退避並再試一次。依預設，AWS 開發套件會使用自動重試邏輯和指數退避。如需詳細資訊，請參閱在 AWS 中錯誤重試與指數退避和指數退避和抖動。

OK to retry? (確定要重試嗎？) 是

VideoTooLargeException

訊息：影片大小 (位元組)：<影片大小> 超過最大限制：<最大大小> 位元組。

所提供媒體的檔案大小或持續時間太大。如需更多詳細資訊，請參閱 Amazon Rekognition 中的限制 (p. 515)。

OK to retry? (確定要重試嗎？) 否

HTTP 狀態碼 5xx

HTTP 5xx 狀態碼指出 AWS 必須解決的問題。這可能是暫時性錯誤。如果確實如此，您可以重試請求直到成功。否則，請前往 AWS 服務運作狀態儀表板，確認服務是否發生任何操作問題。

InternalServerError (HTTP 500)

訊息：內部伺服器錯誤

Amazon Rekognition 發生服務問題。請再次嘗試呼叫。您應以指數方式退避，然後再試一次。依預設，AWS 開發套件會使用自動重試邏輯和指數退避。如需詳細資訊，請參閱在 AWS 中錯誤重試與指數退避和指數退避和抖動。

OK to retry? (確定要重試嗎？) 是

ThrottlingException (HTTP 500)

訊息：服務無法使用

Amazon Rekognition 暫時無法處理請求。請再次嘗試呼叫。我們建議您以指數方式退避並再試一次。依預設，AWS 開發套件會使用自動重試邏輯和指數退避。如需詳細資訊，請參閱在 AWS 中錯誤重試與指數退避和指數退避和抖動。

OK to retry? (確定要重試嗎？) 是

應用程式中的錯誤處理

若要讓您的應用程式順暢執行，您需要新增邏輯來截獲並回應錯誤。一般方式包含使用 `try-catch` 區塊或 `if-then` 陳述式。

AWS 開發套件會執行自己的重試和錯誤檢查。如果您在使用其中一個 AWS 開發套件時發生錯誤，則錯誤碼和說明可以協助您對其進行故障診斷。

您應該也會在回應中看到 `Request ID`。如果您需要與 AWS Support 合作來診斷問題，則 `Request ID` 可能十分有用。

以下 Java 程式碼片段嘗試偵測影像中的物件，並執行早期錯誤處理。(在此情況下，它只會通知請求失敗的使用者。)

```
try {
    DetectLabelsResult result = rekognitionClient.detectLabels(request);
    List <Label> labels = result.getLabels();

    System.out.println("Detected labels for " + photo);
    for (Label label: labels) {
        System.out.println(label.getName() + ": " + label.getConfidence().toString());
    }
}
catch(AmazonRekognitionException e) {
    System.err.println("Could not complete operation");
    System.err.println("Error Message:  " + e.getMessage());
    System.err.println("HTTP Status:    " + e.getStatusCode());
    System.err.println("AWS Error Code: " + e.getErrorCode());
    System.err.println("Error Type:     " + e.getErrorType());
    System.err.println("Request ID:     " + e.getRequestId());
}
catch (AmazonClientException ace) {
    System.err.println("Internal error occurred communicating with Rekognition");
    System.out.println("Error Message:  " + ace.getMessage());
}
```

在此程式碼片段中，`try-catch` 建構會處理兩個不同類型的例外狀況：

- `AmazonRekognitionException` – 如果用戶端請求正確地傳輸至 Amazon Rekognition，但 Amazon Rekognition 無法處理請求並改為傳回錯誤回應，將會發生此例外。

- `AmazonClientException` – 如果用戶端無法從服務取得回應，或者如果用戶端無法剖析來自服務的回應，將會發生此例外。

使用 Amazon Rekognition 作為 FedRAMP 授權服務

AWS FedRAMP 合規計劃將 Amazon Rekognition 納入為 FedRAMP 授權服務。如果您是聯邦或商業客戶，您可以在具有最高為中度影響層級資料的 AWS 美國東部和美國西部區域中，使用此服務來處理和存放機密工作負載。您可以在具有最高為高度影響層級資料的 AWS GovCloud (US) 區域的授權邊界中，使用此服務來處理機密工作負載。如需 FedRAMP 合規的詳細資訊，請參閱 AWS FedRAMP 合規。

若要符合 FedRAMP 規範，您可以使用聯邦資訊處理標準 (FIPS) 端點。當您使用敏感資訊時，這可讓您存取 FIPS 140-2 驗證的密碼編譯模組。如需 FIPS 端點的詳細資訊，請參閱 FIPS 140-2 概觀。

您可以使用 AWS Command Line Interface (AWS CLI) 或其中一個 AWS 開發套件來指定 Amazon Rekognition 使用的端點。

如需可以與 Amazon Rekognition 搭配使用的端點，請參閱 Amazon Rekognition 區域和端點。

以下是《Amazon Rekognition 開發人員指南》中列出集合 (p. 153)主題的範例。它們經過修改，以指定用來存取 Amazon Rekognition 的區域和 FIPS 端點。

Java

對於 Java，在建構 Amazon Rekognition 用戶端時使用 `withEndpointConfiguration` 方法。此範例顯示您在美國東部 (維吉尼亞北部) 區域使用 FIPS 端點的集合：

```
//Copyright 2019 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;

import java.util.List;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.ListCollectionsRequest;
import com.amazonaws.services.rekognition.model.ListCollectionsResult;

public class ListCollections {

   public static void main(String[] args) throws Exception {


      AmazonRekognition amazonRekognition = AmazonRekognitionClientBuilder.standard()
         .withEndpointConfiguration(new
 AwsClientBuilder.EndpointConfiguration("https://rekognition-fips.us-
east-1.amazonaws.com","us-east-1"))
         .build();


      System.out.println("Listing collections");
      int limit = 10;
      ListCollectionsResult listCollectionsResult = null;
      String paginationToken = null;
      do {
         if (listCollectionsResult != null) {
            paginationToken = listCollectionsResult.getNextToken();
         }
         ListCollectionsRequest listCollectionsRequest = new ListCollectionsRequest()
                 .withMaxResults(limit)
```

```
                .withNextToken(paginationToken);

 listCollectionsResult=amazonRekognition.listCollections(listCollectionsRequest);

         List < String > collectionIds = listCollectionsResult.getCollectionIds();
         for (String resultId: collectionIds) {
            System.out.println(resultId);
         }
      } while (listCollectionsResult != null && listCollectionsResult.getNextToken() !=
         null);

   }
}
```

AWS CLI

對於 AWS CLI，使用 `--endpoint-url` 引數來指定端點，透過它存取 Amazon Rekognition。下列範例顯示您在美國東部 (俄亥俄) 區域使用 FIPS 端點的集合：

```
aws rekognition list-collections --endpoint-url https://rekognition-fips.us-
east-2.amazonaws.com --region us-east-2
```

Python

對於 Python，在 boto3.client 函數中使用 `endpoint_url` 引數。將其設定為您要指定的端點。此範例顯示您在美國西部 (奧勒岡) 區域使用 FIPS 端點的集合：

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def list_collections():

    max_results=2

    client=boto3.client('rekognition', endpoint_url='https://rekognition-fips.us-
west-2.amazonaws.com', region_name='us-west-2')

    #Display all the collections
    print('Displaying collections...')
    response=client.list_collections(MaxResults=max_results)
    collection_count=0
    done=False

    while done==False:
        collections=response['CollectionIds']

        for collection in collections:
            print (collection)
            collection_count+=1
        if 'NextToken' in response:
            nextToken=response['NextToken']

 response=client.list_collections(NextToken=nextToken,MaxResults=max_results)

        else:
            done=True

    return collection_count

def main():
```

```
    collection_count=list_collections()
    print("collections: " + str(collection_count))
if __name__ == "__main__":
    main()
```

感應器、輸入影像和影片的最佳實務

本節包含使用 Amazon Rekognition 的最佳實務資訊。

主題

- Amazon Rekognition Image 操作延遲 (p. 103)
- 臉部比較輸入影像的建議 (p. 103)
- 相機設定的建議 (影像和影片) (p. 104)
- 相機設定的建議 (已存放和串流影片) (p. 104)
- 相機設定的建議 (串流影片) (p. 105)

Amazon Rekognition Image 操作延遲

為確保 Amazon Rekognition Image 操作的延遲盡可能降到最低，請考量下列事項：

- 含有您影像的 Amazon S3 儲存貯體區域必須符合您用於 Amazon Rekognition Image API 操作的區域。
- 使用影像位元組呼叫 Amazon Rekognition Image 操作，會比將影像上傳至 Amazon S3 儲存貯體，然後在 Amazon Rekognition Image 操作中參考已上傳影像更快。如果您想要將影像上傳至 Amazon Rekognition Image 以進行近乎即時的處理，請考慮這個方法。例如，從 IP 相機上傳的影像或透過入口網站上傳的影像。
- 如果影像已在 Amazon S3 儲存貯體中，在 Amazon Rekognition Image 操作中參考它可能會比將影像位元組傳遞至操作更快。

臉部比較輸入影像的建議

臉部比較操作所用的模型，是設計成適用各式各樣的姿勢、臉部表情、年齡範圍、旋轉、燈光條件以及大小。為 CompareFaces (p. 273) 選擇參考相片，或使用 IndexFaces (p. 371) 將臉部新增至集合時，我們建議您使用下列準則。

- 使用臉部在建議角度範圍內的影像。臉部俯角應該低於 30 度，而臉部仰角則低於 45 度。任一方向的擺角應該低於 45 度。翻滾沒有限制。
- 使用雙眼睜開且可見雙眼的臉部影像。
- 使用 `IndexFaces` 建立集合時，請使用多個俯仰和偏擺不同的個人臉部影像 (在建議的角度範圍內)。我們建議一個人至少要有 5 個影像製成索引 — 正面、臉部向左偏擺 45 度或更少、臉部向右偏擺 45 度或更少、臉部俯角 30 度或更少，臉部仰角 30 度或更少。如果您想要追蹤這些臉部實例是否屬於同一個人，請在要製成索引的影像中只有一個臉部時，考慮使用外部影像 ID 屬性。例如，可在集合中利用外部影像 ID 追蹤 John Doe 的 5 個影像，例如 John_Doe_1.jpg、... John_Doe_5.jpg。
- 使用臉部沒有遭到隱蔽或緊密裁剪的影像。影像應包含肩膀以上的完整頭像。不應裁剪到臉部週框方塊。
- 避免擋住臉部的物品，如頭帶和口罩。
- 使用臉部佔據影像中大部分比例的影像。臉部佔據影像更大部分的影像比對時的準確性更高。

- 確保影像具有足夠大的解析度。Amazon Rekognition 可以識別小至 50 x 50 像素的臉部，而影像解析度高達 1920 x 1080。影像的解析度愈高，需要的最小臉部大小愈大。大於大小下限的臉部提供一組更準確的臉部比較結果。
- 使用彩色影像。
- 使用臉部正面打光的影像，而不要有陰影等各種暗影。
- 使用與背景充分對比的影像。高對比單色背景的效果很好。
- 對於需要高精確度的應用，使用中性臉部表情的影像，嘴巴閉合，略帶或沒有笑容。
- 使用明亮且銳利的影像。盡可能避免使用由於主體和相機移動而導致模糊的影像。DetectFaces (p. 320) 可用來判斷臉部的亮度和清晰度。
- 確保將最近的臉部影像製成索引。

相機設定的建議 (影像和影片)

除了臉部比較輸入影像的建議 (p. 103)之外，還有以下建議。

- 影像解析度 – 影像解析度沒有最低需求，只要臉部解析度為 50 x 50 像素，而影像解析度總計高達 1920 x 1080。影像的解析度愈高，需要的最小臉部大小愈大。

  Note

  上述建議是根據相機的原生解析度。從低解析度影像產生高解析度影像不會產生臉部搜尋所需的結果 (因為成像是透過影像放大而產生的)。
- 相機角度 – 測量相機角度的方式有三種 — 俯仰、翻滾和偏擺。
  - 俯仰 – 我們建議當相機朝下時臉部俯角低於 30 度，朝上時臉部仰角低於 45 度。
  - 翻滾 – 此參數沒有最低需求。Amazon Rekognition 可以處理任何翻滾量。
  - 偏擺 – 我們建議任一方向的擺角低於 45 度。

  相機沿著任何軸擷取的臉部角度是面對場景的相機角度和主體頭部在場景中的角度的結合。例如，如果相機向下 30 度，而人員頭部又向下 30 度，則相機看到的實際臉部俯角為 60 度。在此情況下，Amazon Rekognition 將無法辨識臉部。我們建議設定相機，以致相機角度根據下列假設：人們通常是以 30 度或更低的整體俯仰角 (臉部和相機的結合) 來看著相機。
- 相機變焦 – 建議的臉部解析度下限 50 x 50 像素應該驅動此相機設定。我們建議使用相機的變焦設定，以便所需臉部的解析度不低於 50 x 50 像素。
- 相機高度 – 建議的相機俯仰應該驅動此參數。

相機設定的建議 (已存放和串流影片)

除了相機設定的建議 (影像和影片) (p. 104)之外，還有以下建議。

- 編解碼器應該是 h.264 編碼。
- 建議的影格率為 30 fps。(它不得低於 5 fps。)
- 建議的編碼器位元速率為 3 Mbps。(它不得低於 1.5 Mbps。)

- 影格率與影格解析度 – 如果編碼器位元速率是一種限制，我們建議偏向使用更高的影格解析度，而非更高的影格率，以取得更好的臉部搜尋結果。這將確保 Amazon Rekognition 在配置的位元速率內取得最佳品質影格。不過，這有一個缺點。由於畫面播放速率較低，攝影機會遺失場景中的快速動作。對於特定的設定，了解這兩個參數之間的權衡是很重要的。例如，如果最大的可能位元速率為 1.5 Mbps，則相機可以 5 fps 擷取 1080p，或以 15 fps 擷取 720p。兩者之間的選擇取決於應用方式，只要符合建議的臉部解析度 50 x 50 像素即可。

相機設定的建議 (串流影片)

除了相機設定的建議 (已存放和串流影片) (p. 104)之外，還有以下建議。

串流應用程式的額外限制是網際網路頻寬。若為即時影片，Amazon Rekognition 只接受 Amazon Kinesis Video Streams 做為輸入。您應該了解編碼器位元速率與可用網路頻寬之間的相依性。可用頻寬應該至少支援相機用來編碼即時串流的相同位元速率。這可確保將透過 Amazon Kinesis Video Streams 轉送相機擷取的任何項目。如果可用頻寬低於編碼器位元速率，則 Amazon Kinesis Streams 就會根據網路頻寬捨棄位元。這會導致低視訊品質。

一般串流設定涉及將多個相機連接到轉送串流的網路集線器。在此情況下，頻寬應該容納來自所有連接到集線器之相機的串流累計總和。例如，如果集線器連接到 5 部以 1.5 Mbps 編碼的相機，則可用的網路頻寬應該至少 7.5 Mbps。若要確保沒有捨棄的封包，您應該考慮將網路頻寬保持高於 7.5 Mbps，以因應由於相機與集線器之間捨棄的連線而產生的抖動。實際值將取決於內部網路的可靠性。

偵測物件和場景

本節提供以Amazon Rekognition Image 和 Amazon Rekognition Video 偵測影像和影片內標籤的資訊。

標籤或標記是指影像中的一個物件、場景或概念，根據影像或視訊的內容決定。例如，一張照片中有一個人在熱帶海灘上，包含的標記可能就有「人」、「水」、「沙」、「棕櫚樹」、「泳裝」(物件)、「海灘」(場景)、「戶外」(概念)。Amazon Rekognition Video 也可以偵測活動，例如人在滑雪或騎單車。Amazon Rekognition Image 不會偵測影像內的活動。

Amazon Rekognition Image 和 Amazon Rekognition Video 可以傳回常見物件標籤的週框方塊，例如人員、汽車、家具、服飾或寵物。對於較不常用的物件標籤，系統不會傳回週框方塊資訊。您可以使用週框方塊，來找出物件在影像中的確切位置、已偵測到物件的計數實例，或使用週框方塊維度測量物件的大小。

Amazon Rekognition Image 和 Amazon Rekognition Video 會使用上階標籤的階層式分類法來分類標籤。例如，走在路上的人員可能會被偵測為行人。行人的父標籤為人員。在回應中會傳回這兩個標籤。系統會傳回所有上階標籤，而且指定的標籤包含一個清單，其中列出其父標籤和其他上階標籤。例如，祖父和曾祖父標籤 (如果它們存在的話)。您可以使用父標籤來建置相關標籤的群組，並允許在一或多個影像中查詢類似標籤。例如，查詢所有 車輛可能從某個影像傳回汽車，而從另一個影像傳回摩托車。

Amazon Rekognition Image 和 Amazon Rekognition Video 都會傳回用來在影像或已儲存影片中偵測標籤的標籤偵測模型版本。

例如，在以下影像中，Amazon Rekognition Image 能夠偵測人物、滑板、停好的車或其他資訊。Amazon Rekognition Image 也會傳回所偵測到人員的週框方塊，以及其他偵測到的物件，例如汽車和車輪。Amazon Rekognition Video 與 Amazon Rekognition Image 也可提供 Amazon Rekognition 對於每個偵測標籤的準確性之可信度分數，以百分比顯示。



主題

- 在影像中偵測標籤 (p. 107)
- 偵測影片中的標籤 (p. 116)
- 偵測自訂標籤 (p. 118)

在影像中偵測標籤

您可以使用 DetectLabels (p. 324) 操作，以偵測影像中的標籤。如需範例，請參閱 分析存放在 Amazon S3 儲存貯體中的影像 (p. 24)。

下列範例使用各種 AWS 開發套件和 AWS CLI 呼叫 `DetectLabels` 操作。如需有關 `DetectLabels` 操作回應的資訊，請參閱 DetectLabels 回應 (p. 112)。

偵測影像中的標籤

1. 如果您尚未：
   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 上傳包含一個或多個物件的影像—例如樹、房子和船—至您的 S3 儲存貯體。影像的格式必須是 .jpg 或 .png 格式。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
3. 使用下列範例來呼叫 `DetectLabels` 操作。

   Java

   此範例顯示一份在輸入影像中偵測到的標籤清單。將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
package com.amazonaws.samples;
import java.util.List;

import com.amazonaws.services.rekognition.model.BoundingBox;
import com.amazonaws.services.rekognition.model.DetectLabelsRequest;
import com.amazonaws.services.rekognition.model.DetectLabelsResult;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.Instance;
import com.amazonaws.services.rekognition.model.Label;
import com.amazonaws.services.rekognition.model.Parent;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;

public class DetectLabels {

    public static void main(String[] args) throws Exception {

        String photo = "photo";
        String bucket = "bucket";

        AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

        DetectLabelsRequest request = new DetectLabelsRequest()
```

```
                .withImage(new Image().withS3Object(new
 S3Object().withName(photo).withBucket(bucket)))
                .withMaxLabels(10).withMinConfidence(75F);

        try {
            DetectLabelsResult result = rekognitionClient.detectLabels(request);
            List<Label> labels = result.getLabels();

            System.out.println("Detected labels for " + photo + "\n");
            for (Label label : labels) {
                System.out.println("Label: " + label.getName());
                System.out.println("Confidence: " +
 label.getConfidence().toString() + "\n");

                List<Instance> instances = label.getInstances();
                System.out.println("Instances of " + label.getName());
                if (instances.isEmpty()) {
                    System.out.println("  " + "None");
                } else {
                    for (Instance instance : instances) {
                        System.out.println("  Confidence: " +
 instance.getConfidence().toString());
                        System.out.println("  Bounding box: " +
 instance.getBoundingBox().toString());
                    }
                }
                System.out.println("Parent labels for " + label.getName() + ":");
                List<Parent> parents = label.getParents();
                if (parents.isEmpty()) {
                    System.out.println("  None");
                } else {
                    for (Parent parent : parents) {
                        System.out.println("  " + parent.getName());
                    }
                }
                System.out.println("----------------------");
                System.out.println();

            }
        } catch (AmazonRekognitionException e) {
            e.printStackTrace();
        }
    }
}
```

AWS CLI

此範例顯示 `detect-labels` CLI 操作的 JSON 輸出。將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
aws rekognition detect-labels \
--image '{"S3Object":{"Bucket":"bucket","Name":"file"}}'
```

Python

此範例顯示在輸入影像中偵測到的標籤。在函數 `main` 中，將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
```

```
def detect_labels(photo, bucket):

    client=boto3.client('rekognition')

    response = client.detect_labels(Image={'S3Object':
{'Bucket':bucket,'Name':photo}},
        MaxLabels=10)

    print('Detected labels for ' + photo)
    print()
    for label in response['Labels']:
        print ("Label: " + label['Name'])
        print ("Confidence: " + str(label['Confidence']))
        print ("Instances:")
        for instance in label['Instances']:
            print ("  Bounding box")
            print ("    Top: " + str(instance['BoundingBox']['Top']))
            print ("    Left: " + str(instance['BoundingBox']['Left']))
            print ("    Width: " +  str(instance['BoundingBox']['Width']))
            print ("    Height: " +  str(instance['BoundingBox']['Height']))
            print ("  Confidence: " + str(instance['Confidence']))
            print()

        print ("Parents:")
        for parent in label['Parents']:
            print ("   " + parent['Name'])
        print ("----------")
        print ()
    return len(response['Labels'])


def main():
    photo=''
    bucket=''
    label_count=detect_labels(photo, bucket)
    print("Labels detected: " + str(label_count))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示一份在輸入影像中偵測到的標籤清單。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectLabels
{
    public static void Example()
    {
        String photo = "input.jpg";
        String bucket = "bucket";
```

```
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectLabelsRequest detectlabelsRequest = new DetectLabelsRequest()
        {
            Image = new Image()
            {
                S3Object = new S3Object()
                {
                    Name = photo,
                    Bucket = bucket
                },
            },
            MaxLabels = 10,
            MinConfidence = 75F
        };

        try
        {
            DetectLabelsResponse detectLabelsResponse =
 rekognitionClient.DetectLabels(detectlabelsRequest);
            Console.WriteLine("Detected labels for " + photo);
            foreach (Label label in detectLabelsResponse.Labels)
                Console.WriteLine("{0}: {1}", label.Name, label.Confidence);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

Ruby

此範例顯示一份在輸入影像中偵測到的標籤清單。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

   # Add to your Gemfile
   # gem 'aws-sdk-rekognition'
   require 'aws-sdk-rekognition'
   credentials = Aws::Credentials.new(
      ENV['AWS_ACCESS_KEY_ID'],
      ENV['AWS_SECRET_ACCESS_KEY']
   )
   bucket = 'bucket' # the bucketname without s3://
   photo  = 'photo'# the name of file
   client   = Aws::Rekognition::Client.new credentials: credentials
   attrs = {
     image: {
       s3_object: {
         bucket: bucket,
         name: photo
       },
     },
     attributes: ['ALL']
   }
   response = client.detect_faces attrs
   puts "Detected faces for: #{photo}"
   response.face_details.each do |face_detail|
     low  = face_detail.age_range.low
     high = face_detail.age_range.high
```

```
    puts "The detected face is between: #{low} and #{high} years old"
    puts "All other attributes:"
    puts "  bounding_box.width:     #{face_detail.bounding_box.width}"
    puts "  bounding_box.height:    #{face_detail.bounding_box.height}"
    puts "  bounding_box.left:      #{face_detail.bounding_box.left}"
    puts "  bounding_box.top:       #{face_detail.bounding_box.top}"
    puts "  age.range.low:          #{face_detail.age_range.low}"
    puts "  age.range.high:         #{face_detail.age_range.high}"
    puts "  smile.value:            #{face_detail.smile.value}"
    puts "  smile.confidence:       #{face_detail.smile.confidence}"
    puts "  eyeglasses.value:       #{face_detail.eyeglasses.value}"
    puts "  eyeglasses.confidence:  #{face_detail.eyeglasses.confidence}"
    puts "  sunglasses.value:       #{face_detail.sunglasses.value}"
    puts "  sunglasses.confidence:  #{face_detail.sunglasses.confidence}"
    puts "  gender.value:           #{face_detail.gender.value}"
    puts "  gender.confidence:      #{face_detail.gender.confidence}"
    puts "  beard.value:            #{face_detail.beard.value}"
    puts "  beard.confidence:       #{face_detail.beard.confidence}"
    puts "  mustache.value:         #{face_detail.mustache.value}"
    puts "  mustache.confidence:    #{face_detail.mustache.confidence}"
    puts "  eyes_open.value:        #{face_detail.eyes_open.value}"
    puts "  eyes_open.confidence:   #{face_detail.eyes_open.confidence}"
    puts "  mout_open.value:        #{face_detail.mouth_open.value}"
    puts "  mout_open.confidence:   #{face_detail.mouth_open.confidence}"
    puts "  emotions[0].type:       #{face_detail.emotions[0].type}"
    puts "  emotions[0].confidence: #{face_detail.emotions[0].confidence}"
    puts "  landmarks[0].type:      #{face_detail.landmarks[0].type}"
    puts "  landmarks[0].x:         #{face_detail.landmarks[0].x}"
    puts "  landmarks[0].y:         #{face_detail.landmarks[0].y}"
    puts "  pose.roll:              #{face_detail.pose.roll}"
    puts "  pose.yaw:               #{face_detail.pose.yaw}"
    puts "  pose.pitch:             #{face_detail.pose.pitch}"
    puts "  quality.brightness:     #{face_detail.quality.brightness}"
    puts "  quality.sharpness:      #{face_detail.quality.sharpness}"
    puts "  confidence:             #{face_detail.confidence}"
    puts "------------"
    puts ""
  end
```

Node.js

此範例顯示一份在輸入影像中偵測到的標籤清單。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)


// Load the SDK and UUID
var AWS = require('aws-sdk');
var uuid = require('node-uuid');


const bucket = 'bucket' // the bucketname without s3://
const photo  = 'photo' // the name of file

 const config = new AWS.Config({
  accessKeyId: process.env.AWS_ACCESS_KEY_ID,
  secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY,
  region: process.env.AWS_REGION
})
const client = new AWS.Rekognition();
const params = {
```

```
  Image: {
    S3Object: {
      Bucket: bucket,
      Name: photo
    },
  },
  MaxLabels: 10
}
client.detectLabels(params, function(err, response) {
  if (err) {
    console.log(err, err.stack); // an error occurred
  } else {
    console.log(`Detected labels for: ${photo}`)
    response.Labels.forEach(label => {
      console.log(`Label:      ${label.Name}`)
      console.log(`Confidence: ${label.Confidence}`)
      console.log("Instances:")
      label.Instances.forEach(instance => {
        let box = instance.BoundingBox
        console.log("  Bounding box:")
        console.log(`    Top:        ${box.Top}`)
        console.log(`    Left:       ${box.Left}`)
        console.log(`    Width:      ${box.Width}`)
        console.log(`    Height:     ${box.Height}`)
        console.log(`  Confidence: ${instance.Confidence}`)
      })
      console.log("Parents:")
      label.Parents.forEach(parent => {
        console.log(`  ${parent.Name}`)
      })
      console.log("------------")
      console.log("")
    }) // for response.labels
  } // if
});
```

DetectLabels 操作要求

`DetectLabel` 的輸入是影像。在此範例 JSON 輸入中，來源影像是從 Amazon S3 儲存貯體載入的。`MaxLabels` 是回應中傳回的最大標籤數量。`MinConfidence` 是 Amazon Rekognition Image 在偵測到的標籤的準確性中必須具有的最低信賴度，以便在回應中傳回。

```
{
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "input.jpg"
        }
    },
    "MaxLabels": 10,
    "MinConfidence": 75
}
```

DetectLabels 回應

`DetectLabels` 的回應是影像中偵測到的一系列標籤，以及偵測所依據的可信度層級。

以下是來自 `DetectLabels` 的回應範例。

回應顯示操作偵測到多個標籤，包括人員、車輛和汽車。每個標籤都有一個相關的可信度等級。例如，偵測演算法對於影像中包含人員的可信度為 98.991432%。

回應也包含 `Parents` 陣列中標籤的上階標籤。例如 Automobile (汽車) 標籤有兩個名為 Vehicle (車輛) 和 Transportation (運輸) 的 父標籤。

常見物件標籤的回應包含輸入影像上標籤位置的週框方塊資訊。例如，人員標籤具有一個實例陣列，其中包含兩個週框方塊。這些是在影像中偵測到的兩個人員位置。

欄位 `LabelModelVersion` 包含 `DetectLabels` 所使用之偵測模型的版本編號。

```
{

    {
    "Labels": [
        {
            "Name": "Vehicle",
            "Confidence": 99.15271759033203,
            "Instances": [],
            "Parents": [
                {
                    "Name": "Transportation"
                }
            ]
        },
        {
            "Name": "Transportation",
            "Confidence": 99.15271759033203,
            "Instances": [],
            "Parents": []
        },
        {
            "Name": "Automobile",
            "Confidence": 99.15271759033203,
            "Instances": [],
            "Parents": [
                {
                    "Name": "Vehicle"
                },
                {
                    "Name": "Transportation"
                }
            ]
        },
        {
            "Name": "Car",
            "Confidence": 99.15271759033203,
            "Instances": [
                {
                    "BoundingBox": {
                        "Width": 0.10616336017847061,
                        "Height": 0.18528179824352264,
                        "Left": 0.0037978808395564556,
                        "Top": 0.5039216876029968
                    },
                    "Confidence": 99.15271759033203
                },
                {
                    "BoundingBox": {
                        "Width": 0.2429988533258438,
                        "Height": 0.21577216684818268,
                        "Left": 0.7309805154800415,
                        "Top": 0.5251884460449219
                    },
                    "Confidence": 99.1286392211914
                },
                {
                    "BoundingBox": {
```

```
            "Width": 0.14233611524105072,
            "Height": 0.15528248250484467,
            "Left": 0.6494812965393066,
            "Top": 0.5333095788955688
        },
        "Confidence": 98.48368072509766
    },
    {
        "BoundingBox": {
            "Width": 0.11086395382881165,
            "Height": 0.10271988064050674,
            "Left": 0.10355594009160995,
            "Top": 0.5354844927787781
        },
        "Confidence": 96.45606231689453
    },
    {
        "BoundingBox": {
            "Width": 0.06254628300666809,
            "Height": 0.053911514580249786,
            "Left": 0.46083059906959534,
            "Top": 0.5573825240135193
        },
        "Confidence": 93.65448760986328
    },
    {
        "BoundingBox": {
            "Width": 0.10105438530445099,
            "Height": 0.12226245552301407,
            "Left": 0.5743985772132874,
            "Top": 0.534368634223938
        },
        "Confidence": 93.06217193603516
    },
    {
        "BoundingBox": {
            "Width": 0.056389667093753815,
            "Height": 0.17163699865341187,
            "Left": 0.9427769780158997,
            "Top": 0.5235804319381714
        },
        "Confidence": 92.6864013671875
    },
    {
        "BoundingBox": {
            "Width": 0.06003860384225845,
            "Height": 0.06737709045410156,
            "Left": 0.22409997880458832,
            "Top": 0.5441341400146484
        },
        "Confidence": 90.4227066040039
    },
    {
        "BoundingBox": {
            "Width": 0.02848697081208229,
            "Height": 0.19150497019290924,
            "Left": 0.0,
            "Top": 0.5107086896896362
        },
        "Confidence": 86.65286254882812
    },
    {
        "BoundingBox": {
            "Width": 0.04067881405353546,
            "Height": 0.03428703173995018,
            "Left": 0.316415935754776,
```

```
                    "Top": 0.5566273927688599
                },
                "Confidence": 85.36471557617188
            },
            {
                "BoundingBox": {
                    "Width": 0.043411049991846085,
                    "Height": 0.0893595889210701,
                    "Left": 0.18293385207653046,
                    "Top": 0.5394920110702515
                },
                "Confidence": 82.21705627441406
            },
            {
                "BoundingBox": {
                    "Width": 0.031183116137981415,
                    "Height": 0.03989990055561066,
                    "Left": 0.2853088080883026,
                    "Top": 0.5579366683959961
                },
                "Confidence": 81.0157470703125
            },
            {
                "BoundingBox": {
                    "Width": 0.031113790348172188,
                    "Height": 0.056484755128622055,
                    "Left": 0.2580395042896271,
                    "Top": 0.5504819750785828
                },
                "Confidence": 56.13441467285156
            },
            {
                "BoundingBox": {
                    "Width": 0.08586374670267105,
                    "Height": 0.08550430089235306,
                    "Left": 0.5128012895584106,
                    "Top": 0.5438792705535889
                },
                "Confidence": 52.37760925292969
            }
        ],
        "Parents": [
            {
                "Name": "Vehicle"
            },
            {
                "Name": "Transportation"
            }
        ]
    },
    {
        "Name": "Human",
        "Confidence": 98.9914321899414,
        "Instances": [],
        "Parents": []
    },
    {
        "Name": "Person",
        "Confidence": 98.9914321899414,
        "Instances": [
            {
                "BoundingBox": {
                    "Width": 0.19360728561878204,
                    "Height": 0.2742200493812561,
                    "Left": 0.43734854459762573,
                    "Top": 0.35072067379951477
```

```
                    },
                    "Confidence": 98.9914321899414
                },
                {
                    "BoundingBox": {
                        "Width": 0.03801717236638069,
                        "Height": 0.06597328186035156,
                        "Left": 0.9155802130699158,
                        "Top": 0.5010883808135986
                    },
                    "Confidence": 85.02790832519531
                }
            ],
            "Parents": []
        }
    ],
    "LabelModelVersion": "2.0"
}


}
```

偵測影片中的標籤

Amazon Rekognition Video 可以偵測影片中的標籤以及偵測到標籤的時間。如需開發套件程式碼範例，請參閱 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。如需 AWS CLI 的範例，請參閱 以 AWS Command Line Interface 分析影片 (p. 66)。

Amazon Rekognition Video視訊中的標籤偵測是一種非同步操作。若要開始在影片中偵測標籤，請呼叫 StartLabelDetection (p. 415)。Amazon Rekognition Video 將發佈影片分析的完成狀態至 Amazon Simple Notification Service 主題。如果視訊分析成功，請呼叫 GetLabelDetection (p. 358) 以取得偵測到的標籤。如需呼叫視訊分析 API 操作的資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

GetLabelDetection 操作回應

`GetLabelDetection` 會傳回陣列 (`Labels`)，其中包含視訊中偵測到之標籤的相關資訊。可指定 `SortBy` 參數依據時間或偵測到的標籤來排序陣列。

以下是 `GetLabelDetection` 的 JSON 回應範例。在回應中，請注意下列事項：

- 排序順序 – 傳回的標籤陣列會依時間排序。若要依標籤排序，請在 `SortBy` 輸入參數中指定 `NAME` 以執行 `GetLabelDetection`。如果標籤在視訊中多次出現，將有 (LabelDetection (p. 478)) 元素的多個執行個體。
- 標籤資訊 – `LabelDetection` 陣列元素包含 (Label (p. 477)) 物件，其中包括標籤名稱和 Amazon Rekognition 對於標籤偵測精確度的可信度分數。`Label` 物件也包含標籤的階層式分類法和常用標籤的週框方塊資訊。`Timestamp` 則是偵測到標籤的時間，從影片開始起算並以毫秒為單位。
- 分頁資訊 – 範例顯示標籤偵測資訊的一頁。您可以指定 `GetLabelDetection` 的 `MaxResults` 輸入參數中要傳回幾個 `LabelDetection` 物件。如果結果數目超過 `MaxResults`，`GetLabelDetection` 會傳回用來取得下一頁結果的字符 (`NextToken`)。如需詳細資訊，請參閱 取得 Amazon Rekognition Video 分析結果 (p. 53)。
- 影片資訊– 該回應包含 `GetLabelDetection` 傳回的每頁資訊中，影片格式 (`VideoMetadata`) 的資訊。

```
{
    "Labels": [
        {
            "Timestamp": 0,
```

```
            "Label": {
                "Instances": [],
                "Confidence": 60.51791763305664,
                "Parents": [],
                "Name": "Electronics"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Instances": [],
                "Confidence": 99.53411102294922,
                "Parents": [],
                "Name": "Human"
            }
        },
        {
            "Timestamp": 0,
            "Label": {
                "Instances": [
                    {
                        "BoundingBox": {
                            "Width": 0.11109819263219833,
                            "Top": 0.08098889887332916,
                            "Left": 0.8881205320358276,
                            "Height": 0.9073750972747803
                        },
                        "Confidence": 99.5831298828125
                    },
                    {
                        "BoundingBox": {
                            "Width": 0.1268676072359085,
                            "Top": 0.14018426835536957,
                            "Left": 0.0003282368124928324,
                            "Height": 0.7993982434272766
                        },
                        "Confidence": 99.46029663085938
                    }
                ],
                "Confidence": 99.53411102294922,
                "Parents": [],
                "Name": "Person"
            }
        },
        .
        .
        .

        {
            "Timestamp": 166,
            "Label": {
                "Instances": [],
                "Confidence": 73.6471176147461,
                "Parents": [
                    {
                        "Name": "Clothing"
                    }
                ],
                "Name": "Sleeve"
            }
        }

    ],
    "LabelModelVersion": "2.0",
    "JobStatus": "SUCCEEDED",
    "VideoMetadata": {
```

```
        "Format": "QuickTime / MOV",
        "FrameRate": 23.976024627685547,
        "Codec": "h264",
        "DurationMillis": 5005,
        "FrameHeight": 674,
        "FrameWidth": 1280
    }
}
```

偵測自訂標籤

Amazon Rekognition 自訂標籤可以識別出影像中專屬於您業務需求的物體和場景，例如標誌或工程機具零件。如需詳細資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南中的什麼是 Amazon Rekognition 自訂標籤？。

偵測與分析臉部

Amazon Rekognition 可以偵測影像與影片中的臉部。本節說明用於分析臉部的非儲存操作。使用 Amazon Rekognition 可獲得臉部在影像或視訊中被偵測到的位置、臉部特徵點 (例如眼睛位置)、以及偵測到的情緒 (例如表現出快樂或悲傷) 等相關資訊。您也可以將影像中的臉部與另一張影像中偵測到的臉孔進行比對。

當您提供含有臉部的影像時，Amazon Rekognition 會偵測影像中的臉部、分析其臉部屬性，然後傳回影像中偵測到之臉部與臉部屬性的信賴分數百分比。



本節提供影像與影片臉部分析的範例。如需使用 Amazon Rekognition API 的詳細資訊，請參閱 使用影像 (p. 23) 和 使用儲存的影片來操作 (p. 49)。

> Note
>
> Amazon Rekognition Image 和 Amazon Rekognition Video 使用的臉部偵測模型不支援偵測卡通/動畫人物或非人類實體。如果您想要偵測影像或視訊中的卡通人物，我們建議您使用 Amazon Rekognition 自訂標籤。如需詳細資訊，請參閱 Amazon Rekognition 自訂標籤開發人員指南。

您可以使用儲存操作，來儲存影像中偵測到之臉部的臉部中繼資料。您之後可以搜尋存放在影像與影片中的臉部。例如，您可以在影片中搜尋特定人員。如需更多詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

主題

- 臉部偵測和臉部辨比較的概觀 (p. 120)
- 臉部屬性的準則 (p. 120)
- 偵測影像中的臉部 (p. 120)
- 比較影像中的臉部 (p. 133)
- 偵測已存放影片中的臉部 (p. 141)

臉部偵測和臉部辨比較的概觀

機器學習有兩種主要應用，其會分析包含臉部的影像：臉部偵測和臉部比較。臉部偵測系統旨在回答此類問題：此圖片有臉部嗎？ 臉部偵測系統會判定靜態影像或視訊影格中呈現的任何臉部的存在、位置、範圍和 (可能) 方向。此系統旨在偵測臉部是否存在，而不管性別、年齡和臉部毛髮之類的屬性。

臉部比較系統旨在回答此類問題：影像中的臉部是否符合另一個影像中的臉部？ 臉部比較系統會取得臉部的影像，並預測臉部是否符合所提供資料庫中的其他臉部。臉部比較系統旨在比較並預測潛在相符的臉部，而不管其表情、臉部毛髮和年齡。

臉部偵測和臉部比較系統都可以使用機率或可信度分數的形式，提供預測可信度的估計值。例如，臉部偵測系統可能預測影像區域是可信度分數為 90% 的臉部，而另一個影像區域是可信度分數為 60% 的臉部。可信度分數更高的區域應該更可能包含臉部。如果臉部偵測系統未適當地偵測到臉部，或提供實際臉部的低可信度預測，這稱為錯失偵測或誤判。如果臉部偵測系統未正確地預測有一個高信度的臉部存在，則這是誤報或誤判。同樣地，臉部比較系統可能無法比對屬於同一個人的兩個臉部 (錯失偵測/誤判)，或者可能不正確地預測來自不同人的兩個臉部是同一個人 (誤報/誤判)。

可信度分數是臉部偵測和比較系統的關鍵元件。這些系統會使用預測對應的可信度，預測臉部是否存在於影像中，或是否符合另一個影像中的臉部。這些系統的使用者應該在設計其應用程式，並根據系統輸出做出決策時，考慮系統所提供的可信度分數/相似度閾值。例如，在應用程式用來識別類似容貌家人的照片中，如果可信度閾值設為 80%，則應用程式會在預測達到 80% 可信度時傳回相符項，但不會傳回低於該可信度的相符項。此閾值是可接受的，因為對於此類型的使用案例，錯失偵測或誤報的風險很低。不過，對於錯失偵測或誤報的風險更高的使用案例，系統應該使用更高的可信度。在臉部需高度準確的案例中，您應該使用 99% 可信度/相似度閾值。如需建議可信度閾值的詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

臉部屬性的準則

Amazon Rekognition 傳回其偵測到之每個臉部的週框方塊、屬性、情緒、特徵點、品質，以及姿勢。每個屬性或情緒都有一值和可信度分數。例如，某張臉部經過預測，性別可能判定為「女性」，可信度分數為 85%，而情緒判定為「快樂」，可信度分數為 90%。

性別二元論 (男性/女性) 預測是根據特定影像中臉部的實體外觀來判定。此方式不會指出一個人的性別認同，因此您不應使用Amazon Rekognition 進行此類判定。我們不建議採用性別二元論預測來制定會影響個人權利、隱私或服務存取的決策。

同樣地，情緒表達的預測僅根據影像中一個人臉部的實體外觀。此方式不會指出一個人的內心情緒狀態，因此您不應使用Amazon Rekognition 進行此類判定。例如，照片中強顏歡笑的人可能看起來很開心，但卻感受不到幸福。

對於分類精確度可能對影像主體產生負面影響的使用案例，我們建議您使用 99% 或更高的閾值。唯一的例外是年齡範圍，其中 Amazon Rekognition 會估計人員的較低和較高年齡。在此情況下，年齡範圍越寬，該預測的可信度越低。做為近似值，您應該使用年齡範圍的中點，來預估所偵測到臉部之年齡的單一值。(實際年齡不一定對應至此數字。)

這些屬性的其中一個最佳用途是產生彙總統計資料。例如，屬性 (例如微笑、姿勢和清晰度) 可以用來在社交媒體應用程式中自動選取「最佳設定檔圖片」。另一個常用案例是使用預測的性別和年齡屬性，以匿名方式預估廣泛範例的人口統計資料 (例如，在活動或零售商店)。

偵測影像中的臉部

Amazon Rekognition Image 提供 DetectFaces (p. 320) 種操作，這可尋找主要的臉部特徵 (例如眼睛、鼻子與嘴巴)，以偵測輸入影像中的臉部。Amazon Rekognition Image 可在影像中偵測到 100 個最大的臉部。

您可以用影像位元組陣列 (Base64 編碼影像位元組) 的方式提供輸入影像，或指定 Amazon S3 物件。在此程序中，您會將影像 (JPEG 或 PNG) 上傳至您的 S3 儲存貯體，並指定物件索引碼名稱。

偵測影像中的臉部

1. 如果您尚未：

   a. 建立或更新具有 AmazonRekognitionFullAccess 和 AmazonS3ReadOnlyAccess 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。

2. 將 (含有一或多個臉部) 的影像上傳至您的 S3 儲存貯體。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。

3. 使用下列範例以呼叫 DetectFaces。

   Java

   此範例顯示偵測到之臉部的估計年齡範圍，並列出所有偵測到之臉部屬性的 JSON。變更影像檔案名稱的 photo 值。變更儲存影像之 Amazon S3 儲存貯體的 bucket 值。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.model.AgeRange;
import com.amazonaws.services.rekognition.model.Attribute;
import com.amazonaws.services.rekognition.model.DetectFacesRequest;
import com.amazonaws.services.rekognition.model.DetectFacesResult;
import com.amazonaws.services.rekognition.model.FaceDetail;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.List;


public class DetectFaces {


   public static void main(String[] args) throws Exception {

      String photo = "input.jpg";
      String bucket = "bucket";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();


      DetectFacesRequest request = new DetectFacesRequest()
         .withImage(new Image()
            .withS3Object(new S3Object()
               .withName(photo)
               .withBucket(bucket)))
         .withAttributes(Attribute.ALL);
      // Replace Attribute.ALL with Attribute.DEFAULT to get default values.
```

```
      try {
         DetectFacesResult result = rekognitionClient.detectFaces(request);
         List < FaceDetail > faceDetails = result.getFaceDetails();

         for (FaceDetail face: faceDetails) {
            if (request.getAttributes().contains("ALL")) {
               AgeRange ageRange = face.getAgeRange();
               System.out.println("The detected face is estimated to be between "
                  + ageRange.getLow().toString() + " and " +
 ageRange.getHigh().toString()
                  + " years old.");
               System.out.println("Here's the complete set of attributes:");
            } else { // non-default attributes have null values.
               System.out.println("Here's the default set of attributes:");
            }

            ObjectMapper objectMapper = new ObjectMapper();

 System.out.println(objectMapper.writerWithDefaultPrettyPrinter().writeValueAsString(face));
         }

      } catch (AmazonRekognitionException e) {
         e.printStackTrace();
      }

   }

}
```

AWS CLI

此範例顯示 `detect-faces` AWS CLI 操作的 JSON 輸出。將 `file` 取代為影像檔案的名稱。將 `bucket` 取代為 Amazon S3 儲存貯體的名稱，其中包含影像檔案。

```
aws rekognition detect-faces \
--image '{"S3Object":{"Bucket":"bucket","Name":"file"}}' \
--attributes "ALL"
```

Python

此範例顯示偵測到之臉部的估計年齡範圍，並列出所有偵測到之臉部屬性的 JSON。變更影像檔案名稱的 `photo` 值。變更儲存影像之 Amazon S3 儲存貯體的 `bucket` 值。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
import json

def detect_faces(photo, bucket):

    client=boto3.client('rekognition')

    response = client.detect_faces(Image={'S3Object':
{'Bucket':bucket,'Name':photo}},Attributes=['ALL'])

    print('Detected faces for ' + photo)
    for faceDetail in response['FaceDetails']:
        print('The detected face is between ' + str(faceDetail['AgeRange']['Low'])
              + ' and ' + str(faceDetail['AgeRange']['High']) + ' years old')
        print('Here are the other attributes:')
```

```
        print(json.dumps(faceDetail, indent=4, sort_keys=True))
    return len(response['FaceDetails'])
def main():
    photo='photo'
    bucket='bucket'
    face_count=detect_faces(photo, bucket)
    print("Faces detected: " + str(face_count))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示偵測到之臉部的估計年齡範圍，並列出所有偵測到之臉部屬性的 JSON。變更影像檔案名稱的 `photo` 值。變更儲存影像之 Amazon S3 儲存貯體的 `bucket` 值。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.Collections.Generic;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectFaces
{
    public static void Example()
    {
        String photo = "input.jpg";
        String bucket = "bucket";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectFacesRequest detectFacesRequest = new DetectFacesRequest()
        {
            Image = new Image()
            {
                S3Object = new S3Object()
                {
                    Name = photo,
                    Bucket = bucket
                },
            },
            // Attributes can be "ALL" or "DEFAULT".
            // "DEFAULT": BoundingBox, Confidence, Landmarks, Pose, and Quality.
            // "ALL": See https://docs.aws.amazon.com/sdkfornet/v3/apidocs/items/
Rekognition/TFaceDetail.html
            Attributes = new List<String>() { "ALL" }
        };

        try
        {
            DetectFacesResponse detectFacesResponse =
 rekognitionClient.DetectFaces(detectFacesRequest);
            bool hasAll = detectFacesRequest.Attributes.Contains("ALL");
            foreach(FaceDetail face in detectFacesResponse.FaceDetails)
            {
                Console.WriteLine("BoundingBox: top={0} left={1} width={2}
 height={3}", face.BoundingBox.Left,
                    face.BoundingBox.Top, face.BoundingBox.Width,
 face.BoundingBox.Height);
```

```
                Console.WriteLine("Confidence: {0}\nLandmarks: {1}\nPose: pitch={2}
 roll={3} yaw={4}\nQuality: {5}",
                    face.Confidence, face.Landmarks.Count, face.Pose.Pitch,
                    face.Pose.Roll, face.Pose.Yaw, face.Quality);
                if (hasAll)
                    Console.WriteLine("The detected face is estimated to be between
 " +
                        face.AgeRange.Low + " and " + face.AgeRange.High + " years
 old.");
            }
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

Ruby

此範例顯示偵測到之臉部的估計年齡範圍，並列出各種臉部屬性。變更影像檔案名稱的 `photo` 值。變更儲存影像之 Amazon S3 儲存貯體的 `bucket` 值。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

   # Add to your Gemfile
   # gem 'aws-sdk-rekognition'
   require 'aws-sdk-rekognition'
   credentials = Aws::Credentials.new(
      ENV['AWS_ACCESS_KEY_ID'],
      ENV['AWS_SECRET_ACCESS_KEY']
   )
   bucket = 'bucket' # the bucketname without s3://
   photo  = 'input.jpg'# the name of file
   client   = Aws::Rekognition::Client.new credentials: credentials
   attrs = {
     image: {
       s3_object: {
         bucket: bucket,
         name: photo
       },
     },
     attributes: ['ALL']
   }
   response = client.detect_faces attrs
   puts "Detected faces for: #{photo}"
   response.face_details.each do |face_detail|
     low  = face_detail.age_range.low
     high = face_detail.age_range.high
     puts "The detected face is between: #{low} and #{high} years old"
     puts "All other attributes:"
     puts "  bounding_box.width:     #{face_detail.bounding_box.width}"
     puts "  bounding_box.height:    #{face_detail.bounding_box.height}"
     puts "  bounding_box.left:      #{face_detail.bounding_box.left}"
     puts "  bounding_box.top:       #{face_detail.bounding_box.top}"
     puts "  age.range.low:          #{face_detail.age_range.low}"
     puts "  age.range.high:         #{face_detail.age_range.high}"
     puts "  smile.value:            #{face_detail.smile.value}"
     puts "  smile.confidence:       #{face_detail.smile.confidence}"
     puts "  eyeglasses.value:       #{face_detail.eyeglasses.value}"
     puts "  eyeglasses.confidence:  #{face_detail.eyeglasses.confidence}"
     puts "  sunglasses.value:       #{face_detail.sunglasses.value}"
```

```
    puts "  sunglasses.confidence:  #{face_detail.sunglasses.confidence}"
    puts "  gender.value:           #{face_detail.gender.value}"
    puts "  gender.confidence:      #{face_detail.gender.confidence}"
    puts "  beard.value:            #{face_detail.beard.value}"
    puts "  beard.confidence:       #{face_detail.beard.confidence}"
    puts "  mustache.value:         #{face_detail.mustache.value}"
    puts "  mustache.confidence:    #{face_detail.mustache.confidence}"
    puts "  eyes_open.value:        #{face_detail.eyes_open.value}"
    puts "  eyes_open.confidence:   #{face_detail.eyes_open.confidence}"
    puts "  mout_open.value:        #{face_detail.mouth_open.value}"
    puts "  mout_open.confidence:   #{face_detail.mouth_open.confidence}"
    puts "  emotions[0].type:       #{face_detail.emotions[0].type}"
    puts "  emotions[0].confidence: #{face_detail.emotions[0].confidence}"
    puts "  landmarks[0].type:      #{face_detail.landmarks[0].type}"
    puts "  landmarks[0].x:         #{face_detail.landmarks[0].x}"
    puts "  landmarks[0].y:         #{face_detail.landmarks[0].y}"
    puts "  pose.roll:              #{face_detail.pose.roll}"
    puts "  pose.yaw:               #{face_detail.pose.yaw}"
    puts "  pose.pitch:             #{face_detail.pose.pitch}"
    puts "  quality.brightness:     #{face_detail.quality.brightness}"
    puts "  quality.sharpness:      #{face_detail.quality.sharpness}"
    puts "  confidence:             #{face_detail.confidence}"
    puts "------------"
    puts ""
  end
```

Node.js

此範例顯示偵測到之臉部的估計年齡範圍，並列出各種臉部屬性。變更影像檔案名稱的 `photo` 值。變更儲存影像之 Amazon S3 儲存貯體的 `bucket` 值。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

  const AWS = require('aws-sdk')
  const bucket = 'bucket' // the bucketname without s3://
  const photo  = 'input.jpg' // the name of file
  const config = new AWS.Config({
    accessKeyId: process.env.AWS_ACCESS_KEY_ID,
    secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY,
    region: process.env.AWS_REGION
  })
  const client = new AWS.Rekognition();
  const params = {
    Image: {
      S3Object: {
        Bucket: bucket,
        Name: photo
      },
    },
    Attributes: ['ALL']
  }
  client.detectFaces(params, function(err, response) {
    if (err) {
      console.log(err, err.stack); // an error occurred
    } else {
      console.log(`Detected faces for: ${photo}`)
      response.FaceDetails.forEach(data => {
        let low  = data.AgeRange.Low
        let high = data.AgeRange.High
        console.log(`The detected face is between: ${low} and ${high} years old`)
        console.log("All other attributes:")
        console.log(`  BoundingBox.Width:      ${data.BoundingBox.Width}`)
```

```
        console.log(`  BoundingBox.Height:     ${data.BoundingBox.Height}`)
        console.log(`  BoundingBox.Left:       ${data.BoundingBox.Left}`)
        console.log(`  BoundingBox.Top:        ${data.BoundingBox.Top}`)
        console.log(`  Age.Range.Low:          ${data.AgeRange.Low}`)
        console.log(`  Age.Range.High:         ${data.AgeRange.High}`)
        console.log(`  Smile.Value:            ${data.Smile.Value}`)
        console.log(`  Smile.Confidence:       ${data.Smile.Confidence}`)
        console.log(`  Eyeglasses.Value:       ${data.Eyeglasses.Value}`)
        console.log(`  Eyeglasses.Confidence:  ${data.Eyeglasses.Confidence}`)
        console.log(`  Sunglasses.Value:       ${data.Sunglasses.Value}`)
        console.log(`  Sunglasses.Confidence:  ${data.Sunglasses.Confidence}`)
        console.log(`  Gender.Value:           ${data.Gender.Value}`)
        console.log(`  Gender.Confidence:      ${data.Gender.Confidence}`)
        console.log(`  Beard.Value:            ${data.Beard.Value}`)
        console.log(`  Beard.Confidence:       ${data.Beard.Confidence}`)
        console.log(`  Mustache.Value:         ${data.Mustache.Value}`)
        console.log(`  Mustache.Confidence:    ${data.Mustache.Confidence}`)
        console.log(`  EyesOpen.Value:         ${data.EyesOpen.Value}`)
        console.log(`  EyesOpen.Confidence:    ${data.EyesOpen.Confidence}`)
        console.log(`  MouthOpen.Value:        ${data.MouthOpen.Value}`)
        console.log(`  MouthOpen.Confidence:   ${data.MouthOpen.Confidence}`)
        console.log(`  Emotions[0].Type:       ${data.Emotions[0].Type}`)
        console.log(`  Emotions[0].Confidence: ${data.Emotions[0].Confidence}`)
        console.log(`  Landmarks[0].Type:      ${data.Landmarks[0].Type}`)
        console.log(`  Landmarks[0].X:         ${data.Landmarks[0].X}`)
        console.log(`  Landmarks[0].Y:         ${data.Landmarks[0].Y}`)
        console.log(`  Pose.Roll:              ${data.Pose.Roll}`)
        console.log(`  Pose.Yaw:               ${data.Pose.Yaw}`)
        console.log(`  Pose.Pitch:             ${data.Pose.Pitch}`)
        console.log(`  Quality.Brightness:     ${data.Quality.Brightness}`)
        console.log(`  Quality.Sharpness:      ${data.Quality.Sharpness}`)
        console.log(`  Confidence:             ${data.Confidence}`)
        console.log("------------")
        console.log("")
      }) // for response.faceDetails
    } // if
  });
```

DetectFaces 操作要求

DetectFaces 的輸入是影像。在此範例中，影像是從 Amazon S3 儲存貯體載入的。Attributes 參數指定所有臉部屬性應該要傳回。如需詳細資訊，請參閱 使用影像 (p. 23)。

```
{
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "input.jpg"
        }
    },
    "Attributes": [
        "ALL"
    ]
}
```

DetectFaces 操作回應

DetectFaces 針對每個偵測到的臉孔傳回以下資訊：

- 週框方塊 – 臉部周圍的週框方塊座標。

- 信賴度 – 週框方塊包含臉部的信賴度。
- 臉部特徵點 – 臉部特徵點的陣列。對於每個特徵點 (例如左眼、右眼與嘴巴)，回應均會提供 x 和 y 座標。
- 臉部屬性 – 一組臉部屬性，例如臉上是否有鬍鬚。對於每個這類屬性，該回應均會提供一個值。該值的類型可能不同，例如布林值類型 (此人是否戴著太陽眼鏡) 或字串 (此人是男性或女性)。此外，對於大部分屬性，該回應也會提供偵測到之屬性值的信賴度。
- 畫質 – 描述臉部的亮度與銳利度。如需盡可能確保最佳臉部偵測的資訊，請參閱 臉部比較輸入影像的建議 (p. 103)。
- 姿態 – 描述影像內臉部的旋轉。
- 情緒 – 一組情緒，分析中會提供其信賴度。

以下是 `DetectFaces` API 呼叫的回應範例。


```
{
    "FaceDetails": [
        {
            "AgeRange": {
                "High": 43,
                "Low": 26
            },
            "Beard": {
                "Confidence": 97.48941802978516,
                "Value": true
            },
            "BoundingBox": {
                "Height": 0.6968063116073608,
                "Left": 0.26937249302864075,
                "Top": 0.11424895375967026,
                "Width": 0.42325547337532043
            },
            "Confidence": 99.99995422363281,
            "Emotions": [
                {
                    "Confidence": 0.042965151369571686,
                    "Type": "DISGUSTED"
                },
                {
                    "Confidence": 0.002022328320890665,
                    "Type": "HAPPY"
                },
                {
                    "Confidence": 0.4482877850532532,
                    "Type": "SURPRISED"
                },
                {
                    "Confidence": 0.007082826923578978,
                    "Type": "ANGRY"
                },
                {
                    "Confidence": 0,
                    "Type": "CONFUSED"
                },
                {
                    "Confidence": 99.47616577148438,
                    "Type": "CALM"
                },
                {
                    "Confidence": 0.017732391133904457,
                    "Type": "SAD"
                }
            ],
            "Eyeglasses": {
```

```
            "Confidence": 99.42405700683594,
            "Value": false
        },
        "EyesOpen": {
            "Confidence": 99.99604797363281,
            "Value": true
        },
        "Gender": {
            "Confidence": 99.722412109375,
            "Value": "Male"
        },
        "Landmarks": [
            {
                "Type": "eyeLeft",
                "X": 0.38549351692199707,
                "Y": 0.3959200084209442
            },
            {
                "Type": "eyeRight",
                "X": 0.5773905515670776,
                "Y": 0.394561767578125
            },
            {
                "Type": "mouthLeft",
                "X": 0.40410104393959045,
                "Y": 0.6479480862617493
            },
            {
                "Type": "mouthRight",
                "X": 0.5623446702957153,
                "Y": 0.647117555141449
            },
            {
                "Type": "nose",
                "X": 0.47763553261756897,
                "Y": 0.5337067246437073
            },
            {
                "Type": "leftEyeBrowLeft",
                "X": 0.3114689588546753,
                "Y": 0.3376390337944031
            },
            {
                "Type": "leftEyeBrowRight",
                "X": 0.4224424660205841,
                "Y": 0.3232649564743042
            },
            {
                "Type": "leftEyeBrowUp",
                "X": 0.36654090881347656,
                "Y": 0.3104579746723175
            },
            {
                "Type": "rightEyeBrowLeft",
                "X": 0.5353175401687622,
                "Y": 0.3223199248313904
            },
            {
                "Type": "rightEyeBrowRight",
                "X": 0.6546239852905273,
                "Y": 0.3348073363304138
            },
            {
                "Type": "rightEyeBrowUp",
                "X": 0.5936762094497681,
                "Y": 0.3080498278141022
```

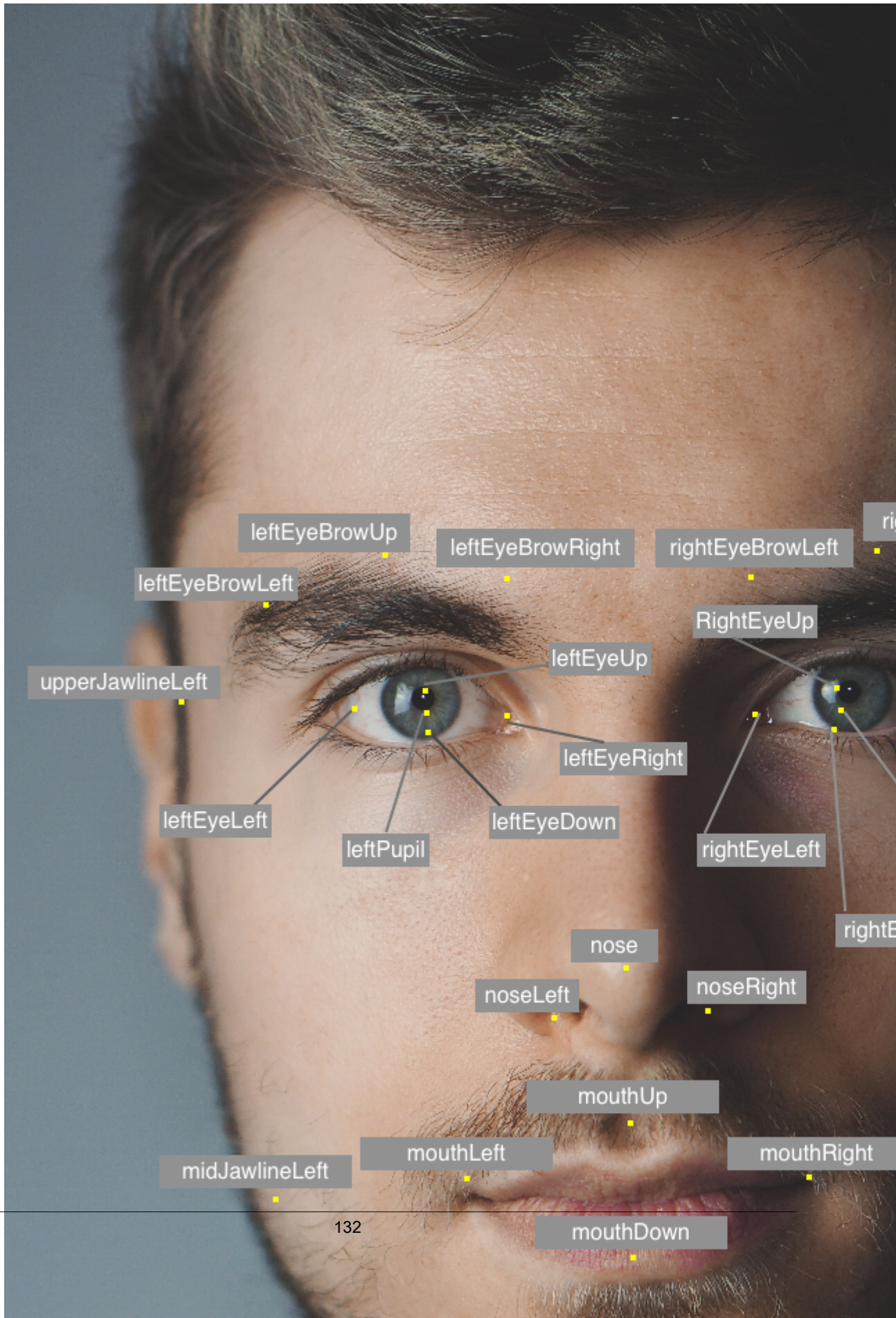
```
        },
        {
            "Type": "leftEyeLeft",
            "X": 0.3524211347103119,
            "Y": 0.3936865031719208
        },
        {
            "Type": "leftEyeRight",
            "X": 0.4229775369167328,
            "Y": 0.3973258435726166
        },
        {
            "Type": "leftEyeUp",
            "X": 0.38467878103256226,
            "Y": 0.3836822807788849
        },
        {
            "Type": "leftEyeDown",
            "X": 0.38629674911499023,
            "Y": 0.40618783235549927
        },
        {
            "Type": "rightEyeLeft",
            "X": 0.5374732613563538,
            "Y": 0.39637991786003113
        },
        {
            "Type": "rightEyeRight",
            "X": 0.609208345413208,
            "Y": 0.391626238822937
        },
        {
            "Type": "rightEyeUp",
            "X": 0.5750962495803833,
            "Y": 0.3821527063846588
        },
        {
            "Type": "rightEyeDown",
            "X": 0.5740782618522644,
            "Y": 0.40471214056015015
        },
        {
            "Type": "noseLeft",
            "X": 0.4441811740398407,
            "Y": 0.5608476400375366
        },
        {
            "Type": "noseRight",
            "X": 0.5155643820762634,
            "Y": 0.5569332242012024
        },
        {
            "Type": "mouthUp",
            "X": 0.47968366742134094,
            "Y": 0.6176465749740601
        },
        {
            "Type": "mouthDown",
            "X": 0.4807897210121155,
            "Y": 0.690782368183136
        },
        {
            "Type": "leftPupil",
            "X": 0.38549351692199707,
            "Y": 0.3959200084209442
        },
```

```
                {
                    "Type": "rightPupil",
                    "X": 0.5773905515670776,
                    "Y": 0.394561767578125
                },
                {
                    "Type": "upperJawlineLeft",
                    "X": 0.27245330810546875,
                    "Y": 0.3902156949043274
                },
                {
                    "Type": "midJawlineLeft",
                    "X": 0.31561678647994995,
                    "Y": 0.6596118807792664
                },
                {
                    "Type": "chinBottom",
                    "X": 0.48385748267173767,
                    "Y": 0.8160444498062134
                },
                {
                    "Type": "midJawlineRight",
                    "X": 0.6625112891197205,
                    "Y": 0.656606137752533
                },
                {
                    "Type": "upperJawlineRight",
                    "X": 0.7042999863624573,
                    "Y": 0.3863988518714905
                }
            ],
            "MouthOpen": {
                "Confidence": 99.83820343017578,
                "Value": false
            },
            "Mustache": {
                "Confidence": 72.20288848876953,
                "Value": false
            },
            "Pose": {
                "Pitch": -4.970901966094971,
                "Roll": -1.4911699295043945,
                "Yaw": -10.983647346496582
            },
            "Quality": {
                "Brightness": 73.81391906738281,
                "Sharpness": 86.86019134521484
            },
            "Smile": {
                "Confidence": 99.93638610839844,
                "Value": false
            },
            "Sunglasses": {
                "Confidence": 99.81478881835938,
                "Value": false
            }
        }
    ]
}
```


請注意以下內容：

- `Pose` 資料描述偵測到之臉部的旋轉。您可以使用 `BoundingBox` 與 `Pose` 資料的組合，在您的應用程式所顯示的臉部周圍繪製週框方塊。

- Quality 描述臉部的亮度與銳利度。您可能會發現這有助於在影像之間比較臉部，並找出最佳臉部。
- 上述回應顯示服務可偵測到的所有臉部 landmarks、所有臉部屬性與情緒。若要在回應中取得所有項目，您必須將 attributes 參數的值指定為 ALL。根據預設，DetectFaces API 只會傳回下列五個臉部屬性：BoundingBox、Confidence、Pose、Quality 和 landmarks。傳回的預設特徵點如下：eyeLeft、eyeRight、nose、mouthLeft 和 mouthRight。
- 下圖顯示 DetectFaces API 操作傳回的臉部上臉部特徵點 (Landmarks) 的相對位置。


leftEyeBrowUp
leftEyeBrowRight
rightEyeBrowLeft
leftEyeBrowLeft
RightEyeUp
leftEyeUp
upperJawlineLeft
leftEyeRight
leftEyeLeft
leftEyeDown
leftPupil
rightEyeLeft
nose
noseLeft
noseRight
mouthUp
mouthLeft
mouthRight
midJawlineLeft
mouthDown

比較影像中的臉部

若要將來源影像中的臉部與目標影像中的每個臉部比較，請使用 CompareFaces (p. 273) 操作。

若要指定您想要在回應中傳回之符合項的最低信賴度，請在要求中使用 `similarityThreshold`。如需詳細資訊，請參閱 CompareFaces (p. 273)。

如果您提供的來源影像含有多個臉部，服務會偵測出最大的臉部，並用其與目標影像中偵測到的每個臉部比較。

您可以用影像位元組陣列的方式提供來源與目標影像 (Base64 編碼影像位元組)，或指定 Amazon S3 物件。在 AWS CLI 範例中，您會將兩張 JPEG 影像上傳至您的 Amazon S3 儲存貯體，並指定物件索引碼名稱。在其他範例中，您會從本機檔案系統載入兩個檔案，並以影像位元組陣列的方式加以輸入。

比較臉部

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (其具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` (僅 AWS CLI 範例) 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列程式碼範例來呼叫 `CompareFaces` 操作。

   Java

   此範例顯示有關來源和目標影像 (從本機檔案系統載入) 中相符臉孔的資訊，表面載入在從本機檔案系統。

   將 `sourceImage` 與 `targetImage` 的值取代為來源和目標影像的路徑和檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.BoundingBox;
import com.amazonaws.services.rekognition.model.CompareFacesMatch;
import com.amazonaws.services.rekognition.model.CompareFacesRequest;
import com.amazonaws.services.rekognition.model.CompareFacesResult;
import com.amazonaws.services.rekognition.model.ComparedFace;
import java.util.List;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.nio.ByteBuffer;
import com.amazonaws.util.IOUtils;

public class CompareFaces {

   public static void main(String[] args) throws Exception{
       Float similarityThreshold = 70F;
       String sourceImage = "source.jpg";
       String targetImage = "target.jpg";
       ByteBuffer sourceImageBytes=null;
       ByteBuffer targetImageBytes=null;
```

```
      AmazonRekognition rekognitionClient =
AmazonRekognitionClientBuilder.defaultClient();

      //Load source and target images and create input parameters
      try (InputStream inputStream = new FileInputStream(new File(sourceImage))) {
         sourceImageBytes = ByteBuffer.wrap(IOUtils.toByteArray(inputStream));
      }
      catch(Exception e)
      {
          System.out.println("Failed to load source image " + sourceImage);
          System.exit(1);
      }
      try (InputStream inputStream = new FileInputStream(new File(targetImage))) {
          targetImageBytes = ByteBuffer.wrap(IOUtils.toByteArray(inputStream));
      }
      catch(Exception e)
      {
          System.out.println("Failed to load target images: " + targetImage);
          System.exit(1);
      }

      Image source=new Image()
           .withBytes(sourceImageBytes);
      Image target=new Image()
           .withBytes(targetImageBytes);

      CompareFacesRequest request = new CompareFacesRequest()
              .withSourceImage(source)
              .withTargetImage(target)
              .withSimilarityThreshold(similarityThreshold);

      // Call operation
      CompareFacesResult
compareFacesResult=rekognitionClient.compareFaces(request);


      // Display results
      List <CompareFacesMatch> faceDetails = compareFacesResult.getFaceMatches();
      for (CompareFacesMatch match: faceDetails){
        ComparedFace face= match.getFace();
        BoundingBox position = face.getBoundingBox();
        System.out.println("Face at " + position.getLeft().toString()
              + " " + position.getTop()
              + " matches with " + match.getSimilarity().toString()
              + "% confidence.");

      }
      List<ComparedFace> uncompared = compareFacesResult.getUnmatchedFaces();

      System.out.println("There was " + uncompared.size()
           + " face(s) that did not match");
   }
}
```

AWS CLI

此範例顯示 compare-faces AWS CLI 操作的 JSON 輸出。

將 bucket-name 取代為 Amazon S3 儲存貯體的名稱，其中包含來源和目標影像。將 source.jpg 和 target.jpg 取代為來源和目標影像的檔案名稱。

```
aws rekognition compare-faces \
--source-image '{"S3Object":{"Bucket":"bucket-name","Name":"source.jpg"}}' \
```

```
--target-image '{"S3Object":{"Bucket":"bucket-name","Name":"target.jpg"}}'
```

Python

此範例顯示有關來源和目標影像 (從本機檔案系統載入) 中相符臉孔的資訊，表面載入在從本機檔案系統。

將 source_file 與 target_file 的值取代為來源和目標影像的路徑和檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def compare_faces(sourceFile, targetFile):

    client=boto3.client('rekognition')

    imageSource=open(sourceFile,'rb')
    imageTarget=open(targetFile,'rb')

    response=client.compare_faces(SimilarityThreshold=80,
                                  SourceImage={'Bytes': imageSource.read()},
                                  TargetImage={'Bytes': imageTarget.read()})

    for faceMatch in response['FaceMatches']:
        position = faceMatch['Face']['BoundingBox']
        similarity = str(faceMatch['Similarity'])
        print('The face at ' +
               str(position['Left']) + ' ' +
               str(position['Top']) +
               ' matches with ' + similarity + '% confidence')

    imageSource.close()
    imageTarget.close()
    return len(response['FaceMatches'])

def main():
    source_file='source'
    target_file='target'
    face_matches=compare_faces(source_file, target_file)
    print("Face matches: " + str(face_matches))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示有關來源和目標影像 (從本機檔案系統載入) 中相符臉孔的資訊，表面載入在從本機檔案系統。

將 sourceImage 與 targetImage 的值取代為來源和目標影像的路徑和檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.IO;
using Amazon.Rekognition;
```

```
using Amazon.Rekognition.Model;

public class CompareFaces
{
    public static void Example()
    {
        float similarityThreshold = 70F;
        String sourceImage = "source.jpg";
        String targetImage = "target.jpg";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        Amazon.Rekognition.Model.Image imageSource = new
 Amazon.Rekognition.Model.Image();
        try
        {
            using (FileStream fs = new FileStream(sourceImage, FileMode.Open,
 FileAccess.Read))
            {
                byte[] data = new byte[fs.Length];
                fs.Read(data, 0, (int)fs.Length);
                imageSource.Bytes = new MemoryStream(data);
            }
        }
        catch (Exception)
        {
            Console.WriteLine("Failed to load source image: " + sourceImage);
            return;
        }

        Amazon.Rekognition.Model.Image imageTarget = new
 Amazon.Rekognition.Model.Image();
        try
        {
            using (FileStream fs = new FileStream(targetImage, FileMode.Open,
 FileAccess.Read))
            {
                byte[] data = new byte[fs.Length];
                data = new byte[fs.Length];
                fs.Read(data, 0, (int)fs.Length);
                imageTarget.Bytes = new MemoryStream(data);
            }
        }
        catch (Exception)
        {
            Console.WriteLine("Failed to load target image: " + targetImage);
            return;
        }

        CompareFacesRequest compareFacesRequest = new CompareFacesRequest()
        {
            SourceImage = imageSource,
            TargetImage = imageTarget,
            SimilarityThreshold = similarityThreshold
        };

        // Call operation
        CompareFacesResponse compareFacesResponse =
 rekognitionClient.CompareFaces(compareFacesRequest);

        // Display results
        foreach(CompareFacesMatch match in compareFacesResponse.FaceMatches)
        {
            ComparedFace face = match.Face;
            BoundingBox position = face.BoundingBox;
            Console.WriteLine("Face at " + position.Left
```

```
                + " " + position.Top
                + " matches with " + face.Confidence
                + "% confidence.");
        }

        Console.WriteLine("There was " + compareFacesResponse.UnmatchedFaces.Count
 + " face(s) that did not match");
        Console.WriteLine("Source image rotation: " +
 compareFacesResponse.SourceImageOrientationCorrection);
        Console.WriteLine("Target image rotation: " +
 compareFacesResponse.TargetImageOrientationCorrection);
    }
}
```

Ruby

此範例顯示有關來源和目標影像 (從本機檔案系統載入) 中相符臉孔的資訊，表面載入在從本機檔案系統。

將 `photo_source` 與 `photo_target` 的值取代為來源和目標影像的路徑和檔案名稱。

```
 # Add to your Gemfile
  # gem 'aws-sdk-rekognition'
  require 'aws-sdk-rekognition'
  credentials = Aws::Credentials.new(
     ENV['AWS_ACCESS_KEY_ID'],
     ENV['AWS_SECRET_ACCESS_KEY']
  )
  bucket        = 'bucket' # the bucketname without s3://
  photo_source  = 'source.jpg'
  photo_target  = 'target.jpg'
  client   = Aws::Rekognition::Client.new credentials: credentials
  attrs = {
    source_image: {
      s3_object: {
        bucket: bucket,
        name: photo_source
      },
    },
    target_image: {
      s3_object: {
        bucket: bucket,
        name: photo_target
      },
    },
    similarity_threshold: 70
  }
  response = client.compare_faces attrs
  response.face_matches.each do |face_match|
    position   = face_match.face.bounding_box
    similarity = face_match.similarity
    puts "The face at: #{position.left}, #{position.top} matches with
#{similarity} % confidence"
  end
```

Node.js

此範例顯示有關來源和目標影像 (從本機檔案系統載入) 中相符臉孔的資訊，表面載入在從本機檔案系統。

將 `photo_source` 與 `photo_target` 的值取代為來源和目標影像的路徑和檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)


   const AWS = require('aws-sdk')
   const bucket        = 'bucket' // the bucketname without s3://
   const photo_source  = 'source.jpg'
   const photo_target  = 'target.jpg'
   const config = new AWS.Config({
     accessKeyId: process.env.AWS_ACCESS_KEY_ID,
     secretAccessKey: process.env.AWS_SECRET_ACCESS_KEY,
     region: process.env.AWS_REGION
   })
   const client = new AWS.Rekognition();
   const params = {
     SourceImage: {
       S3Object: {
         Bucket: bucket,
         Name: photo_source
       },
     },
     TargetImage: {
       S3Object: {
         Bucket: bucket,
         Name: photo_target
       },
     },
     SimilarityThreshold: 70
   }
   client.compareFaces(params, function(err, response) {
     if (err) {
       console.log(err, err.stack); // an error occurred
     } else {
       response.FaceMatches.forEach(data => {
         let position   = data.Face.BoundingBox
         let similarity = data.Similarity
         console.log(`The face at: ${position.Left}, ${position.Top} matches with
 ${similarity} % confidence`)
       }) // for response.faceDetails
     } // if
   });
```

CompareFaces 操作要求

CompareFaces 的輸入是影像。在此範例中，來源和目標影像從本機檔案系統載入。SimilarityThreshold 輸入參數指定比較臉孔必須符合的最低可信度應包含在回應中。如需詳細資訊，請參閱 使用影像 (p. 23)。

```
{
    "SourceImage": {
        "Bytes": "/9j/4AAQSk2Q==..."
    },
    "TargetImage": {
        "Bytes": "/9j/4O1Q==..."
    },
    "SimilarityThreshold": 70
}
```

CompareFaces 操作回應

在回應中，您會取得臉部相符項的陣列、來源臉部資訊、來源與目標影像方向，以及不相符臉部的陣列。對於目標影像中每個相符臉孔，該回應提供相似度分數 (臉孔與來源資料的相似程度) 和臉孔中繼資料。臉孔中繼資料包含資訊，例如相符臉孔的週框方塊，和一系列的臉部特徵點。不相符臉部的陣列包括臉部中繼資料。

在下列範例回應中，請注意以下事項：

- 臉部符合資訊 – 範例顯示在目標影像中找到一個相符的臉部。對於該相符的臉部，該回應會提供週框方塊與信賴度值 (Amazon Rekognition 對週框方塊包含臉部的信賴度)。`similarity` 分數 99.99 表示臉部非常相似。臉部符合資訊也包含特徵點位置的陣列。

  如果多個臉部相符，`faceMatches` 陣列會包含所有相符的臉部。
- 來源臉部資訊 – 該回應包含用於比較之來源影像中臉部的資訊，包括週框方塊與信賴度值。
- 不符臉部符合資訊 – 範例顯示 Amazon Rekognition 在目標影像中找到一個臉部與來源影像中所分析的臉部不相符。對於該臉部，回應會提供週框方塊與信賴度值，表示出 Amazon Rekognition 對週框方塊包含臉部的信賴度。臉部資訊也包含特徵點位置的陣列。

  如果 Amazon Rekognition 找到多個不相符的臉部，`UnmatchedFaces` 陣列會包含所有不相符的臉部。


```
{
    "FaceMatches": [{
        "Face": {
            "BoundingBox": {
                "Width": 0.5521978139877319,
                "Top": 0.1203877404332161,
                "Left": 0.23626373708248138,
                "Height": 0.3126954436302185
            },
            "Confidence": 99.98751068115234,
            "Pose": {
                "Yaw": -82.36799621582031,
                "Roll": -62.13221740722656,
                "Pitch": 0.8652129173278809
            },
            "Quality": {
                "Sharpness": 99.99880981445312,
                "Brightness": 54.49755096435547
            },
            "Landmarks": [{
                    "Y": 0.2996366024017334,
                    "X": 0.41685718297958374,
                    "Type": "eyeLeft"
                },
                {
                    "Y": 0.2658946216106415,
                    "X": 0.4414493441581726,
                    "Type": "eyeRight"
                },
                {
                    "Y": 0.3465650677680969,
                    "X": 0.48636093735694885,
                    "Type": "nose"
                },
                {
                    "Y": 0.30935320258140564,
                    "X": 0.6251809000968933,
                    "Type": "mouthLeft"
                },
                {
```

```
                    "Y": 0.26942989230155945,
                    "X": 0.6454493403434753,
                    "Type": "mouthRight"
                }
            ]
        },
        "Similarity": 100.0
    }],
    "SourceImageOrientationCorrection": "ROTATE_90",
    "TargetImageOrientationCorrection": "ROTATE_90",
    "UnmatchedFaces": [{
        "BoundingBox": {
            "Width": 0.4890109896659851,
            "Top": 0.6566604375839233,
            "Left": 0.10989011079072952,
            "Height": 0.278298944234848
        },
        "Confidence": 99.99992370605469,
        "Pose": {
            "Yaw": 51.51519012451172,
            "Roll": -110.32493591308594,
            "Pitch": -2.322134017944336
        },
        "Quality": {
            "Sharpness": 99.99671173095703,
            "Brightness": 57.23163986206055
        },
        "Landmarks": [{
                "Y": 0.8288310766220093,
                "X": 0.3133862614631653,
                "Type": "eyeLeft"
            },
            {
                "Y": 0.7632885575294495,
                "X": 0.28091415762901306,
                "Type": "eyeRight"
            },
            {
                "Y": 0.7417283654212952,
                "X": 0.3631140887737274,
                "Type": "nose"
            },
            {
                "Y": 0.8081989884376526,
                "X": 0.48565614223480225,
                "Type": "mouthLeft"
            },
            {
                "Y": 0.7548204660415649,
                "X": 0.46090251207351685,
                "Type": "mouthRight"
            }
        ]
    }],
    "SourceImageFace": {
        "BoundingBox": {
            "Width": 0.5521978139877319,
            "Top": 0.1203877404332161,
            "Left": 0.23626373708248138,
            "Height": 0.3126954436302185
        },
        "Confidence": 99.98751068115234
    }
}
```

偵測已存放影片中的臉部

Amazon Rekognition Video 可以偵測存放在 Amazon S3 儲存貯體之影片中的臉部，並提供下列資訊：

- 在影片中偵測到臉部的次數。
- 偵測到臉部時臉部在影片影格中的位置。
- 臉部特徵點，例如左眼的位置。

Amazon Rekognition Video 已儲存影片中的臉部偵測是一項非同步操作。若要開始在影片中偵測臉部，請呼叫 StartFaceDetection (p. 407)。Amazon Rekognition Video 將發佈影片分析的完成狀態至 Amazon Simple Notification Service (Amazon SNS) 主題。如果影片分析成功，您可以呼叫 GetFaceDetection (p. 348) 取得影片分析的結果。如需開始影片分析並取得結果的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

此程序會展開「使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)」中的程式碼，該作業使用 Amazon Simple Queue Service (Amazon SQS) 佇列來取得影片分析要求的完成狀態。

偵測存放在 Amazon S3 儲存貯體 (SDK) 之影片中的臉部

1. 執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。
2. 將下列程式碼新增至您在步驟 1 中建立的類別 `VideoDetect`。

Java

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)


private static void StartFaceDetection(String bucket, String video) throws
 Exception{

    NotificationChannel channel= new NotificationChannel()
            .withSNSTopicArn(snsTopicArn)
            .withRoleArn(roleArn);

    StartFaceDetectionRequest req = new StartFaceDetectionRequest()
            .withVideo(new Video()
                    .withS3Object(new S3Object()
                        .withBucket(bucket)
                        .withName(video)))
            .withNotificationChannel(channel);



    StartFaceDetectionResult startLabelDetectionResult =
 rek.startFaceDetection(req);
    startJobId=startLabelDetectionResult.getJobId();

}

private static void GetFaceDetectionResults() throws Exception{

    int maxResults=10;
    String paginationToken=null;
    GetFaceDetectionResult faceDetectionResult=null;

    do{
        if (faceDetectionResult !=null){
            paginationToken = faceDetectionResult.getNextToken();
```

```
        }

        faceDetectionResult = rek.getFaceDetection(new GetFaceDetectionRequest()
             .withJobId(startJobId)
             .withNextToken(paginationToken)
             .withMaxResults(maxResults));

        VideoMetadata videoMetaData=faceDetectionResult.getVideoMetadata();

        System.out.println("Format: " + videoMetaData.getFormat());
        System.out.println("Codec: " + videoMetaData.getCodec());
        System.out.println("Duration: " + videoMetaData.getDurationMillis());
        System.out.println("FrameRate: " + videoMetaData.getFrameRate());


        //Show faces, confidence and detection times
        List<FaceDetection> faces= faceDetectionResult.getFaces();

        for (FaceDetection face: faces) {
            long seconds=face.getTimestamp()/1000;
            System.out.print("Sec: " + Long.toString(seconds) + " ");
            System.out.println(face.getFace().toString());
            System.out.println();
        }
    } while (faceDetectionResult !=null && faceDetectionResult.getNextToken() !=
 null);


}
```

在函數 `main` 中，將下行:

```
        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();
```

取代為:

```
        StartFaceDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetFaceDetectionResults();
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # ============== Faces===============
    def StartFaceDetection(self):
        response=self.rek.start_face_detection(Video={'S3Object': {'Bucket':
 self.bucket, 'Name': self.video}},
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
 self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)

    def GetFaceDetectionResults(self):
        maxResults = 10
```

```
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_face_detection(JobId=self.startJobId,
                                                   MaxResults=maxResults,
                                                   NextToken=paginationToken)

            print('Codec: ' + response['VideoMetadata']['Codec'])
            print('Duration: ' + str(response['VideoMetadata']['DurationMillis']))
            print('Format: ' + response['VideoMetadata']['Format'])
            print('Frame rate: ' + str(response['VideoMetadata']['FrameRate']))
            print()

            for faceDetection in response['Faces']:
                print('Face: ' + str(faceDetection['Face']))
                print('Confidence: ' + str(faceDetection['Face']['Confidence']))
                print('Timestamp: ' + str(faceDetection['Timestamp']))
                print()

            if 'NextToken' in response:
                paginationToken = response['NextToken']
            else:
                finished = True
```

在函數 `main` 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    analyzer.StartFaceDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetFaceDetectionResults()
```

Note

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的視訊範例，要取代的函數名稱會不同。

3. 執行程式碼。影片中偵測到之臉部的資訊便會顯示。

GetFaceDetection 操作回應

`GetFaceDetection` 會傳回陣列 (`Faces`)，其中包含視訊中偵測到之臉孔的相關資訊。每當在影片中偵測到臉部時，就會有一個陣列元素 FaceDetection (p. 462)。傳回的陣列元素會依從影片開頭起的時間 (以毫秒為單位) 順序排序。

下列範例是 `GetFaceDetection` 的部分 JSON 回應。在回應中，請注意下列事項：

- 臉部資訊 – `FaceDetection`陣列元素包含偵測到之臉部的資訊 (FaceDetail (p. 459))，以及在影片中偵測到臉部的時間 (`Timestamp`)。
- 分頁資訊 – 該範例顯示一頁臉部偵測資訊。您可以在 `GetFaceDetection` 的 `MaxResults` 輸入參數中指定要傳回的人物元素數目。如果結果數目超過 `MaxResults`，`GetFaceDetection` 會傳回用來取得下一頁結果的字符 (`NextToken`)。如需詳細資訊，請參閱 取得 Amazon Rekognition Video 分析結果 (p. 53)。

- 視訊資訊 – 回應包含 GetFaceDetection 所傳回之每頁資訊中視訊格式 (VideoMetadata) 的相關資訊。


```
{
    "Faces": [
        {
            "Face": {
                "BoundingBox": {
                    "Height": 0.23000000417232513,
                    "Left": 0.42500001192092896,
                    "Top": 0.16333332657814026,
                    "Width": 0.12937499582767487
                },
                "Confidence": 99.97504425048828,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.46415066719055176,
                        "Y": 0.2572723925113678
                    },
                    {
                        "Type": "eyeRight",
                        "X": 0.5068183541297913,
                        "Y": 0.23705792427062988
                    },
                    {
                        "Type": "nose",
                        "X": 0.49765899777412415,
                        "Y": 0.28383663296699524
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.487221896648407,
                        "Y": 0.3452930748462677
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.5142884850502014,
                        "Y": 0.33167609572410583
                    }
                ],
                "Pose": {
                    "Pitch": 15.966927528381348,
                    "Roll": -15.547388076782227,
                    "Yaw": 11.34195613861084
                },
                "Quality": {
                    "Brightness": 44.80223083496094,
                    "Sharpness": 99.95819854736328
                }
            },
            "Timestamp": 0
        },
        {
            "Face": {
                "BoundingBox": {
                    "Height": 0.20000000298023224,
                    "Left": 0.029999999329447746,
                    "Top": 0.2199999988079071,
                    "Width": 0.11249999701976776
                },
                "Confidence": 99.85971069335938,
                "Landmarks": [
                    {
```

```
                    "Type": "eyeLeft",
                    "X": 0.06842322647571564,
                    "Y": 0.3010137975215912
                },
                {
                    "Type": "eyeRight",
                    "X": 0.10543643683195114,
                    "Y": 0.29697132110595703
                },
                {
                    "Type": "nose",
                    "X": 0.09569807350635529,
                    "Y": 0.33701086044311523
                },
                {
                    "Type": "mouthLeft",
                    "X": 0.0732642263174057,
                    "Y": 0.3757539987564087
                },
                {
                    "Type": "mouthRight",
                    "X": 0.10589495301246643,
                    "Y": 0.3722417950630188
                }
            ],
            "Pose": {
                "Pitch": -0.5589138865470886,
                "Roll": -5.1093974113464355,
                "Yaw": 18.69594955444336
            },
            "Quality": {
                "Brightness": 43.052337646484375,
                "Sharpness": 99.68138885498047
            }
        },
        "Timestamp": 0
    },
    {
        "Face": {
            "BoundingBox": {
                "Height": 0.2177777737379074,
                "Left": 0.7593749761581421,
                "Top": 0.13333334028720856,
                "Width": 0.12250000238418579
            },
            "Confidence": 99.63436889648438,
            "Landmarks": [
                {
                    "Type": "eyeLeft",
                    "X": 0.8005779385566711,
                    "Y": 0.20915353298187256
                },
                {
                    "Type": "eyeRight",
                    "X": 0.8391435146331787,
                    "Y": 0.21049551665782928
                },
                {
                    "Type": "nose",
                    "X": 0.8191410899162292,
                    "Y": 0.2523227035999298
                },
                {
                    "Type": "mouthLeft",
                    "X": 0.8093273043632507,
                    "Y": 0.29053622484207153
```

```
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.8366993069648743,
                        "Y": 0.29101791977882385
                    }
                ],
                "Pose": {
                    "Pitch": 3.165884017944336,
                    "Roll": 1.4182015657424927,
                    "Yaw": -11.151537895202637
                },
                "Quality": {
                    "Brightness": 28.910892486572266,
                    "Sharpness": 97.61507415771484
                }
            },
            "Timestamp": 0
        }.......

    ],
    "JobStatus": "SUCCEEDED",
    "NextToken": "i7fj5XPV/
fwviXqz0eag9Ow332Jd5G8ZGWf7hooirD/6V1qFmjKFOQZ6QPWUiqv29HbyuhMNqQ==",
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 67301,
        "FileExtension": "mp4",
        "Format": "QuickTime / MOV",
        "FrameHeight": 1080,
        "FrameRate": 29.970029830932617,
        "FrameWidth": 1920
    }
}
```

在集合中搜尋臉部

Amazon Rekognition 可將偵測到的臉部相關資訊儲存於伺服器端的容器中，也就是集合。您可以使用儲存在集合中的臉部資訊在影像、儲存的影片與串流視訊中搜尋已知的臉部。Amazon Rekognition 支援 IndexFaces (p. 371) 操作。您可以使用此操作來偵測影像中臉孔，並將偵測到的臉部特徵的資訊保留於集合中。此為以儲存體為基礎的 API 操作範例，因服務將在伺服器中保留資訊。

若要儲存臉部資訊，您必須先在帳戶中其中一個 AWS 區域中建立 (CreateCollection (p. 279)) 臉部集合。當您呼叫 `IndexFaces` 操作時可指定此臉部集合。在建立臉孔集合並儲存所有臉孔的臉部特徵資訊後，即可搜尋集合來比對臉孔。若要在影像中搜尋臉部，請呼叫 SearchFacesByImage (p. 395)。若要在儲存的影片中搜尋臉部，請呼叫 StartFaceSearch (p. 411)。若要在串流視訊中搜尋臉部，請呼叫 CreateStreamProcessor (p. 289)。

> Note
>
> 此服務不會保留實際的影像位元組。反之，底層偵測演算法將先偵測輸入影像中的臉孔，將臉孔特徵擷取至每個臉孔的特徵向量中，接著存放於集合。Amazon Rekognition 將在執行臉部比對時使用這些特徵向量。

您可以在各種不同情況中使用集合。例如，您可以建立一個臉部集合並使用 `IndexFaces` 操作來儲存掃描到的名牌影像。當員工進入大樓時，將拍攝員工臉部的影像並傳送至 `SearchFacesByImage` 操作。如果臉部比對的相似性分數達到設定的目標 (例如 99%)，即可驗證該員工身分。

管理集合

臉部集合是主要的 Amazon Rekognition 資源，每個您所建立的臉部集合都有一組專有的 Amazon Resource Name (ARN)。在帳戶中的特定 AWS 區域建立每個臉部集合。當建立集合時，將採用最新版的臉部偵測模型版本。如需詳細資訊，請參閱 模型版本控制 (p. 9)。

您可以對集合執行以下管理操作。

- 用 the section called “CreateCollection” (p. 279) 建立一個集合。如需詳細資訊，請參閱 建立集合 (p. 150)。
- 用 the section called “ListCollections” (p. 379) 列出可用的集合。如需詳細資訊，請參閱 列出集合 (p. 153)。
- 用 the section called “DescribeCollection” (p. 302) 描述一個集合。如需詳細資訊，請參閱 描述集合 (p. 156)。
- 用 the section called “DeleteCollection” (p. 292) 刪除一個集合。如需詳細資訊，請參閱 刪除集合 (p. 159)。

管理集合中的臉孔

在您建立臉部集合時，您可以將臉部存放在其中。Amazon Rekognition 提供下列在集合中管理臉部的操作。

- the section called “IndexFaces” (p. 371) 操作可偵測輸入影像 (JPEG 或 PNG) 中的臉孔，並將它們新增到指定的臉部集合。對於在影像中偵測到的每個臉部傳回唯一的臉部 ID。存入臉部之後，便可以搜尋臉部集合來比對臉部。如需詳細資訊，請參閱 新增臉部到集合 (p. 162)。
- the section called “ListFaces” (p. 382) 操作列出集合中的臉部。如需詳細資訊，請參閱 新增臉部到集合 (p. 162)。
- the section called “DeleteFaces” (p. 294) 操作會刪除集合中的臉部。如需詳細資訊，請參閱 刪除集合中的臉部 (p. 176)。

使用 IndexFaces 的指導

以下是在常見案例中使用 `IndexFaces` 的指導。

關鍵或公共安全應用

- 呼叫具有多張影像的 IndexFaces (p. 371)，其中每一張影像只包含一個臉部，且會將傳回的臉部 ID 與影像主體的識別符建立關聯。
- 您可以在製作索引之前使用 DetectFaces (p. 320)，以確認影像中只有一個臉部。如果偵測到多個臉部，請在檢閱只有一個臉部之後重新提交影像。這可防止不小心製作多個臉部的索引，以及將它們與同一個人建立關聯。

照片共享和社交媒體應用程式

- 您應該呼叫 `IndexFaces`，而不限制在使用案例中包含多個臉部的影像，例如家庭相冊。在這類情況下，您需要識別每張照片中的每一個人，並使用該資訊依其中呈現的人員將照片分組。

一般用法

- 製作同一個人之不同影像 (尤其是具有不同臉部屬性 (臉部姿勢、臉部毛髮等等)) 的索引，以改善比對品質。
- 包含檢閱程序，以便可以使用正確的臉部識別符來為失敗的比對製作索引，來提高後續的臉部比對能力。
- 如需影像品質的相關資訊，請參閱 臉部比較輸入影像的建議 (p. 103)。

在集合中搜尋臉部

在建立臉部集合並儲存所有臉部的臉部特徵資訊後，即可搜尋臉部集合來比對臉部。有了 Amazon Rekognition，您可以在符合的集合中搜尋臉部：

- 提供的臉部 ID (the section called “SearchFaces” (p. 392))。如需詳細資訊，請參閱 使用臉部 ID 來搜尋臉孔 (p. 179)。
- 在提供的影像 (the section called “SearchFacesByImage” (p. 395)) 中最大的臉孔。如需詳細資訊，請參閱 使用影像來搜尋臉孔 (p. 183)。
- 已存放影片中的臉孔。如需詳細資訊，請參閱 在儲存的視訊中搜尋臉部 (p. 187)。
- 串流影片中的臉孔。如需詳細資訊，請參閱 處理串流影片 (p. 76)。

`CompareFaces` 操作與臉部搜尋操作的差異如下所示：

- `CompareFaces` 操作將原始影像中的臉孔與目標影像中的臉孔進行比較。這項比對的範圍僅限於在目標影像中所偵測到的臉孔。如需詳細資訊，請參閱 比較影像中的臉部 (p. 133)。
- `SearchFaces` 與 `SearchFacesByImage` 將臉孔 (可由 `FaceId` 或輸入影像辨識) 與指定的臉部集合中的所有臉孔進行比對。因此，這種搜尋方法範圍更廣。此外，若是已儲存在臉部集合中的臉孔，相關臉部特徵資訊將保留，因此您可以多次搜尋相符的臉部。

使用相似性臨界值來比對臉部

我們提供相似閾值做為輸入參數，讓您控制所有搜尋結果操作 (CompareFaces (p. 273)、SearchFaces (p. 392) 和 SearchFacesByImage (p. 395))。

SearchFaces 和 SearchFacesByImage FaceMatchThreshold 相似性閾值輸入屬性控制的返回結果數量，是根據匹配的臉部相似度。(這屬性是適用於 CompareFaces 的 SimilarityThreshold。) 含低於臨界值之 Similarity 回應屬性值的回應不會傳回。請務必利用此閾值校正您的使用案例，因為其可決定您的相符結果包含多少錯誤肯定。這可控制您搜尋結果的回呼—閾值越低，回呼越高。

所有機器學習系統是機率性的。您應根據使用案例，在設定適當的相似性閾值時使用判斷。例如，如果您想要建立照片應用程式來辨識樣貌相似度家庭成員，您可以選擇較低的閾值 (例如 80%)。另一方面，對於許多強制執行法律使用案例，建議您使用 99% 或更高的臨界值，以降低意外辨識錯誤。

除 FaceMatchThreshold 外，您還可以使用 Similarity 回應屬性做為降低意外辨識錯誤的方法。例如，您可以選擇使用較低閾值 (例如 80%) 來傳回更多結果。接著您可以使用回應屬性 Similarity (相似性百分比)，來縮小選擇範圍並篩選應用程式中適當的回應。同樣地，使用更高的相似度 (如 99% 或更高) 將會降低辨識錯誤的風險。

涉及公共安全的使用案例

除了感應器、輸入影像和影片的最佳實務 (p. 103)和使用 IndexFaces 的指導 (p. 148)中列出的建議之外，在涉及公共安全的使用案例中部署臉部偵測和比較系統時，您應該使用以下最佳實務。首先，您應該使用 99% 或更高的可信度閾值，以降低錯誤和誤報。其次，您應該包含人工檢閱，以驗證從臉部偵測或比較系統收到的結果，而且若沒有額外的人工檢閱，您不應該根據系統輸出做出決策。臉部偵測和比較系統應該做為一個工具，以協助縮小範圍，並允許人們迅速檢閱和考慮選項。第三，我們建議您在這些使用案例中使用臉部偵測或比較系統時應該透明，包括儘可能將使用這些系統的情形通知終端使用者和主體、獲得此類使用的同意，以及提供一個機制，讓最終使用者和主體可以提供意見回饋以改善系統。

如果您是執法部門，在進行刑事調查時需用到 Amazon Rekognition 臉部比較功能，則您必須遵守 AWS 服務條款中列出的要求。這包含下列項目：

- 由經過適當培訓的人員審查所有決定，採取可能影響公民自由或同等人權的行動。
- 培訓人員負責使用臉部辨識系統。
- 公開您使用臉部辨識系統的方式。
- 未經獨立審查或若非緊急情況，不得使用 Amazon Rekognition 持續監控一對象。

在所有情況下，應在具有其他強而有力的證據下檢視臉部比較配對，而不應將其做為採取行動的唯一決定因素。不過，如果將臉部比較用於非執法部門的案例 (例如，解除電話的鎖定或在進入安全、私有辦公室建築物時驗證員工的身分)，這些決策就不需要手動稽核，因為不會影響一個人的公民權利。

如果您打算針對涉及公共安全的使用案例使用臉部偵測或臉部比較系統，則應該採用先前提及的最佳實務。此外，您應該參考已發佈的資源，了解如何使用臉部比較。其中包括司法部司法援助局所提供的 Face Recognition Policy Development Template For Use In Criminal Intelligence and Investigative Activities。此範本提供多種臉部比較和生物相關資源，旨在提供執法和公共安全部門一個框架，以開發遵守適用法律、降低隱私風險，並建立實體責任和監督的臉部比較政策。其他資源包括美國國家電信暨資訊管理局提供的 Best Privacy Practices for Commercial Use of Facial Recognition，以及美國聯邦貿易委員會全體提供的 Best Practices for Common Uses of Facial Recognition。可能會在未來開發並發佈其他資源，而且您應該持續對此重要主題自我教育。

提醒您，您在使用 AWS 服務時必須遵守所有適用的法律，而且若使用的方式違反其他人的權利或可能有害其他人的話，您可能會無法使用任何 AWS 服務。這意味著您不得以非法歧視個人或侵犯個人正當程序、隱私或公民自由的方式將 AWS 服務用於公共安全使用案例。您應該視需要取得適當的法律建議，以查閱關於使用案例的任何法律要求或問題。

使用Amazon Rekognition 以協助公共安全

Amazon Rekognition 可協助公共安全與執法部門要求案例—例如尋找失蹤兒童、打擊人口販賣或預防犯罪。在公共安全與執法部門要求案例中，請考慮以下：

- 使用 Amazon Rekognition 做為尋找可能配對的第一步。透過 Amazon Rekognition 臉部操作的回應可讓您快速地取得一組潛在配對，可供您進一步考量。
- 請勿使用 Amazon Rekognition 回應，來在需要人類分析的情境中進行自主決策。如果您是執法部門，需使用 Amazon Rekognition 協助識別與刑事調查相關的對象，且所採取的身分類型行動可能影響該人的公民自由或同等人權，則負責決定採取行動的人員必須針對獨立驗證身分證明接受適當培訓。
- 將 99% 相似度的臨界值用於需要高準確臉部相似度比對的案例。此類範例即是進入建築物時的存取驗證。
- 當有公民權利這方面的考量時，例如包含執法部門的使用案例，請使用 99% 或更高的臨界值，以及採用人工的方式檢閱臉部比較預測結果，以確保未侵犯人員的公民權利。
- 將低於 99% 相似度臨界值用於受益於大型潛在相符集合的案例中。此類範例其中之一便是尋找失蹤人口。如有需要，您可以使用相似度回應屬性，來判斷與您希望辨識之人員的潛在相符的相似程度。
- 擬訂計畫，來判斷 Amazon Rekognition 所傳回之相符臉孔的對錯。例如，在您使用 IndexFaces (p. 371) 操作建置索引時，透過使用相同人員的多個影像來改善相符程度。如需詳細資訊，請參閱 使用 IndexFaces 的指導 (p. 148)。

在其他使用案例 (例如社交媒體)，我們建議您使用最佳判斷，以評估 Amazon Rekognition 結果是否需要人工檢閱。此外，根據應用程式的要求，相似度臨界值可能較低。

建立集合

您可以使用 CreateCollection (p. 279) 操作來建立集合。

如需更多詳細資訊，請參閱 管理集合 (p. 147)。

若要建立集合 (SDK)

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 `AmazonRekognitionFullAccess` 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 `CreateCollection` 操作。

   Java

   以下範例建立集合和顯示其 Amazon Resource Name (ARN)。

   將 `collectionId` 的值變更為您要建立集合的名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.CreateCollectionRequest;
import com.amazonaws.services.rekognition.model.CreateCollectionResult;


public class CreateCollection {

   public static void main(String[] args) throws Exception {
```

```
      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();


      String collectionId = "MyCollection";
            System.out.println("Creating collection: " +
         collectionId );

        CreateCollectionRequest request = new CreateCollectionRequest()
                    .withCollectionId(collectionId);

      CreateCollectionResult createCollectionResult =
 rekognitionClient.createCollection(request);
      System.out.println("CollectionArn : " +
         createCollectionResult.getCollectionArn());
      System.out.println("Status code : " +
         createCollectionResult.getStatusCode().toString());

   }

}
```

AWS CLI

此 AWS CLI 命令顯示 `create-collection` CLI 操作的 JSON 輸出。

將 `collection-id` 的值取代為您要建立集合的名稱。

```
aws rekognition create-collection \
    --collection-id "collectionname"
```

Python

以下範例建立集合和顯示其 Amazon Resource Name (ARN)。

將 `collection_id` 的值變更為您要建立集合的名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def create_collection(collection_id):

    client=boto3.client('rekognition')

    #Create a collection
    print('Creating collection:' + collection_id)
    response=client.create_collection(CollectionId=collection_id)
    print('Collection ARN: ' + response['CollectionArn'])
    print('Status code: ' + str(response['StatusCode']))
    print('Done...')

def main():
    collection_id='Collection'
    create_collection(collection_id)

if __name__ == "__main__":
    main()
```

.NET

以下範例建立集合和顯示其 Amazon Resource Name (ARN)。

將 `collectionId` 的值變更為您要建立集合的名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class CreateCollection
{
    public static void Example()
    {
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        String collectionId = "MyCollection";
        Console.WriteLine("Creating collection: " + collectionId);

        CreateCollectionRequest createCollectionRequest = new
 CreateCollectionRequest()
        {
            CollectionId = collectionId
        };

        CreateCollectionResponse createCollectionResponse =
 rekognitionClient.CreateCollection(createCollectionRequest);
        Console.WriteLine("CollectionArn : " +
 createCollectionResponse.CollectionArn);
        Console.WriteLine("Status code : " + createCollectionResponse.StatusCode);

    }
}
```

CreateCollection 操作要求

`CreationCollection` 的輸入是您要建立的集合的名稱。

```
{
    "CollectionId": "MyCollection"
}
```

CreateCollection 操作回應

Amazon Rekognition 建立集合並傳回新建立的集合之 Amazon Resource Name (ARN)。

```
{
   "CollectionArn": "aws:rekognition:us-east-1:acct-id:collection/examplecollection",
   "StatusCode": 200
}
```

列出集合

您可以使用 ListCollections (p. 379) 操作列出您正在使用的區域中的集合。

如需詳細資訊，請參閱 管理集合 (p. 147)。

若要列出集合 (SDK)

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 AmazonRekognitionFullAccess 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 ListCollections 操作。

   Java

   以下範例列出目前區域中的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;

import java.util.List;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.ListCollectionsRequest;
import com.amazonaws.services.rekognition.model.ListCollectionsResult;

public class ListCollections {

   public static void main(String[] args) throws Exception {


      AmazonRekognition amazonRekognition =
 AmazonRekognitionClientBuilder.defaultClient();


      System.out.println("Listing collections");
      int limit = 10;
      ListCollectionsResult listCollectionsResult = null;
      String paginationToken = null;
      do {
         if (listCollectionsResult != null) {
            paginationToken = listCollectionsResult.getNextToken();
         }
         ListCollectionsRequest listCollectionsRequest = new
 ListCollectionsRequest()
                 .withMaxResults(limit)
                 .withNextToken(paginationToken);

 listCollectionsResult=amazonRekognition.listCollections(listCollectionsRequest);

         List < String > collectionIds = listCollectionsResult.getCollectionIds();
         for (String resultId: collectionIds) {
            System.out.println(resultId);
         }
      } while (listCollectionsResult != null &&
 listCollectionsResult.getNextToken() !=
```

```
         null);

   }
}
```

AWS CLI

此 AWS CLI 命令顯示 `list-collections` CLI 操作的 JSON 輸出。

```
aws rekognition list-collections
```

Python

以下範例列出目前區域中的集合。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def list_collections():

    max_results=2

    client=boto3.client('rekognition')

    #Display all the collections
    print('Displaying collections...')
    response=client.list_collections(MaxResults=max_results)
    collection_count=0
    done=False

    while done==False:
        collections=response['CollectionIds']

        for collection in collections:
            print (collection)
            collection_count+=1
        if 'NextToken' in response:
            nextToken=response['NextToken']

 response=client.list_collections(NextToken=nextToken,MaxResults=max_results)

        else:
            done=True

    return collection_count

def main():

    collection_count=list_collections()
    print("collections: " + str(collection_count))
if __name__ == "__main__":
    main()
```

.NET

以下範例列出目前區域中的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
```

```
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class ListCollections
{
    public static void Example()
    {
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        Console.WriteLine("Listing collections");
        int limit = 10;

        ListCollectionsResponse listCollectionsResponse = null;
        String paginationToken = null;
        do
        {
            if (listCollectionsResponse != null)
                paginationToken = listCollectionsResponse.NextToken;

            ListCollectionsRequest listCollectionsRequest = new
 ListCollectionsRequest()
            {
                MaxResults = limit,
                NextToken = paginationToken
            };

            listCollectionsResponse =
 rekognitionClient.ListCollections(listCollectionsRequest);

            foreach (String resultId in listCollectionsResponse.CollectionIds)
                Console.WriteLine(resultId);
        } while (listCollectionsResponse != null &&
 listCollectionsResponse.NextToken != null);
    }
}
```

ListCollections 操作要求

`ListCollections` 的輸入是要傳回的最大集合數。

```
{
    "MaxResults": 2
}
```

如果回應的臉孔多於 `MaxResults` 請求的臉孔，則會傳回一個字符，您可以在後續的 `ListCollections` 呼叫中使用該字符獲取下一組結果。例如：

```
{
    "NextToken": "MGYZLAHX1T5a....",
    "MaxResults": 2
}
```

ListCollections 操作回應

Amazon Rekognition 傳回一系列 (`CollectionIds`) 集合。個別陣列 (`FaceModelversions`) 提供用於分析每個集合中的臉部的臉部模型的版本。例如，在下列 JSON 回應中，集合 `MyCollection` 使用臉部模型

的 2.0 版分析臉部。集合 AnotherCollection 使用臉部模型的 3.0 版本。如需詳細資訊，請參閱 模型版本控制 (p. 9)。

NextToken 是在後續的 ListCollections 呼叫中用於取得下一組結果的字符。

```
{
    "CollectionIds": [
        "MyCollection",
        "AnotherCollection"
    ],
    "FaceModelVersions": [
        "2.0",
        "3.0"
    ],
    "NextToken": "MGYZLAHX1T5a...."
}
```

描述集合

您可以使用 DescribeCollection (p. 302) 操作，來取得下列集合的相關資訊：

- 編制為集合索引的臉部數。
- 集合用來進行臉部偵測的模型版本。如需詳細資訊，請參閱 the section called “模型版本控制” (p. 9)。
- 集合的 Amazon Resource Name (ARN)。
- 集合的建立日期和時間。

描述集合 (SDK)

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 AmazonRekognitionFullAccess 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 DescribeCollection 操作。

   Java

   此範例描述一個集合。

   將 collectionId 的值變更為所需集合的 ID。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package com.amazonaws.samples;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.DescribeCollectionRequest;
import com.amazonaws.services.rekognition.model.DescribeCollectionResult;


public class DescribeCollection {

   public static void main(String[] args) throws Exception {
```

```
      String collectionId = "CollectionID";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();


      System.out.println("Describing collection: " +
         collectionId );


        DescribeCollectionRequest request = new DescribeCollectionRequest()
                    .withCollectionId(collectionId);

      DescribeCollectionResult describeCollectionResult =
 rekognitionClient.describeCollection(request);
      System.out.println("Collection Arn : " +
         describeCollectionResult.getCollectionARN());
      System.out.println("Face count : " +
         describeCollectionResult.getFaceCount().toString());
      System.out.println("Face model version : " +
         describeCollectionResult.getFaceModelVersion());
      System.out.println("Created : " +
         describeCollectionResult.getCreationTimestamp().toString());

   }

}
```

AWS CLI

此 AWS CLI 命令顯示 `describe-collection` CLI 操作的 JSON 輸出。將 `collection-id` 的值變更為所需集合的 ID。

```
aws rekognition describe-collection --collection-id collectionname
```

Python

此範例描述一個集合。

將 `collection_id` 的值變更為所需集合的 ID。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
from botocore.exceptions import ClientError

def describe_collection(collection_id):

    print('Attempting to describe collection ' + collection_id)
    client=boto3.client('rekognition')

    try:
        response=client.describe_collection(CollectionId=collection_id)
        print("Collection Arn: "  + response['CollectionARN'])
        print("Face Count: "  + str(response['FaceCount']))
        print("Face Model Version: "  + response['FaceModelVersion'])
        print("Timestamp: "  + str(response['CreationTimestamp']))
```

```
    except ClientError as e:
        if e.response['Error']['Code'] == 'ResourceNotFoundException':
            print ('The collection ' + collection_id + ' was not found ')
        else:
            print ('Error other than Not Found occurred: ' + e.response['Error']
['Message'])
    print('Done...')


def main():
    collection_id='MyCollection'
    describe_collection(collection_id)

if __name__ == "__main__":
    main()
```

.NET

此範例描述一個集合。

將 `collectionId` 的值變更為所需集合的 ID。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DescribeCollection
{
    public static void Example()
    {
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        String collectionId = "CollectionID";
        Console.WriteLine("Describing collection: " + collectionId);

        DescribeCollectionRequest describeCollectionRequest = new
 DescribeCollectionRequest()
        {
            CollectionId = collectionId
        };

        DescribeCollectionResponse describeCollectionResponse =
 rekognitionClient.DescribeCollection(describeCollectionRequest);
        Console.WriteLine("Collection ARN: " +
 describeCollectionResponse.CollectionARN);
        Console.WriteLine("Face count: " + describeCollectionResponse.FaceCount);
        Console.WriteLine("Face model version: " +
 describeCollectionResponse.FaceModelVersion);
        Console.WriteLine("Created: " +
 describeCollectionResponse.CreationTimestamp);
    }
}
```

DescribeCollection 操作要求

DescribeCollection 的輸入是所需集合的 ID，如以下 JSON 範例所示。

```
{
    "CollectionId": "MyCollection"
}
```

DescribeCollection 操作回應

回應包括：

- 編制為集合索引的臉部數，FaceCount。
- 用來偵測臉部的臉部模型版本，FaceModelVersion。
- 集合 Amazon Resource Name，CollectionARN。
- 集合的建立日期和時間，CreationTimestamp。CreationTimestamp 的值是 Unix epoch 時間到集合建立時所經的毫秒數。Unix epoch 時間為 1970 年 1 月 1 日星期四 00:00:00 國際標準時間 (UTC)。如需詳細資訊，請參閱 Unix 時間。

```
{
    "CollectionARN": "arn:aws:rekognition:us-east-1:nnnnnnnnnnnn:collection/MyCollection",
    "CreationTimestamp": 1.533422155042E9,
    "FaceCount": 200,
    "FaceModelVersion": "1.0"
}
```

刪除集合

您可以使用 DeleteCollection (p. 292) 操作來刪除集合。

如需詳細資訊，請參閱 管理集合 (p. 147)。

若要刪除集合 (SDK)

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 AmazonRekognitionFullAccess 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 DeleteCollection 操作。

   Java

   此範例刪除一個集合。

   將值 collectionId 變更為您要刪除的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
```

```
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.DeleteCollectionRequest;
import com.amazonaws.services.rekognition.model.DeleteCollectionResult;


public class DeleteCollection {

   public static void main(String[] args) throws Exception {

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      String collectionId = "MyCollection";

      System.out.println("Deleting collections");

      DeleteCollectionRequest request = new DeleteCollectionRequest()
         .withCollectionId(collectionId);
      DeleteCollectionResult deleteCollectionResult =
 rekognitionClient.deleteCollection(request);

      System.out.println(collectionId + ": " +
 deleteCollectionResult.getStatusCode()
         .toString());

   }

}
```

AWS CLI

此 AWS CLI 命令顯示 `delete-collection` CLI 操作的 JSON 輸出。將 `collection-id` 的值取代為您要刪除的集合的名稱。

```
aws rekognition delete-collection \
    --collection-id "collectionname"
```

Python

此範例刪除一個集合。

將值 `collection_id` 變更為您要刪除的集合。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
from botocore.exceptions import ClientError
from os import environ

def delete_collection(collection_id):


    print('Attempting to delete collection ' + collection_id)
    client=boto3.client('rekognition')
    status_code=0
    try:
        response=client.delete_collection(CollectionId=collection_id)
```

```
        status_code=response['StatusCode']

    except ClientError as e:
        if e.response['Error']['Code'] == 'ResourceNotFoundException':
            print ('The collection ' + collection_id + ' was not found ')
        else:
            print ('Error other than Not Found occurred: ' + e.response['Error']
['Message'])
        status_code=e.response['ResponseMetadata']['HTTPStatusCode']
    return(status_code)


def main():
    collection_id='UnitTestCollection'
    status_code=delete_collection(collection_id)
    print('Status code: ' + str(status_code))

if __name__ == "__main__":
    main()
```

.NET

此範例刪除一個集合。

將值 `collectionId` 變更為您要刪除的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DeleteCollection
{
    public static void Example()
    {
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        String collectionId = "MyCollection";
        Console.WriteLine("Deleting collection: " + collectionId);

        DeleteCollectionRequest deleteCollectionRequest = new
 DeleteCollectionRequest()
        {
            CollectionId = collectionId
        };

        DeleteCollectionResponse deleteCollectionResponse =
 rekognitionClient.DeleteCollection(deleteCollectionRequest);
        Console.WriteLine(collectionId + ": " +
 deleteCollectionResponse.StatusCode);
    }
}
```

DeleteCollection 操作要求

`DeleteCollection` 的輸入是要刪除的集合 ID，如以下 JSON 範例所示。

```
{
    "CollectionId": "MyCollection"
}
```

DeleteCollection 操作回應

DeleteCollection 回應包含指示操作成功或失敗的 HTTP 狀態碼的。若集合成功刪除則傳回 200。

```
{"StatusCode":200}
```

新增臉部到集合

您可以使用 IndexFaces (p. 371) 操作來偵測影像中的臉孔，並將它們新增至集合。對於每個偵測到的臉孔，Amazon Rekognition 將擷取臉部特徵並將特徵資訊存放於資料庫中。此外，命令將為每個在指定臉部集合中偵測到的臉部存放中繼資料。Amazon Rekognition 不會存放實際影像位元組。

如需為編制索引提供適合臉部的相關資訊，請參閱 臉部比較輸入影像的建議 (p. 103)。

IndexFaces 操作將保留每個臉部的下列資訊：

- 多維度臉部特徵 – IndexFaces 使用臉部分析來擷取臉部特徵的多維度資訊，並將資訊存放於臉部集合中。您無法直接存取此資訊。不過，Amazon Rekognition 會在搜尋臉部集合並比對臉孔時使用此資訊。

- 中繼資料 – 每個臉孔的中繼資料包括週框方塊、可信度等級 (週框方塊標註的臉部)、由 Amazon Rekognition 指定的 ID (臉部 ID 與影像 ID)、以及請求中的外部影像 ID (若您提供此資訊)。此資訊會在對於 IndexFaces API 呼叫的回應中回傳給您。如需範例，請參閱下列範例回應中的 face 元素。

  此服務會傳回此中繼資料，以回應下列 API 呼叫：

  - the section called “ListFaces” (p. 382)
  - 搜尋臉部操作 – 對於 the section called “SearchFaces” (p. 392) 與 the section called “SearchFacesByImage” (p. 395) 的回應將傳回對於每個相符臉孔的配對可信度，同時提供配對的臉部之中繼資料。

由 IndexFaces 索引的臉部數量取決於與輸入集合相關聯的臉部偵測模型的版本。如需詳細資訊，請參閱 模型版本控制 (p. 9)。

有索引之臉孔的相關資訊會以 the section called “FaceRecord” (p. 464) 物件陣列傳回。

您可能想要將編入索引的臉部與偵測到這些臉部的影像連結。例如，您可能想要維護影像的用戶端索引以及影像中的臉部。若要將臉部與影像連結，請在 ExternalImageId 請求參數中指定一個影像 ID。影像 ID 可以是檔案名稱或您建立的另一個 ID。

除了上述關於 API 將保留於臉部集合中的資訊外，API 也會傳回未保留於集合中的臉部詳細資訊。(請參閱下列範例回應中的 faceDetail 元素)。

Note

DetectFaces 將傳回相同的資訊，因此您不需要同時呼叫 DetectFaces 和 IndexFaces。

篩選臉孔

這個 IndexFaces 操作可讓您篩選根據圖像建立索引的臉孔。運用 `IndexFaces`，您可以指定索引建立時所要使用的最多臉孔數目，或者選擇只對高品質偵測臉孔建立索引。

使用 `MaxFaces` 輸入參數，您可以指定 `IndexFaces` 在建立索引時所用的最多臉孔數目。若您在建立索引時想要運用圖像中尺寸最大的臉孔，而不要使用尺寸最小的臉孔時，例如，背景中站立人群的臉孔，就能使用這項參數。

在預設情況下，`IndexFaces` 選擇用於篩選出臉部的品質列。您可以使用 `QualityFilter` 輸入參數，以明確設定品質列。這些值為：

- `AUTO` — Amazon Rekognition 選擇用於篩選出臉部的品質列 (預設值)。
- `LOW` — 除了品質最低的臉部之外，所有臉部都會加以索引。
- `MEDIUM`
- `HIGH` — 僅對品質最高的臉部行索引。
- `NONE` - 不會根據品質來篩選掉任何臉部。

`IndexFaces` 篩選臉部的原因如下：

- 相較於圖像尺寸，臉部過小。
- 臉部過於模糊。
- 圖像過暗。
- 臉部表情過於誇張。
- 臉部沒有足夠的細節，不適合臉部搜尋。

> **Note**
>
> 若要使用品質篩選，您必須使用臉部模型第 3 版或更高版本的相關聯集合。若要取得與集合有關聯之臉孔模型的版本，請呼叫 the section called “DescribeCollection” (p. 302)。

未經 `IndexFaces` 建立索引之臉孔的相關資訊，將透過 the section called “UnindexedFace” (p. 511) 物件陣列傳回。`Reasons` 陣列包含為何無法為臉孔建立索引的原因清單。例如，當值為 `EXCEEDS_MAX_FACES` 時，表示因為已偵測到透過 `MaxFaces` 指定的臉孔數目，所以無法為臉孔建立索引。

如需詳細資訊，請參閱 管理集合中的臉孔 (p. 147)。

若要新增臉部到集合 (SDK)

1. 如果您尚未：
   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 將影像 (含有一或多個臉部) 上傳至您的 Amazon S3 儲存貯體。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
3. 使用下列範例來呼叫 `IndexFaces` 操作。

   Java

   此範例顯示新增到集合的臉部的臉部識別符。

將 collectionId 的值變更為要新增到臉部的集合的名稱。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。.withMaxFaces(1) 參數會將用於建立索引的臉孔數目限制為 1。移除或變更其值以符合您的需求。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;

import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.FaceRecord;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.IndexFacesRequest;
import com.amazonaws.services.rekognition.model.IndexFacesResult;
import com.amazonaws.services.rekognition.model.QualityFilter;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.model.UnindexedFace;
import java.util.List;

public class AddFacesToCollection {
    public static final String collectionId = "MyCollection";
    public static final String bucket = "bucket";
    public static final String photo = "input.jpg";

    public static void main(String[] args) throws Exception {

        AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

        Image image = new Image()
                .withS3Object(new S3Object()
                .withBucket(bucket)
                .withName(photo));

        IndexFacesRequest indexFacesRequest = new IndexFacesRequest()
                .withImage(image)
                .withQualityFilter(QualityFilter.AUTO)
                .withMaxFaces(1)
                .withCollectionId(collectionId)
                .withExternalImageId(photo)
                .withDetectionAttributes("DEFAULT");

        IndexFacesResult indexFacesResult =
 rekognitionClient.indexFaces(indexFacesRequest);

        System.out.println("Results for " + photo);
        System.out.println("Faces indexed:");
        List<FaceRecord> faceRecords = indexFacesResult.getFaceRecords();
        for (FaceRecord faceRecord : faceRecords) {
            System.out.println("  Face ID: " + faceRecord.getFace().getFaceId());
            System.out.println("  Location:" +
 faceRecord.getFaceDetail().getBoundingBox().toString());
        }

        List<UnindexedFace> unindexedFaces = indexFacesResult.getUnindexedFaces();
        System.out.println("Faces not indexed:");
        for (UnindexedFace unindexedFace : unindexedFaces) {
            System.out.println("  Location:" +
 unindexedFace.getFaceDetail().getBoundingBox().toString());
            System.out.println("  Reasons:");
            for (String reason : unindexedFace.getReasons()) {
                System.out.println("   " + reason);
```

```
            }
        }
    }
}
```

AWS CLI

此 AWS CLI 命令顯示 index-faces CLI 操作的 JSON 輸出。

以您要儲存臉部的集合之名稱取代 collection-id 的值。將 Bucket 與 Name 的值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。max-faces 參數會將用於建立索引的臉孔數目限制為 1。移除或變更其值以符合您的需求。

```
aws rekognition index-faces \
      --image '{"S3Object":{"Bucket":"bucket-name","Name":"file-name"}}' \
      --collection-id "collection-id" \
      --max-faces 1 \
      --quality-filter "AUTO" \
      --detection-attributes "ALL" \
      --external-image-id "example-image.jpg"
```

Python

此範例顯示新增到集合的臉部的臉部識別符。

將 collectionId 的值變更為要新增到臉部的集合的名稱。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。MaxFaces 輸入參數會將用於建立索引的臉孔數目限制為 1。移除或變更其值以符合您的需求。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def add_faces_to_collection(bucket,photo,collection_id):



    client=boto3.client('rekognition')

    response=client.index_faces(CollectionId=collection_id,
                                Image={'S3Object':{'Bucket':bucket,'Name':photo}},
                                ExternalImageId=photo,
                                MaxFaces=1,
                                QualityFilter="AUTO",
                                DetectionAttributes=['ALL'])

    print ('Results for ' + photo)
    print('Faces indexed:')
    for faceRecord in response['FaceRecords']:
         print('  Face ID: ' + faceRecord['Face']['FaceId'])
         print('  Location: {}'.format(faceRecord['Face']['BoundingBox']))

    print('Faces not indexed:')
    for unindexedFace in response['UnindexedFaces']:
        print(' Location: {}'.format(unindexedFace['FaceDetail']['BoundingBox']))
        print(' Reasons:')
        for reason in unindexedFace['Reasons']:
            print('   ' + reason)
    return len(response['FaceRecords'])
```

```
def main():
    bucket='bucket'
    collection_id='collection'
    photo='photo'


    indexed_faces_count=add_faces_to_collection(bucket, photo, collection_id)
    print("Faces indexed count: " + str(indexed_faces_count))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示新增到集合的臉部的臉部識別符。

將 collectionId 的值變更為要新增到臉部的集合的名稱。將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.Collections.Generic;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class AddFaces
{
    public static void Example()
    {
        String collectionId = "MyCollection";
        String bucket = "bucket";
        String photo = "input.jpg";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        Image image = new Image()
        {
            S3Object = new S3Object()
            {
                Bucket = bucket,
                Name = photo
            }
        };

        IndexFacesRequest indexFacesRequest = new IndexFacesRequest()
        {
            Image = image,
            CollectionId = collectionId,
            ExternalImageId = photo,
            DetectionAttributes = new List<String>(){ "ALL" }
        };

        IndexFacesResponse indexFacesResponse =
 rekognitionClient.IndexFaces(indexFacesRequest);

        Console.WriteLine(photo + " added");
        foreach (FaceRecord faceRecord in indexFacesResponse.FaceRecords)
            Console.WriteLine("Face detected: Faceid is " +
               faceRecord.Face.FaceId);
    }
```

```
}
```

IndexFaces 操作要求

IndexFaces 的輸入是要索引的影像和新增臉部或臉部的集合。

```
{
    "CollectionId": "MyCollection",
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "input.jpg"
        }
    },
    "ExternalImageId": "input.jpg",
    "DetectionAttributes": [
        "DEFAULT"
    ],
    "MaxFaces": 1,
    "QualityFilter": "AUTO"
}
```

IndexFaces 操作回應

IndexFaces 傳回有關在影像中偵測到的臉部的資訊。例如，以下 JSON 回應包含輸入影像中偵測到的臉部的預設偵測屬性。範例當中還會示範當 MaxFaces 輸入參數的值已經超過 —Reasons 陣列包含 EXCEEDS_MAX_FACES，導致無法為臉孔建立索引。當 Reasons 包含像是 LOW_SHARPNESS 或 LOW_BRIGHTNESS 等品質原因時，導致無法為臉孔建立索引。如需詳細資訊，請參閱 the section called “UnindexedFace” (p. 511)。

```
{
    "FaceModelVersion": "3.0",
    "FaceRecords": [
        {
            "Face": {
                "BoundingBox": {
                    "Height": 0.3247932195663452,
                    "Left": 0.5055555701255798,
                    "Top": 0.2743072211742401,
                    "Width": 0.21444444358348846
                },
                "Confidence": 99.99998474121094,
                "ExternalImageId": "input.jpg",
                "FaceId": "b86e2392-9da1-459b-af68-49118dc16f87",
                "ImageId": "09f43d92-02b6-5cea-8fbd-9f187db2050d"
            },
            "FaceDetail": {
                "BoundingBox": {
                    "Height": 0.3247932195663452,
                    "Left": 0.5055555701255798,
                    "Top": 0.2743072211742401,
                    "Width": 0.21444444358348846
                },
                "Confidence": 99.99998474121094,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.5751981735229492,
                        "Y": 0.4010535478591919
                    },
```

```
                    {
                        "Type": "eyeRight",
                        "X": 0.6511467099189758,
                        "Y": 0.4017036259174347
                    },
                    {
                        "Type": "nose",
                        "X": 0.6314528584480286,
                        "Y": 0.4710812568664551
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.5879443287849426,
                        "Y": 0.5171778798103333
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.6444502472877502,
                        "Y": 0.5164633989334106
                    }
                ],
                "Pose": {
                    "Pitch": -10.313642501831055,
                    "Roll": -1.0316886901855469,
                    "Yaw": 18.079818725585938
                },
                "Quality": {
                    "Brightness": 71.2919921875,
                    "Sharpness": 78.74752044677734
                }
            }
        }
    ],
    "OrientationCorrection": "",
    "UnindexedFaces": [
        {
            "FaceDetail": {
                "BoundingBox": {
                    "Height": 0.1329464465379715,
                    "Left": 0.5611110925674438,
                    "Top": 0.6832437515258789,
                    "Width": 0.08777777850627899
                },
                "Confidence": 92.37225341796875,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.5796897411346436,
                        "Y": 0.7452847957611084
                    },
                    {
                        "Type": "eyeRight",
                        "X": 0.6078574657440186,
                        "Y": 0.742687463760376
                    },
                    {
                        "Type": "nose",
                        "X": 0.597953200340271,
                        "Y": 0.7620673179626465
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.5884202122688293,
                        "Y": 0.7920381426811218
                    },
                    {
```

```
                        "Type": "mouthRight",
                        "X": 0.60627681016922,
                        "Y": 0.7919750809669495
                    }
                ],
                "Pose": {
                    "Pitch": 15.658954620361328,
                    "Roll": -4.583454608917236,
                    "Yaw": 10.558992385864258
                },
                "Quality": {
                    "Brightness": 42.54612350463867,
                    "Sharpness": 86.93206024169922
                }
            },
            "Reasons": [
                "EXCEEDS_MAX_FACES"
            ]
        }
    ]
}
```

若要取得所有臉部資訊，請指定 `DetectionAttributes` 請求參數為『ALL』。例如，在以下範例回應中可找到 `faceDetail` 元素中的其它資訊，而這些資訊不會保存在伺服器上：

- 25 個臉部特徵點 (相較於上述範例中只有 5 個特徵點)
- 九種臉部屬性 (眼鏡、鬍子等)
- 情緒 (請參閱 `emotion` 元素)

`face` 元素提供保留在伺服器上的中繼資料。

`FaceModelVersion` 是與集合相關聯的臉部模型版本。如需詳細資訊，請參閱 模型版本控制 (p. 9)。

`OrientationCorrection` 是影像的估計方向。如果您使用的臉部偵測模型版本大於第 3 版，則不會傳回方向更正資訊。如需詳細資訊，請參閱 取得影像方向與週框方塊座標 (p. 42)。

```
{
    "FaceModelVersion": "3.0",
    "FaceRecords": [
        {
            "Face": {
                "BoundingBox": {
                    "Height": 0.06333333253860474,
                    "Left": 0.17185185849666595,
                    "Top": 0.7366666793823242,
                    "Width": 0.11061728745698929
                },
                "Confidence": 99.99999237060547,
                "ExternalImageId": "input.jpg",
                "FaceId": "578e2e1b-d0b0-493c-aa39-ba476a421a34",
                "ImageId": "9ba38e68-35b6-5509-9d2e-fcffa75d1653"
            },
            "FaceDetail": {
                "AgeRange": {
                    "High": 25,
                    "Low": 15
                },
                "Beard": {
                    "Confidence": 99.98077392578125,
                    "Value": false
                },
                "BoundingBox": {
```

```
            "Height": 0.06333333253860474,
            "Left": 0.17185185849666595,
            "Top": 0.7366666793823242,
            "Width": 0.11061728745698929
        },
        "Confidence": 99.99999237060547,
        "Emotions": [
            {
                "Confidence": 95.40877532958984,
                "Type": "HAPPY"
            },
            {
                "Confidence": 6.6088080406188965,
                "Type": "CALM"
            },
            {
                "Confidence": 0.7385611534118652,
                "Type": "SAD"
            }
        ],
        "Eyeglasses": {
            "Confidence": 99.96795654296875,
            "Value": false
        },
        "EyesOpen": {
            "Confidence": 64.0671157836914,
            "Value": true
        },
        "Gender": {
            "Confidence": 100,
            "Value": "Female"
        },
        "Landmarks": [
            {
                "Type": "eyeLeft",
                "X": 0.21361233294010162,
                "Y": 0.757106363773346
            },
            {
                "Type": "eyeRight",
                "X": 0.2518567442893982,
                "Y": 0.7599404454231262
            },
            {
                "Type": "nose",
                "X": 0.2262365221977234,
                "Y": 0.7711842060089111
            },
            {
                "Type": "mouthLeft",
                "X": 0.2050037682056427,
                "Y": 0.7801263332366943
            },
            {
                "Type": "mouthRight",
                "X": 0.2430567592382431,
                "Y": 0.7836716771125793
            },
            {
                "Type": "leftPupil",
                "X": 0.2161938101053238,
                "Y": 0.756662905216217
            },
            {
                "Type": "rightPupil",
                "X": 0.2523181438446045,
```

```
            "Y": 0.7603650689125061
        },
        {
            "Type": "leftEyeBrowLeft",
            "X": 0.20066319406032562,
            "Y": 0.7501518130302429
        },
        {
            "Type": "leftEyeBrowUp",
            "X": 0.2130996286869049,
            "Y": 0.7480520606040955
        },
        {
            "Type": "leftEyeBrowRight",
            "X": 0.22584207355976105,
            "Y": 0.7504606246948242
        },
        {
            "Type": "rightEyeBrowLeft",
            "X": 0.24509544670581818,
            "Y": 0.7526801824569702
        },
        {
            "Type": "rightEyeBrowUp",
            "X": 0.2582615911960602,
            "Y": 0.7516844868659973
        },
        {
            "Type": "rightEyeBrowRight",
            "X": 0.26881539821624756,
            "Y": 0.7554477453231812
        },
        {
            "Type": "leftEyeLeft",
            "X": 0.20624476671218872,
            "Y": 0.7568746209144592
        },
        {
            "Type": "leftEyeRight",
            "X": 0.22105035185813904,
            "Y": 0.7582521438598633
        },
        {
            "Type": "leftEyeUp",
            "X": 0.21401576697826385,
            "Y": 0.7553104162216187
        },
        {
            "Type": "leftEyeDown",
            "X": 0.21317370235919952,
            "Y": 0.7584449648857117
        },
        {
            "Type": "rightEyeLeft",
            "X": 0.24393919110298157,
            "Y": 0.7600628137588501
        },
        {
            "Type": "rightEyeRight",
            "X": 0.2598416209220886,
            "Y": 0.7605880498886108
        },
        {
            "Type": "rightEyeUp",
            "X": 0.2519053518772125,
            "Y": 0.7582084536552429
```

```
                    },
                    {
                        "Type": "rightEyeDown",
                        "X": 0.25177454948425293,
                        "Y": 0.7612871527671814
                    },
                    {
                        "Type": "noseLeft",
                        "X": 0.2185886949300766,
                        "Y": 0.774715781211853
                    },
                    {
                        "Type": "noseRight",
                        "X": 0.23328955471515656,
                        "Y": 0.7759330868721008
                    },
                    {
                        "Type": "mouthUp",
                        "X": 0.22446128726005554,
                        "Y": 0.7805567383766174
                    },
                    {
                        "Type": "mouthDown",
                        "X": 0.22087252140045166,
                        "Y": 0.7891407608985901
                    }
                ],
                "MouthOpen": {
                    "Confidence": 95.87068939208984,
                    "Value": false
                },
                "Mustache": {
                    "Confidence": 99.9828109741211,
                    "Value": false
                },
                "Pose": {
                    "Pitch": -0.9409101605415344,
                    "Roll": 7.233824253082275,
                    "Yaw": -2.3602254390716553
                },
                "Quality": {
                    "Brightness": 32.01998519897461,
                    "Sharpness": 93.67259216308594
                },
                "Smile": {
                    "Confidence": 86.7142105102539,
                    "Value": true
                },
                "Sunglasses": {
                    "Confidence": 97.38925170898438,
                    "Value": false
                }
            }
        }
    ],
    "OrientationCorrection": "ROTATE_0"
    "UnindexedFaces": []
}
```


在集合中列出臉部

您可以使用 ListFaces (p. 382) 操作來列出集合中的臉部。

如需詳細資訊，請參閱 管理集合中的臉孔 (p. 147)。

若要列出集合中的臉部 (SDK)

1. 如果您尚未：

   a. 建立或更新 IAM 使用者 (具有 AmazonRekognitionFullAccess 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。

2. 使用下列範例來呼叫 ListFaces 操作。

   Java

   此範例顯示集合中的臉部的清單。

   將 collectionId 的值變更為所需的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.Face;
import com.amazonaws.services.rekognition.model.ListFacesRequest;
import com.amazonaws.services.rekognition.model.ListFacesResult;
import java.util.List;
import com.fasterxml.jackson.databind.ObjectMapper;



public class ListFacesInCollection {
    public static final String collectionId = "MyCollection";

   public static void main(String[] args) throws Exception {

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      ObjectMapper objectMapper = new ObjectMapper();

      ListFacesResult listFacesResult = null;
      System.out.println("Faces in collection " + collectionId);

      String paginationToken = null;
      do {
         if (listFacesResult != null) {
            paginationToken = listFacesResult.getNextToken();
         }

         ListFacesRequest listFacesRequest = new ListFacesRequest()
                 .withCollectionId(collectionId)
                 .withMaxResults(1)
                 .withNextToken(paginationToken);

         listFacesResult =  rekognitionClient.listFaces(listFacesRequest);
         List < Face > faces = listFacesResult.getFaces();
         for (Face face: faces) {
            System.out.println(objectMapper.writerWithDefaultPrettyPrinter()
                .writeValueAsString(face));
         }
```

```
      } while (listFacesResult != null && listFacesResult.getNextToken() !=
         null);
   }

}
```

AWS CLI

此 AWS CLI 命令顯示 `list-faces` CLI 操作的 JSON 輸出。將 `collection-id` 的值取代為要列出的集合的名稱。

```
aws rekognition list-faces \
      --collection-id "collection-id"
```

Python

此範例顯示集合中的臉部的清單。

將 `collectionId` 的值變更為所需的集合。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def list_faces_in_collection(collection_id):


    maxResults=2
    faces_count=0
    tokens=True

    client=boto3.client('rekognition')
    response=client.list_faces(CollectionId=collection_id,
                               MaxResults=maxResults)

    print('Faces in collection ' + collection_id)


    while tokens:

        faces=response['Faces']

        for face in faces:
            print (face)
            faces_count+=1
        if 'NextToken' in response:
            nextToken=response['NextToken']
            response=client.list_faces(CollectionId=collection_id,
                                       NextToken=nextToken,MaxResults=maxResults)
        else:
            tokens=False
    return faces_count
def main():

    collection_id='collection'

    faces_count=list_faces_in_collection(collection_id)
    print("faces count: " + str(faces_count))
if __name__ == "__main__":
```

```
    main()
```

.NET

此範例顯示集合中的臉部的清單。

將 collectionId 的值變更為所需的集合。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class ListFaces
{
    public static void Example()
    {
        String collectionId = "MyCollection";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        ListFacesResponse listFacesResponse = null;
        Console.WriteLine("Faces in collection " + collectionId);

        String paginationToken = null;
        do
        {
            if (listFacesResponse != null)
                paginationToken = listFacesResponse.NextToken;

            ListFacesRequest listFacesRequest = new ListFacesRequest()
            {
                CollectionId = collectionId,
                MaxResults = 1,
                NextToken = paginationToken
            };

            listFacesResponse = rekognitionClient.ListFaces(listFacesRequest);
            foreach(Face face in listFacesResponse.Faces)
                Console.WriteLine(face.FaceId);
        } while (listFacesResponse != null && !
String.IsNullOrEmpty(listFacesResponse.NextToken));
    }
}
```

ListFaces 操作要求

ListFaces 的輸入是您要為其列出臉部的集合 ID。MaxResults 是要傳回之臉部的最大值。

```
{
    "CollectionId": "MyCollection",
    "MaxResults": 1
}
```

如果回應的臉孔多於 MaxResults 請求的臉孔，則會傳回一個字符，您可以在後續的 ListFaces 呼叫中使用該字符獲取下一組結果。例如：

```
{
    "CollectionId": "MyCollection",
    "NextToken": "sm+5ythT3aeEVIR4WA....",
    "MaxResults": 1
}
```

ListFaces 操作回應

`ListFaces` 的回應是有關存放在指定集合中的臉部中繼資料的資訊。

- FaceModelVersion – 與集合相關聯的臉部模型版本。如需詳細資訊，請參閱 模型版本控制 (p. 9)。
- 臉部 – 有關集合中的臉部資訊。這包括有關 the section called “BoundingBox” (p. 440)、信心、影像識別符，以及臉部 ID 的資訊。如需詳細資訊，請參閱 the section called “Face” (p. 457)。
- NextToken – 用於取得下一組結果的字元。

```
{
    "FaceModelVersion": "3.0",
    "Faces": [
        {
            "BoundingBox": {
                "Height": 0.06333330273628235,
                "Left": 0.1718519926071167,
                "Top": 0.7366669774055481,
                "Width": 0.11061699688434601
            },
            "Confidence": 100,
            "ExternalImageId": "input.jpg",
            "FaceId": "0b683aed-a0f1-48b2-9b5e-139e9cc2a757",
            "ImageId": "9ba38e68-35b6-5509-9d2e-fcffa75d1653"
        }
    ],
    "NextToken": "sm+5ythT3aeEVIR4WA...."
}
```

刪除集合中的臉部

您可以使用 DeleteFaces (p. 294) 操作刪除集合中的臉部。如需詳細資訊，請參閱 管理集合中的臉孔 (p. 147)。

刪除集合中的臉部

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 `AmazonRekognitionFullAccess` 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 `DeleteFaces` 操作。

   Java

   此範例刪除集合中的單一臉部。

   將 `collectionId` 的值變更為包含要刪除的臉部的集合。將 `faces` 的值變更為要刪除的臉部 ID。如果要刪除多個臉部相符，可新增臉部 ID 至 `faces` 陣列。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.DeleteFacesRequest;
import com.amazonaws.services.rekognition.model.DeleteFacesResult;

import java.util.List;


public class DeleteFacesFromCollection {
   public static final String collectionId = "MyCollection";
   public static final String faces[] = {"xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"};

   public static void main(String[] args) throws Exception {

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();


      DeleteFacesRequest deleteFacesRequest = new DeleteFacesRequest()
              .withCollectionId(collectionId)
              .withFaceIds(faces);

      DeleteFacesResult
 deleteFacesResult=rekognitionClient.deleteFaces(deleteFacesRequest);


      List < String > faceRecords = deleteFacesResult.getDeletedFaces();
      System.out.println(Integer.toString(faceRecords.size()) + " face(s)
 deleted:");
      for (String face: faceRecords) {
         System.out.println("FaceID: " + face);
      }
   }
}
```

AWS CLI

此 AWS CLI 命令顯示 `delete-faces` CLI 操作的 JSON 輸出。將 `collection-id` 的值取代為包含要刪除的臉部的集合的名稱。將 `face-ids` 的值取代為要刪除的臉部 ID 陣列。

```
aws rekognition delete-faces --collection-id "collectionname" --face-ids
 '["faceid"]'
```

Python

此範例刪除集合中的單一臉部。

將 `collectionId` 的值變更為包含要刪除的臉部的集合。將 `faces` 的值變更為要刪除的臉部 ID。如果要刪除多個臉部相符，可新增臉部 ID 至 `faces` 陣列。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def delete_faces_from_collection(collection_id, faces):

    client=boto3.client('rekognition')

    response=client.delete_faces(CollectionId=collection_id,
                                 FaceIds=faces)

    print(str(len(response['DeletedFaces'])) + ' faces deleted:')
    for faceId in response['DeletedFaces']:
         print (faceId)
    return len(response['DeletedFaces'])

def main():

    collection_id='collection'
    faces=[]
    faces.append("xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx")

    faces_count=delete_faces_from_collection(collection_id, faces)
    print("deleted faces count: " + str(faces_count))

if __name__ == "__main__":
    main()
```

.NET

此範例刪除集合中的單一臉部。

將 collectionId 的值變更為包含要刪除的臉部的集合。將 faces 的值變更為要刪除的臉部 ID。如果要刪除多個臉部相符，可新增臉部 ID 至 faces 清單。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.Collections.Generic;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DeleteFaces
{
    public static void Example()
    {
        String collectionId = "MyCollection";
        List<String> faces = new List<String>() { "xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx" };

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DeleteFacesRequest deleteFacesRequest = new DeleteFacesRequest()
        {
            CollectionId = collectionId,
            FaceIds = faces
        };
```

```
        DeleteFacesResponse deleteFacesResponse =
 rekognitionClient.DeleteFaces(deleteFacesRequest);
        foreach (String face in deleteFacesResponse.DeletedFaces)
            Console.WriteLine("FaceID: " + face);
    }
}
```

DeleteFaces 操作要求

DeleteFaces 的輸入是包含臉部的集合 ID，以及要刪除的臉部的臉部 ID 陣列。

```
{
    "CollectionId": "MyCollection",
    "FaceIds": [
        "daf29cac-f910-41e9-851f-6eeb0e08f973"
    ]
}
```

DeleteFaces 操作回應

DeleteFaces 回應包含已刪除臉部的臉部 ID 陣列。

```
{
    "DeletedFaces": [
        "daf29cac-f910-41e9-851f-6eeb0e08f973"
    ]
}
```

使用臉部 ID 來搜尋臉孔

您可以使用 SearchFaces (p. 392) 操作來搜尋集合中符合指定臉部 ID 的臉孔。

當偵測到臉部並加入至集合時，臉部 ID 將隨 IndexFaces (p. 371) 操作的回應一同回傳。如需詳細資訊，請參閱 管理集合中的臉孔 (p. 147)。

若要使用臉部 ID 來搜尋集合中的臉孔 (SDK)

1. 如果您尚未：
   a. 建立或更新 IAM 使用者 (具有 AmazonRekognitionFullAccess 許可)。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 SearchFaces 操作。

   Java

   此範例顯示有關與由其 ID 識別的臉部符合的臉部資訊。

   將 collectionID 的值變更為包含所需臉部的集合。將 faceId 的值變更為要尋找的臉部識別符。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)
```

```
package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.amazonaws.services.rekognition.model.FaceMatch;
import com.amazonaws.services.rekognition.model.SearchFacesRequest;
import com.amazonaws.services.rekognition.model.SearchFacesResult;
import java.util.List;


  public class SearchFaceMatchingIdCollection {
      public static final String collectionId = "MyCollection";
      public static final String faceId = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx";

     public static void main(String[] args) throws Exception {

         AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

         ObjectMapper objectMapper = new ObjectMapper();
       // Search collection for faces matching the face id.

       SearchFacesRequest searchFacesRequest = new SearchFacesRequest()
               .withCollectionId(collectionId)
               .withFaceId(faceId)
               .withFaceMatchThreshold(70F)
               .withMaxFaces(2);

        SearchFacesResult searchFacesByIdResult =
                rekognitionClient.searchFaces(searchFacesRequest);

        System.out.println("Face matching faceId " + faceId);
       List < FaceMatch > faceImageMatches = searchFacesByIdResult.getFaceMatches();
       for (FaceMatch face: faceImageMatches) {
          System.out.println(objectMapper.writerWithDefaultPrettyPrinter()
                  .writeValueAsString(face));

          System.out.println();
       }
     }

}
```

執行程式碼範例。顯示有關臉部符合的資訊。

AWS CLI

此 AWS CLI 命令顯示 `search-faces` CLI 操作的 JSON 輸出。以您想要搜尋的臉部識別碼來取代`face-id` 的值，並以您想要搜尋的集合來取代 `collection-id` 的值。

```
aws rekognition search-faces \
    --face-id face-id \
    --collection-id "collection-id"
```

Python

此範例顯示有關與由其 ID 識別的臉部符合的臉部資訊。

將 `collectionID` 的值變更為包含所需臉部的集合。將 `faceId` 的值變更為要尋找的臉部識別符。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def search_face_in_collection(face_id,collection_id):
    threshold = 90
    max_faces=2
    client=boto3.client('rekognition')


    response=client.search_faces(CollectionId=collection_id,
                                 FaceId=face_id,
                                 FaceMatchThreshold=threshold,
                                 MaxFaces=max_faces)


    face_matches=response['FaceMatches']
    print ('Matching faces')
    for match in face_matches:
            print ('FaceId:' + match['Face']['FaceId'])
            print ('Similarity: ' + "{:.2f}".format(match['Similarity']) + "%")
            print
    return len(face_matches)

def main():

    face_id='xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx'
    collection_id='MyCollection'

    faces=[]
    faces.append(face_id)

    faces_count=search_face_in_collection(face_id, collection_id)
    print("faces found: " + str(faces_count))

if __name__ == "__main__":
    main()
```

.NET

此範例顯示有關與由其 ID 識別的臉部符合的臉部資訊。

將 `collectionID` 的值變更為包含所需臉部的集合。將 `faceId` 的值變更為要尋找的臉部識別符。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class SearchFacesMatchingId
{
    public static void Example()
    {
        String collectionId = "MyCollection";
        String faceId = "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx";
```

```
        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        // Search collection for faces matching the face id.

        SearchFacesRequest searchFacesRequest = new SearchFacesRequest()
        {
            CollectionId = collectionId,
            FaceId = faceId,
            FaceMatchThreshold = 70F,
            MaxFaces = 2
        };

        SearchFacesResponse searchFacesResponse =
 rekognitionClient.SearchFaces(searchFacesRequest);

        Console.WriteLine("Face matching faceId " + faceId);

        Console.WriteLine("Matche(s): ");
        foreach (FaceMatch face in searchFacesResponse.FaceMatches)
            Console.WriteLine("FaceId: " + face.Face.FaceId + ", Similarity: " +
 face.Similarity);
    }
}
```

執行程式碼範例。顯示有關臉部符合的資訊。

SearchFaces 操作要求

SearchFaces 將根據臉部 ID (每個儲存在臉部集合中的臉孔都有臉部 ID) 在指定的臉部集合中搜尋相似的臉孔。回應不包含您要搜尋的臉部。它只包含類似的臉部。在預設情況下，SearchFaces 傳回在使用演算法偵測後相似度超過 80% 的臉部。相似度代表偵測到的臉孔與輸入臉部間符合的程度。或者，您可以使用 FaceMatchThreshold 來指定不同的值。

```
{
    "CollectionId": "MyCollection",
    "FaceId": "0b683aed-a0f1-48b2-9b5e-139e9cc2a757",
    "MaxFaces": 2,
    "FaceMatchThreshold": 70
}
```

SearchFaces 操作回應

此操作會傳回一系列找到的臉部配對以及您提供的臉部 ID 來做為輸入檔。

```
{
    "SearchedFaceId": "7ecf8c19-5274-5917-9c91-1db9ae0449e2",
    "FaceMatches": [ list of face matches found ]
}
```

對於每個找到的臉部配對，回應將包含相似度以及臉部中繼資料，如下方範例回應所示：

```
{
   ...
    "FaceMatches": [
        {
            "Similarity": 100.0,
            "Face": {
                "BoundingBox": {
                    "Width": 0.6154,
```

```
                    "Top": 0.2442,
                    "Left": 0.1765,
                    "Height": 0.4692
                },
                "FaceId": "84de1c86-5059-53f2-a432-34ebb704615d",
                "Confidence": 99.9997,
                "ImageId": "d38ebf91-1a11-58fc-ba42-f978b3f32f60"
            }
        },
        {
            "Similarity": 84.6859,
            "Face": {
                "BoundingBox": {
                    "Width": 0.2044,
                    "Top": 0.2254,
                    "Left": 0.4622,
                    "Height": 0.3119
                },
                "FaceId": "6fc892c7-5739-50da-a0d7-80cc92c0ba54",
                "Confidence": 99.9981,
                "ImageId": "5d913eaf-cf7f-5e09-8c8f-cb1bdea8e6aa"
            }
        }
    ]
}
```

使用影像來搜尋臉孔

您可以使用 SearchFacesByImage (p. 395) 操作來搜尋集合中符合指定影像中最大臉部的臉孔。

如需詳細資訊，請參閱 在集合中搜尋臉部 (p. 148)。

若要使用影像來搜尋集合中的臉孔 (SDK)

1. 如果您尚未：

   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。

2. 將 (含有一或多個臉部) 的影像上傳至您的 S3 儲存貯體。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。

3. 使用下列範例來呼叫 `SearchFacesByImage` 操作。

   Java

   此範例顯示有關與影像中最大臉部符合的臉部資訊。程式碼範例指定 `FaceMatchThreshold` 和 `MaxFaces` 參數以限制回應中傳回的結果。

   在下列範例中，變更下列內容：將 `collectionId` 的值變更為要搜尋的集合，將 `bucket` 的值變更為包含輸入影像的儲存貯體，然後將 `photo` 的值變更為輸入影像。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
```

```
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.FaceMatch;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.model.SearchFacesByImageRequest;
import com.amazonaws.services.rekognition.model.SearchFacesByImageResult;
import java.util.List;
import com.fasterxml.jackson.databind.ObjectMapper;


public class SearchFaceMatchingImageCollection {
    public static final String collectionId = "MyCollection";
    public static final String bucket = "bucket";
    public static final String photo = "input.jpg";

    public static void main(String[] args) throws Exception {

       AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      ObjectMapper objectMapper = new ObjectMapper();

       // Get an image object from S3 bucket.
      Image image=new Image()
              .withS3Object(new S3Object()
                      .withBucket(bucket)
                      .withName(photo));

      // Search collection for faces similar to the largest face in the image.
      SearchFacesByImageRequest searchFacesByImageRequest = new
 SearchFacesByImageRequest()
              .withCollectionId(collectionId)
              .withImage(image)
              .withFaceMatchThreshold(70F)
              .withMaxFaces(2);

       SearchFacesByImageResult searchFacesByImageResult =
               rekognitionClient.searchFacesByImage(searchFacesByImageRequest);

       System.out.println("Faces matching largest face in image from" + photo);
      List < FaceMatch > faceImageMatches =
 searchFacesByImageResult.getFaceMatches();
      for (FaceMatch face: faceImageMatches) {
          System.out.println(objectMapper.writerWithDefaultPrettyPrinter()
                  .writeValueAsString(face));
         System.out.println();
      }
   }
}
```

AWS CLI

此 AWS CLI 命令顯示 `search-faces-by-image` CLI 操作的 JSON 輸出。以您在步驟 2 中所使用的 S3 儲存貯體來取代 `Bucket` 的值。以您在步驟 2 中所使用的影像檔案名稱來取代 `Name` 的值。以您要搜尋的集合名稱取代 `collection-id` 的值。

```
aws rekognition search-faces-by-image \
    --image '{"S3Object":{"Bucket":"bucket-name","Name":"Example.jpg"}}' \
    --collection-id "collection-id"
```

Python

此範例顯示有關與影像中最大臉部符合的臉部資訊。程式碼範例指定 FaceMatchThreshold 和 MaxFaces 參數以限制回應中傳回的結果。

在下列範例中，變更下列內容：將 collectionId 的值變更為要搜尋的集合，並將 bucket 和 photo 的值取代為步驟 2 中使用的 Amazon S3 儲存貯體和影像的名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

if __name__ == "__main__":

    bucket='bucket'
    collectionId='MyCollection'
    fileName='input.jpg'
    threshold = 70
    maxFaces=2

    client=boto3.client('rekognition')


    response=client.search_faces_by_image(CollectionId=collectionId,
                                Image={'S3Object':
{'Bucket':bucket,'Name':fileName}},
                                FaceMatchThreshold=threshold,
                                MaxFaces=maxFaces)


    faceMatches=response['FaceMatches']
    print ('Matching faces')
    for match in faceMatches:
            print ('FaceId:' + match['Face']['FaceId'])
            print ('Similarity: ' + "{:.2f}".format(match['Similarity']) + "%")
            print
```

.NET

此範例顯示有關與影像中最大臉部符合的臉部資訊。程式碼範例指定 FaceMatchThreshold 和 MaxFaces 參數以限制回應中傳回的結果。

在下列範例中，變更下列內容：將 collectionId 的值變更為要搜尋的集合，並將 bucket 和 photo 的值取代為步驟 2 中使用的 Amazon S3 儲存貯體和影像的名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class SearchFacesMatchingImage
{
    public static void Example()
    {
        String collectionId = "MyCollection";
        String bucket = "bucket";
```

```
        String photo = "input.jpg";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        // Get an image object from S3 bucket.
        Image image = new Image()
        {
            S3Object = new S3Object()
            {
                Bucket = bucket,
                Name = photo
            }
        };

        SearchFacesByImageRequest searchFacesByImageRequest = new
 SearchFacesByImageRequest()
        {
            CollectionId = collectionId,
            Image = image,
            FaceMatchThreshold = 70F,
            MaxFaces = 2
        };

        SearchFacesByImageResponse searchFacesByImageResponse =
 rekognitionClient.SearchFacesByImage(searchFacesByImageRequest);

        Console.WriteLine("Faces matching largest face in image from " + photo);
        foreach (FaceMatch face in searchFacesByImageResponse.FaceMatches)
            Console.WriteLine("FaceId: " + face.Face.FaceId + ", Similarity: " +
 face.Similarity);
    }
}
```

SearchFacesByImage 操作要求

`SearchFacesImageByImage` 的輸入參數是要搜尋的集合和來源影像位置。在此範例中，來源影像存放在 Amazon S3 儲存貯體 (`S3Object`) 中。另外指定要傳回的最大臉數 (`Maxfaces`) 以及必須與要傳回的臉孔 (`FaceMatchThreshold`) 符合的最小信心度。

```
{
    "CollectionId": "MyCollection",
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "input.jpg"
        }
    },
    "MaxFaces": 2,
    "FaceMatchThreshold": 70
}
```

SearchFacesByImage 操作回應

指定輸入影像 (.jpeg 或 .png) 時，操作首先將偵側輸入影像中的臉孔，接著搜尋指定的臉部集合以尋找相似臉孔。

> **Note**
>
> 如果服務在輸入影像中偵測到多個臉部，將使用偵測到的最大臉孔來搜尋臉部集合。

此操作會傳回一系列找到的臉部配對以及有關輸入臉部的資訊。這包括邊框等資訊，以及表示邊框包含臉部的信心度水平的信心度值。

在預設情況下，SearchFacesByImage 傳回在使用演算法偵測後相似度超過 80% 的臉部。相似度代表偵測到的臉孔與輸入臉部間符合的程度。或者，您可以使用 FaceMatchThreshold 來指定不同的值。對於每個找到的臉部配對，回應將包含相似度以及臉部中繼資料，如下方範例回應所示：

```
{
    "FaceMatches": [
        {
            "Face": {
                "BoundingBox": {
                    "Height": 0.06333330273628235,
                    "Left": 0.1718519926071167,
                    "Top": 0.7366669774055481,
                    "Width": 0.11061699688434601
                },
                "Confidence": 100,
                "ExternalImageId": "input.jpg",
                "FaceId": "578e2e1b-d0b0-493c-aa39-ba476a421a34",
                "ImageId": "9ba38e68-35b6-5509-9d2e-fcffa75d1653"
            },
            "Similarity": 99.9764175415039
        }
    ],
    "FaceModelVersion": "3.0",
    "SearchedFaceBoundingBox": {
        "Height": 0.06333333253860474,
        "Left": 0.17185185849666595,
        "Top": 0.7366666793823242,
        "Width": 0.11061728745698929
    },
    "SearchedFaceConfidence": 99.99999237060547
}
```

在儲存的視訊中搜尋臉部

您可以在集合搜尋與儲存的影片或串流視訊中偵測到的人物相符的臉部。此節涵蓋在儲存的影片中搜尋臉部的操作。如需在串流視訊搜尋臉孔的相關資訊，請參閱 處理串流影片 (p. 76)。

必須使用 IndexFaces (p. 371) 先將搜尋的臉孔在集合中編制為索引。如需詳細資訊，請參閱 新增臉部到集合 (p. 162)。

Amazon Rekognition Video 臉部搜尋與分析儲存於 Amazon S3 儲存貯體中的影片之其他 Amazon Rekognition Video 操作採用相同的非同步工作流程。若要開始在存放的影片中搜尋臉部，呼叫 StartFaceSearch (p. 411) 並提供您要搜尋之集合的 ID。Amazon Rekognition Video 會將影片分析的完成狀態發佈至 Amazon Simple Notification Service (Amazon SNS) 主題。如果影片分析成功，請呼叫 GetFaceSearch (p. 353) 以取得搜尋結果。如需開始影片分析並取得結果的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

下列程序說明如何在集合中搜尋與影片中偵測到的人物相符之臉孔。程序也將說明如何取得影片中相符人物的追蹤資料。此程序會展開「使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)」中的程式碼，該作業使用 Amazon Simple Queue Service (Amazon SQS) 佇列來取得影片分析要求的完成狀態。

搜尋與臉部相符的影片 (開發套件)

1. 建立集合 (p. 150)。

2. 將臉部編入集合索引 (p. 162)。
3. 執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。
4. 將下列程式碼新增至您在步驟 3 中建立的類別 `VideoDetect`。

Java

```
   //Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
   //PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

       //Face collection search in video
 ======================================================================
       private static void StartFaceSearchCollection(String bucket, String video,
 String collection) throws Exception{

        NotificationChannel channel= new NotificationChannel()
                .withSNSTopicArn(snsTopicArn)
                .withRoleArn(roleArn);

        StartFaceSearchRequest req = new StartFaceSearchRequest()
                .withCollectionId(collection)
                .withVideo(new Video()
                        .withS3Object(new S3Object()
                                .withBucket(bucket)
                                .withName(video)))
                .withNotificationChannel(channel);



        StartFaceSearchResult startPersonCollectionSearchResult =
 rek.startFaceSearch(req);
        startJobId=startPersonCollectionSearchResult.getJobId();

    }

    //Face collection search in video
 ======================================================================
    private static void GetFaceSearchCollectionResults() throws Exception{

       GetFaceSearchResult faceSearchResult=null;
       int maxResults=10;
       String paginationToken=null;

       do {

           if (faceSearchResult !=null){
               paginationToken = faceSearchResult.getNextToken();
           }


           faceSearchResult  = rek.getFaceSearch(
                   new GetFaceSearchRequest()
                   .withJobId(startJobId)
                   .withMaxResults(maxResults)
                   .withNextToken(paginationToken)
                   .withSortBy(FaceSearchSortBy.TIMESTAMP)
                   );


           VideoMetadata videoMetaData=faceSearchResult.getVideoMetadata();

           System.out.println("Format: " + videoMetaData.getFormat());
           System.out.println("Codec: " + videoMetaData.getCodec());
           System.out.println("Duration: " + videoMetaData.getDurationMillis());
```

```
            System.out.println("FrameRate: " + videoMetaData.getFrameRate());
            System.out.println();


            //Show search results
            List<PersonMatch> matches=
                    faceSearchResult.getPersons();

            for (PersonMatch match: matches) {
                long milliSeconds=match.getTimestamp();
                System.out.print("Timestamp: " + Long.toString(milliSeconds));
                System.out.println(" Person number: " +
 match.getPerson().getIndex());
                List <FaceMatch> faceMatches = match.getFaceMatches();
                if (faceMatches != null) {
                    System.out.println("Matches in collection...");
                    for (FaceMatch faceMatch: faceMatches){
                        Face face=faceMatch.getFace();
                        System.out.println("Face Id: "+ face.getFaceId());
                        System.out.println("Similarity: " +
 faceMatch.getSimilarity().toString());
                        System.out.println();
                    }
                }
                System.out.println();
            }

            System.out.println();

          } while (faceSearchResult !=null && faceSearchResult.getNextToken() !=
 null);

      }
```

在函數 `main` 中，將下行:

```
        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();
```

取代為:

```
        String collection="collection";
        StartFaceSearchCollection(bucket, video, collection);

        if (GetSQSMessageSuccess()==true)
         GetFaceSearchCollectionResults();
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # ============== Face Search ===============
    def StartFaceSearchCollection(self,collection):
        response = self.rek.start_face_search(Video={'S3Object':
{'Bucket':self.bucket,'Name':self.video}},
            CollectionId=collection,
            NotificationChannel={'RoleArn':self.roleArn,
 'SNSTopicArn':self.snsTopicArn})
```

```
        self.startJobId=response['JobId']

        print('Start Job Id: ' + self.startJobId)


    def GetFaceSearchCollectionResults(self):
        maxResults = 10
        paginationToken = ''

        finished = False

        while finished == False:
            response = self.rek.get_face_search(JobId=self.startJobId,
                                                MaxResults=maxResults,
                                                NextToken=paginationToken)

            print(response['VideoMetadata']['Codec'])
            print(str(response['VideoMetadata']['DurationMillis']))
            print(response['VideoMetadata']['Format'])
            print(response['VideoMetadata']['FrameRate'])

            for personMatch in response['Persons']:
                print('Person Index: ' + str(personMatch['Person']['Index']))
                print('Timestamp: ' + str(personMatch['Timestamp']))

                if ('FaceMatches' in personMatch):
                    for faceMatch in personMatch['FaceMatches']:
                        print('Face ID: ' + faceMatch['Face']['FaceId'])
                        print('Similarity: ' + str(faceMatch['Similarity']))
                print()
            if 'NextToken' in response:
                paginationToken = response['NextToken']
            else:
                finished = True
            print()
```

在函數 `main` 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    collection='tests'
    analyzer.StartFaceSearchCollection(collection)

    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetFaceSearchCollectionResults()
```

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的影片範例，要取代的程式碼可能會不同。

5. 將 `collection` 的值變更為您已在步驟 1 建立的集合名稱。
6. 執行程式碼。影片中的人物清單，而這些人物的臉孔符合輸入集合中顯示的臉部。也將顯示每個符合的人物之追蹤資料。

GetFaceSearch 操作回應

以下是 GetFaceSearch 的 JSON 回應範例。

回應包含一系列的人物 (Persons)，這些在影片中偵測到的人物臉孔符合輸入集合中的臉部。只要在視訊中出現符合的人物，就會出現一個陣列元素 PersonMatch (p. 488)。每個 PersonMatch 包含來自輸入集合的一系列臉部配對 FaceMatch (p. 463)、配對人物的資訊 PersonDetail (p. 486)、以及在視訊中配對該人物的時間點。


```
{
    "JobStatus": "SUCCEEDED",
    "NextToken": "IJdbzkZfvBRqj8GPV82BPiZKkLOGCqDIsNZG/gQsEE5faTVK9JHOz/xxxxxxxxxxxxxxx",
    "Persons": [
        {
            "FaceMatches": [
                {
                    "Face": {
                        "BoundingBox": {
                            "Height": 0.527472972869873,
                            "Left": 0.33530598878860474,
                            "Top": 0.2161169946193695,
                            "Width": 0.35503000020980835
                        },
                        "Confidence": 99.90239715576172,
                        "ExternalImageId": "image.PNG",
                        "FaceId": "a2f2e224-bfaa-456c-b360-7c00241e5e2d",
                        "ImageId": "eb57ed44-8d8d-5ec5-90b8-6d190daff4c3"
                    },
                    "Similarity": 98.40909576416016
                }
            ],
            "Person": {
                "BoundingBox": {
                    "Height": 0.8694444298744202,
                    "Left": 0.2473958283662796,
                    "Top": 0.10092592239379883,
                    "Width": 0.49427083134651184
                },
                "Face": {
                    "BoundingBox": {
                        "Height": 0.23000000417232513,
                        "Left": 0.42500001192092896,
                        "Top": 0.16333332657814026,
                        "Width": 0.12937499582767487
                    },
                    "Confidence": 99.97504425048828,
                    "Landmarks": [
                        {
                            "Type": "eyeLeft",
                            "X": 0.46415066719055176,
                            "Y": 0.2572723925113678
                        },
                        {
                            "Type": "eyeRight",
                            "X": 0.5068183541297913,
                            "Y": 0.23705792427062988
                        },
                        {
                            "Type": "nose",
                            "X": 0.49765899777412415,
                            "Y": 0.28383663296699524
                        },
                        {
                            "Type": "mouthLeft",
```

```
                        "X": 0.487221896648407,
                        "Y": 0.3452930748462677
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.5142884850502014,
                        "Y": 0.33167609572410583
                    }
                ],
                "Pose": {
                    "Pitch": 15.966927528381348,
                    "Roll": -15.547388076782227,
                    "Yaw": 11.34195613861084
                },
                "Quality": {
                    "Brightness": 44.80223083496094,
                    "Sharpness": 99.95819854736328
                }
            },
            "Index": 0
        },
        "Timestamp": 0
    },
    {
        "Person": {
            "BoundingBox": {
                "Height": 0.2177777737379074,
                "Left": 0.7593749761581421,
                "Top": 0.13333334028720856,
                "Width": 0.12250000238418579
            },
            "Face": {
                "BoundingBox": {
                    "Height": 0.2177777737379074,
                    "Left": 0.7593749761581421,
                    "Top": 0.13333334028720856,
                    "Width": 0.12250000238418579
                },
                "Confidence": 99.63436889648438,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.8005779385566711,
                        "Y": 0.20915353298187256
                    },
                    {
                        "Type": "eyeRight",
                        "X": 0.8391435146331787,
                        "Y": 0.21049551665782928
                    },
                    {
                        "Type": "nose",
                        "X": 0.8191410899162292,
                        "Y": 0.2523227035999298
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.8093273043632507,
                        "Y": 0.29053622484207153
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.8366993069648743,
                        "Y": 0.29101791977882385
                    }
                ],
```

```
                "Pose": {
                    "Pitch": 3.165884017944336,
                    "Roll": 1.4182015657424927,
                    "Yaw": -11.151537895202637
                },
                "Quality": {
                    "Brightness": 28.910892486572266,
                    "Sharpness": 97.61507415771484
                }
            },
            "Index": 1
        },
        "Timestamp": 0
    },
    {
        "Person": {
            "BoundingBox": {
                "Height": 0.8388888835906982,
                "Left": 0,
                "Top": 0.15833333134651184,
                "Width": 0.2369791716337204
            },
            "Face": {
                "BoundingBox": {
                    "Height": 0.20000000298023224,
                    "Left": 0.029999999329447746,
                    "Top": 0.2199999988079071,
                    "Width": 0.11249999701976776
                },
                "Confidence": 99.85971069335938,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.06842322647571564,
                        "Y": 0.3010137975215912
                    },
                    {
                        "Type": "eyeRight",
                        "X": 0.10543643683195114,
                        "Y": 0.29697132110595703
                    },
                    {
                        "Type": "nose",
                        "X": 0.09569807350635529,
                        "Y": 0.33701086044311523
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.0732642263174057,
                        "Y": 0.3757539987564087
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.10589495301246643,
                        "Y": 0.3722417950630188
                    }
                ],
                "Pose": {
                    "Pitch": -0.5589138865470886,
                    "Roll": -5.1093974113464355,
                    "Yaw": 18.69594955444336
                },
                "Quality": {
                    "Brightness": 43.052337646484375,
                    "Sharpness": 99.68138885498047
                }
```

```
                },
                "Index": 2
            },
            "Timestamp": 0
        }......

    ],
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 67301,
        "Format": "QuickTime / MOV",
        "FrameHeight": 1080,
        "FrameRate": 29.970029830932617,
        "FrameWidth": 1920
    }
}
```

人物路徑

Amazon Rekognition Video能在影片中建立人物的路徑並提供資訊，例如：

- 偵測到路徑時，該人物出現在視訊影格中的位置。
- 偵測到的臉部特徵點，例如左眼的位置。

Amazon Rekognition Video已儲存影片中的人物路徑是一種非同步操作。若要開始在影片中偵測路徑，請呼叫StartPersonTracking (p. 419)。Amazon Rekognition Video將發佈影片分析的完成狀態至 Amazon Simple Notification Service 主題。如果視訊分析成功，請呼叫 GetPersonTracking (p. 362) 以取得視訊分析的結果。如需更多呼叫 Amazon Rekognition Video API 操作的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

下列程序說明如何在儲存於 Amazon S3 儲存貯體的影片中追蹤人物路徑。此範例會展開 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 中的程式碼，這會使用 Amazon Simple Queue Service 佇列來取得視訊分析要求的完成狀態。

若要偵測存放在 Amazon S3 儲存貯體 (SDK) 之視訊中的人物

1. 執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。
2. 將下列程式碼新增至您在步驟 1 中建立的類別 `VideoDetect`。

   Java

```
       //Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
       //PDX-License-Identifier: MIT-0 (For details, see https://github.com/
awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

        //
Persons========================================================================
        private static void StartPersonDetection(String bucket, String video)
 throws Exception{


            NotificationChannel channel= new NotificationChannel()
                    .withSNSTopicArn(snsTopicArn)
                    .withRoleArn(roleArn);

         StartPersonTrackingRequest req = new StartPersonTrackingRequest()
                 .withVideo(new Video()
                         .withS3Object(new S3Object()
                             .withBucket(bucket)
                             .withName(video)))
                 .withNotificationChannel(channel);



         StartPersonTrackingResult startPersonDetectionResult =
 rek.startPersonTracking(req);
         startJobId=startPersonDetectionResult.getJobId();

        }

        private static void GetPersonDetectionResults() throws Exception{
            int maxResults=10;
            String paginationToken=null;
            GetPersonTrackingResult personTrackingResult=null;

            do{
```

```
                if (personTrackingResult !=null){
                    paginationToken = personTrackingResult.getNextToken();
                }

                personTrackingResult = rek.getPersonTracking(new
GetPersonTrackingRequest()
                     .withJobId(startJobId)
                     .withNextToken(paginationToken)
                     .withSortBy(PersonTrackingSortBy.TIMESTAMP)
                     .withMaxResults(maxResults));

                VideoMetadata
videoMetaData=personTrackingResult.getVideoMetadata();

                System.out.println("Format: " + videoMetaData.getFormat());
                System.out.println("Codec: " + videoMetaData.getCodec());
                System.out.println("Duration: " +
videoMetaData.getDurationMillis());
                System.out.println("FrameRate: " + videoMetaData.getFrameRate());


                //Show persons, confidence and detection times
                List<PersonDetection> detectedPersons=
personTrackingResult.getPersons();

                for (PersonDetection detectedPerson: detectedPersons) {

                   long seconds=detectedPerson.getTimestamp()/1000;
                   System.out.print("Sec: " + Long.toString(seconds) + " ");
                   System.out.println("Person Identifier: "  +
detectedPerson.getPerson().getIndex());
                       System.out.println();
                }
            }  while (personTrackingResult !=null &&
personTrackingResult.getNextToken() != null);

        }
```

在函數 `main` 中，將下行:

```
        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();
```

取代為:

```
        StartPersonDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetPersonDetectionResults();
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # ============== People pathing ===============
    def StartPersonPathing(self):
        response=self.rek.start_person_tracking(Video={'S3Object': {'Bucket':
 self.bucket, 'Name': self.video}},
```

```
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)

    def GetPersonPathingResults(self):
        maxResults = 10
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_person_tracking(JobId=self.startJobId,
                                        MaxResults=maxResults,
                                        NextToken=paginationToken)

            print('Codec: ' + response['VideoMetadata']['Codec'])
            print('Duration: ' + str(response['VideoMetadata']['DurationMillis']))
            print('Format: ' + response['VideoMetadata']['Format'])
            print('Frame rate: ' + str(response['VideoMetadata']['FrameRate']))
            print()

            for personDetection in response['Persons']:
                print('Index: ' + str(personDetection['Person']['Index']))
                print('Timestamp: ' + str(personDetection['Timestamp']))
                print()

            if 'NextToken' in response:
                paginationToken = response['NextToken']
            else:
                finished = True
```

在函數 `main` 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    analyzer.StartPersonPathing()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetPersonPathingResults()
```

Note

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的影片範例， 要取代的程式碼可能會不同。

3. 執行程式碼。將顯示追蹤到的人物之專有識別碼，同時以秒為單位顯示人物路徑被追蹤到的時間。

GetPersonTracking 操作回應

`GetPersonTracking` 傳回 `Persons` 物件的 PersonDetection (p. 487) 陣列，其中包含影片中追蹤到的人物詳細資訊，以及追蹤到他們路徑的時間。

您可以使用 `Persons` 輸入參數來排序 `SortBy`。追蹤到影片中的人物路徑時，指定 `TIMESTAMP` 來排序元素。指定 `INDEX` 來排序影片中追蹤到的人物。在每個人物的結果組合中，依據追蹤路徑精確度的可信度遞減

排序元素。在預設情況下，將根據 Persons 的排序傳回 TIMESTAMP。以下是來自 GetPersonDetection 的 JSON 回應範例。結果會依據人物路徑從影片開始後在視訊中被追蹤到的時間排序，以毫秒為單位。在回應中，請注意下列事項：

- 人物資訊–PersonDetection 陣列元素包含有關偵測到的人物之資訊。例如，該人物被偵測到的時間 (Timestamp)、該人物被偵測到時所在的視訊影格位置 (BoundingBox)、以及 Amazon Rekognition Video 對於正確偵測該人物的可信度 (Confidence)。

  不會在每次追蹤到人物路徑的時間戳記時傳回臉部特徵。此外在部分情況下，被追蹤到的人物身體可能無法顯示，因此只會傳回他們的臉部位置。
- 分頁資訊–範例顯示人物偵測資訊的一頁。您可以在 GetPersonTracking 的 MaxResults 輸入參數中指定要傳回的人物元素數目。如果結果數目超過 MaxResults，GetPersonTracking 會傳回用來取得下一頁結果的字符 (NextToken)。如需詳細資訊，請參閱 取得 Amazon Rekognition Video 分析結果 (p. 53)。
- 索引–在影片中識別人物的專有識別碼。
- 影片資訊– 該回應包含 GetPersonDetection 傳回的每頁資訊中，影片格式 (VideoMetadata) 的資訊。


```
{
    "JobStatus": "SUCCEEDED",
    "NextToken": "AcDymG0fSSoaI6+BBYpka5wVlqttysSPP8VvWcujMDluj1QpFo/vf
+mrMoqBGk8eUEiFlllR6g==",
    "Persons": [
        {
            "Person": {
                "BoundingBox": {
                    "Height": 0.8787037134170532,
                    "Left": 0.00572916679084301,
                    "Top": 0.12129629403352737,
                    "Width": 0.21666666865348816
                },
                "Face": {
                    "BoundingBox": {
                        "Height": 0.20000000298023224,
                        "Left": 0.029999999329447746,
                        "Top": 0.2199999988079071,
                        "Width": 0.11249999701976776
                    },
                    "Confidence": 99.85971069335938,
                    "Landmarks": [
                        {
                            "Type": "eyeLeft",
                            "X": 0.06842322647571564,
                            "Y": 0.3010137975215912
                        },
                        {
                            "Type": "eyeRight",
                            "X": 0.10543643683195114,
                            "Y": 0.29697132110595703
                        },
                        {
                            "Type": "nose",
                            "X": 0.09569807350635529,
                            "Y": 0.33701086044311523
                        },
                        {
                            "Type": "mouthLeft",
                            "X": 0.0732642263174057,
                            "Y": 0.3757539987564087
                        },
                        {
```

```
                        "Type": "mouthRight",
                        "X": 0.10589495301246643,
                        "Y": 0.3722417950630188
                    }
                ],
                "Pose": {
                    "Pitch": -0.5589138865470886,
                    "Roll": -5.1093974113464355,
                    "Yaw": 18.69594955444336
                },
                "Quality": {
                    "Brightness": 43.052337646484375,
                    "Sharpness": 99.68138885498047
                }
            },
            "Index": 0
        },
        "Timestamp": 0
    },
    {
        "Person": {
            "BoundingBox": {
                "Height": 0.9074074029922485,
                "Left": 0.24791666865348816,
                "Top": 0.09259258955717087,
                "Width": 0.375
            },
            "Face": {
                "BoundingBox": {
                    "Height": 0.23000000417232513,
                    "Left": 0.42500001192092896,
                    "Top": 0.16333332657814026,
                    "Width": 0.12937499582767487
                },
                "Confidence": 99.97504425048828,
                "Landmarks": [
                    {
                        "Type": "eyeLeft",
                        "X": 0.46415066719055176,
                        "Y": 0.2572723925113678
                    },
                    {
                        "Type": "eyeRight",
                        "X": 0.5068183541297913,
                        "Y": 0.23705792427062988
                    },
                    {
                        "Type": "nose",
                        "X": 0.49765899777412415,
                        "Y": 0.28383663296699524
                    },
                    {
                        "Type": "mouthLeft",
                        "X": 0.487221896648407,
                        "Y": 0.3452930748462677
                    },
                    {
                        "Type": "mouthRight",
                        "X": 0.5142884850502014,
                        "Y": 0.33167609572410583
                    }
                ],
                "Pose": {
                    "Pitch": 15.966927528381348,
                    "Roll": -15.547388076782227,
                    "Yaw": 11.34195613861084
```

```
                    },
                    "Quality": {
                        "Brightness": 44.80223083496094,
                        "Sharpness": 99.95819854736328
                    }
                },
                "Index": 1
            },
            "Timestamp": 0
        }.....

    ],
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 67301,
        "FileExtension": "mp4",
        "Format": "QuickTime / MOV",
        "FrameHeight": 1080,
        "FrameRate": 29.970029830932617,
        "FrameWidth": 1920
    }
}
```

辨識名人

Amazon Rekognition 可以辨識各領域中數千位名人，例如娛樂與媒體界、體育界、商業界以及政治界。透過 Amazon Rekognition，您可以辨識影像與已存放影片中的名人。您也可以取得已辨識之名人的其他資訊。

Amazon Rekognition 名人辨識 API 已調校成可在不同設定、裝飾、化妝和其他條件下偵測名人。社交、媒體和娛樂客戶可建置使用名人辨識的應用程式。例如，娛樂應用程式會辨識名人類似的外觀或應用程式會在自動化的影片標記過程中辨識名人。Amazon Rekognition 名人辨識是設計來在影像或影片中有知名名人時專門使用。名人辨識 API 會從找到的相符項目中，傳回最接近的項目以及相似度分數。如需辨識非名人臉部的資訊，請參閱 在集合中搜尋臉部 (p. 147)。

> Note
>
> 如果您是名人，而且不希望加入此功能，請聯絡 AWS 支援或傳送電子郵件至 `<rekognition-celebrity-opt-out@amazon.com>`。

主題

- 相較於臉部搜尋的名人辨識 (p. 201)
- 辨識影像中的名人 (p. 201)
- 辨識已存放影片中的名人 (p. 207)
- 取得名人的相關資訊 (p. 211)

相較於臉部搜尋的名人辨識

Amazon Rekognition 同時提供名人辨識和臉部辨識功能。這些功能在其使用案例和最佳實務中有一些主要差異。

名人辨識已經過預先訓練而能夠辨識各種領域 (例如運動、媒體、政界及商場) 中成千上萬的名人。其設計旨在比對名人臉部與其各種第二個身分 (例如，沒有化妝的強尼戴普、將強尼戴普看成「剪刀手愛德華」，以及將強尼戴普看成「傑克史派羅」)。此功能旨在協助您搜尋大量影像或影片，以識別可能包含特定名人的小集。它不是用來比對非名人的各種人臉。在正確比對名人很重要的情況下，我們建議也使用人工操作員來查看這些較少量的標記內容，以協助確保高水平的準確性，並適當應用人類的判斷。使用名人辨識的方式不應該對名人造成負面影響。

另一方面，臉部辨識是更通用的功能，可讓您建立自己的臉部集合與專屬的臉部向量，以驗證身分或搜尋任何人員 (不只是名人)。臉部辨識可以用於驗證大樓進出、公共安全和社交媒體之類的應用。在所有這些情況下，建議您使用最佳實務、適當的可信度閾值 (包括 99%，適用於公共安全使用案例)，以及在正確比對很重要之情況下的人工檢閱。

如需詳細資訊，請參閱 在集合中搜尋臉部 (p. 147)。

辨識影像中的名人

若要辨識影像中的名人並取得已辨識之名人的其他資訊，請使用 RecognizeCelebrities (p. 388) 非儲存 API 操作。例如，在社交媒體或新聞與娛樂業中，時間對資訊收集而言至關重要，您可以使用 `RecognizeCelebrities` 操作，來識別影像中多達 100 位名人，並傳回名人的網頁連結 (如果有)。Amazon Rekognition 不會記得名人偵測所在的影像。您的應用程式必須存放此資訊。

如果您尚未存放 `RecognizeCelebrities` 傳回的名人其他資訊，又不想重新分析影像來取得該資訊，請使用 GetCelebrityInfo (p. 337)。若要呼叫 `GetCelebrityInfo`，您需要 Amazon Rekognition 指派給每位名人的唯一識別符。對於影像中已辨識的每位名人，都會在 `RecognizeCelebrities` 回應中傳回此識別符。

如果您要處理大型影像集合以辨識名人，請考慮使用 AWS Batch 在背景批次處理 `RecognizeCelebrities` 的呼叫。當您將新的影像新增至集合時，可以使用 AWS Lambda 函數辨識名人，方法是在影像上傳至 S3 儲存貯體時，呼叫 `RecognizeCelebrities`。

呼叫 RecognizeCelebrities

您可以使用 AWS Command Line Interface (AWS CLI) 或 AWS 開發套件，以影像位元組陣列 (Base64 編碼影像位元組) 或 Amazon S3 物件的方式提供輸入影像。在AWS CLI 程序中，您會將 .jpg 或 .png 格式的影像上傳至 S3 儲存貯體。在 AWS SDK for Java 程序中，您則會使用從本機檔案系統載入的影像。如需輸入影像建議的資訊，請參閱 使用影像 (p. 23)。

若要執行此程序，您需要含有一或多個名人臉部的影片檔案。

辨識影像中的名人

1. 如果您尚未：
   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 `RecognizeCelebrities` 操作。

   Java

   此範例顯示與影像中偵測到的名人有關的資訊。

   將 `photo` 的值變更為某個影像檔案的路徑和檔案名稱，而該影像檔案含有一個或多個名人臉孔。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.BoundingBox;
import com.amazonaws.services.rekognition.model.Celebrity;
import com.amazonaws.services.rekognition.model.RecognizeCelebritiesRequest;
import com.amazonaws.services.rekognition.model.RecognizeCelebritiesResult;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.nio.ByteBuffer;
import com.amazonaws.util.IOUtils;
import java.util.List;


public class RecognizeCelebrities {

   public static void main(String[] args) {
      String photo = "moviestars.jpg";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      ByteBuffer imageBytes=null;
      try (InputStream inputStream = new FileInputStream(new File(photo))) {
         imageBytes = ByteBuffer.wrap(IOUtils.toByteArray(inputStream));
      }
```

```
      catch(Exception e)
      {
          System.out.println("Failed to load file " + photo);
          System.exit(1);
      }


      RecognizeCelebritiesRequest request = new RecognizeCelebritiesRequest()
         .withImage(new Image()
         .withBytes(imageBytes));

      System.out.println("Looking for celebrities in image " + photo + "\n");

      RecognizeCelebritiesResult
 result=rekognitionClient.recognizeCelebrities(request);

      //Display recognized celebrity information
      List<Celebrity> celebs=result.getCelebrityFaces();
      System.out.println(celebs.size() + " celebrity(s) were recognized.\n");

      for (Celebrity celebrity: celebs) {
          System.out.println("Celebrity recognized: " + celebrity.getName());
          System.out.println("Celebrity ID: " + celebrity.getId());
          BoundingBox boundingBox=celebrity.getFace().getBoundingBox();
          System.out.println("position: " +
             boundingBox.getLeft().toString() + " " +
             boundingBox.getTop().toString());
          System.out.println("Further information (if available):");
          for (String url: celebrity.getUrls()){
             System.out.println(url);
          }
          System.out.println();
       }
       System.out.println(result.getUnrecognizedFaces().size() + " face(s) were
 unrecognized.");
    }
}
```

AWS CLI

此 AWS CLI 命令顯示 `recognize-celebrities` CLI 操作的 JSON 輸出。

將 `bucketname` 變更為 Amazon S3 儲存貯體的名稱，其中包含影像。將 `input.jpg` 變更為某個影像的檔案名稱，而該影像中含有一個或多個名人臉孔。

```
aws rekognition recognize-celebrities \
  --image "S3Object={Bucket=bucketname,Name=input.jpg}"
```

Python

此範例顯示與影像中偵測到的名人有關的資訊。

將 `photo` 的值變更為某個影像檔案的路徑和檔案名稱，而該影像檔案含有一個或多個名人臉孔。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3
import json

def recognize_celebrities(photo):
```

```
    client=boto3.client('rekognition')

    with open(photo, 'rb') as image:
        response = client.recognize_celebrities(Image={'Bytes': image.read()})

    print('Detected faces for ' + photo)
    for celebrity in response['CelebrityFaces']:
        print ('Name: ' + celebrity['Name'])
        print ('Id: ' + celebrity['Id'])
        print ('Position:')
        print ('   Left: ' + '{:.2f}'.format(celebrity['Face']['BoundingBox']
['Height']))
        print ('   Top: ' + '{:.2f}'.format(celebrity['Face']['BoundingBox']
['Top']))
        print ('Info')
        for url in celebrity['Urls']:
            print ('   ' + url)
        print
    return len(response['CelebrityFaces'])

def main():
    photo='moviestars.jpg'

    celeb_count=recognize_celebrities(photo)
    print("Celebrities detected: " + str(celeb_count))


if __name__ == "__main__":
    main()
```

.NET

此範例顯示與影像中偵測到的名人有關的資訊。

將 photo 的值變更為某個影像檔案的路徑和檔案名稱，而該影像檔案含有一個或多個名人臉孔 (.jpg 或 .png 格式)。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using System.IO;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class CelebritiesInImage
{
    public static void Example()
    {
        String photo = "moviestars.jpg";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        RecognizeCelebritiesRequest recognizeCelebritiesRequest = new
 RecognizeCelebritiesRequest();

        Amazon.Rekognition.Model.Image img = new Amazon.Rekognition.Model.Image();
        byte[] data = null;
        try
        {
            using (FileStream fs = new FileStream(photo, FileMode.Open,
 FileAccess.Read))
```

```
            {
                data = new byte[fs.Length];
                fs.Read(data, 0, (int)fs.Length);
            }
        }
        catch(Exception)
        {
            Console.WriteLine("Failed to load file " + photo);
            return;
        }

        img.Bytes = new MemoryStream(data);
        recognizeCelebritiesRequest.Image = img;

        Console.WriteLine("Looking for celebrities in image " + photo + "\n");

        RecognizeCelebritiesResponse recognizeCelebritiesResponse =
rekognitionClient.RecognizeCelebrities(recognizeCelebritiesRequest);

        Console.WriteLine(recognizeCelebritiesResponse.CelebrityFaces.Count + "
celebrity(s) were recognized.\n");
        foreach (Celebrity celebrity in
recognizeCelebritiesResponse.CelebrityFaces)
        {
            Console.WriteLine("Celebrity recognized: " + celebrity.Name);
            Console.WriteLine("Celebrity ID: " + celebrity.Id);
            BoundingBox boundingBox = celebrity.Face.BoundingBox;
            Console.WriteLine("position: " +
               boundingBox.Left + " " + boundingBox.Top);
            Console.WriteLine("Further information (if available):");
            foreach (String url in celebrity.Urls)
                Console.WriteLine(url);
        }
        Console.WriteLine(recognizeCelebritiesResponse.UnrecognizedFaces.Count + "
face(s) were unrecognized.");
    }
}
```

3. 記錄所顯示之其中一個名人 ID 的值。您在「取得名人的相關資訊 (p. 211)」中會需要用到。

RecognizeCelebrities 操作要求

輸入至 `RecognizeCelebrities` 的是影像。在此範例中，影像以影像位元組傳遞。如需詳細資訊，請參閱使用影像 (p. 23)。

```
{
    "Image": {
        "Bytes": "/AoSiyvFpm....."
    }
}
```

RecognizeCelebrities 操作回應

以下是 `RecognizeCelebrities` 的 JSON 輸入和輸出範例。

`RecognizeCelebrities` 會傳回已辨識之名人的陣列與無法辨識之臉部的陣列。在 範例中，請注意下列事項：

- 已辨識的名人 – `Celebrities` 是已辨識之名人的陣列。陣列中每個 Celebrity (p. 442) 物件包含名人名稱與 URL 的清單，該 URL 指向相關的內容—例如，名人的 IMDB 連結。Amazon Rekognition 會傳回您

的應用程式可以使用的 ComparedFace (p. 446) 物件來判斷名人臉孔在影像中所在的位置與名人的唯一識別符。您之後可透過 GetCelebrityInfo (p. 337) API 操作，使用唯一識別符來擷取名人資訊。

- 無法辨識的臉部 – `UnrecognizedFaces` 是未與任何已知名人相符之臉部的陣列。陣列中的每個 ComparedFace (p. 446) 物件都包含週框方塊 (及其他資訊)，可讓您用來尋找影像中的臉部。
- 影像方向 – `OrientationCorrection` 是可用於正確顯示影像的影像方向資訊。如需詳細資訊，請參閱 取得影像方向與週框方塊座標 (p. 42)。


```
{
    "CelebrityFaces": [{
        "Face": {
            "BoundingBox": {
                "Height": 0.617123007774353,
                "Left": 0.15641026198863983,
                "Top": 0.10864841192960739,
                "Width": 0.3641025722026825
            },
            "Confidence": 99.99589538574219,
            "Landmarks": [{
                "Type": "eyeLeft",
                "X": 0.2837241291999817,
                "Y": 0.3637104034423828
            }, {
                "Type": "eyeRight",
                "X": 0.4091649055480957,
                "Y": 0.37378931045532227
            }, {
                "Type": "nose",
                "X": 0.35267341136932373,
                "Y": 0.49657556414604187
            }, {
                "Type": "mouthLeft",
                "X": 0.2786353826522827,
                "Y": 0.5455248355865479
            }, {
                "Type": "mouthRight",
                "X": 0.39566439390182495,
                "Y": 0.5597742199897766
            }],
            "Pose": {
                "Pitch": -7.749263763427734,
                "Roll": 2.004552125930786,
                "Yaw": 9.012002944946289
            },
            "Quality": {
                "Brightness": 32.69192123413086,
                "Sharpness": 99.9305191040039
            }
        },
        "Id": "3Ir0du6",
        "MatchConfidence": 98.0,
        "Name": "Jeff Bezos",
        "Urls": ["www.imdb.com/name/nm1757263"]
    }],
    "OrientationCorrection": "ROTATE_0",
    "UnrecognizedFaces": [{
        "BoundingBox": {
            "Height": 0.5345501899719238,
            "Left": 0.48461538553237915,
            "Top": 0.16949152946472168,
            "Width": 0.3153846263885498
        },
```

```
        "Confidence": 99.92860412597656,
        "Landmarks": [{
            "Type": "eyeLeft",
            "X": 0.5863404870033264,
            "Y": 0.36940744519233704
        }, {
            "Type": "eyeRight",
            "X": 0.6999204754829407,
            "Y": 0.3769848346710205
        }, {
            "Type": "nose",
            "X": 0.6349524259567261,
            "Y": 0.4804527163505554
        }, {
            "Type": "mouthLeft",
            "X": 0.5872702598571777,
            "Y": 0.5535582304000854
        }, {
            "Type": "mouthRight",
            "X": 0.6952020525932312,
            "Y": 0.5600858926773071
        }],
        "Pose": {
            "Pitch": -7.386096477508545,
            "Roll": 2.304218292236328,
            "Yaw": -6.175624370574951
        },
        "Quality": {
            "Brightness": 37.16635513305664,
            "Sharpness": 99.9305191040039
        }
    }]
}
```

辨識已存放影片中的名人

在已存放影片的 Amazon Rekognition Video 名人辨識是一種非同步操作。為了識別名人雲集，請使用 StartCelebrityRecognition (p. 400) 在存放影片開始影片分析。完成狀態 Amazon Rekognition Video 發佈到 Amazon Simple Notification Service 主題的影片分析。如果影片分析成功，請呼叫 GetCelebrityRecognition (p. 339) 以取得分析結果。如需開始影片分析並取得結果的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

此程序會展開「使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)」中的程式碼，該作業使用 Amazon SQS 佇列來取得影片分析要求的完成狀態。若要執行此程序，您需要含有一或多個名人臉部的影片檔案。

偵測存放在 Amazon S3 儲存貯體 (SDK) 之影片中的名人

1. 執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。
2. 將下列程式碼新增至您在步驟 1 中建立的類別 `VideoDetect`。

   Java

```
        //Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
        //PDX-License-Identifier: MIT-0 (For details, see https://github.com/
awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

      //
 Celebrities========================================================================
```

```
     private static void StartCelebrityDetection(String bucket, String video)
throws Exception{

           NotificationChannel channel= new NotificationChannel()
                   .withSNSTopicArn(snsTopicArn)
                   .withRoleArn(roleArn);

          StartCelebrityRecognitionRequest req = new
StartCelebrityRecognitionRequest()
                .withVideo(new Video()
                      .withS3Object(new S3Object()
                            .withBucket(bucket)
                            .withName(video)))
                .withNotificationChannel(channel);



          StartCelebrityRecognitionResult startCelebrityRecognitionResult =
rek.startCelebrityRecognition(req);
          startJobId=startCelebrityRecognitionResult.getJobId();

       }

       private static void GetCelebrityDetectionResults() throws Exception{

          int maxResults=10;
          String paginationToken=null;
          GetCelebrityRecognitionResult celebrityRecognitionResult=null;

          do{
             if (celebrityRecognitionResult !=null){
                paginationToken = celebrityRecognitionResult.getNextToken();
             }
             celebrityRecognitionResult = rek.getCelebrityRecognition(new
GetCelebrityRecognitionRequest()
                   .withJobId(startJobId)
                   .withNextToken(paginationToken)
                   .withSortBy(CelebrityRecognitionSortBy.TIMESTAMP)
                   .withMaxResults(maxResults));


             System.out.println("File info for page");
             VideoMetadata
videoMetaData=celebrityRecognitionResult.getVideoMetadata();

             System.out.println("Format: " + videoMetaData.getFormat());
             System.out.println("Codec: " + videoMetaData.getCodec());
             System.out.println("Duration: " + videoMetaData.getDurationMillis());
             System.out.println("FrameRate: " + videoMetaData.getFrameRate());

             System.out.println("Job");

             System.out.println("Job status: " +
celebrityRecognitionResult.getJobStatus());


             //Show celebrities
             List<CelebrityRecognition> celebs=
celebrityRecognitionResult.getCelebrities();

             for (CelebrityRecognition celeb: celebs) {
                long seconds=celeb.getTimestamp()/1000;
                System.out.print("Sec: " + Long.toString(seconds) + " ");
                CelebrityDetail details=celeb.getCelebrity();
                System.out.println("Name: " + details.getName());
                System.out.println("Id: " + details.getId());
```

```
                System.out.println();
             }
          } while (celebrityRecognitionResult !=null &&
celebrityRecognitionResult.getNextToken() != null);

       }
```

在函數 main 中，將下行:

```
       StartLabelDetection(bucket, video);

       if (GetSQSMessageSuccess()==true)
        GetLabelDetectionResults();
```

取代為:

```
       StartCelebrityDetection(bucket, video);

       if (GetSQSMessageSuccess()==true)
        GetCelebrityDetectionResults();
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # ============== Celebrities ===============
    def StartCelebrityDetection(self):
        response=self.rek.start_celebrity_recognition(Video={'S3Object': {'Bucket':
 self.bucket, 'Name': self.video}},
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
 self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)

    def GetCelebrityDetectionResults(self):
        maxResults = 10
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_celebrity_recognition(JobId=self.startJobId,
                                                MaxResults=maxResults,
                                                NextToken=paginationToken)

            print(response['VideoMetadata']['Codec'])
            print(str(response['VideoMetadata']['DurationMillis']))
            print(response['VideoMetadata']['Format'])
            print(response['VideoMetadata']['FrameRate'])

            for celebrityRecognition in response['Celebrities']:
                print('Celebrity: ' +
                    str(celebrityRecognition['Celebrity']['Name']))
                print('Timestamp: ' + str(celebrityRecognition['Timestamp']))
                print()

            if 'NextToken' in response:
                paginationToken = response['NextToken']
```

```
            else:
                finished = True
```

在函數 `main` 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    analyzer.StartCelebrityDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetCelebrityDetectionResults()
```

Note

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的影片範例，要取代的程式碼可能會不同。

3. 執行程式碼。在影片中辨識出之名人的資訊便會顯示。

GetCelebrityRecognition 操作回應

以下是 JSON 回應的範例。回應包含下列內容：

- 已辨識的名人 – `Celebrities` 是影片中已辨識之名人的陣列與辨識時間。每當在影片中辨識出該名人時，就會有一個 CelebrityRecognition (p. 445) 物件。每個 `CelebrityRecognition` 包含經識別名人的資訊 (CelebrityDetail (p. 443)) 和名人在影片中識別的時間 (`Timestamp`)。會在影片開頭量測 `Timestamp` (單位：毫秒)。
- CelebrityDetail – 包含已辨識之名人的資訊。其包括名人名稱 (`Name`)、識別符 (`ID`) 與指向相關內容的 URL 清單 (`Urls`)。同時也包括名人身體的週框方塊、Amazon Rekognition Video 對辨識正確性的信賴度，以及名人臉部細節 FaceDetail (p. 459)。如果您之後需要取得相關內容，可以搭配 GetCelebrityInfo (p. 337) 使用 `ID`。
- VideoMetadata – 已分析影片的資訊。

```
{
    "Celebrities": [
        {
            "Celebrity": {
                "BoundingBox": {
                    "Height": 0.8842592835426331,
                    "Left": 0,
                    "Top": 0.11574073880910873,
                    "Width": 0.24427083134651184
                },
                "Confidence": 0.699999988079071,
                "Face": {
                    "BoundingBox": {
                        "Height": 0.20555555820465088,
                        "Left": 0.029374999925494194,
                        "Top": 0.22333332896232605,
                        "Width": 0.11562500149011612
                    },
                    "Confidence": 99.89837646484375,
                    "Landmarks": [
```

```
                        {
                            "Type": "eyeLeft",
                            "X": 0.06857934594154358,
                            "Y": 0.30842265486717224
                        },
                        {
                            "Type": "eyeRight",
                            "X": 0.10396526008844376,
                            "Y": 0.300625205039978
                        },
                        {
                            "Type": "nose",
                            "X": 0.0966852456331253,
                            "Y": 0.34081998467445374
                        },
                        {
                            "Type": "mouthLeft",
                            "X": 0.075217105448246,
                            "Y": 0.3811396062374115
                        },
                        {
                            "Type": "mouthRight",
                            "X": 0.10744428634643555,
                            "Y": 0.37407416105270386
                        }
                    ],
                    "Pose": {
                        "Pitch": -0.9784082174301147,
                        "Roll": -8.808176040649414,
                        "Yaw": 20.28228759765625
                    },
                    "Quality": {
                        "Brightness": 43.312068939208984,
                        "Sharpness": 99.9305191040039
                    }
                },
                "Id": "XXXXXX",
                "Name": "Celeb A",
                "Urls": []
            },
            "Timestamp": 367
        },......
    ],
    "JobStatus": "SUCCEEDED",
    "NextToken": "XfXnZKiyMOGDhzBzYUhS5puM+g1IgezqFeYpv/H/+5noP/LmM57FitUAwSQ5D6G4AB/
PNwolrw==",
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 67301,
        "FileExtension": "mp4",
        "Format": "QuickTime / MOV",
        "FrameHeight": 1080,
        "FrameRate": 29.970029830932617,
        "FrameWidth": 1920
    }
}
```


取得名人的相關資訊

在這些程序中，您會使用 GetCelebrityInfo (p. 337) API 操作來取得名人資訊。使用之前 the section called "RecognizeCelebrities" (p. 388) 呼叫所傳回的名人 ID，即可識別名人。

呼叫 GetCelebrityInfo

這些程序也需要 Amazon Rekognition 已知名人的名人 ID。使用您在「辨識影像中的名人 (p. 201)」中記下的名人 ID。

取得名人資訊 (開發套件)

1. 如果您尚未：
   a. 建立或更新具有 AmazonRekognitionFullAccess 和 AmazonS3ReadOnlyAccess 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝及設定 AWS CLI 與 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 使用下列範例來呼叫 GetCelebrityInfo 操作。

   Java

   此範例顯示與名人相關的名稱和資訊。

   將 id 取代為「辨識影像中的名人 (p. 201)」中所顯示的其中一個名人 ID。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.GetCelebrityInfoRequest;
import com.amazonaws.services.rekognition.model.GetCelebrityInfoResult;


public class CelebrityInfo {

   public static void main(String[] args) {
      String id = "nnnnnnnn";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      GetCelebrityInfoRequest request = new GetCelebrityInfoRequest()
         .withId(id);

      System.out.println("Getting information for celebrity: " + id);

      GetCelebrityInfoResult result=rekognitionClient.getCelebrityInfo(request);

      //Display celebrity information
      System.out.println("celebrity name: " + result.getName());
      System.out.println("Further information (if available):");
      for (String url: result.getUrls()){
         System.out.println(url);
      }
   }
}
```

   AWS CLI

   此 AWS CLI 命令顯示 get-celebrity-info CLI 操作的 JSON 輸出。將 ID 取代為「辨識影像中的名人 (p. 201)」中所顯示的其中一個名人 ID。

```
aws rekognition get-celebrity-info --id ID
```

Python

此範例顯示與名人相關的名稱和資訊。

將 id 取代為「辨識影像中的名人 (p. 201)」中所顯示的其中一個名人 ID。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def get_celebrity_info(id):


    client=boto3.client('rekognition')

    #Display celebrity info
    print('Getting celebrity info for celebrity: ' + id)

    response=client.get_celebrity_info(Id=id)

    print (response['Name'])
    print ('Further information (if available):')
    for url in response['Urls']:
        print (url)

def main():
    id="nnnnnnnn"
    celebrity_info=get_celebrity_info(id)



if __name__ == "__main__":
    main()
```

.NET

此範例顯示與名人相關的名稱和資訊。

將 id 取代為「辨識影像中的名人 (p. 201)」中所顯示的其中一個名人 ID。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;


public class CelebrityInfo
{
    public static void Example()
    {
        String id = "nnnnnnnn";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();
```

```
        GetCelebrityInfoRequest celebrityInfoRequest = new
 GetCelebrityInfoRequest()
        {
            Id = id
        };

        Console.WriteLine("Getting information for celebrity: " + id);

        GetCelebrityInfoResponse celebrityInfoResponse =
 rekognitionClient.GetCelebrityInfo(celebrityInfoRequest);

        //Display celebrity information
        Console.WriteLine("celebrity name: " + celebrityInfoResponse.Name);
        Console.WriteLine("Further information (if available):");
        foreach (String url in celebrityInfoResponse.Urls)
            Console.WriteLine(url);
    }
}
```

GetCelebrityInfo 操作要求

以下是 GetCelebrityInfo 的 JSON 輸入和輸出範例。

輸入 GetCelebrityInfo 的是所需名人的 ID。

```
{
    "Id": "nnnnnnn"
}
```

GetCelebrityInfo 將連結的陣列 (Urls) 傳回至有關所要求之名人的相關資訊。

```
{
    "Name": "Celebrity Name",
    "Urls": [
        "www.imdb.com/name/nmnnnnnnn"
    ]
}
```

偵測不安全的內容

您可以使用 Amazon Rekognition 來判斷影像或儲存的影片是否包含不安全的內容，例如露骨的成人內容或暴力內容。

您可以在社交媒體、線上市場和專業媒體等各種使用案例中使用影像和影片管制 API。使用 Amazon Rekognition 來偵測不安全的內容，即可降低對不安全內容進行人為審查的需求。如果您想對不安全內容進行人工檢閱，可以使用 Amazon Augmented AI。

主題

- 使用影像和影片管制 API (p. 215)
- 偵測不安全的影像 (p. 216)
- 偵測不安全的已存放影片 (p. 220)
- 使用 Amazon Augmented AI (預覽版) 檢閱不安全的內容 (p. 224)

使用影像和影片管制 API

在 Amazon Rekognition Image API 中，您可以使用 DetectModerationLabels (p. 329) 操作來偵測影像中不安全的內容。您可以在 Amazon Rekognition Video API 中，使用 StartContentModeration (p. 403) 和 GetContentModeration (p. 344) 操作以非同步方式偵測不安全的內容。

Amazon Rekognition 使用兩層階層式分類法來標記不安全內容的類別。每個最上層類別都有數個第二層類別。

最上層類別	第二層類別
露骨裸露	裸露
	男性裸露圖片
	女性裸露圖片
	性行為
	附有插圖的裸露或性行為
	成人玩具
暗示性	女性泳衣或內衣褲
	男性泳衣或內衣褲
	部分裸露
	暴露的服裝
暴力	暴力畫面或血腥
	身體暴力

最上層類別	第二層類別
	武器暴力
	武器
	自我傷害
令人反感	異常消瘦的身體
	屍體
	絞刑

您可以為應用程式決定內容的適合性。例如，可接受影像本身具有暗示性內容，但不接受含有裸露內容。若要篩選影像，請使用 `DetectModerationLabels` (影像) 與 `GetContentModeration` (影片) 傳回的 ModerationLabel (p. 480) 標籤陣列。

您可指定 `MinConfidence` 輸入參數，以設定 Amazon Rekognition 用於偵測不安全內容的可信度閾值。系統不會對偵測到可信度比 `MinConfidence` 低的不安全內容傳回標籤。

指定低於 50% 的 `MinConfidence` 值可能會傳回大量錯誤肯定的結果。我們建議您，能使用低精確度進行偵測時，才使用低於 50% 的值。如果您不指定 `MinConfidence` 的值，Amazon Rekognition 會以至少 50% 的可信度，傳回偵測到的不安全內容標籤。

`ModerationLabel` 陣列包含前述類別中的標籤，以及對已辨識內容之正確性所估計的信賴度。最上層標籤會與任何已識別的第二層標籤一起傳回。例如，Amazon Rekognition 傳回的最上層標籤可能是具有高信賴分數的「露骨裸露」。這可能足以滿足您的篩選需求。不過，如果有必要，您也可以使用第二層標籤的信賴度分數 (例如「男性裸露圖片」)，來取得更精細的篩選。如需範例，請參閱 偵測不安全的影像 (p. 216)。

Amazon Rekognition Image 和 Amazon Rekognition Video 都會傳回用於偵測不安全內容的管制偵測模型版本 (`ModerationModelVersion`)。

> **Note**
>
> Amazon Rekognition 不是授權機構，也不會以任何方式聲稱是不安全內容的詳盡篩選條件。此外，影像和影片管制 API 不會偵測影像是否包含非法內容，例如兒童色情。

偵測不安全的影像

您可以使用 DetectModerationLabels (p. 329) 操作來判斷影像是否包含不安全的內容。

偵測影像中不安全的內容

影像的格式必須是 .jpg 或 .png。您可以用影像位元組陣列 (Base64 編碼影像位元組) 的方式提供輸入影像，或指定 Amazon S3 物件。在這些程序中，您會將影像 (.jpg 或 .png) 上傳至 S3 儲存貯體。

若要執行這些程序，您需要安裝 AWS CLI 與 AWS SDK for Java。如需詳細資訊，請參閱 Amazon Rekognition 入門 (p. 10)。您使用的 AWS 帳戶必須有 Amazon Rekognition API 的存取許可。如需詳細資訊，請參閱 Amazon Rekognition 定義的動作。

偵測影像中的管制標籤 (SDK)

1. 如果您尚未：

a. 建立或更新具有 AmazonRekognitionFullAccess 和 AmazonS3ReadOnlyAccess 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。

b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。

2. 將影像上傳至您的 S3 儲存貯體。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。

3. 使用下列範例來呼叫 DetectModerationLabels 操作。

   Java

   此範例輸出偵測到的不安全內容標籤名稱、信心度以及偵測到的管制標籤的父標籤。

   將 bucket 與 photo 的值取代為您在步驟 2 中所使用的 S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.DetectModerationLabelsRequest;
import com.amazonaws.services.rekognition.model.DetectModerationLabelsResult;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.ModerationLabel;
import com.amazonaws.services.rekognition.model.S3Object;

import java.util.List;

public class DetectModerationLabels
{
   public static void main(String[] args) throws Exception
   {
      String photo = "input.jpg";
      String bucket = "bucket";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();

      DetectModerationLabelsRequest request = new DetectModerationLabelsRequest()
        .withImage(new Image().withS3Object(new
 S3Object().withName(photo).withBucket(bucket)))
        .withMinConfidence(60F);
      try
      {
           DetectModerationLabelsResult result =
 rekognitionClient.detectModerationLabels(request);
           List<ModerationLabel> labels = result.getModerationLabels();
           System.out.println("Detected labels for " + photo);
           for (ModerationLabel label : labels)
           {
              System.out.println("Label: " + label.getName()
               + "\n Confidence: " + label.getConfidence().toString() + "%"
               + "\n Parent:" + label.getParentName());
           }
       }
       catch (AmazonRekognitionException e)
       {
         e.printStackTrace();
```

```
        }
    }
}
```

AWS CLI

此 AWS CLI 命令顯示 `detect-moderation-labels` CLI 操作的 JSON 輸出。

將 `bucket` 與 `input.jpg` 取代為您在步驟 2 中所使用的 S3 儲存貯體名稱與影像檔案名稱。

```
aws rekognition detect-moderation-labels \
--image '{"S3Object":{"Bucket":"bucket","Name":"input.jpg"}}'
```

Python

此範例輸出偵測到的不安全內容標籤名稱、信心度以及偵測到的不安全內容標籤的父標籤。

在函數 `main` 中，將 `bucket` 與 `photo` 的值取代為您在步驟 2 中所使用的 S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def moderate_image(photo, bucket):

    client=boto3.client('rekognition')

    response = client.detect_moderation_labels(Image={'S3Object':
{'Bucket':bucket,'Name':photo}})

    print('Detected labels for ' + photo)
    for label in response['ModerationLabels']:
        print (label['Name'] + ' : ' + str(label['Confidence']))
        print (label['ParentName'])
    return len(response['ModerationLabels'])



def main():
    photo='photo'
    bucket='bucket'
    label_count=moderate_image(photo, bucket)
    print("Labels detected: " + str(label_count))


if __name__ == "__main__":
    main()
```

.NET

此範例輸出偵測到的不安全內容標籤名稱、信心度以及偵測到的管制標籤的父標籤。

將 `bucket` 與 `photo` 的值取代為您在步驟 2 中所使用的 S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectModerationLabels
{
    public static void Example()
    {
        String photo = "input.jpg";
        String bucket = "bucket";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectModerationLabelsRequest detectModerationLabelsRequest = new
 DetectModerationLabelsRequest()
        {
            Image = new Image()
            {
                S3Object = new S3Object()
                {
                    Name = photo,
                    Bucket = bucket
                },
            },
            MinConfidence = 60F
        };

        try
        {
            DetectModerationLabelsResponse detectModerationLabelsResponse =
 rekognitionClient.DetectModerationLabels(detectModerationLabelsRequest);
            Console.WriteLine("Detected labels for " + photo);
            foreach (ModerationLabel label in
 detectModerationLabelsResponse.ModerationLabels)
                Console.WriteLine("Label: {0}\n Confidence: {1}\n Parent: {2}",
                    label.Name, label.Confidence, label.ParentName);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

DetectModerationLabels 操作要求

`DetectModerationLabels` 的輸入是影像。在此範例 JSON 輸入中，來源影像是從 Amazon S3 儲存貯體載入的。`MinConfidence` 是在偵測到的標籤的準確性中 Amazon Rekognition Image 必須具有的最低信賴度，以便在回應中傳回。

```
{
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "input.jpg"
        }
    },
```

```
    "MinConfidence": 60
}
```

DetectModerationLabels 操作回應

`DetectModerationLabels` 可從 S3 儲存貯體擷取輸入影像，您也可以用影像位元組的方式提供輸入影像。下列範例是來自 `DetectModerationLabels` 呼叫的回應。

在下列範例 JSON 回應中，請注意以下事項：

- 不安全的影像偵測資訊 – 此範例顯示在影像中找到之不安全內容的標籤清單。此清單包含最上層標籤與影像中偵測到的每個第二層標籤。

  標籤 – 每個標籤都具有名稱、Amazon Rekognition 對標籤正確性估計的信賴度，以及其父標籤的名稱。最上層標籤的父名稱為 ""。

  標籤信賴度 – 每個標籤都有一個介於 0 到 100 之間的信賴度數值，該數值表示 Amazon Rekognition 對標籤正確性的信賴度百分比。您可以為要在 API 操作要求回應中傳回的標籤指定要求的信賴度。

```
{
"ModerationLabels": [
    {
        "Confidence": 99.24723052978516,
        "ParentName": "",
        "Name": "Explicit Nudity"
    },
    {
        "Confidence": 99.24723052978516,
        "ParentName": "Explicit Nudity",
        "Name": "Graphic Male Nudity"
    },
    {
        "Confidence": 88.25341796875,
        "ParentName": "Explicit Nudity",
        "Name": "Sexual Activity"
    }
]
}
```

偵測不安全的已存放影片

在已存放影片的 Amazon Rekognition Video 不安全內容偵測是一種非同步操作。若要開始進行不安全內容偵測，請呼叫 StartContentModeration (p. 403)。Amazon Rekognition Video 將發佈影片分析的完成狀態至 Amazon Simple Notification Service 主題。如果影片分析成功，請呼叫 GetContentModeration (p. 344) 以取得分析結果。如需開始影片分析並取得結果的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

此程序會展開「使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)」中的程式碼，該作業使用 Amazon Simple Queue Service 佇列來取得影片分析要求的完成狀態。

偵測存放在 Amazon S3 儲存貯體 (SDK) 之影片中的不安全內容

1. 執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)。
2. 將下列程式碼新增至您在步驟 1 中建立的類別 `VideoDetect`。

Java

```
    //Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
    //PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

        //Content moderation
 ==================================================================
        private static void StartUnsafeContentDetection(String bucket, String
 video) throws Exception{

            NotificationChannel channel= new NotificationChannel()
                    .withSNSTopicArn(snsTopicArn)
                    .withRoleArn(roleArn);

            StartContentModerationRequest req = new StartContentModerationRequest()
                    .withVideo(new Video()
                            .withS3Object(new S3Object()
                                .withBucket(bucket)
                                .withName(video)))
                    .withNotificationChannel(channel);



             StartContentModerationResult startModerationLabelDetectionResult =
 rek.startContentModeration(req);
             startJobId=startModerationLabelDetectionResult.getJobId();

         }

         private static void GetUnsafeContentDetectionResults() throws Exception{

             int maxResults=10;
             String paginationToken=null;
             GetContentModerationResult moderationLabelDetectionResult =null;

             do{
                 if (moderationLabelDetectionResult !=null){
                     paginationToken =
 moderationLabelDetectionResult.getNextToken();
                 }

                 moderationLabelDetectionResult = rek.getContentModeration(
                         new GetContentModerationRequest()
                             .withJobId(startJobId)
                             .withNextToken(paginationToken)
                             .withSortBy(ContentModerationSortBy.TIMESTAMP)
                             .withMaxResults(maxResults));



                 VideoMetadata
 videoMetaData=moderationLabelDetectionResult.getVideoMetadata();

                 System.out.println("Format: " + videoMetaData.getFormat());
                 System.out.println("Codec: " + videoMetaData.getCodec());
                 System.out.println("Duration: " +
 videoMetaData.getDurationMillis());
                 System.out.println("FrameRate: " + videoMetaData.getFrameRate());


                 //Show moderated content labels, confidence and detection times
                 List<ContentModerationDetection> moderationLabelsInFrames=
                         moderationLabelDetectionResult.getModerationLabels();
```

```
                for (ContentModerationDetection label: moderationLabelsInFrames)
{
                    long seconds=label.getTimestamp()/1000;
                    System.out.print("Sec: " + Long.toString(seconds));
                    System.out.println(label.getModerationLabel().toString());
                    System.out.println();
                }
            } while (moderationLabelDetectionResult !=null &&
moderationLabelDetectionResult.getNextToken() != null);

        }
```

在函數 `main` 中，將下行:

```
        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();
```

取代為:

```
        StartUnsafeContentDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetUnsafeContentDetectionResults();
```

Python

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    # ============== Unsafe content ===============
    def StartUnsafeContent(self):
        response=self.rek.start_content_moderation(Video={'S3Object': {'Bucket':
 self.bucket, 'Name': self.video}},
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
 self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)

    def GetUnsafeContentResults(self):
        maxResults = 10
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_content_moderation(JobId=self.startJobId,
                                                MaxResults=maxResults,
                                                NextToken=paginationToken)

            print('Codec: ' + response['VideoMetadata']['Codec'])
            print('Duration: ' + str(response['VideoMetadata']['DurationMillis']))
            print('Format: ' + response['VideoMetadata']['Format'])
            print('Frame rate: ' + str(response['VideoMetadata']['FrameRate']))
            print()

            for contentModerationDetection in response['ModerationLabels']:
                print('Label: ' +
```

```
                    str(contentModerationDetection['ModerationLabel']['Name']))
                print('Confidence: ' +
                    str(contentModerationDetection['ModerationLabel']
['Confidence']))
                print('Parent category: ' +
                    str(contentModerationDetection['ModerationLabel']
['ParentName']))
                print('Timestamp: ' + str(contentModerationDetection['Timestamp']))
                print()

            if 'NextToken' in response:
                paginationToken = response['NextToken']
            else:
                finished = True
```

在函數 `main` 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    analyzer.StartUnsafeContent()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetUnsafeContentResults()
```

Note

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的影片範例， 要取代的程式碼可能會不同。

3. 執行程式碼。在影片中偵測到的不安全內容標籤清單便會顯示。

GetContentModeration 操作回應

`GetContentModeration` 的回應是 ContentModerationDetection (p. 449) 物件的陣列 `ModerationLabels`。此陣列在每次偵測到不安全內容標籤時就會包含一個元素。在 `ContentModerationDetectionObject` 物件中，ModerationLabel (p. 480) 包含已偵測到不安全內容之項目的資訊。`Timestamp` 是偵測到標籤的時間，會從影片開始起算並以毫秒為單位。標籤會按照階層排列，排列方式和不安全內容影像分析偵測到的標籤相同。如需詳細資訊，請參閱 偵測不安全的內容 (p. 215)。

以下是來自 `GetContentModeration` 的回應範例。

```
{
    "JobStatus": "SUCCEEDED",
    "ModerationLabels": [
        {
            "ModerationLabel": {
                "Confidence": 93.02153015136719,
                "Name": "Male Swimwear Or Underwear",
                "ParentName": "Suggestive"
            },
            "Timestamp": 0
        },
        {
            "ModerationLabel": {
```

```
                "Confidence": 93.02153015136719,
                "Name": "Suggestive",
                "ParentName": ""
            },
            "Timestamp": 0
        },
        {
            "ModerationLabel": {
                "Confidence": 98.29075622558594,
                "Name": "Male Swimwear Or Underwear",
                "ParentName": "Suggestive"
            },
            "Timestamp": 1000
        },
        {
            "ModerationLabel": {
                "Confidence": 98.29075622558594,
                "Name": "Suggestive",
                "ParentName": ""
            },
            "Timestamp": 1000
        },
        {
            "ModerationLabel": {
                "Confidence": 97.91191101074219,
                "Name": "Male Swimwear Or Underwear",
                "ParentName": "Suggestive"
            },
            "Timestamp": 1999
        }
    ],
    "NextToken": "w5xfYx64+QvCdGTidVVWtczKHe0JAcUFu2tJ1RgDevHRovJ
+1xej2GUDfTMWrTVn1nwSMHi9",
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 3533,
        "Format": "QuickTime / MOV",
        "FrameHeight": 1080,
        "FrameRate": 30,
        "FrameWidth": 1920
    }
}
```


使用 Amazon Augmented AI (預覽版) 檢閱不安全的內容

Amazon Augmented AI 目前為預覽版本，並可能有所變更。我們建議僅搭配測試資料庫使用此工具，不要在生產環境中使用。

Amazon Augmented AI (Amazon A2I) 可讓您建立人工檢閱機器學習預測所需的工作流程。

Amazon Rekognition 直接與 Amazon A2I 整合，因此您可以輕鬆對偵測不安全影像的使用案例實作人工檢閱。Amazon A2I 提供對於影像仲裁的人工檢閱工作流程。這可讓您輕鬆檢閱 Amazon Rekognition 的預測。您可以定義使用案例的信賴度閾值，並隨著時間調整它們。透過 Amazon A2I，您可以使用自己組織內的檢閱者集區或 Amazon Mechanical Turk。您也可以使用 AWS 預先篩選的人力廠商，以確保品質並嚴守安全程序。

下列步驟會引導您完成使用 Amazon Rekognition 設定 Amazon A2I。首先，使用 Amazon A2I 建立流程定義，該流程定義具有觸發人工檢閱的條件。然後，將流程定義的 Amazon Resource Name (ARN) 傳遞給 Amazon Rekognition `DetectModerationLabel` 操作。在 `DetectModerationLabel` 回應中，您可以查看是否需要人工檢閱。人工檢閱的結果可以在流程定義所設定的 Amazon S3 儲存貯體中取得。

使用 Amazon A2I 執行 DetectModerationLabels (預覽)

1. 完成 Amazon SageMaker 文件中 Amazon Augmented AI 入門中所列的先決條件。

   此外，請記得在 Amazon SageMaker 文件中的 Amazon Augmented AI 的許可和安全頁面中設置您的 IAM 許可。
2. 請遵循 Amazon SageMaker 文件中建立人工檢閱工作流程的指示。

   人工檢閱工作流程可管理影像的處理。它包含觸發人工檢閱的條件、影像傳送目標的工作團隊、工作團隊使用的 UI 範本，以及工作團隊結果傳送目標的 Amazon S3 儲存貯體。

   在您的 `CreateFlowDefinition` 呼叫中，您需要將 `HumanLoopRequestSource` 設定為 "AWS/Rekognition/DetectModerationLabels/Image/V3"。之後，你需要決定如何設定會觸發人工檢閱的條件。

   對於 Amazon Rekognition，所使用的 `ConditionType` 是 `ModerationLabelConfidenceCheck`，它的 `ConditionParameters` 是 `ModerationLabelName`、ConfidenceLessThan、GreaterThan、EqualTo、GreaterThanOrEqualTo 和 LessThanOrEqualTo。這些參數可讓您設定要檢查哪個仲裁標籤，以及觸發人工檢閱的閾值。除了這些選項之外，您還可以使用邏輯運算子，例如 “And”、“Or” 或 “Not” 來建立更複雜的條件。若要將已分析的所有影像傳送至人工檢閱，您可以將標籤設定為 “*”，將 ConfidenceGreater 參數設定為 0。

   下列程式碼範例是 `CreateFlowDefinition` 的部分呼叫。如果在「暗示性」標籤上的評分低於 98%，並在「女性泳衣或內衣褲」標籤上超過 95%，則會傳送影像進行人工檢閱。這意味著，如果影像不被認為是暗示性的，但是有一個穿著內衣褲或泳衣的女人，你可以藉由人工檢閱仔細檢查影像。

```
def create_flow_definition():
'''
Creates a Flow Definition resource

Returns:
struct: FlowDefinitionArn
'''
humanLoopActivationConditions = json.dumps(
    {
        "Conditions": [
            {
              "And": [
                {
                    "ConditionType": "ModerationLabelConfidenceCheck",
                    "ConditionParameters": {
                        "ModerationLabelName": "Suggestive",
                        "ConfidenceLessThan": 98
                    }
                },
                {
                    "ConditionType": "ModerationLabelConfidenceCheck",
                    "ConditionParameters": {
                        "ModerationLabelName": "Female Swimwear Or Underwear",
                        "ConfidenceGreaterThan": 95
                    }
                }
              ]
           }
        ]
    }
)
```

```
```

`CreateFlowDefinition` 會傳回 `FlowDefinitionArn`，您會在下一個步驟中呼叫 `DetectModerationLabels` 時使用。

如需詳細資訊，請參閱 Amazon SageMaker API 參考中的 CreateFlowDefinition。

3. 呼叫 `DetectModerationLabels` 時設定 `HumanLoopConfig` 參數，如同 偵測不安全的影像 (p. 216) 中所示。

   a. 在 `HumanLoopConfig` 參數內，將 `FlowDefinitionArn` 設定為您在步驟 2 中所建立流量定義的 ARN。
   b. 設定您的 `HumanLoopName`。這應該是區域中唯一的，且必須是小寫的。
   c. (選用) 您可以使用 `DataAttributes` 來設定傳遞至 Amazon Rekognition 的影像是否沒有個人識別資訊。您必須設定此參數才能將資訊傳送至 Amazon Mechanical Turk。

   以下是對於 `DetectModerationLabels` 呼叫之外觀的範例 (已在 Python 中設定 `HumanLoopConfig`)。

```
        client.detect_moderation_labels(Image={'S3Object':
{'Bucket':Bucket,'Name':photo}},
        HumanLoopConfig={'FlowDefinitionArn':string,'HumanLoopName':string})
```

4. 執行 `DetectModerationLabels`。

   當您在已啟用 `HumanLoopConfig` 的情況下執行 `DetectModerationLabels` 時，Amazon Rekognition 會呼叫 Amazon SageMaker API 操作 `StartHumanLoop`。此命令會從 `DetectModerationLabels` 中取得回應，並根據範例中的流量定義條件來檢查它。如果它符合檢閱的條件，則會傳回 `HumanLoopArn`。這表示您在流程定義中設定的工作團隊成員現在可以檢閱影像。呼叫 Amazon Augmented AI 執行時間操作 `DescribeHumanLoop` 會提供迴圈結果的相關資訊。如需詳細資訊，請參閱 Amazon Augmented AI API 參考文件中的 DescribeHumanLoop。

   檢閱影像之後，您可以在流程定義輸出路徑中指定的儲存貯體中查看結果。當檢閱完成時，Amazon A2I 也會通知您有關 Amazon CloudWatch Events。若要查看要尋找的事件，請參閱 Amazon SageMaker 文件中的 CloudWatch Events。

   如需詳細資訊，請參閱 Amazon SageMaker 文件中的 Amazon Augmented AI 入門。

偵測文字

Amazon Rekognition 文字偵測可以偵測影像和視訊中的文字。然後，它可以將偵測到的文字轉換為機器可讀取的文字。您可以使用影像中的機器可讀取文字偵測來實作解決方案，例如：

- 視覺搜尋。例如，擷取和顯示包含相同文字的影像。
- 內容見解。例如，針對發生在擷取的視訊影格中所辨識到的文字之主題提供見解。您的應用程式可以搜尋辨識出的文字以找出相關內容—例如，新聞、運動比數、運動員號碼和字幕。
- 導覽。例如，開發支援語音功能的手機應用程式，協助視障使用者辨識餐廳名稱、商店或街道指標。
- 公共安全與交通協助。例如，從交通監視器影像中偵測車牌號碼。
- 篩選。例如，從影像中篩選出可辨識個人身分的資訊。

對於視訊中的文字偵測，您可以實作解決方案，例如：

- 搜尋視訊中包含特定文字關鍵字的片段，例如新聞節目畫面中的來賓姓名
- 合規與管制 - 偵測意外的文字、褻瀆或垃圾郵件
- 尋找視訊時間軸上的所有文字浮水印以執行進一步處理，例如用其他語言替換文字以達成內容國際化
- 尋找文字位置，以便可以相應地對齊其他圖形

DetectText (p. 333) 偵測 .jpeg 或 .png 格式影像中的文字。您可以使用 Amazon Rekognition Video API 來偵測與 the section called “StartTextDetection” (p. 427) 和 the section called “GetTextDetection” (p. 367) 非同步視訊中的文字。影像和視訊文字偵測 API 都支援大多數字型，包括高度風格化的字型。偵測文字之後，Amazon Rekognition 會建立偵測到的字詞和文字行的代表法，顯示兩者間的關聯，並顯示文字在影像或視訊影格中的位置。

`DetectText` 和 `GetTextDetection` 可偵測字詞和文字行。

字詞是一或多個 ISO 基本拉丁文字字元，不能以空格分隔。 `DetectText` 最多可以在一個影格中偵測 50 個字詞，而且 `GetTextDetection` 每個視訊影格最多可偵測 50 個字詞。Amazon Rekognition 也可以偵測數字和常見符號，例如 @、/、$、%、-、_、+、* 以及 #。

文字行是一串等距的字詞。一行文字不一定為完整句子。例如，駕照號碼偵測結果為一行文字。一行文字的結尾後方不會有對齊的文字。同時，當單字間的間隔與單字的長度相比出現較大間隔時，便是一行文字的結尾。根據單字間的間隔，這表示 Amazon Rekognition 可能會偵測以相同方向對齊的多行文字。句點不代表一行文字的結尾。如果句子跨越多行，此操作會傳回多行文字。

請參考下列影像：



藍色方塊代表關於偵測到的文字和 `DetectText` 作業 (在影像中) 或 `GetTextDetection` 作業 (在視訊的單一影格中) 傳回之文字位置的資訊。在此範例中，Amazon Rekognition 會將「IT's」、「MONDAY」、「but」、「keep」和「Smiling」偵測為單字。文字必須在水平軸 +/- 90 度方向的範圍內才可被偵測到。

如需範例，請參閱 在影像中偵測文字 (p. 228)。

在影像中偵測文字

您提供的輸入影像可以是影像位元組陣列 (base64 編碼影像位元組)，或者 Amazon S3 物件。在此步驟中，需上傳 .jpeg 或 .png 影像到 S3 儲存貯體，並指定檔案名稱。

若要偵測影像中的文字 (API)

1. 如果您尚未：
   a. 建立或更新具有 `AmazonRekognitionFullAccess` 和 `AmazonS3ReadOnlyAccess` 許可的 IAM 使用者。如需詳細資訊，請參閱 步驟 1：設定 AWS 帳戶，建立一個 IAM 使用者 (p. 10)。
   b. 安裝並設定 AWS CLI 和 AWS SDK。如需詳細資訊，請參閱 步驟 2：設定 AWS CLI 和 AWS 開發套件 (p. 11)。
2. 上傳包含文字的影像到您的 S3 儲存貯體。

   如需指示，請參閱 Amazon Simple Storage Service Console User Guide 中的將物件上傳至 Amazon S3。
3. 使用下列範例來呼叫 `DetectText` 操作。

   Java

   以下範例程式碼會顯示在影像中偵測到的行和文字。

   將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

package aws.example.rekognition.image;
import com.amazonaws.services.rekognition.AmazonRekognition;
import com.amazonaws.services.rekognition.AmazonRekognitionClientBuilder;
import com.amazonaws.services.rekognition.model.AmazonRekognitionException;
import com.amazonaws.services.rekognition.model.Image;
import com.amazonaws.services.rekognition.model.S3Object;
import com.amazonaws.services.rekognition.model.DetectTextRequest;
import com.amazonaws.services.rekognition.model.DetectTextResult;
import com.amazonaws.services.rekognition.model.TextDetection;
import java.util.List;



public class DetectText {

   public static void main(String[] args) throws Exception {


      String photo = "inputtext.jpg";
      String bucket = "bucket";

      AmazonRekognition rekognitionClient =
 AmazonRekognitionClientBuilder.defaultClient();



      DetectTextRequest request = new DetectTextRequest()
              .withImage(new Image()
              .withS3Object(new S3Object()
              .withName(photo)
              .withBucket(bucket)));
```

```
      try {
         DetectTextResult result = rekognitionClient.detectText(request);
         List<TextDetection> textDetections = result.getTextDetections();

         System.out.println("Detected lines and words for " + photo);
         for (TextDetection text: textDetections) {

                 System.out.println("Detected: " + text.getDetectedText());
                 System.out.println("Confidence: " +
 text.getConfidence().toString());
                 System.out.println("Id : " + text.getId());
                 System.out.println("Parent Id: " + text.getParentId());
                 System.out.println("Type: " + text.getType());
                 System.out.println();
         }
      } catch(AmazonRekognitionException e) {
         e.printStackTrace();
      }
   }
}
```

AWS CLI

此 AWS CLI 命令顯示 `detect-text` CLI 操作的 JSON 輸出。

將 `Bucket` 與 `Name` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
aws rekognition detect-text \
--image "S3Object={Bucket=bucketname,Name=input.jpg}"
```

Python

以下範例程式碼會顯示在影像中偵測到的行和文字。

將 `bucket` 與 `photo` 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
#Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

import boto3

def detect_text(photo, bucket):

    client=boto3.client('rekognition')

    response=client.detect_text(Image={'S3Object':{'Bucket':bucket,'Name':photo}})

    textDetections=response['TextDetections']
    print ('Detected text\n----------')
    for text in textDetections:
            print ('Detected text:' + text['DetectedText'])
            print ('Confidence: ' + "{:.2f}".format(text['Confidence']) + "%")
            print ('Id: {}'.format(text['Id']))
            if 'ParentId' in text:
                print ('Parent Id: {}'.format(text['ParentId']))
            print ('Type:' + text['Type'])
            print()
```

```
    return len(textDetections)

def main():

    bucket='bucket'
    photo='photo'
    text_count=detect_text(photo,bucket)
    print("Text detected: " + str(text_count))


if __name__ == "__main__":
    main()
```

.NET

以下範例程式碼會顯示在影像中偵測到的行和文字。

將 bucket 與 photo 的數值取代為您在步驟 2 中所使用的 Amazon S3 儲存貯體名稱與影像檔案名稱。

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

using System;
using Amazon.Rekognition;
using Amazon.Rekognition.Model;

public class DetectText
{
    public static void Example()
    {
        String photo = "input.jpg";
        String bucket = "bucket";

        AmazonRekognitionClient rekognitionClient = new AmazonRekognitionClient();

        DetectTextRequest detectTextRequest = new DetectTextRequest()
        {
            Image = new Image()
            {
                S3Object = new S3Object()
                {
                    Name = photo,
                    Bucket = bucket
                }
            }
        };

        try
        {
            DetectTextResponse detectTextResponse =
 rekognitionClient.DetectText(detectTextRequest);
            Console.WriteLine("Detected lines and words for " + photo);
            foreach (TextDetection text in detectTextResponse.TextDetections)
            {
                Console.WriteLine("Detected: " + text.DetectedText);
                Console.WriteLine("Confidence: " + text.Confidence);
                Console.WriteLine("Id : " + text.Id);
                Console.WriteLine("Parent Id: " + text.ParentId);
                Console.WriteLine("Type: " + text.Type);
            }
        }
        catch (Exception e)
```

```
        {
            Console.WriteLine(e.Message);
        }
    }
}
```

DetectText 操作要求

在 DetectText 操作中，提供 base64 編碼位元組陣列的輸入影像或是存放於 Amazon S3 儲存貯體中的影像。以下範例 JSON 請求顯示從 Amazon S3 儲存貯體載入的影像。

```
{
    "Image": {
        "S3Object": {
            "Bucket": "bucket",
            "Name": "inputtext.jpg"
        }
    }
}
```

篩選條件

按文字區域、大小和可信度分數進行篩選可為您提供更大的靈活性來控製文字偵測輸出。藉由使用感興趣的區域，您可以輕鬆地將文字偵測限制在與您相關的區域，例如，從機器影像讀取零件編號時，個人檔案相片右上角或相對於參考點的固定位置 。文字週框方塊大小篩選器可用於避免產生嘈雜或無關緊要的小背景文字。最後，文字可信度篩選器讓您可以消除因朦朧或模糊而導致的不可靠結果。您可以使用下列篩選器：

- MinConfidence – 設定文字偵測的可信度。可信度低於此等級的文字會從結果中排除。值應該介於 0.5 和 1 之間。預設的 MinConfidence 是 0.5。
- MinBoundingBoxWidth – 設定文字週框方塊的最小寬度。週邊方塊小於此值的文字會從結果中排除。此值相對於影像影格寬度。
- MinBoundingBoxHeight – 設定文字週邊方塊的最小高度。週邊方塊高度小於此值的文字會從結果中排除。此值相對於影像影格高度。
- RegionsOfInterest – 將偵測限制為影像影格的特定區域。這些值相對於影格尺寸。對於僅部分區域內的文字，回應是未定義的。

DetectText 操作回應

DetectText 操作將分析影像並傳回陣列 TextDetections，其中每個元素 (TextDetection (p. 506)) 代表在影像中偵測到的一行文字或單字。針對每個元素，DetectText 會傳回以下資訊：

- 偵測到的文字 (DetectedText)
- 單字與文字行之間的關係 (Id 和 ParentId)
- 文字在影像中的位置 (Geometry)
- Amazon Rekognition 對於文字與週框方塊偵測精確度的可信度 (Confidence)
- 偵測到的文字類型 (Type)

偵測到的文字

每個 TextDetection 元素包含辨識到的文字 (單字或行) DetectedText 欄位。傳回的文字可能包含字元，讓單字無法辨識。例如，C@t 而不是 Cat。若要判斷 TextDetection 元素代表的是一行文字或單字，請使用 Type 欄位。

每個 TextDetection 元素都包含百分比值，代表 Amazon Rekognition 對於文字與文字周圍的週框方塊之偵測精確度的可信度。

單字和行的關係

每個 TextDetection 元素都有一個辨識碼欄位，Id。Id 會顯示單字在行中的位置。如果元素是一個單字，父系識別碼欄位 ParentId 將找出偵測到該單字的行。行的 ParentId 為 null。例如，範例影像中的「but keep」文字行具有下列 Id 和 ParentId 值：

文字	ID	父系 ID
但保留	3	
但	8	3
保留	9	3

文字在影像中的位置

若要判定辨識到的文字在影像中的位置，請使用 DetectText 傳回的週框方塊 (Geometry (p. 467)) 資訊。Geometry 物件包含用於偵測到的行和單字之兩種類型的週框方塊資訊：

- 在 BoundingBox (p. 440) 物件中的對齊軸粗略矩形外框
- 在 Point (p. 489) 陣列中由多個 X 與 Y 座標組成的精密多邊形

週框方塊與多邊形座標將顯示文字在影像中所在的位置。座標值為整體影像大小的比例。如需詳細資訊，請參閱 BoundingBox (p. 440)。

下列來自 DetectText 操作的 JSON 回應將顯示在下列影像中偵測到的單字與行。




```
{
    "TextDetections": [
        {
            "Confidence": 90.54900360107422,
            "DetectedText": "IT'S",
            "Geometry": {
                "BoundingBox": {
                    "Height": 0.10317354649305344,
                    "Left": 0.6677391529083252,
                    "Top": 0.17569075524806976,
                    "Width": 0.15113449096679688
                },
                "Polygon": [
                    {
                        "X": 0.6677391529083252,
                        "Y": 0.17569075524806976
                    },
                    {
                        "X": 0.8188736438751221,
                        "Y": 0.17574213445186615
                    },
```

```
                {
                    "X": 0.8188582062721252,
                    "Y": 0.278915673494339
                },
                {
                    "X": 0.6677237153053284,
                    "Y": 0.2788642942905426
                }
            ]
        },
        "Id": 0,
        "Type": "LINE"
    },
    {
        "Confidence": 59.411651611328125,
        "DetectedText": "I",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.05955825746059418,
                "Left": 0.2763049304485321,
                "Top": 0.394121915102005,
                "Width": 0.026684552431106567
            },
            "Polygon": [
                {
                    "X": 0.2763049304485321,
                    "Y": 0.394121915102005
                },
                {
                    "X": 0.30298948287963867,
                    "Y": 0.3932435214519501
                },
                {
                    "X": 0.30385109782218933,
                    "Y": 0.45280176401138306
                },
                {
                    "X": 0.27716654539108276,
                    "Y": 0.453680157661438
                }
            ]
        },
        "Id": 1,
        "Type": "LINE"
    },
    {
        "Confidence": 92.76634979248047,
        "DetectedText": "MONDAY",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.11997425556182861,
                "Left": 0.5545867085456848,
                "Top": 0.34920141100883484,
                "Width": 0.39841532707214355
            },
            "Polygon": [
                {
                    "X": 0.5545867085456848,
                    "Y": 0.34920141100883484
                },
                {
                    "X": 0.9530020356178284,
                    "Y": 0.3471102714538574
                },
                {
                    "X": 0.9532787799835205,
```

```
                    "Y": 0.46708452701568604
                },
                {
                    "X": 0.554863452911377,
                    "Y": 0.46917566657066345
                }
            ]
        },
        "Id": 2,
        "Type": "LINE"
    },
    {
        "Confidence": 96.7636489868164,
        "DetectedText": "but keep",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.0756164938211441,
                "Left": 0.634815514087677,
                "Top": 0.5181083083152771,
                "Width": 0.20877975225448608
            },
            "Polygon": [
                {
                    "X": 0.634815514087677,
                    "Y": 0.5181083083152771
                },
                {
                    "X": 0.8435952663421631,
                    "Y": 0.52589350938797
                },
                {
                    "X": 0.8423560857772827,
                    "Y": 0.6015099883079529
                },
                {
                    "X": 0.6335763335227966,
                    "Y": 0.59372478723526
                }
            ]
        },
        "Id": 3,
        "Type": "LINE"
    },
    {
        "Confidence": 99.47185516357422,
        "DetectedText": "Smiling",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.2814019024372101,
                "Left": 0.48475268483161926,
                "Top": 0.6823741793632507,
                "Width": 0.47539761662483215
            },
            "Polygon": [
                {
                    "X": 0.48475268483161926,
                    "Y": 0.6823741793632507
                },
                {
                    "X": 0.9601503014564514,
                    "Y": 0.587857186794281
                },
                {
                    "X": 0.9847385287284851,
                    "Y": 0.8692590594291687
                },
```

```
                {
                    "X": 0.5093409419059753,
                    "Y": 0.9637760519981384
                }
            ]
        },
        "Id": 4,
        "Type": "LINE"
    },
    {
        "Confidence": 90.54900360107422,
        "DetectedText": "IT'S",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.10387301445007324,
                "Left": 0.6685508489608765,
                "Top": 0.17597118020057678,
                "Width": 0.14985692501068115
            },
            "Polygon": [
                {
                    "X": 0.6677391529083252,
                    "Y": 0.17569075524806976
                },
                {
                    "X": 0.8188736438751221,
                    "Y": 0.17574213445186615
                },
                {
                    "X": 0.8188582062721252,
                    "Y": 0.278915673494339
                },
                {
                    "X": 0.6677237153053284,
                    "Y": 0.2788642942905426
                }
            ]
        },
        "Id": 5,
        "ParentId": 0,
        "Type": "WORD"
    },
    {
        "Confidence": 92.76634979248047,
        "DetectedText": "MONDAY",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.11929994821548462,
                "Left": 0.5540683269500732,
                "Top": 0.34858056902885437,
                "Width": 0.3998897075653076
            },
            "Polygon": [
                {
                    "X": 0.5545867085456848,
                    "Y": 0.34920141100883484
                },
                {
                    "X": 0.9530020356178284,
                    "Y": 0.3471102714538574
                },
                {
                    "X": 0.9532787799835205,
                    "Y": 0.46708452701568604
                },
                {
```

```
                    "X": 0.554863452911377,
                    "Y": 0.46917566657066345
                }
            ]
        },
        "Id": 7,
        "ParentId": 2,
        "Type": "WORD"
    },
    {
        "Confidence": 59.411651611328125,
        "DetectedText": "I",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.05981886386871338,
                "Left": 0.2779299318790436,
                "Top": 0.3935416042804718,
                "Width": 0.02624112367630005
            },
            "Polygon": [
                {
                    "X": 0.2763049304485321,
                    "Y": 0.394121915102005
                },
                {
                    "X": 0.30298948287963867,
                    "Y": 0.3932435214519501
                },
                {
                    "X": 0.30385109782218933,
                    "Y": 0.45280176401138306
                },
                {
                    "X": 0.27716654539108276,
                    "Y": 0.453680157661438
                }
            ]
        },
        "Id": 6,
        "ParentId": 1,
        "Type": "WORD"
    },
    {
        "Confidence": 95.33189392089844,
        "DetectedText": "but",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.06849122047424316,
                "Left": 0.6350157260894775,
                "Top": 0.5214487314224243,
                "Width": 0.08413040637969971
            },
            "Polygon": [
                {
                    "X": 0.6347596645355225,
                    "Y": 0.5215170383453369
                },
                {
                    "X": 0.719483494758606,
                    "Y": 0.5212655067443848
                },
                {
                    "X": 0.7195737957954407,
                    "Y": 0.5904868841171265
                },
                {
```

```
                    "X": 0.6348499655723572,
                    "Y": 0.5907384157180786
                }
            ]
        },
        "Id": 8,
        "ParentId": 3,
        "Type": "WORD"
    },
    {
        "Confidence": 98.1954116821289,
        "DetectedText": "keep",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.07207882404327393,
                "Left": 0.7295929789543152,
                "Top": 0.5265749096870422,
                "Width": 0.11196041107177734
            },
            "Polygon": [
                {
                    "X": 0.7290706038475037,
                    "Y": 0.5251666903495789
                },
                {
                    "X": 0.842876672744751,
                    "Y": 0.5268880724906921
                },
                {
                    "X": 0.8423973917961121,
                    "Y": 0.5989891886711121
                },
                {
                    "X": 0.7285913228988647,
                    "Y": 0.5972678065299988
                }
            ]
        },
        "Id": 9,
        "ParentId": 3,
        "Type": "WORD"
    },
    {
        "Confidence": 99.47185516357422,
        "DetectedText": "Smiling",
        "Geometry": {
            "BoundingBox": {
                "Height": 0.3739858865737915,
                "Left": 0.48920923471450806,
                "Top": 0.5900818109512329,
                "Width": 0.5097314119338989
            },
            "Polygon": [
                {
                    "X": 0.48475268483161926,
                    "Y": 0.6823741793632507
                },
                {
                    "X": 0.9601503014564514,
                    "Y": 0.587857186794281
                },
                {
                    "X": 0.9847385287284851,
                    "Y": 0.8692590594291687
                },
                {
```

```
                    "X": 0.5093409419059753,
                    "Y": 0.9637760519981384
                }
            ]
        },
        "Id": 10,
        "ParentId": 4,
        "Type": "WORD"
    }
  ]
}
```

偵測已儲存影片中的文字

Amazon Rekognition Video 已儲存影片中的文字偵測是一項非同步操作。若要開始偵測文字，請呼叫 the section called “StartTextDetection” (p. 427)。Amazon Rekognition Video 將發佈影片分析的完成狀態至 Amazon SNS 主題。如果影片分析成功，請呼叫 the section called “GetTextDetection” (p. 367) 以取得分析結果。如需開始影片分析並取得結果的詳細資訊，請參閱 呼叫 Amazon Rekognition Video 操作 (p. 51)。

此程序會在 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 中展開程式碼。它使用 Amazon SQS 佇列來獲取視訊分析要求的完成狀態。

偵測存放在 Amazon S3 儲存貯體 (SDK) 之影片中的文字

1. 請執行使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56)中的步驟。
2. 將下列程式碼加入至步驟 1 中的類別 `VideoDetect`。

   Java

```
//Copyright 2018 Amazon.com, Inc. or its affiliates. All Rights Reserved.
//PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/
amazon-rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)


private static void StartTextDetection(String bucket, String video) throws
 Exception{

    NotificationChannel channel= new NotificationChannel()
            .withSNSTopicArn(snsTopicArn)
            .withRoleArn(roleArn);

    StartTextDetectionRequest req = new StartTextDetectionRequest()
            .withVideo(new Video()
                    .withS3Object(new S3Object()
                        .withBucket(bucket)
                        .withName(video)))
            .withNotificationChannel(channel);


    StartTextDetectionResult startTextDetectionResult =
 rek.startTextDetection(req);
    startJobId=startTextDetectionResult.getJobId();

}

private static void GetTextDetectionResults() throws Exception{

    int maxResults=10;
    String paginationToken=null;
    GetTextDetectionResult textDetectionResult=null;
```

```
    do{
        if (textDetectionResult !=null){
            paginationToken = textDetectionResult.getNextToken();

        }


        textDetectionResult = rek.getTextDetection(new GetTextDetectionRequest()
             .withJobId(startJobId)
             .withNextToken(paginationToken)
             .withMaxResults(maxResults));

        VideoMetadata videoMetaData=textDetectionResult.getVideoMetadata();

        System.out.println("Format: " + videoMetaData.getFormat());
        System.out.println("Codec: " + videoMetaData.getCodec());
        System.out.println("Duration: " + videoMetaData.getDurationMillis());
        System.out.println("FrameRate: " + videoMetaData.getFrameRate());


        //Show text, confidence values
        List<TextDetectionResult> textDetections =
 textDetectionResult.getTextDetections();


        for (TextDetectionResult text: textDetections) {
            long seconds=text.getTimestamp()/1000;
            System.out.println("Sec: " + Long.toString(seconds) + " ");
            TextDetection detectedText=text.getTextDetection();

            System.out.println("Text Detected: " + detectedText.getDetectedText());
                System.out.println("Confidence: " +
 detectedText.getConfidence().toString());
                System.out.println("Id : " + detectedText.getId());
                System.out.println("Parent Id: " + detectedText.getParentId());
                System.out.println("Bounding Box" +
 detectedText.getGeometry().getBoundingBox().toString());
                System.out.println("Type: " + detectedText.getType());
                System.out.println();
        }
    } while (textDetectionResult !=null && textDetectionResult.getNextToken() !=
 null);


}
```

在函數 `main` 中，將下行:

```
        StartLabelDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetLabelDetectionResults();
```

取代為:

```
        StartTextDetection(bucket, video);

        if (GetSQSMessageSuccess()==true)
         GetTextDetectionResults();
```

Python

```
#Copyright 2019 Amazon.com, Inc. or its affiliates. All Rights Reserved.
#PDX-License-Identifier: MIT-0 (For details, see https://github.com/awsdocs/amazon-
rekognition-developer-guide/blob/master/LICENSE-SAMPLECODE.)

    def StartTextDetection(self):
        response=self.rek.start_text_detection(Video={'S3Object': {'Bucket':
 self.bucket, 'Name': self.video}},
            NotificationChannel={'RoleArn': self.roleArn, 'SNSTopicArn':
 self.snsTopicArn})

        self.startJobId=response['JobId']
        print('Start Job Id: ' + self.startJobId)

    def GetTextDetectionResults(self):
        maxResults = 10
        paginationToken = ''
        finished = False

        while finished == False:
            response = self.rek.get_text_detection(JobId=self.startJobId,
                                        MaxResults=maxResults,
                                        NextToken=paginationToken)

            print('Codec: ' + response['VideoMetadata']['Codec'])

            print('Duration: ' + str(response['VideoMetadata']['DurationMillis']))
            print('Format: ' + response['VideoMetadata']['Format'])
            print('Frame rate: ' + str(response['VideoMetadata']['FrameRate']))
            print()

            for textDetection in response['TextDetections']:
                text=textDetection['TextDetection']

                print("Timestamp: " + str(textDetection['Timestamp']))
                print("   Text Detected: " + text['DetectedText'])
                print("   Confidence: " +  str(text['Confidence']))
                print ("      Bounding box")
                print ("        Top: " + str(text['Geometry']['BoundingBox']
['Top']))
                print ("        Left: " + str(text['Geometry']['BoundingBox']
['Left']))
                print ("        Width: " +  str(text['Geometry']['BoundingBox']
['Width']))
                print ("        Height: " +  str(text['Geometry']['BoundingBox']
['Height']))
                print ("   Type: " + str(text['Type']) )
                print()

                if 'NextToken' in response:
                    paginationToken = response['NextToken']
                else:
                    finished = True
```

在函數 main 中，將下行:

```
    analyzer.StartLabelDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetLabelDetectionResults()
```

取代為:

```
    analyzer.StartTextDetection()
    if analyzer.GetSQSMessageSuccess()==True:
        analyzer.GetTextDetectionResults()
```

Note

如果您已執行 使用 Java 或 Python (開發套件) 分析儲存在 Amazon S3 儲存貯體中的影片 (p. 56) 以外的影片範例，要取代的程式碼可能會不同。

3. 執行程式碼。在視訊中偵測到的文字會顯示在清單中。

篩選條件

篩選器是可選的要求參數，當您呼叫 `StartTextDetection` 時，會使用此篩選器。按文字區域、大小和可信度分數進行篩選可為您提供更大的靈活性來控製文字偵測輸出。透過感興趣的區域，您可以輕鬆地將文字偵測限制在與您相關的區域，例如，圖形的底部第三區域或足球遊戲中讀取記分牌的左上角。文字週框方塊大小篩選器可用於避免產生嘈雜或無關緊要的小背景文字。最後，文字可信度篩選器讓您可以消除因朦朧或模糊而導致的不可靠結果。您可以使用下列篩選器：

- MinConfidence – 設定文字偵測的可信度。可信度低於此等級的文字會從結果中排除。值應該介於 0.5 和 1 之間。預設的 MinConfidence 是 0.8
- MinBoundingBoxWidth – 設定文字週框方塊的最小寬度。週邊方塊小於此值的文字會從結果中排除。此值相對於視訊影格寬度。
- MinBoundingBoxHeight – 設定文字週邊方塊的最小高度。週邊方塊高度小於此值的文字會從結果中排除。此值相對於視訊影格高度。
- RegionsOfInterest – 將偵測限制為影格的特定區域。這些值是對於影格尺寸。對於僅部分區域內的物件，回應是未定義的。

GetTextDetection 回應

`GetTextDetection` 會傳回陣列 (`TextDetectionResults`)，其中包含視訊中偵測到之文字的相關資訊。每當在影片中偵測到文字或文字行時，就會有一個陣列元素 TextDetection (p. 506)。陣列元素會從影片開頭起依時間 (以毫秒為單位) 排序。

以下是 `GetTextDetection` 的部分 JSON 回應。在回應中，請注意下列事項：

- 文字資訊 – `TextDetectionResult`陣列元素包含偵測到之文字的資訊 (TextDetection (p. 506))，以及在影片中偵測到文字的時間 (`Timestamp`)。
- 分頁資訊 – 該範例顯示一頁文字偵測資訊。您可以在 `GetTextDetection` 的 `MaxResults` 輸入參數中指定要傳回的文字元素數目。如果結果數目超過 `MaxResults`，或結果數目超過預設最大值，`GetTextDetection` 會傳回用來取得下一頁結果的字符 (`NextToken`)。如需詳細資訊，請參閱 取得 Amazon Rekognition Video 分析結果 (p. 53)。
- 視訊資訊 – 回應包含 `GetTextDetection` 傳回之每頁資訊中視訊格式 (`VideoMetadata`) 的相關資訊。

```
{
    "JobStatus": "SUCCEEDED",
    "VideoMetadata": {
        "Codec": "h264",
        "DurationMillis": 174441,
        "Format": "QuickTime / MOV",
        "FrameRate": 29.970029830932617,
```

```
        "FrameHeight": 480,
        "FrameWidth": 854
    },
    "TextDetections": [
        {
            "Timestamp": 967,
            "TextDetection": {
                "DetectedText": "Twinkle Twinkle Little Star",
                "Type": "LINE",
                "Id": 0,
                "Confidence": 99.91780090332031,
                "Geometry": {
                    "BoundingBox": {
                        "Width": 0.8337579369544983,
                        "Height": 0.08365312218666077,
                        "Left": 0.08313830941915512,
                        "Top": 0.4663468301296234
                    },
                    "Polygon": [
                        {
                            "X": 0.08313830941915512,
                            "Y": 0.4663468301296234
                        },
                        {
                            "X": 0.9168962240219116,
                            "Y": 0.4674469828605652
                        },
                        {
                            "X": 0.916861355304718,
                            "Y": 0.5511001348495483
                        },
                        {
                            "X": 0.08310343325138092,
                            "Y": 0.5499999523162842
                        }
                    ]
                }
            }
        },
        {
            "Timestamp": 967,
            "TextDetection": {
                "DetectedText": "Twinkle",
                "Type": "WORD",
                "Id": 1,
                "ParentId": 0,
                "Confidence": 99.98338317871094,
                "Geometry": {
                    "BoundingBox": {
                        "Width": 0.2423887550830841,
                        "Height": 0.0833333358168602,
                        "Left": 0.08313817530870438,
                        "Top": 0.46666666865348816
                    },
                    "Polygon": [
                        {
                            "X": 0.08313817530870438,
                            "Y": 0.46666666865348816
                        },
                        {
                            "X": 0.3255269229412079,
                            "Y": 0.46666666865348816
                        },
                        {
                            "X": 0.3255269229412079,
                            "Y": 0.550000011920929
```

```
                    },
                    {
                        "X": 0.08313817530870438,
                        "Y": 0.550000011920929
                    }
                ]
            }
        }
    },
    {
        "Timestamp": 967,
        "TextDetection": {
            "DetectedText": "Twinkle",
            "Type": "WORD",
            "Id": 2,
            "ParentId": 0,
            "Confidence": 99.982666015625,
            "Geometry": {
                "BoundingBox": {
                    "Width": 0.2423887550830841,
                    "Height": 0.08124999701976776,
                    "Left": 0.3454332649707794,
                    "Top": 0.46875
                },
                "Polygon": [
                    {
                        "X": 0.3454332649707794,
                        "Y": 0.46875
                    },
                    {
                        "X": 0.5878220200538635,
                        "Y": 0.46875
                    },
                    {
                        "X": 0.5878220200538635,
                        "Y": 0.550000011920929
                    },
                    {
                        "X": 0.3454332649707794,
                        "Y": 0.550000011920929
                    }
                ]
            }
        }
    },
    {
        "Timestamp": 967,
        "TextDetection": {
            "DetectedText": "Little",
            "Type": "WORD",
            "Id": 3,
            "ParentId": 0,
            "Confidence": 99.8787612915039,
            "Geometry": {
                "BoundingBox": {
                    "Width": 0.16627635061740875,
                    "Height": 0.08124999701976776,
                    "Left": 0.6053864359855652,
                    "Top": 0.46875
                },
                "Polygon": [
                    {
                        "X": 0.6053864359855652,
                        "Y": 0.46875
                    },
                    {
```

```
                            "X": 0.7716627717018127,
                            "Y": 0.46875
                        },
                        {
                            "X": 0.7716627717018127,
                            "Y": 0.550000011920929
                        },
                        {
                            "X": 0.6053864359855652,
                            "Y": 0.550000011920929
                        }
                    ]
                }
            }
        },
        {
            "Timestamp": 967,
            "TextDetection": {
                "DetectedText": "Star",
                "Type": "WORD",
                "Id": 4,
                "ParentId": 0,
                "Confidence": 99.82640075683594,
                "Geometry": {
                    "BoundingBox": {
                        "Width": 0.12997658550739288,
                        "Height": 0.08124999701976776,
                        "Left": 0.7868852615356445,
                        "Top": 0.46875
                    },
                    "Polygon": [
                        {
                            "X": 0.7868852615356445,
                            "Y": 0.46875
                        },
                        {
                            "X": 0.9168618321418762,
                            "Y": 0.46875
                        },
                        {
                            "X": 0.9168618321418762,
                            "Y": 0.550000011920929
                        },
                        {
                            "X": 0.7868852615356445,
                            "Y": 0.550000011920929
                        }
                    ]
                }
            }
        }
    ],
    "NextToken": "NiHpGbZFnkM/S8kLcukMni15wb05iKtquu/Mwc+Qg1LVlMjjKNOD0Z0GusSPg7TONLe
+OZ3P",
    "TextModelVersion": "3.0"
}
```

Amazon Rekognition 安全性

AWS中的雲端安全最為優先。身為 AWS 的客戶，您將能從資料中心和網路架構中獲益，這些都是專為最重視安全的組織而設計的。

請使用下列主題來了解如何保護您的 Amazon Rekognition 資源。

主題

- 適用於 Amazon Rekognition 的 Identity and Access Management (p. 245)
- Amazon Rekognition 的資料保護 (p. 256)
- 監控 Rekognition (p. 258)
- 使用 AWS CloudTrail 記錄 Amazon Rekognition API 呼叫 (p. 261)
- 使用 Amazon Rekognition 搭配 Amazon VPC 端點 (p. 267)
- Amazon Rekognition 的合規驗證 (p. 268)
- Amazon Rekognition 中的彈性 (p. 269)
- Amazon Rekognition 中的組態與漏洞分析 (p. 269)
- Amazon Rekognition 中的基礎設施安全 (p. 269)

適用於 Amazon Rekognition 的 Identity and Access Management

AWS Identity and Access Management (IAM) is an AWS service that helps an administrator securely control access to AWS resources. IAM administrators control who can be authenticated (signed in) and authorized (have permissions) to use Amazon Rekognition resources. IAM is an AWS service that you can use with no additional charge.

主題

- 對象 (p. 245)
- 使用身分來驗證 (p. 246)
- 使用政策管理存取權 (p. 247)
- Amazon Rekognition 如何搭配 IAM 運作 (p. 249)
- Amazon Rekognition 身分類型政策範例 (p. 251)
- 針對 Amazon Rekognition Identity and Access 進行故障診斷 (p. 254)

對象

How you use AWS Identity and Access Management (IAM) differs, depending on the work you do in Amazon Rekognition.

Service user – If you use the Amazon Rekognition service to do your job, then your administrator provides you with the credentials and permissions that you need. As you use more Amazon Rekognition features to do your work, you might need additional permissions. Understanding how access is managed can help you request the right permissions from your administrator. If you cannot access a feature in Amazon Rekognition, see 針對 Amazon Rekognition Identity and Access 進行故障診斷 (p. 254).

Service administrator – If you're in charge of Amazon Rekognition resources at your company, you probably have full access to Amazon Rekognition. It's your job to determine which Amazon Rekognition features

and resources your employees should access. You must then submit requests to your IAM administrator to change the permissions of your service users. Review the information on this page to understand the basic concepts of IAM. To learn more about how your company can use IAM with Amazon Rekognition, see Amazon Rekognition 如何搭配 IAM 運作 (p. 249).

IAM administrator – If you're an IAM administrator, you might want to learn details about how you can write policies to manage access to Amazon Rekognition. To view example Amazon Rekognition identity-based policies that you can use in IAM, see Amazon Rekognition 身分類型政策範例 (p. 251).

使用身分來驗證

Authentication is how you sign in to AWS using your identity credentials. For more information about signing in using the AWS Management Console, see The IAM Console and Sign-in Page in the IAM User Guide.

You must be authenticated (signed in to AWS) as the AWS account root user, an IAM user, or by assuming an IAM role. You can also use your company's single sign-on authentication, or even sign in using Google or Facebook. In these cases, your administrator previously set up identity federation using IAM roles. When you access AWS using credentials from another company, you are assuming a role indirectly.

To sign in directly to the AWS Management Console, use your password with your root user email or your IAM user name. You can access AWS programmatically using your root user or IAM user access keys. AWS provides SDK and command line tools to cryptographically sign your request using your credentials. If you don’t use AWS tools, you must sign the request yourself. Do this using Signature Version 4, a protocol for authenticating inbound API requests. For more information about authenticating requests, see Signature Version 4 Signing Process in the AWS General Reference.

Regardless of the authentication method that you use, you might also be required to provide additional security information. For example, AWS recommends that you use multi-factor authentication (MFA) to increase the security of your account. To learn more, see Using Multi-Factor Authentication (MFA) in AWS in the IAM User Guide.

AWS Account Root User

When you first create an AWS account, you begin with a single sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account root user and is accessed by signing in with the email address and password that you used to create the account. We strongly recommend that you do not use the root user for your everyday tasks, even the administrative ones. Instead, adhere to the best practice of using the root user only to create your first IAM user. Then securely lock away the root user credentials and use them to perform only a few account and service management tasks.

IAM 使用者和群組

An IAM user is an identity within your AWS account that has specific permissions for a single person or application. An IAM user can have long-term credentials such as a user name and password or a set of access keys. To learn how to generate access keys, see Managing Access Keys for IAM Users in the IAM User Guide. When you generate access keys for an IAM user, make sure you view and securely save the key pair. You cannot recover the secret access key in the future. Instead, you must generate a new access key pair.

An IAM group is an identity that specifies a collection of IAM users. You can't sign in as a group. You can use groups to specify permissions for multiple users at a time. Groups make permissions easier to manage for large sets of users. For example, you could have a group named IAMAdmins and give that group permissions to administer IAM resources.

Users are different from roles. A user is uniquely associated with one person or application, but a role is intended to be assumable by anyone who needs it. Users have permanent long-term credentials, but roles

provide temporary credentials. To learn more, see When to Create an IAM User (Instead of a Role) in the IAM User Guide.

IAM 角色

An IAM role is an identity within your AWS account that has specific permissions. It is similar to an IAM user, but is not associated with a specific person. You can temporarily assume an IAM role in the AWS Management Console by switching roles. You can assume a role by calling an AWS CLI or AWS API operation or by using a custom URL. For more information about methods for using roles, see Using IAM Roles in the IAM User Guide.

IAM roles with temporary credentials are useful in the following situations:

- Temporary IAM user permissions – An IAM user can assume an IAM role to temporarily take on different permissions for a specific task.
- Federated user access – Instead of creating an IAM user, you can use existing identities from AWS Directory Service, your enterprise user directory, or a web identity provider. These are known as federated users. AWS assigns a role to a federated user when access is requested through an identity provider. For more information about federated users, see Federated Users and Roles in the IAM User Guide.
- Cross-account access – You can use an IAM role to allow someone (a trusted principal) in a different account to access resources in your account. Roles are the primary way to grant cross-account access. However, with some AWS services, you can attach a policy directly to a resource (instead of using a role as a proxy). To learn the difference between roles and resource-based policies for cross-account access, see How IAM Roles Differ from Resource-based Policies in the IAM User Guide.
- AWS service access – A service role is an IAM role that a service assumes to perform actions in your account on your behalf. When you set up some AWS service environments, you must define a role for the service to assume. This service role must include all the permissions that are required for the service to access the AWS resources that it needs. Service roles vary from service to service, but many allow you to choose your permissions as long as you meet the documented requirements for that service. Service roles provide access only within your account and cannot be used to grant access to services in other accounts. You can create, modify, and delete a service role from within IAM. For example, you can create a role that allows Amazon Redshift to access an Amazon S3 bucket on your behalf and then load data from that bucket into an Amazon Redshift cluster. For more information, see Creating a Role to Delegate Permissions to an AWS Service in the IAM User Guide.
- Applications running on Amazon EC2 – You can use an IAM role to manage temporary credentials for applications that are running on an EC2 instance and making AWS CLI or AWS API requests. This is preferable to storing access keys within the EC2 instance. To assign an AWS role to an EC2 instance and make it available to all of its applications, you create an instance profile that is attached to the instance. An instance profile contains the role and enables programs that are running on the EC2 instance to get temporary credentials. For more information, see Using an IAM Role to Grant Permissions to Applications Running on Amazon EC2 Instances in the IAM User Guide.

To learn whether to use IAM roles, see When to Create an IAM Role (Instead of a User) in the IAM User Guide.

使用政策管理存取權

You control access in AWS by creating policies and attaching them to IAM identities or AWS resources. A policy is an object in AWS that, when associated with an identity or resource, defines their permissions. AWS evaluates these policies when an entity (root user, IAM user, or IAM role) makes a request. Permissions in the policies determine whether the request is allowed or denied. Most policies are stored in AWS as JSON documents. For more information about the structure and contents of JSON policy documents, see Overview of JSON Policies in the IAM User Guide.

An IAM administrator can use policies to specify who has access to AWS resources, and what actions they can perform on those resources. Every IAM entity (user or role) starts with no permissions. In other words, by default, users can do nothing, not even change their own password. To give a user permission to do something, an administrator must attach a permissions policy to a user. Or the administrator can add the user to a group that has the intended permissions. When an administrator gives permissions to a group, all users in that group are granted those permissions.

IAM policies define permissions for an action regardless of the method that you use to perform the operation. For example, suppose that you have a policy that allows the `iam:GetRole` action. A user with that policy can get role information from the AWS Management Console, the AWS CLI, or the AWS API.

身分型政策

Identity-based policies are JSON permissions policy documents that you can attach to an identity, such as an IAM user, role, or group. These policies control what actions that identity can perform, on which resources, and under what conditions. To learn how to create an identity-based policy, see Creating IAM Policies in the IAM User Guide.

Identity-based policies can be further categorized as inline policies or managed policies. Inline policies are embedded directly into a single user, group, or role. Managed policies are standalone policies that you can attach to multiple users, groups, and roles in your AWS account. Managed policies include AWS managed policies and customer managed policies. To learn how to choose between a managed policy or an inline policy, see Choosing Between Managed Policies and Inline Policies in the IAM User Guide.

資源型政策

Resource-based policies are JSON policy documents that you attach to a resource such as an Amazon S3 bucket. Service administrators can use these policies to define what actions a specified principal (account member, user, or role) can perform on that resource and under what conditions. Resource-based policies are inline policies. There are no managed resource-based policies.

存取控制清單 (ACL)

Access control lists (ACLs) are a type of policy that controls which principals (account members, users, or roles) have permissions to access a resource. ACLs are similar to resource-based policies, although they do not use the JSON policy document format. Amazon S3, AWS WAF, and Amazon VPC are examples of services that support ACLs. To learn more about ACLs, see Access Control List (ACL) Overview in the Amazon Simple Storage Service Developer Guide.

其他政策類型

AWS supports additional, less-common policy types. These policy types can set the maximum permissions granted to you by the more common policy types.

- Permissions boundaries – A permissions boundary is an advanced feature in which you set the maximum permissions that an identity-based policy can grant to an IAM entity (IAM user or role). You can set a permissions boundary for an entity. The resulting permissions are the intersection of entity's identity-based policies and its permissions boundaries. Resource-based policies that specify the user or role in the `Principal` field are not limited by the permissions boundary. An explicit deny in any of these policies overrides the allow. For more information about permissions boundaries, see Permissions Boundaries for IAM Entities in the IAM User Guide.
- Service control policies (SCPs) – SCPs are JSON policies that specify the maximum permissions for an organization or organizational unit (OU) in AWS Organizations. AWS Organizations is a service for grouping and centrally managing multiple AWS accounts that your business owns. If you enable all features in an organization, then you can apply service control policies (SCPs) to any or all of your accounts. The SCP limits permissions for entities in member accounts, including each AWS account

root user. For more information about Organizations and SCPs, see How SCPs Work in the AWS Organizations User Guide.

- Session policies – Session policies are advanced policies that you pass as a parameter when you programmatically create a temporary session for a role or federated user. The resulting session's permissions are the intersection of the user or role's identity-based policies and the session policies. Permissions can also come from a resource-based policy. An explicit deny in any of these policies overrides the allow. For more information, see Session Policies in the IAM User Guide.

多種政策類型

When multiple types of policies apply to a request, the resulting permissions are more complicated to understand. To learn how AWS determines whether to allow a request when multiple policy types are involved, see Policy Evaluation Logic in the IAM User Guide.

Amazon Rekognition 如何搭配 IAM 運作

在您使用 IAM 管理 Amazon Rekognition 的存取權前，建議您先了解可搭配 Amazon Rekognition 使用的 IAM 功能有哪些。若要取得 Amazon Rekognition 和其他 AWS 服務與 IAM 搭配運作的高層級檢視，請參閱《IAM User Guide》中的與 IAM 搭配運作的 AWS 服務。

主題

- Amazon Rekognition 身分型政策 (p. 249)
- Amazon Rekognition 資源型政策 (p. 250)
- Amazon Rekognition IAM 角色 (p. 251)

Amazon Rekognition 身分型政策

使用 IAM 身分型政策，您可以指定允許或拒絕的動作和資源，以及在何種條件下會允許或拒絕動作。Amazon Rekognition 支援特定動作、資源及條件金鑰。若要了解您在 JSON 政策中使用的所有元素，請參閱 IAM User Guide中的 IAM JSON 政策元素參考。

動作

The `Action` element of an IAM identity-based policy describes the specific action or actions that will be allowed or denied by the policy. Policy actions usually have the same name as the associated AWS API operation. The action is used in a policy to grant permissions to perform the associated operation.

Amazon Rekognition 中的政策動作會在動作前使用以下前綴：`rekognition:`。例如，若要授予某人使用 Amazon Rekognition`DeleteLabels` API 操作的許可，來偵測影像中的物件、場景或概念，請在其政策中加入 `rekognition:DetectLabels` 動作。政策陳述式必須包含 `Action` 或 `NotAction` 元素。Amazon Rekognition 會定義一組自己的動作，來描述您可以使用此服務執行的任務。

若要在單一陳述式中指定多個動作，請用逗號分隔，如下所示：

```
"Action": [
      "rekognition:action1",
      "rekognition:action2"
```

您也可以使用萬用字元 (*) 來指定多個動作。例如，若要指定開頭是 `Describe` 文字的所有動作，請包含以下動作：

```
"Action": "rekognition:Describe*"
```

若要查看 Amazon Rekognition 動作的清單，請參閱 IAM User Guide中的 Actions Defined by Amazon Rekognition。

資源

The `Resource` element specifies the object or objects to which the action applies. Statements must include either a `Resource` or a `NotResource` element. You specify a resource using an ARN or using the wildcard (*) to indicate that the statement applies to all resources.

如需 ARN 格式的詳細資訊，請參閱 Amazon Resource Name (ARN) 和 AWS 服務命名空間。

例如，若要在陳述式中指定 `MyCollection` 集合，請使用以下 ARN。

```
"Resource": "arn:aws:rekognition:us-east-1:123456789012:collection/MyCollection"
```

若要指定屬於特定帳戶的所有執行個體，請使用萬用字元 (*)：

```
"Resource": "arn:aws:rekognition:us-east-1:123456789012:collection/*"
```

有些 Amazon Rekognition 動作 (例如用來建立資源的動作) 無法在特定資源上執行。在這些情況下，您必須使用萬用字元 (*)。

```
"Resource": "*"
```

若要查看 Amazon Rekognition 資源類型的清單及其 ARN，請參閱《IAM User Guide》中的 Resources Defined by Amazon Rekognition。您若要了解您可以使用哪些動作指定每項資源的 ARN，請參閱 Actions Defined by Amazon Rekognition。

條件金鑰

Amazon Rekognition 不提供任何服務專用條件，但它支援使用一些全域條件索引鍵。若要查看所有 AWS 全域條件索引鍵，請參閱 IAM User Guide中的 AWS 全域條件內容索引鍵。

範例

若要檢視 Amazon Rekognition 身分型政策範例，請參閱Amazon Rekognition 身分類型政策範例 (p. 251)。

Amazon Rekognition 資源型政策

Amazon Rekognition 不支援資源型政策。

其他服務 (例如 Amazon S3) 也支援資源型許可政策。例如，您可以將政策連接至 S3 儲存貯體，以管理該儲存貯體的存取許可。

若要存取存放在 Amazon S3 儲存貯體中的影像，您必須具備存取 S3 儲存貯體中物件的許可。若有此許可，Amazon Rekognition 就可以從 S3 儲存貯體下載影像。下列政策範例可讓使用者在名為 Tests3bucket 的 S3 儲存貯體上執行 `s3:GetObject` 動作。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "s3:GetObject",
            "Resource": [
```

```
                "arn:aws:s3:::Tests3bucket"
            ]
        }
    ]
}
```

若要使用啟用版本控制的 S3 儲存貯體，請新增 `s3:GetObjectVersion` 動作，如下列範例所示。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetObject",
                "s3:GetObjectVersion"
            ],
            "Resource": [
                "arn:aws:s3:::Tests3bucket"
            ]
        }
    ]
```

Amazon Rekognition IAM 角色

IAM 角色是您 AWS 帳戶中的一種實體，具備特定許可。

搭配 Amazon Rekognition 使用暫時登入資料

您可以搭配聯合使用暫時登入資料登入、取得 IAM 角色，或是取得跨帳戶角色。您取得暫時安全登入資料的方式是透過呼叫 AWS STS API 操作 (例如，AssumeRole 或 GetFederationToken)。

Amazon Rekognition 支援使用暫時登入資料。

服務連結角色

服務連結角色可讓 AWS 服務存取其他服務中的資源，以代您完成動作。服務連結角色會顯示在您的 IAM 帳戶中，並由該服務所擁有。IAM 管理員可以檢視，但不能編輯服務連結角色的許可。

Amazon Rekognition 不支援服務連結角色。

服務角色

此功能可讓服務代表您擔任服務角色。此角色可讓服務存取其他服務中的資源，以代表您完成動作。服務角色會出現在您的 IAM 帳戶，且由該帳戶所擁有。這表示 IAM 管理員可以變更此角色的許可。不過，這樣可能會破壞此服務的功能。

Amazon Rekognition 支援服務角色。

在 Amazon Rekognition 中選擇 IAM 角色

當您設定 Amazon Rekognition 來分析儲存影片時，必須選擇角色，以允許 Amazon Rekognition 代您存取 Amazon SNS。如果您之前已建立服務角色或服務連結角色，Amazon Rekognition 會提供您角色清單讓您選擇。如需詳細資訊，請參閱the section called “設定 Amazon Rekognition Video” (p. 55)。

Amazon Rekognition 身分類型政策範例

根據預設，IAM 使用者和角色不具備建立或修改 Amazon Rekognition 資源的許可。他們也無法使用 AWS Management Console、AWS CLI 或 AWS API 執行任務。IAM 管理員必須建立 IAM 政策，授予使用者和角

色在他們所需指定資源上執行特定 API 操作的許可。管理員接著必須將這些政策連接至需要這些許可的 IAM 使用者或群組。

若要了解如何使用這些範例 JSON 政策文件建立 IAM 身分型政策，請參閱 IAM User Guide中的在 JSON 標籤上建立政策。

主題

- 政策最佳實務 (p. 252)
- 使用 Amazon Rekognition 主控台 (p. 252)
- Amazon Rekognition 的 AWS 受管 (預先定義) 政策 (p. 252)
- Amazon Rekognition 自訂標籤政策範例 (p. 253)
- 範例 1：允許資源的使用者唯讀存取 (p. 253)
- 範例 2：允許資源的使用者完整存取 (p. 253)
- 允許使用者檢視他們自己的許可 (p. 254)

政策最佳實務

Identity-based policies are very powerful. They determine whether someone can create, access, or delete Amazon Rekognition resources in your account. These actions can incur costs for your AWS account. When you create or edit identity-based policies, follow these guidelines and recommendations:

- Get Started Using AWS Managed Policies – To start using Amazon Rekognition quickly, use AWS managed policies to give your employees the permissions they need. These policies are already available in your account and are maintained and updated by AWS. For more information, see Get Started Using Permissions With AWS Managed Policies in the IAM User Guide.
- Grant Least Privilege – When you create custom policies, grant only the permissions required to perform a task. Start with a minimum set of permissions and grant additional permissions as necessary. Doing so is more secure than starting with permissions that are too lenient and then trying to tighten them later. For more information, see Grant Least Privilege in the IAM User Guide.
- Enable MFA for Sensitive Operations – For extra security, require IAM users to use multi-factor authentication (MFA) to access sensitive resources or API operations. For more information, see Using Multi-Factor Authentication (MFA) in AWS in the IAM User Guide.
- Use Policy Conditions for Extra Security – To the extent that it's practical, define the conditions under which your identity-based policies allow access to a resource. For example, you can write conditions to specify a range of allowable IP addresses that a request must come from. You can also write conditions to allow requests only within a specified date or time range, or to require the use of SSL or MFA. For more information, see IAM JSON Policy Elements: Condition in the IAM User Guide.

使用 Amazon Rekognition 主控台

除了 Amazon Rekognition「自訂標籤」功能之外，使用 Amazon Rekognition 主控台時，Amazon Rekognition 不需要任何額外許可。如需有關 Amazon Rekognition 自訂標籤的詳細資訊，請參閱步驟 5：設定 Amazon Rekognition 自訂標籤主控台許可。

對於僅呼叫 AWS CLI 或 AWS API 的使用者，您不需要允許最基本的主控台許可。反之，只需允許存取符合您嘗試執行之 API 操作的動作就可以了。

Amazon Rekognition 的 AWS 受管 (預先定義) 政策

AWS 獨立的 IAM 政策由 AWS 所建立與管理，可用來解決許多常用案例。這些 AWS 受管政策會授予常用案例所需的許可，讓您免於調查所需的許可。如需詳細資訊，請參閱 IAM User Guide 中的 AWS 受管政策。

以下是專屬於 Amazon Rekognition，並可連接至您帳戶中使用者的 AWS 受管政策：

- AmazonRekognitionFullAccess – 授予 Amazon Rekognition 資源的完整存取，包括建立與刪除集合。
- AmazonRekognitionReadOnlyAccess – 授予 Amazon Rekognition 資源的唯讀存取。
- AmazonRekognitionServiceRole – 允許 Amazon Rekognition 代表您呼叫 Amazon Kinesis Data Streams 與 Amazon SNS 服務。

> **Note**
>
> 您可以登入 IAM 主控台並在該處搜尋特定政策，來檢閱這些許可政策。
> 這些政策會在您使用 AWS 開發套件或 AWS CLI 時運作。

您也可以建立專有的自訂 IAM 政策，來允許 Amazon Rekognition 動作與資源的相關許可。您可以將這些自訂政策連接至需要這些許可的 IAM 使用者或群組。

Amazon Rekognition 自訂標籤政策範例

您可以為 Amazon Rekognition 自訂標籤建立以身分類型政策。如需詳細資訊，請參閱安全性。

範例 1：允許資源的使用者唯讀存取

下列範例會授予 Amazon Rekognition 資源的唯讀存取。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "rekognition:CompareFaces",
                "rekognition:DetectFaces",
                "rekognition:DetectLabels",
                "rekognition:ListCollections",
                "rekognition:ListFaces",
                "rekognition:SearchFaces",
                "rekognition:SearchFacesByImage",
                "rekognition:DetectText",
                "rekognition:GetCelebrityInfo",
                "rekognition:RecognizeCelebrities",
                "rekognition:DetectModerationLabels",
                "rekognition:GetLabelDetection",
                "rekognition:GetFaceDetection",
                "rekognition:GetContentModeration",
                "rekognition:GetPersonTracking",
                "rekognition:GetCelebrityRecognition",
                "rekognition:GetFaceSearch",
                "rekognition:GetTextDetection",
                "rekognition:DescribeStreamProcessor",
                "rekognition:ListStreamProcessors",
                "rekognition:DescribeProjects",
                "rekognition:DescribeProjectVersions",
                "rekognition:DetectCustomLabels"
            ],
            "Resource": "*"
        }
    ]
}
```

範例 2：允許資源的使用者完整存取

下列範例授予對 Amazon Rekognition 資源的完整存取權。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "rekognition:*"
            ],
            "Resource": "*"
        }
    ]
}
```

允許使用者檢視他們自己的許可

This example shows how you might create a policy that allows IAM users to view the inline and managed policies that are attached to their user identity. This policy includes permissions to complete this action on the console or programmatically using the AWS CLI or AWS API.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "ViewOwnUserInfo",
            "Effect": "Allow",
            "Action": [
                "iam:GetUserPolicy",
                "iam:ListGroupsForUser",
                  "iam:ListAttachedUserPolicies",
                "iam:ListUserPolicies",
                "iam:GetUser"
            ],
            "Resource": ["arn:aws:iam::*:user/${aws:username}"]
        },
        {
            "Sid": "NavigateInConsole",
            "Effect": "Allow",
            "Action": [
                "iam:GetGroupPolicy",
                "iam:GetPolicyVersion",
                "iam:GetPolicy",
                "iam:ListAttachedGroupPolicies",
                "iam:ListGroupPolicies",
                "iam:ListPolicyVersions",
                "iam:ListPolicies",
                "iam:ListUsers"
            ],
            "Resource": "*"
        }
    ]
}
```

針對 Amazon Rekognition Identity and Access 進行故障診斷

請使用以下資訊來協助您診斷和修復使用 Amazon Rekognition 和 IAM 時發生的常見問題。

主題

- 我未獲得在 Amazon Rekognition 中執行動作的授權 (p. 255)

- 我未獲得執行 iam:PassRole 的授權 (p. 255)
- 我想要檢視我的存取金鑰 (p. 255)
- 我是管理員，而且我想要允許其他人存取 Amazon Rekognition (p. 256)
- 我想要允許我的 AWS 帳戶之外的人存取我的 Amazon Rekognition 資源 (p. 256)

我未獲得在 Amazon Rekognition 中執行動作的授權

若 AWS Management Console 告知您並未獲得執行動作的授權，您必須聯絡您的管理員以取得協助。您的管理員是提供您使用者名稱和密碼的人員。

以下範例錯誤會在 `mateojackson` IAM 使用者嘗試使用主控台檢視 *`widget`* 的詳細資訊，但卻沒有 `rekognition:`*`GetWidget`* 許可時發生。

```
User: arn:aws:iam::123456789012:user/mateojackson is not authorized to perform:
 rekognition:GetWidget on resource: my-example-widget
```

在此情況下，Mateo 會請求管理員更新他的政策，允許他使用 `rekognition:`*`GetWidget`* 動作存取 *`my-example-widget`* 資源。

我未獲得執行 iam:PassRole 的授權

If you receive an error that you're not authorized to perform the `iam:PassRole` action, then you must contact your administrator for assistance. Your administrator is the person that provided you with your user name and password. Ask that person to update your policies to allow you to pass a role to Amazon Rekognition.

Some AWS services allow you to pass an existing role to that service, instead of creating a new service role or service-linked role. To do this, you must have permissions to pass the role to the service.

The following example error occurs when an IAM user named `marymajor` tries to use the console to perform an action in Amazon Rekognition. However, the action requires the service to have permissions granted by a service role. Mary does not have permissions to pass the role to the service.

```
User: arn:aws:iam::123456789012:user/marymajor is not authorized to perform: iam:PassRole
```

In this case, Mary asks her administrator to update her policies to allow her to perform the `iam:PassRole` action.

我想要檢視我的存取金鑰

After you create your IAM user access keys, you can view your access key ID at any time. However, you can't view your secret access key again. If you lose your secret key, you must create a new access key pair.

Access keys consist of two parts: an access key ID (for example, `AKIAIOSFODNN7EXAMPLE`) and a secret access key (for example, `wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY`). Like a user name and password, you must use both the access key ID and secret access key together to authenticate your requests. Manage your access keys as securely as you do your user name and password.

> Important
>
> Do not provide your access keys to a third party, even to help find your canonical user ID. By doing this, you might give someone permanent access to your account.

When you create an access key pair, you are prompted to save the access key ID and secret access key in a secure location. The secret access key is available only at the time you create it. If you lose your secret access key, you must add new access keys to your IAM user. You can have a maximum of two access keys. If you already have two, you must delete one key pair before creating a new one. To view instructions, see Managing Access Keys in the IAM User Guide.

我是管理員，而且我想要允許其他人存取 Amazon Rekognition

To allow others to access Amazon Rekognition, you must create an IAM entity (user or role) for the person or application that needs access. They will use the credentials for that entity to access AWS. You must then attach a policy to the entity that grants them the correct permissions in Amazon Rekognition.

To get started right away, see Creating Your First IAM Delegated User and Group in the IAM User Guide.

我想要允許我的 AWS 帳戶之外的人存取我的 Amazon Rekognition 資源

You can create a role that users in other accounts or people outside of your organization can use to access your resources. You can specify who is trusted to assume the role. For services that support resource-based policies or access control lists (ACLs), you can use those policies to grant people access to your resources.

To learn more, consult the following:

- To learn whether Amazon Rekognition supports these features, see Amazon Rekognition 如何搭配 IAM 運作 (p. 249).
- To learn how to provide access to your resources across AWS accounts that you own, see Providing Access to an IAM User in Another AWS Account That You Own in the IAM User Guide.
- To learn how to provide access to your resources to third-party AWS accounts, see Providing Access to AWS Accounts Owned by Third Parties in the IAM User Guide.
- To learn how to provide access through identity federation, see Providing Access to Externally Authenticated Users (Identity Federation) in the IAM User Guide.
- To learn the difference between using roles and resource-based policies for cross-account access, see How IAM Roles Differ from Resource-based Policies in the IAM User Guide.

Amazon Rekognition 的資料保護

Amazon Rekognition 會遵循 AWS 共同責任模型，此模型包含資料保護的法規和指導。AWS 會負責保護執行所有 AWS 服務的全球基礎設施。AWS 會維持對此基礎設施上託管資料的控制，包含處理客戶內容和個人資料的安全性組態控制。身為資料控制者或資料處理者的 AWS 客戶和 APN 合作夥伴都需負責保護在 AWS 雲端中放置的任何個人資料。

基於資料保護目的，我們建議您保護 AWS 帳戶登入資料，並使用 AWS Identity and Access Management (IAM) 設定個別使用者帳戶，以便每個使用者都只獲得完成其任務所需的許可。此外，您可以設定 S3 儲存貯體、Amazon Kinesis 視訊串流和臉部集合等 AWS 資源，以符合 最低權限的原則。我們也建議您以下列方式保護資料：

- 每個帳戶都使用多重驗證 (MFA)。
- 使用 SSL/TLS 與 AWS 資源通訊。
- 使用 AWS CloudTrail 設定 API 和使用者活動記錄。
- 使用 AWS 加密解決方案，以及 AWS 服務內的所有預設安全控制。

- 使用進階的受管安全服務，例如 Amazon Macie，其可協助探索和保護存放在 Amazon S3 的個人資料。

我們強烈建議您絕對不要將客戶帳戶號碼等敏感的識別資訊，放在自由格式的欄位中，如 Name (名稱) 欄位。這包括當您使用 Amazon Rekognition 或使用主控台、API、AWS CLI 或 AWS 開發套件的其他 AWS 服務。您輸入 Amazon Rekognition 或其他服務的任何資料都可能被選入診斷日誌中。當您提供外部伺服器的 URL 時，請勿在 URL 中包含登入資料資訊，以驗證您對該伺服器的請求。

如需資料保護的詳細資訊，請參閱「AWS 安全部落格」上的 AWS 共同責任模型和 GDPR 部落格文章。

資料加密

下列資訊說明 Amazon Rekognition 使用資料加密來保護您的資料的位置。

靜態加密

影像

除非您已連絡 AWS支援團隊並遵循提供給您的程序並選擇退出，否則傳遞至 Amazon Rekognition API 操作的影像可以儲存並用於改善服務。儲存的影像會在靜態時使用 AWS Key Management Service (SSE-KMS) 進行加密 (Amazon S3)。

集合

對於在集合中存放資訊的臉部比較操作，底層偵測演算法將先偵測輸入影像中的臉孔，擷取至每個臉孔的特徵向量中，接著將臉孔向量存放於集合。Amazon Rekognition 將在執行臉部比較時使用這些臉孔向量。臉孔向量儲存為浮點數的陣列。資料本身並無無意義，通常作為有效雜湊，且無法進行反向工程。資料不會進一步加密。

影片

若要分析影片，Amazon Rekognition 會將影片複製到服務中進行處理。除非您已連絡 AWS支援團隊並遵循提供給您的程序並選擇退出，否則可以儲存影片並用於改善服務。這些影片會在靜態時使用 AWS Key Management Service (SSE-KMS) 進行加密 (Amazon S3)。

Amazon Rekognition 自訂標籤模型

經過培訓的模型和其他客戶資料的加密是使用伺服器端的加密搭配 Amazon S3 受管金鑰 (SSE-S3)。如需詳細資訊，請參閱使用伺服器端加密保護資料。

傳輸中加密

Amazon Rekognition API 端點僅支援透過 HTTPS 的安全連線。所有通訊都是使用 Transport Layer Security (TLS) 加密。

金鑰管理

您可以使用 AWS Key Management Service (KMS) 來管理存放在 Amazon S3 儲存貯體中輸入影像和影片的金鑰。

網際網路流量隱私權

Amazon Rekognition 的 Amazon Virtual Private Cloud (Amazon VPC) 端點是 VPC 內的邏輯實體，僅允許連接到 Amazon Rekognition。Amazon VPC 會將請求路由到 Amazon Rekognition 並將回應路由傳回 VPC。如需詳細資訊，請參閱 Amazon VPC 使用者指南中的 VPC 端點。如需有關搭配 Amazon

Rekognition 使用 Amazon VPC 端點的詳細資訊，請參閱使用 Amazon Rekognition 搭配 Amazon VPC 端點 (p. 267)。

監控 Rekognition

使用 CloudWatch 可獲得個別 Rekognition 操作的指標或帳戶的全域 Rekognition 指標。您可以使用指標來追蹤以 Rekognition 為基礎的解決方案之運作狀態，並設定當一個或多個指標超出既定閾值時傳送警示通知給自己。例如，您可以查看發生的伺服器錯誤次數指標，或者偵測到的臉孔數量指標。您也可以查看特定 Rekognition 操作的成功次數指標。若要查看指標，您可以使用 Amazon CloudWatch、Amazon AWS Command Line Interface 或 CloudWatch API。

您也可以使用 Rekognition 主控台來查看指定的時間帶期間的彙總指標。如需詳細資訊，請參閱 練習 4：查看彙總指標 (主控台) (p. 22)。

使用 Rekognition 的 CloudWatch 指標

要使用指標，您必須指定下列資訊：

- 指標維度，或者無維度。維度是一組用來單獨辨識指標的名稱值組。Rekognition 有一個維度，稱為操作。它提供特定操作的指標。如果您未指定維度，指標範圍將適用於帳戶內的所有 Rekognition 操作。
- 指標名稱，例如 `UserErrorCount`。

您可以使用 AWS Management Console、AWS CLI 或 CloudWatch API 來取得 Rekognition 的監控資料。您也可以透過其中一個 Amazon AWS 軟體開發套件 (SDK) 或 CloudWatch API 工具來使用 CloudWatch API。主控台會根據 CloudWatch API 的原始資料顯示一系列圖形。根據需求，您可能偏好使用顯示於主控台中的圖形或自 API 擷取的圖形。

下列清單顯示一些常見的指標用途。這些是協助您開始的建議，而不是完整清單。

我要如何？	相關指標
我要如何追蹤辨識到的臉孔數量？	監控 `DetectedFaceCount` 指標的 `Sum` 統計資料。
我要如何得知我的應用程式已達每秒最高請求數量？	監控 `ThrottledCount` 指標的 `Sum` 數據。
我要如何監控請求錯誤？	使用 `UserErrorCount` 指標的 `Sum` 統計資料。
我要如何找到請求總數？	使用 `ResponseTime` 指標的 `ResponseTime` 與 `Data Samples` 統計資料。其中包括任何產生錯誤的請求。如果您只想要查看成功操作呼叫，請使用 `SuccessfulRequestCount` 指標。
我要如何監控 `Rekognition` 操作呼叫的延遲？	使用 `ResponseTime` 指標。
我要如何監控 `IndexFaces` 成功新增臉部到 Rekognition 集合的次數？	使用 `SuccessfulRequestCount` 指標與 `IndexFaces` 操作來監控 `Sum` 統計資料。使用 `Operation` 維度來選擇操作與指標。

您必須擁有適當的 CloudWatch 許可才可使用 Rekognition 來監控 CloudWatch。如需詳細資訊，請參閱 Amazon CloudWatch 的身份驗證和存取控制。

存取 Rekognition 指標

以下範例說明如何使用 CloudWatch 主控台、AWS CLI 以及 CloudWatch API 來存取 Rekognition 指標。

檢視指標 (主控台)

1. 於 https://console.aws.amazon.com/cloudwatch/ 開啟 CloudWatch 主控台。
2. 選擇 Metrics (指標)、選擇 All Metrics (所有指標) 標籤，然後選擇 Rekognition。
3. 選擇 Metrics with no dimensions (無維度的指標)，然後選擇一個指標。

   例如，選擇 DetectedFace 指標來計算已偵測到幾個臉孔。
4. 選擇日期範圍的值。指標計數顯示於圖形中。

若要查看一段時間內的成功指標 **`DetectFaces`** 操作呼叫 (CLI)。

- 開啟 AWS CLI，然後輸入下列命令：

  ```
  aws cloudwatch get-metric-statistics --metric-name SuccessfulRequestCount
  --start-time 2017-1-1T19:46:20 --end-time 2017-1-6T19:46:57 --
  period 3600 --namespace AWS/Rekognition --statistics Sum --dimensions
  Name=Operation,Value=DetectFaces --region us-west-2
  ```

  此範例顯示在一段時間內執行的成功 `DetectFaces` 操作呼叫。如需詳細資訊，請參閱 get-metric-statistics。

若要存取指標 (CloudWatch API)

- 呼叫 `GetMetricStatistics`。如需詳細資訊，請參閱 Amazon CloudWatch API Reference.

建立警示

您可以建立 CloudWatch 警示，在警示變更狀態時傳送 Amazon Simple Notification Service (Amazon SNS) 訊息。警示會監看指定時段內的單一指標，並根據與多個時段內指定閾值相對的指標值來執行一或多個動作。此動作是傳送到 Amazon SNS 主題或 Auto Scaling 政策的通知。

警示叫用動作只適用於持續狀態變更。CloudWatch 警示，因為它們是不叫用動作，只需在特定的狀態。狀態必須發生變更並維持一段指定的時間。

若要設定警示 (主控台)

1. Sign in to the AWS Management Console and open the CloudWatch console at https://console.aws.amazon.com/cloudwatch/.
2. 選擇Create Alarm (建立警示)。這會啟動 Create Alarm Wizard (建立警示精靈)。
3. 在 Metrics with no dimensions (無維度的指標) 指標清單中，選擇 Rekognition Metrics (Rekognition 指標)，然後選擇一個指標。

   例如，選擇 DetectedFaceCount 來設定偵測到的臉孔之最高數量警示。
4. 在 Time Range (時間範圍) 區域內，選擇包含您已呼叫的臉部偵測操作之日期範圍值。選擇 Next (下一步)
5. 填入 Name (名稱) 和 Description (說明)。對於 Whenever (每當) 選項，請選擇 >= 並輸入您所選擇的最大值。
6. 如果您希望 CloudWatch 在達到警示狀態時傳送電子郵件給您，請在 Whenever this alarm: (每當此警示：) 選項中請選擇 State is ALARM (狀態為警示)。若要傳送警示到現有的 Amazon SNS 主題，請在

Send notification to: (傳送通知至) 選項中選擇現有的 SNS 主題。若要設定新的電子郵件訂閱清單的名稱與電子郵件地址，請選擇 Create topic (建立主題)，CloudWatch 將會儲存清單並顯示於欄位中，可提供未來設定警示時使用。

Note

如果您使用 Create topic (建立主題) 建立新 Amazon SNS 主題，必須先驗證電子郵件地址，指定的收件人才能收到通知。Amazon SNS 只在警示進入警示狀態下傳送電子郵件。如果此警示狀態在驗證電子郵件地址之前發生變更，目標收件人就不會收到通知。

7. 在 Alarm Preview (警示預覽) 區段中預覽警示。選擇Create Alarm (建立警示)。

設定警示 (AWS CLI)

- 開啟 AWS CLI，然後輸入下列命令。變更 `alarm-actions` 參數值，以連結您之前建立的 Amazon SNS 主題。

```
aws cloudwatch put-metric-alarm --alarm-name UserErrors --alarm-description
"Alarm when more than 10 user errors occur" --metric-name UserErrorCount --
namespace AWS/Rekognition --statistic Average --period 300 --threshold 10
--comparison-operator GreaterThanThreshold --evaluation-periods 2 --alarm-
actions arn:aws:sns:us-west-2:111111111111:UserError --unit Count
```

此範例說明如何建立警示，當 5 分鐘內發生超過 10 個使用者錯誤時通知。如需詳細資訊，請參閱 put-metric-alarm。

設定警示 (CloudWatch API)

- 呼叫 `PutMetricAlarm`。如需詳細資訊，請參閱 Amazon CloudWatch API Reference。

Rekognition 的 CloudWatch 指標

本節包含有關可提供 Amazon Rekognition 使用的 Amazon CloudWatch 指標和操作維度資訊。

您也可以從 Rekognition 主控台查看 Rekognition 指標的彙總檢視。如需詳細資訊，請參閱 練習 4：查看彙總指標 (主控台) (p. 22)。

Rekognition 的 CloudWatch 指標

下表顯示 Rekognition 指標摘要。

指標	敘述
SuccessfulRequestCount	成功請求的數量。成功請求的回應碼範圍是 200 到 299。 單位：計數 有效的統計資訊：`Sum,Average`
ThrottledCount	已調節的請求數目。Rekognition 會在收到的請求超過為您的帳戶每秒所設的交易上限時節制請求。如果經常超過為您的帳戶所設的限制，您可以請求提高上限。若要請求提高，請參閱 AWS Service Limits。 單位：計數 有效的統計資訊：`Sum,Average`

指標	敘述
ResponseTime	以毫秒為單位的時間，提供 Rekognition 來運算回應。 個單位: 1. `Data Samples` 統計資料的計數 2. `Average` 統計資料的毫秒 有效的統計資訊：`Data Samples,Average` Note `ResponseTime` 指標不包含在 Rekognition 指標窗格中。
DetectedFaceCount	以 `IndexFaces` 或 `DetectFaces` 操作偵測到臉部的次數。 單位：計數 有效的統計資訊：`Sum,Average`
DetectedLabelCount	以 `DetectLabels` 操作偵測到標籤的次數。 單位：計數 有效的統計資訊：`Sum,Average`
ServerErrorCount	伺服器錯誤的次數。伺服器錯誤的回應碼範圍是 500 到 599。 單位：計數 有效的統計資訊：`Sum, Average`
UserErrorCount	使用者錯誤次數 (無效參數、無效影像、無權限等)。使用者錯誤的回應碼範圍是 400 到 499。 單位：計數 有效的統計資訊：`Sum,Average`

Rekognition 的 CloudWatch 維度

若要擷取特定操作的指標，請使用 `Rekognition` 命名空間並提供操作維度。如需維度的詳細資訊，請參閱 Amazon CloudWatch User Guide 中的維度。

使用 AWS CloudTrail 記錄 Amazon Rekognition API 呼叫

Amazon Rekognition 與 AWS CloudTrail 服務整合，可記錄 Amazon Rekognition 中使用者、角色或 AWS 採取的動作。CloudTrail 會將 Amazon Rekognition API 呼叫的子集擷取為事件，包括從 Amazon Rekognition 主控台的呼叫，和從程式碼對 Amazon Rekognition API 的呼叫。如果您建立追蹤記錄，就可以持續傳送 CloudTrail 事件至 Amazon S3 儲存貯體，包括 Amazon Rekognition 的事件。如果您不設定追蹤記錄，仍然可以透過 CloudTrail 主控台中的 Event history (事件歷史記錄) 檢視最新的事件。使用由 CloudTrail 收集的資訊，您就可以判斷送至 Amazon Rekognition 的請求、提出請求的 IP 地址、誰提出請求、提出請求的時間，以及其他詳細資訊。

若要進一步了解 CloudTrail，包括如何設定及啟用，請參閱 AWS CloudTrail User Guide。

Amazon RekognitionCloudTrail 中的資訊

當您建立帳戶時，CloudTrail系統會在您的 AWS 帳戶中啟用。當 Amazon Rekognition 中發生支援的事件活動時，該活動會記錄在 CloudTrail 事件，和其他 AWS 服務事件一起記錄於 Event history (事件歷程記錄)中。您可以檢視、搜尋和下載 AWS 帳戶的最新事件。如需詳細資訊，請參閱使用 CloudTrail 事件歷程記錄檢視事件。

如需您 AWS 帳戶中正在進行事件的記錄 (包含 Amazon Rekognition 的事件)，請建立線索。追蹤記錄可讓 CloudTrail 將日誌檔案交付到 Amazon S3 儲存貯體。根據預設，當您在主控台建立追蹤記錄時，追蹤記錄會套用到所有區域。該追蹤會記錄來自 AWS 分割區中所有區域的事件，並將日誌檔案交付到您指定的 Amazon S3 儲存貯體。此外，您可以設定其他 AWS 服務，以進一步分析和處理 CloudTrail 日誌中所收集的事件資料。如需詳細資訊，請參閱：

- 建立追蹤的概觀
- CloudTrail 支援的服務和整合
- 設定 CloudTrail 的 Amazon SNS 通知
- 接收多個區域的 CloudTrail 日誌檔案及接收多個帳戶的 CloudTrail 日誌檔案

Amazon Rekognition 支援將下列動作記錄為 CloudTrail 日誌檔中的事件：

- the section called “CreateCollection” (p. 279)
- the section called “DeleteCollection” (p. 292)
- the section called “CreateStreamProcessor” (p. 289)
- the section called “DeleteStreamProcessor” (p. 300)
- the section called “DescribeStreamProcessor” (p. 312)
- the section called “ListStreamProcessors” (p. 385)
- the section called “ListCollections” (p. 379)

每一筆事件或記錄項目都會包含產生請求者的資訊。身分資訊可協助您判斷下列事項：

- 該請求是否使用根或 IAM 使用者登入資料提出。
- 提出該請求時，是否使用了特定角色或聯合身分使用者的暫時安全登入資料.
- 該請求是否由另一項 AWS 服務提出。

如需詳細資訊，請參閱 CloudTrail 使用者身分元素。

範例：Amazon Rekognition 日誌檔案項目

追蹤記錄是一種組態，能讓事件以日誌檔案的形式交付至您指定的 Amazon S3 儲存貯體。CloudTrail 日誌檔案包含一個或多個日誌項目。事件代表從任何來源的單一請求，並包含有關請求的動作、動作的日期和時間、請求參數等資訊。CloudTrail 日誌檔不是公有 API 呼叫的排序堆疊追蹤記錄，因此不會出現以任何特定順序顯示。

以下範例顯示含有下列 API 之動作的 CloudTrail 日誌項目：`CreateCollection`、`DeleteCollection`、`CreateStreamProcessor`、`DeleteStreamProcessor`、`Descri`和 `ListCollections`。

```
{
    "Records": [
        {
```

```
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "IAMUser",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:iam::111122223333:user/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "userName": "Alice"
            },
            "eventTime": "2018-03-26T21:46:22Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "CreateCollection",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-internal/3",
            "requestParameters": {
                "collectionId": "8fa6aa65-cab4-4d0a-b976-7fd5df63f38a"
            },
            "responseElements": {
                "collectionArn": "aws:rekognition:us-
east-1:111122223333:collection/8fa6aa65-cab4-4d0a-b976-7fd5df63f38a",
                "faceModelVersion": "2.0",
                "statusCode": 200
            },
            "requestID": "1e77d2d5-313f-11e8-8c0e-75c0272f31a4",
            "eventID": "c6da4992-a9a1-4962-93b6-7d0483d95c30",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "IAMUser",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:iam::111122223333:user/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "userName": "Alice"
            },
            "eventTime": "2018-03-26T21:46:25Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "DeleteCollection",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-internal/3",
            "requestParameters": {
                "collectionId": "8fa6aa65-cab4-4d0a-b976-7fd5df63f38a"
            },
            "responseElements": {
                "statusCode": 200
            },
            "requestID": "213d5b78-313f-11e8-8c0e-75c0272f31a4",
            "eventID": "3ed4f4c9-22f8-4de4-a051-0d9d0c2faec9",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "AssumedRole",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:sts::111122223333:assumed-role/Admin/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "sessionContext": {
                    "attributes": {
```

```
                        "mfaAuthenticated": "false",
                        "creationDate": "2018-03-26T21:48:49Z"
                    },
                    "sessionIssuer": {
                        "type": "Role",
                        "principalId": "EX_PRINCIPAL_ID",
                        "arn": "arn:aws:iam::111122223333:role/Admin",
                        "accountId": "111122223333",
                        "userName": "Admin"
                    }
                }
            },
            "eventTime": "2018-03-26T21:53:09Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "ListCollections",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-cli/1.14.63 Python/3.4.7
 Linux/3.2.45-0.6.wd.971.49.326.metal1.x86_64 botocore/1.9.16",
            "requestParameters": null,
            "responseElements": null,
            "requestID": "116a57f5-3140-11e8-8c0e-75c0272f31a4",
            "eventID": "94bb5ddd-7836-4fb1-a63e-a782eb009824",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "AssumedRole",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:sts::111122223333:assumed-role/Admin/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "sessionContext": {
                    "attributes": {
                        "mfaAuthenticated": "false",
                        "creationDate": "2018-03-26T21:48:49Z"
                    },
                    "sessionIssuer": {
                        "type": "Role",
                        "principalId": "EX_PRINCIPAL_ID",
                        "arn": "arn:aws:iam::111122223333:role/Admin",
                        "accountId": "111122223333",
                        "userName": "Admin"
                    }
                }
            },
            "eventTime": "2018-03-26T22:06:19Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "CreateStreamProcessor",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-cli/1.14.63 Python/3.4.7
 Linux/3.2.45-0.6.wd.971.49.326.metal1.x86_64 botocore/1.9.16",
            "requestParameters": {
                "roleArn": "arn:aws:iam::111122223333:role/AmazonRekognition-
StreamProcessorRole",
                "settings": {
                    "faceSearch": {
                        "collectionId": "test"
                    }
                },
                "name": "ProcessorName",
                "input": {
                    "kinesisVideoStream": {
```

```
                        "arn": "arn:aws:kinesisvideo:us-east-1:111122223333:stream/
VideoStream"
                    }
                },
                "output": {
                    "kinesisDataStream": {
                        "arn": "arn:aws:kinesis:us-east-1:111122223333:stream/
AmazonRekognition-DataStream"
                    }
                }
            },
            "responseElements": {
                "StreamProcessorArn": "arn:aws:rekognition:us-
east-1:111122223333:streamprocessor/ProcessorName"
            },
            "requestID": "e8fb2b3c-3141-11e8-8c0e-75c0272f31a4",
            "eventID": "44ff8f90-fcc2-4740-9e57-0c47610df8e3",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "AssumedRole",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:sts::111122223333:assumed-role/Admin/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "sessionContext": {
                    "attributes": {
                        "mfaAuthenticated": "false",
                        "creationDate": "2018-03-26T21:48:49Z"
                    },
                    "sessionIssuer": {
                        "type": "Role",
                        "principalId": "EX_PRINCIPAL_ID",
                        "arn": "arn:aws:iam::111122223333:role/Admin",
                        "accountId": "111122223333",
                        "userName": "Admin"
                    }
                }
            },
            "eventTime": "2018-03-26T22:09:42Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "DeleteStreamProcessor",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-cli/1.14.63 Python/3.4.7
 Linux/3.2.45-0.6.wd.971.49.326.metal1.x86_64 botocore/1.9.16",
            "requestParameters": {
                "name": "ProcessorName"
            },
            "responseElements": null,
            "requestID": "624c4c3e-3142-11e8-8c0e-75c0272f31a4",
            "eventID": "27fd784e-fbf3-4163-9f0b-0006c6eed39f",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "AssumedRole",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:sts::111122223333:assumed-role/Admin/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
```

```
                "sessionContext": {
                    "attributes": {
                        "mfaAuthenticated": "false",
                        "creationDate": "2018-03-26T21:48:49Z"
                    },
                    "sessionIssuer": {
                        "type": "Role",
                        "principalId": "EX_PRINCIPAL_ID",
                        "arn": "arn:aws:iam::111122223333:role/Admin",
                        "accountId": "111122223333",
                        "userName": "Admin"
                    }
                }
            },
            "eventTime": "2018-03-26T21:56:14Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "ListStreamProcessors",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-cli/1.14.63 Python/3.4.7
Linux/3.2.45-0.6.wd.971.49.326.metal1.x86_64 botocore/1.9.16",
            "requestParameters": null,
            "responseElements": null,
            "requestID": "811735f9-3140-11e8-8c0e-75c0272f31a4",
            "eventID": "5cfc86a6-758c-4fb9-af13-557b04805c4e",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        },
        {
            "eventVersion": "1.05",
            "userIdentity": {
                "type": "AssumedRole",
                "principalId": "EX_PRINCIPAL_ID",
                "arn": "arn:aws:sts::111122223333:assumed-role/Admin/Alice",
                "accountId": "111122223333",
                "accessKeyId": "EXAMPLE_KEY_ID",
                "sessionContext": {
                    "attributes": {
                        "mfaAuthenticated": "false",
                        "creationDate": "2018-03-26T22:55:22Z"
                    },
                    "sessionIssuer": {
                        "type": "Role",
                        "principalId": "EX_PRINCIPAL_ID",
                        "arn": "arn:aws:iam::111122223333:role/Admin",
                        "accountId": "111122223333",
                        "userName": "Admin"
                    }
                }
            },
            "eventTime": "2018-03-26T22:57:38Z",
            "eventSource": "rekognition.amazonaws.com",
            "eventName": "DescribeStreamProcessor",
            "awsRegion": "us-east-1",
            "sourceIPAddress": "127.0.0.1",
            "userAgent": "aws-cli/1.14.63 Python/3.4.7
Linux/3.2.45-0.6.wd.971.49.326.metal1.x86_64 botocore/1.9.16",
            "requestParameters": {
                "name": "ProcessorName"
            },
            "responseElements": null,
            "requestID": "14b2dd34-3149-11e8-8c0e-75c0272f31a4",
            "eventID": "0db3498c-e084-4b30-b5fb-aa0b71ef9b7b",
            "eventType": "AwsApiCall",
            "recipientAccountId": "111122223333"
        }
```

```
    ]
}
```

使用 Amazon Rekognition 搭配 Amazon VPC 端點

如果您使用 Amazon Virtual Private Cloud (Amazon VPC) 來託管 AWS 資源，您可以在 VPC 和 Amazon Rekognition 之間建立私有連線。您可以使用此連線來啟用 Amazon Rekognition 不用透過公有網際網路在 VPC 與您的資源進行通訊。

Amazon VPC 是一項 AWS 服務，您可用來在自己定義的虛擬網路中啟動 AWS 資源。您可利用 VPC 來控制您的網路設定，例如 IP 地址範圍、子網路、路由表和網路閘道。AWS 網路使用 VPC 端點處理 VPC 與 AWS 服務之間的路由。

若要將您的 VPC 連接到 Amazon Rekognition，請定義 Amazon Rekognition 的 VPC 端點介面。界面端點是包含私有 IP 地址的彈性網路界面，做為通往支援之 AWS 服務的流量進入點。端點能為 Amazon Rekognition—提供可靠、可擴展性的連線，且無須使用網際網路閘道、網路位址轉譯 (NAT) 執行個體或 VPN 連接。如需詳細資訊，請參閱 Amazon VPC 使用者指南中的什麼是 Amazon VPC？

界面 VPC 端點以 AWS PrivateLink 啟用。此 AWS 技術讓私有通訊使用於彈性網路介面與 AWS 服務之間的私有 IP 地址。如需詳細資訊，請參閱 AWS PrivateLink。

> Note
>
> AWS PrivateLink 支援所有 Amazon Rekognition 美國聯邦資訊處理標準 (FIPS) 端點。

建立適用於 Amazon Rekognition 的 Amazon VPC 端點

您可以建立兩種 Amazon VPC 端點類型，搭配 Amazon Rekognition 使用。

- 搭配 Amazon Rekognition 操作的 VPC 端點。這是適用於大部份使用者的最合適 VPC 端點類型。
- 使用符合聯邦資訊處理標準 (Federal Information Processing Standard，FIPS) 第 140-2 號美國政府標準的端點，為 Amazon Rekognition 操作建立 VPC 端點。如需詳細資訊，請參閱適用於 AWS GovCloud (美國) 區域的端點。

若要搭配您的 VPC 開始使用 Amazon Rekognition，請使用 Amazon VPC 主控台建立 Amazon Rekognition 的界面 VPC 端點。如需說明，請參閱建立界面端點中的「使用主控台建立 AWS 服務的界面端點」。請記下下列步驟：

- 步驟 3 – 在 Service category (服務類別) 中，選擇 AWS services (AWS 服務)。
- 步驟 4 – 在 Service Name (服務名稱) 中，選擇以下其中一個選項：
  - com.amazonaws.region.rekognition – 為 Amazon Rekognition 操作建立 VPC 端點。
  - com.amazonaws.region.rekognition-fips – 使用符合聯邦資訊處理標準 (Federal Information Processing Standard，FIPS) 第 140-2 號美國政府標準的端點，為 Amazon Rekognition 操作建立 VPC 端點。

如需詳細資訊，請參閱 Amazon VPC 使用者指南中的 入門。

建立 Amazon Rekognition 的 VPC 端點政策

您可以為 Amazon Rekognition 的 Amazon VPC 端點建立政策，藉以指定下列設定：

- 可執行動作的委託人。
- 可執行的動作。

- 可供執行動作的資源。

如需詳細資訊，請參閱 Amazon VPC 使用者指南中的使用 VPC 端點控制對服務的存取。

以下範例政策可讓使用者透過 VPC 端點連線到 Amazon Rekognition，以呼叫 `DetectFaces` API 操作。此政策可防止使用者，使其無法透過 VPC 端點執行其他 Amazon Rekognition API 操作。

使用者仍然可以在 VPC 以外呼叫其他 Amazon Rekognition API 操作。如需如何拒絕存取 VPC 以外之 Amazon Rekognition API 操作的詳細資訊，請參閱Amazon Rekognition 身分型政策 (p. 249)。

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "rekognition:DetectFaces"
            ],
            "Resource": "*",
            "Effect": "Allow",
            "Principal": "*"
        }
    ]
}
```

修改 Amazon Rekognition 的 VPC 端點政策

1. Open the Amazon VPC console at https://console.aws.amazon.com/vpc/.
2. 如果您尚未建立 Amazon Rekognition 的端點，請選擇 Create Endpoint (建立端點)。然後選取 com.amazonaws.***Region***.rekognition，然後選擇 Create endpoint (建立端點)。
3. 在導覽窗格中選擇 Endpoints (端點)。
4. 選取 com.amazonaws.***Region***.rekognition 端點，然後選擇螢幕下半部的 Policy (政策) 標籤。
5. 選擇 Edit Policy (編輯政策)，並對政策做出變更。

Amazon Rekognition 的合規驗證

在多個 AWS 合規計劃中，第三方稽核人員會評估 Amazon Rekognition 的安全與合規。這些計劃包括 SOC、PCI、FedRAMP、HIPAA 等等。

如需特定合規計劃範圍內的 AWS 服務清單，請參閱合規計劃內的 AWS 服務。如需一般資訊，請參閱 AWS 合規計劃。

您可使用 AWS Artifact 下載第三方稽核報告。如需詳細資訊，請參閱在 AWS Artifact 中下載報告。

您使用 Amazon Rekognition 時的合規責任取決於資料的敏感度、您的公司的合規目標，以及適用的法律和法規。AWS 提供以下資源協助您處理合規事宜：

- 安全與合規快速入門指南 – 這些部署指南討論在 AWS 上部署以安全及合規為重心之基準環境的架構考量和步驟。
- HIPAA 安全與合規架構白皮書 – 本白皮書說明公司可如何運用 AWS 來建立 HIPAA 合規的應用程式。
- AWS 合規資源 – 這組手冊和指南可能適用於您的產業和位置。
- AWS Config – 此 AWS 服務可評定資源組態與內部實務、業界準則和法規的合規狀態。
- AWS Security Hub – 此 AWS 服務可供您檢視 AWS 中的安全狀態，可助您檢查是否符合安全產業標準和最佳實務。

Amazon Rekognition 中的彈性

AWS 全球基礎設施是以 AWS 區域與可用區域為中心建置的。AWS 區域提供多個分開且隔離的實際可用區域，並以低延遲、高輸送量和高度備援網路連線相互連結。透過可用區域，您可以設計與操作的應用程式和資料庫，在可用區域之間自動容錯移轉而不會發生中斷。可用區域的可用性、容錯能力和擴充能力，均較單一或多個資料中心的傳統基礎設施還高。

如需 AWS 區域與可用區域的詳細資訊，請參閱 AWS 全球基礎設施。

除了 AWS 全球基礎設施，Amazon Rekognition 還提供數種功能，可協助支援資料的彈性和備份需求。

Amazon Rekognition 中的組態與漏洞分析

組態和 IT 控制是 AWS 與身為我們客戶的您共同的責任。如需詳細資訊，請參閱 AWS 共同責任模型。

Amazon Rekognition 中的基礎設施安全

Amazon Rekognition 為受管服務，受到 Amazon Web Services：安全程序概觀白皮書所述的 AWS 全球網路安全程序所保護。

您可使用 AWS 發佈的 API 呼叫，透過網路存取 Amazon Rekognition。用戶端必須支援 Transport Layer Security (TLS) 1.0 或更新版本。建議使用 TLS 1.2 或更新版本。用戶端也必須支援具備完美轉送私密 (PFS) 的密碼套件，例如臨時 Diffie-Hellman (DHE) 或橢圓曲線臨時 Diffie-Hellman (ECDHE)。現代系統 (如 Java 7 和更新版本) 大多會支援這些模式。

此外，請求必須使用存取金鑰 ID 和與 IAM 委託人相關聯的私密存取金鑰來簽署。或者，您可以使用 AWS Security Token Service (AWS STS) 來產生暫時安全登入資料來簽署請求。

API 參考

本節提供 Amazon Rekognition API 操作的文件。

Amazon Rekognition Image

- the section called “CompareFaces” (p. 273)
- the section called “CreateCollection” (p. 279)
- the section called “DeleteCollection” (p. 292)
- the section called “DeleteFaces” (p. 294)
- the section called “DescribeCollection” (p. 302)
- the section called “DetectFaces” (p. 320)
- the section called “DetectLabels” (p. 324)
- the section called “DetectModerationLabels” (p. 329)
- the section called “DetectText” (p. 333)
- the section called “GetCelebrityInfo” (p. 337)
- the section called “IndexFaces” (p. 371)
- the section called “ListCollections” (p. 379)
- the section called “ListFaces” (p. 382)
- the section called “RecognizeCelebrities” (p. 388)
- the section called “SearchFaces” (p. 392)
- the section called “SearchFacesByImage” (p. 395)

Amazon Rekognition 自訂標籤

- the section called “CreateProject” (p. 282)
- the section called “CreateProjectVersion” (p. 285)
- the section called “DescribeProjects” (p. 305)
- the section called “DescribeProjectVersions” (p. 308)
- the section called “DetectCustomLabels” (p. 316)
- the section called “StartProjectVersion” (p. 422)
- the section called “StopProjectVersion” (p. 431)

Amazon Rekognition Video 已存放影片

- the section called “GetCelebrityRecognition” (p. 339)
- the section called “GetContentModeration” (p. 344)
- the section called “GetFaceDetection” (p. 348)
- the section called “GetFaceSearch” (p. 353)
- the section called “GetLabelDetection” (p. 358)
- the section called “GetPersonTracking” (p. 362)

- the section called “GetTextDetection” (p. 367)
- the section called “StartCelebrityRecognition” (p. 400)
- the section called “StartContentModeration” (p. 403)
- the section called “StartFaceDetection” (p. 407)
- the section called “StartFaceSearch” (p. 411)
- the section called “StartLabelDetection” (p. 415)
- the section called “StartPersonTracking” (p. 419)
- the section called “StartTextDetection” (p. 427)

Amazon Rekognition Video 串流視訊

- the section called “CreateStreamProcessor” (p. 289)
- the section called “DeleteStreamProcessor” (p. 300)
- the section called “DescribeStreamProcessor” (p. 312)
- the section called “ListStreamProcessors” (p. 385)
- the section called “StartStreamProcessor” (p. 425)
- the section called “StopStreamProcessor” (p. 433)

主題

- Actions (p. 271)
- Data Types (p. 434)

Actions

The following actions are supported:

- CompareFaces (p. 273)
- CreateCollection (p. 279)
- CreateProject (p. 282)
- CreateProjectVersion (p. 285)
- CreateStreamProcessor (p. 289)
- DeleteCollection (p. 292)
- DeleteFaces (p. 294)
- DeleteProject (p. 296)
- DeleteProjectVersion (p. 298)
- DeleteStreamProcessor (p. 300)
- DescribeCollection (p. 302)
- DescribeProjects (p. 305)
- DescribeProjectVersions (p. 308)
- DescribeStreamProcessor (p. 312)
- DetectCustomLabels (p. 316)
- DetectFaces (p. 320)
- DetectLabels (p. 324)
- DetectModerationLabels (p. 329)

- DetectText (p. 333)
- GetCelebrityInfo (p. 337)
- GetCelebrityRecognition (p. 339)
- GetContentModeration (p. 344)
- GetFaceDetection (p. 348)
- GetFaceSearch (p. 353)
- GetLabelDetection (p. 358)
- GetPersonTracking (p. 362)
- GetTextDetection (p. 367)
- IndexFaces (p. 371)
- ListCollections (p. 379)
- ListFaces (p. 382)
- ListStreamProcessors (p. 385)
- RecognizeCelebrities (p. 388)
- SearchFaces (p. 392)
- SearchFacesByImage (p. 395)
- StartCelebrityRecognition (p. 400)
- StartContentModeration (p. 403)
- StartFaceDetection (p. 407)
- StartFaceSearch (p. 411)
- StartLabelDetection (p. 415)
- StartPersonTracking (p. 419)
- StartProjectVersion (p. 422)
- StartStreamProcessor (p. 425)
- StartTextDetection (p. 427)
- StopProjectVersion (p. 431)
- StopStreamProcessor (p. 433)

CompareFaces

Compares a face in the source input image with each of the 100 largest faces detected in the target input image.

> Note
>
> If the source image contains multiple faces, the service detects the largest face and compares it with each face detected in the target image.

You pass the input and target images either as base64-encoded image bytes or as references to images in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes isn't supported. The image must be formatted as a PNG or JPEG file.

In response, the operation returns an array of face matches ordered by similarity score in descending order. For each face match, the response provides a bounding box of the face, facial landmarks, pose details (pitch, role, and yaw), quality (brightness and sharpness), and confidence value (indicating the level of confidence that the bounding box contains a face). The response also provides a similarity score, which indicates how closely the faces match.

> Note
>
> By default, only faces with a similarity score of greater than or equal to 80% are returned in the response. You can change this value by specifying the `SimilarityThreshold` parameter.

`CompareFaces` also returns an array of faces that don't match the source image. For each face, it returns a bounding box, confidence value, landmarks, pose details, and quality. The response also returns information about the face in the source image, including the bounding box of the face and confidence value.

The `QualityFilter` input parameter allows you to filter out detected faces that don’t meet a required quality bar. The quality bar is based on a variety of common use cases. Use `QualityFilter` to set the quality bar by specifying `LOW`, `MEDIUM`, or `HIGH`. If you do not want to filter detected faces, specify `NONE`. The default value is `NONE`.

> Note
>
> To use quality filtering, you need a collection associated with version 3 of the face model or higher. To get the version of the face model associated with a collection, call DescribeCollection (p. 302).

If the image doesn't contain Exif metadata, `CompareFaces` returns orientation information for the source and target images. Use these values to display the images with the correct image orientation.

If no faces are detected in the source or target images, `CompareFaces` returns an `InvalidParameterException` error.

> Note
>
> This is a stateless API operation. That is, data returned by this operation doesn't persist.

For an example, see 比較影像中的臉部 (p. 133).

This operation requires permissions to perform the `rekognition:CompareFaces` action.

Request Syntax

```
{
   "QualityFilter": "string",
```

```
   "SimilarityThreshold": number,
   "SourceImage": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "TargetImage": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

QualityFilter (p. 273)

A filter that specifies a quality bar for how much filtering is done to identify faces. Filtered faces aren't compared. If you specify `AUTO`, Amazon Rekognition chooses the quality bar. If you specify `LOW`, `MEDIUM`, or `HIGH`, filtering removes all faces that don't meet the chosen quality bar. The quality bar is based on a variety of common use cases. Low-quality detections can occur for a number of reasons. Some examples are an object that's misidentified as a face, a face that's too blurry, or a face with a pose that's too extreme to use. If you specify `NONE`, no filtering is performed. The default value is `NONE`.

To use quality filtering, the collection you are using must be associated with version 3 of the face model or higher.

Type: String

Valid Values: `NONE | AUTO | LOW | MEDIUM | HIGH`

Required: No

SimilarityThreshold (p. 273)

The minimum level of confidence in the face matches that a match must meet to be included in the `FaceMatches` array.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

SourceImage (p. 273)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

TargetImage (p. 273)

The target image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

Response Syntax

```
{
   "FaceMatches": [
      {
         "Face": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Landmarks": [
               {
                  "Type": "string",
                  "X": number,
                  "Y": number
               }
            ],
            "Pose": {
               "Pitch": number,
               "Roll": number,
               "Yaw": number
            },
            "Quality": {
               "Brightness": number,
               "Sharpness": number
            }
         },
         "Similarity": number
      }
   ],
   "SourceImageFace": {
      "BoundingBox": {
         "Height": number,
         "Left": number,
         "Top": number,
         "Width": number
      },
      "Confidence": number
   },
   "SourceImageOrientationCorrection": "string",
   "TargetImageOrientationCorrection": "string",
   "UnmatchedFaces": [
      {
         "BoundingBox": {
            "Height": number,
            "Left": number,
            "Top": number,
            "Width": number
```

```
         },
         "Confidence": number,
         "Landmarks": [
            {
               "Type": "string",
               "X": number,
               "Y": number
            }
         ],
         "Pose": {
            "Pitch": number,
            "Roll": number,
            "Yaw": number
         },
         "Quality": {
            "Brightness": number,
            "Sharpness": number
         }
      }
   ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceMatches (p. 275)

An array of faces in the target image that match the source image face. Each `CompareFacesMatch` object provides the bounding box, the confidence level that the bounding box contains a face, and the similarity score for the face in the bounding box and the face in the source image.

Type: Array of CompareFacesMatch (p. 448) objects

SourceImageFace (p. 275)

The face in the source image that was used for comparison.

Type: ComparedSourceImageFace (p. 447) object

SourceImageOrientationCorrection (p. 275)

The value of `SourceImageOrientationCorrection` is always null.

If the input image is in .jpeg format, it might contain exchangeable image file format (Exif) metadata that includes the image's orientation. Amazon Rekognition uses this orientation information to perform image correction. The bounding box coordinates are translated to represent object locations after the orientation information in the Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata.

Amazon Rekognition doesn't perform image correction for images in .png format and .jpeg images without orientation information in the image Exif metadata. The bounding box coordinates aren't translated and represent the object locations before the image is rotated.

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

TargetImageOrientationCorrection (p. 275)

The value of `TargetImageOrientationCorrection` is always null.

If the input image is in .jpeg format, it might contain exchangeable image file format (Exif) metadata that includes the image's orientation. Amazon Rekognition uses this orientation information to perform image correction. The bounding box coordinates are translated to represent object locations after the orientation information in the Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata.

Amazon Rekognition doesn't perform image correction for images in .png format and .jpeg images without orientation information in the image Exif metadata. The bounding box coordinates aren't translated and represent the object locations before the image is rotated.

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

UnmatchedFaces (p. 275)

An array of faces in the target image that did not match the source image face.

Type: Array of ComparedFace (p. 446) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

CreateCollection

Creates a collection in an AWS Region. You can add faces to the collection using the IndexFaces (p. 371) operation.

For example, you might create collections, one for each of your application users. A user can then index faces using the `IndexFaces` operation and persist results in a specific collection. Then, a user can search the collection for faces in the user-specific container.

When you create a collection, it is associated with the latest version of the face model version.

> Note
>
> Collection names are case-sensitive.

This operation requires permissions to perform the `rekognition:CreateCollection` action.

Request Syntax

```
{
   "CollectionId": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 279)

ID for the collection that you are creating.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "CollectionArn": "string",
   "FaceModelVersion": "string",
   "StatusCode": number
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CollectionArn (p. 279)

Amazon Resource Name (ARN) of the collection. You can use this to manage permissions on your resources.

Type: String

FaceModelVersion (p. 279)

Version number of the face detection model associated with the collection you are creating.

Type: String

StatusCode (p. 279)

HTTP status code indicating the result of the operation.

Type: Integer

Valid Range: Minimum value of 0.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceAlreadyExistsException

A collection with the specified ID already exists.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go

- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

CreateProject

Creates a new Amazon Rekognition Custom Labels project. A project is a logical grouping of resources (images, Labels, models) and operations (training, evaluation and detection).

This operation requires permissions to perform the `rekognition:CreateProject` action.

Request Syntax

```
{
   "ProjectName": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

ProjectName (p. 282)

The name of the project to create.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "ProjectArn": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

ProjectArn (p. 282)

The Amazon Resource Name (ARN) of the new project. You can use the ARN to configure IAM access to the project.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

CreateProjectVersion

Creates a new version of a model and begins training. Models are managed as part of an Amazon Rekognition Custom Labels project. You can specify one training dataset and one testing dataset. The response from `CreateProjectVersion` is an Amazon Resource Name (ARN) for the version of the model.

Training takes a while to complete. You can get the current status by calling DescribeProjectVersions (p. 308).

Once training has successfully completed, call DescribeProjectVersions (p. 308) to get the training results and evaluate the model.

After evaluating the model, you start the model by calling StartProjectVersion (p. 422).

This operation requires permissions to perform the `rekognition:CreateProjectVersion` action.

Request Syntax

```
{
   "OutputConfig": {
      "S3Bucket": "string",
      "S3KeyPrefix": "string"
   },
   "ProjectArn": "string",
   "TestingData": {
      "Assets": [
         {
            "GroundTruthManifest": {
               "S3Object": {
                  "Bucket": "string",
                  "Name": "string",
                  "Version": "string"
               }
            }
         }
      ],
      "AutoCreate": boolean
   },
   "TrainingData": {
      "Assets": [
         {
            "GroundTruthManifest": {
               "S3Object": {
                  "Bucket": "string",
                  "Name": "string",
                  "Version": "string"
               }
            }
         }
      ]
   },
   "VersionName": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

OutputConfig (p. 285)

The Amazon S3 location to store the results of training.

Type: OutputConfig (p. 484) object

Required: Yes

ProjectArn (p. 285)

The ARN of the Amazon Rekognition Custom Labels project that manages the model that you want to train.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

TestingData (p. 285)

The dataset to use for testing.

Type: TestingData (p. 504) object

Required: Yes

TrainingData (p. 285)

The dataset to use for training.

Type: TrainingData (p. 509) object

Required: Yes

VersionName (p. 285)

A name for the version of the model. This value must be unique.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "ProjectVersionArn": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

ProjectVersionArn (p. 286)

The ARN of the model version that was created. Use `DescribeProjectVersion` to get the current status of the training operation.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

CreateStreamProcessor

Creates an Amazon Rekognition stream processor that you can use to detect and recognize faces in a streaming video.

Amazon Rekognition Video is a consumer of live video from Amazon Kinesis Video Streams. Amazon Rekognition Video sends analysis results to Amazon Kinesis Data Streams.

You provide as input a Kinesis video stream (`Input`) and a Kinesis data stream (`Output`) stream. You also specify the face recognition criteria in `Settings`. For example, the collection containing faces that you want to recognize. Use `Name` to assign an identifier for the stream processor. You use `Name` to manage the stream processor. For example, you can start processing the source video by calling StartStreamProcessor (p. 425) with the `Name` field.

After you have finished analyzing a streaming video, use StopStreamProcessor (p. 433) to stop processing. You can delete the stream processor by calling DeleteStreamProcessor (p. 300).

Request Syntax

```
{
   "Input": {
      "KinesisVideoStream": {
         "Arn": "string"
      }
   },
   "Name": "string",
   "Output": {
      "KinesisDataStream": {
         "Arn": "string"
      }
   },
   "RoleArn": "string",
   "Settings": {
      "FaceSearch": {
         "CollectionId": "string",
         "FaceMatchThreshold": number
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

Input (p. 289)

Kinesis video stream stream that provides the source streaming video. If you are using the AWS CLI, the parameter name is `StreamProcessorInput`.

Type: StreamProcessorInput (p. 499) object

Required: Yes

Name (p. 289)

An identifier you assign to the stream processor. You can use `Name` to manage the stream processor. For example, you can get the current status of the stream processor by calling DescribeStreamProcessor (p. 312). `Name` is idempotent.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Output (p. 289)

Kinesis data stream stream to which Amazon Rekognition Video puts the analysis results. If you are using the AWS CLI, the parameter name is `StreamProcessorOutput`.

Type: StreamProcessorOutput (p. 500) object

Required: Yes

RoleArn (p. 289)

ARN of the IAM role that allows access to the stream processor.

Type: String

Pattern: `arn:aws:iam::\d{12}:role/?[a-zA-Z_0-9+=,.@\-_/]+`

Required: Yes

Settings (p. 289)

Face recognition input parameters to be used by the stream processor. Includes the collection to use for face recognition and the face attributes to detect.

Type: StreamProcessorSettings (p. 501) object

Required: Yes

Response Syntax

```
{
   "StreamProcessorArn": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

StreamProcessorArn (p. 290)

ARN for the newly create stream processor.

Type: String

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:streamprocessor\/.+$)`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DeleteCollection

Deletes the specified collection. Note that this operation removes all faces in the collection. For an example, see 刪除集合 (p. 159).

This operation requires permissions to perform the `rekognition:DeleteCollection` action.

Request Syntax

```
{
   "CollectionId": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 292)

ID of the collection to delete.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "StatusCode": number
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

StatusCode (p. 292)

HTTP status code that indicates the result of the operation.

Type: Integer

Valid Range: Minimum value of 0.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DeleteFaces

Deletes faces from a collection. You specify a collection ID and an array of face IDs to remove from the collection.

This operation requires permissions to perform the `rekognition:DeleteFaces` action.

Request Syntax

```
{
   "CollectionId": "string",
   "FaceIds": [ "string" ]
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 294)

Collection from which to remove the specific faces.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

FaceIds (p. 294)

An array of face IDs to delete.

Type: Array of strings

Array Members: Minimum number of 1 item. Maximum number of 4096 items.

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Required: Yes

Response Syntax

```
{
   "DeletedFaces": [ "string" ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

DeletedFaces (p. 294)

An array of strings (face IDs) of the faces that were deleted.

Type: Array of strings

Array Members: Minimum number of 1 item. Maximum number of 4096 items.

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DeleteProject

Deletes an Amazon Rekognition Custom Labels project. To delete a project you must first delete all models associated with the project. To delete a model, see DeleteProjectVersion (p. 298).

This operation requires permissions to perform the `rekognition:DeleteProject` action.

Request Syntax

```
{
   "ProjectArn": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

ProjectArn (p. 296)

The Amazon Resource Name (ARN) of the project that you want to delete.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

Response Syntax

```
{
   "Status": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Status (p. 296)

The current status of the delete project operation.

Type: String

Valid Values: `CREATING | CREATED | DELETING`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DeleteProjectVersion

Deletes an Amazon Rekognition Custom Labels model.

You can't delete a model if it is running or if it is training. To check the status of a model, use the `Status` field returned from DescribeProjectVersions (p. 308). To stop a running model call StopProjectVersion (p. 431). If the model is training, wait until it finishes.

This operation requires permissions to perform the `rekognition:DeleteProjectVersion` action.

Request Syntax

```
{
   "ProjectVersionArn": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

ProjectVersionArn (p. 298)

The Amazon Resource Name (ARN) of the model version that you want to delete.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

Response Syntax

```
{
   "Status": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Status (p. 298)

The status of the deletion operation.

Type: String

Valid Values: `TRAINING_IN_PROGRESS | TRAINING_COMPLETED | TRAINING_FAILED | STARTING | RUNNING | FAILED | STOPPING | STOPPED | DELETING`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DeleteStreamProcessor

Deletes the stream processor identified by `Name`. You assign the value for `Name` when you create the stream processor with CreateStreamProcessor (p. 289). You might not be able to use the same name for a stream processor for a few seconds after calling `DeleteStreamProcessor`.

Request Syntax

```
{
   "Name": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Name (p. 300)

The name of the stream processor you want to delete.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Elements

If the action is successful, the service sends back an HTTP 200 response with an empty HTTP body.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DescribeCollection

Describes the specified collection. You can use `DescribeCollection` to get information, such as the number of faces indexed into a collection and the version of the model used by the collection for face detection.

For more information, see 描述集合 (p. 156).

Request Syntax

```
{
   "CollectionId": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 302)

The ID of the collection to describe.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "CollectionARN": "string",
   "CreationTimestamp": number,
   "FaceCount": number,
   "FaceModelVersion": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CollectionARN (p. 302)

The Amazon Resource Name (ARN) of the collection.

Type: String

CreationTimestamp (p. 302)

The number of milliseconds since the Unix epoch time until the creation of the collection. The Unix epoch time is 00:00:00 Coordinated Universal Time (UTC), Thursday, 1 January 1970.

Type: Timestamp

FaceCount (p. 302)

The number of faces that are indexed into the collection. To index faces into a collection, use IndexFaces (p. 371).

Type: Long

Valid Range: Minimum value of 0.

FaceModelVersion (p. 302)

The version of the face model that's used by the collection for face detection.

For more information, see 模型版本控制 (p. 9).

Type: String

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DescribeProjects

Lists and gets information about your Amazon Rekognition Custom Labels projects.

This operation requires permissions to perform the `rekognition:DescribeProjects` action.

Request Syntax

```
{
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

MaxResults (p. 305)

The maximum number of results to return per paginated call. The largest value you can specify is 100. If you specify a value greater than 100, a ValidationException error occurs. The default value is 100.

Type: Integer

Valid Range: Minimum value of 1. Maximum value of 100.

Required: No

NextToken (p. 305)

If the previous response was incomplete (because there is more results to retrieve), Amazon Rekognition Custom Labels returns a pagination token in the response. You can use this pagination token to retrieve the next set of results.

Type: String

Length Constraints: Maximum length of 1024.

Required: No

Response Syntax

```
{
   "NextToken": "string",
   "ProjectDescriptions": [
      {
         "CreationTimestamp": number,
         "ProjectArn": "string",
         "Status": "string"
      }
   ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

NextToken (p. 305)

If the previous response was incomplete (because there is more results to retrieve), Amazon Rekognition Custom Labels returns a pagination token in the response. You can use this pagination token to retrieve the next set of results.

Type: String

Length Constraints: Maximum length of 1024.

ProjectDescriptions (p. 305)

A list of project descriptions. The list is sorted by the date and time the projects are created.

Type: Array of ProjectDescription (p. 491) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DescribeProjectVersions

Lists and describes the models in an Amazon Rekognition Custom Labels project. You can specify up to 10 model versions in `ProjectVersionArns`. If you don't specify a value, descriptions for all models are returned.

This operation requires permissions to perform the `rekognition:DescribeProjectVersions` action.

Request Syntax

```
{
   "MaxResults": number,
   "NextToken": "string",
   "ProjectArn": "string",
   "VersionNames": [ "string" ]
}
```

Request Parameters

The request accepts the following data in JSON format.

MaxResults (p. 308)

The maximum number of results to return per paginated call. The largest value you can specify is 100. If you specify a value greater than 100, a ValidationException error occurs. The default value is 100.

Type: Integer

Valid Range: Minimum value of 1. Maximum value of 100.

Required: No

NextToken (p. 308)

If the previous response was incomplete (because there is more results to retrieve), Amazon Rekognition Custom Labels returns a pagination token in the response. You can use this pagination token to retrieve the next set of results.

Type: String

Length Constraints: Maximum length of 1024.

Required: No

ProjectArn (p. 308)

The Amazon Resource Name (ARN) of the project that contains the models you want to describe.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

VersionNames (p. 308)

A list of model version names that you want to describe. You can add up to 10 model version names to the list. If you don't specify a value, all model descriptions are returned. A version name is part of a

model (ProjectVersion) ARN. For example, `my-model.2020-01-21T09.10.15` is the version name in the following ARN. `arn:aws:rekognition:us-east-1:123456789012:project/getting-started/version/`*`my-model.2020-01-21T09.10.15`*`/1234567890123`.

Type: Array of strings

Array Members: Minimum number of 1 item. Maximum number of 10 items.

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: No

Response Syntax

```
{
   "NextToken": "string",
   "ProjectVersionDescriptions": [
      {
         "BillableTrainingTimeInSeconds": number,
         "CreationTimestamp": number,
         "EvaluationResult": {
            "F1Score": number,
            "Summary": {
               "S3Object": {
                  "Bucket": "string",
                  "Name": "string",
                  "Version": "string"
               }
            }
         },
         "MinInferenceUnits": number,
         "OutputConfig": {
            "S3Bucket": "string",
            "S3KeyPrefix": "string"
         },
         "ProjectVersionArn": "string",
         "Status": "string",
         "StatusMessage": "string",
         "TestingDataResult": {
            "Input": {
               "Assets": [
                  {
                     "GroundTruthManifest": {
                        "S3Object": {
                           "Bucket": "string",
                           "Name": "string",
                           "Version": "string"
                        }
                     }
                  }
               ],
               "AutoCreate": boolean
            },
            "Output": {
               "Assets": [
                  {
                     "GroundTruthManifest": {
                        "S3Object": {
                           "Bucket": "string",
                           "Name": "string",
```

```
                        "Version": "string"
                     }
                  }
               }
            ],
            "AutoCreate": boolean
         }
      },
      "TrainingDataResult": {
         "Input": {
            "Assets": [
               {
                  "GroundTruthManifest": {
                     "S3Object": {
                        "Bucket": "string",
                        "Name": "string",
                        "Version": "string"
                     }
                  }
               }
            ]
         },
         "Output": {
            "Assets": [
               {
                  "GroundTruthManifest": {
                     "S3Object": {
                        "Bucket": "string",
                        "Name": "string",
                        "Version": "string"
                     }
                  }
               }
            ]
         }
      },
      "TrainingEndTimestamp": number
   }
]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

NextToken (p. 309)

If the previous response was incomplete (because there is more results to retrieve), Amazon Rekognition Custom Labels returns a pagination token in the response. You can use this pagination token to retrieve the next set of results.

Type: String

Length Constraints: Maximum length of 1024.

ProjectVersionDescriptions (p. 309)

A list of model descriptions. The list is sorted by the creation date and time of the model versions, latest to earliest.

Type: Array of ProjectVersionDescription (p. 492) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DescribeStreamProcessor

Provides information about a stream processor created by CreateStreamProcessor (p. 289). You can get information about the input and output streams, the input parameters for the face recognition being performed, and the current status of the stream processor.

Request Syntax

```
{
   "Name": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Name (p. 312)

Name of the stream processor for which you want information.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Syntax

```
{
   "CreationTimestamp": number,
   "Input": {
      "KinesisVideoStream": {
         "Arn": "string"
      }
   },
   "LastUpdateTimestamp": number,
   "Name": "string",
   "Output": {
      "KinesisDataStream": {
         "Arn": "string"
      }
   },
   "RoleArn": "string",
   "Settings": {
      "FaceSearch": {
         "CollectionId": "string",
         "FaceMatchThreshold": number
      }
   },
   "Status": "string",
   "StatusMessage": "string",
   "StreamProcessorArn": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CreationTimestamp (p. 312)

Date and time the stream processor was created

Type: Timestamp

Input (p. 312)

Kinesis video stream that provides the source streaming video.

Type: StreamProcessorInput (p. 499) object

LastUpdateTimestamp (p. 312)

The time, in Unix format, the stream processor was last updated. For example, when the stream processor moves from a running state to a failed state, or when the user starts or stops the stream processor.

Type: Timestamp

Name (p. 312)

Name of the stream processor.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Output (p. 312)

Kinesis data stream to which Amazon Rekognition Video puts the analysis results.

Type: StreamProcessorOutput (p. 500) object

RoleArn (p. 312)

ARN of the IAM role that allows access to the stream processor.

Type: String

Pattern: `arn:aws:iam::\d{12}:role/?[a-zA-Z_0-9+=,.@\-_/]+`

Settings (p. 312)

Face recognition input parameters that are being used by the stream processor. Includes the collection to use for face recognition and the face attributes to detect.

Type: StreamProcessorSettings (p. 501) object

Status (p. 312)

Current status of the stream processor.

Type: String

Valid Values: `STOPPED | STARTING | RUNNING | FAILED | STOPPING`

StatusMessage (p. 312)

Detailed status message about the stream processor.

Type: String

StreamProcessorArn (p. 312)

ARN of the stream processor.

Type: String

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:streamprocessor\/.+$)`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DetectCustomLabels

Detects custom labels in a supplied image by using an Amazon Rekognition Custom Labels model.

You specify which version of a model version to use by using the `ProjectVersionArn` input parameter.

You pass the input image as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

For each object that the model version detects on an image, the API returns a (`CustomLabel`) object in an array (`CustomLabels`). Each `CustomLabel` object provides the label name (`Name`), the level of confidence that the image contains the object (`Confidence`), and object location information, if it exists, for the label on the image (`Geometry`).

During training model calculates a threshold value that determines if a prediction for a label is true. By default, `DetectCustomLabels` doesn't return labels whose confidence value is below the model's calculated threshold value. To filter labels that are returned, specify a value for `MinConfidence` that is higher than the model's calculated threshold. You can get the model's calculated threshold from the model's training results shown in the Amazon Rekognition Custom Labels console. To get all labels, regardless of confidence, specify a `MinConfidence` value of 0.

You can also add the `MaxResults` parameter to limit the number of labels returned.

This is a stateless API operation. That is, the operation does not persist any data.

This operation requires permissions to perform the `rekognition:DetectCustomLabels` action.

Request Syntax

```
{
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "MaxResults": number,
   "MinConfidence": number,
   "ProjectVersionArn": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Image (p. 316)

Provides the input image either as bytes or an S3 object.

You pass image bytes to an Amazon Rekognition API operation by using the `Bytes` property. For example, you would use the `Bytes` property to pass an image loaded from a local file system. Image bytes passed by using the `Bytes` property must be base64-encoded. Your code may not need to encode image bytes if you are using an AWS SDK to call Amazon Rekognition API operations.

For more information, see 分析從本機檔案系統載入的影像 (p. 29).

You pass images stored in an S3 bucket to an Amazon Rekognition API operation by using the `S3Object` property. Images stored in an S3 bucket do not need to be base64-encoded.

The region for the S3 bucket containing the S3 object must match the region you use for Amazon Rekognition operations.

If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes using the Bytes property is not supported. You must first upload the image to an Amazon S3 bucket and then call the operation using the S3Object property.

For Amazon Rekognition to process an S3 object, the user must have permission to access the S3 object. For more information, see Amazon Rekognition 資源型政策 (p. 250).

Type: Image (p. 472) object

Required: Yes

MaxResults (p. 316)

Maximum number of results you want the service to return in the response. The service returns the specified number of highest confidence labels ranked from highest confidence to lowest.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

MinConfidence (p. 316)

Specifies the minimum confidence level for the labels to return. Amazon Rekognition doesn't return any labels with a confidence lower than this specified value. If you specify a value of 0, all labels are return, regardless of the default thresholds that the model version applies.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

ProjectVersionArn (p. 316)

The ARN of the model version that you want to use.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

Response Syntax

```
{
   "CustomLabels": [
      {
         "Confidence": number,
         "Geometry": {
            "BoundingBox": {
               "Height": number,
```

```
                "Left": number,
                "Top": number,
                "Width": number
             },
             "Polygon": [
                {
                   "X": number,
                   "Y": number
                }
             ]
          },
          "Name": "string"
       }
   ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CustomLabels (p. 317)

An array of custom labels detected in the input image.

Type: Array of CustomLabel (p. 450) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ResourceNotReadyException

The requested resource isn't ready. For example, this exception occurs when you call `DetectCustomLabels` with a model version that isn't deployed.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DetectFaces

Detects faces within an image that is provided as input.

`DetectFaces` detects the 100 largest faces in the image. For each face detected, the operation returns face details. These details include a bounding box of the face, a confidence value (that the bounding box contains a face), and a fixed set of attributes such as facial landmarks (for example, coordinates of eye and mouth), presence of beard, sunglasses, and so on.

The face-detection algorithm is most effective on frontal faces. For non-frontal or obscured faces, the algorithm might not detect the faces or might detect faces with lower confidence.

You pass the input image either as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

> Note
>
> This is a stateless API operation. That is, the operation does not persist any data.

This operation requires permissions to perform the `rekognition:DetectFaces` action.

Request Syntax

```
{
   "Attributes": [ "string" ],
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

Attributes (p. 320)

An array of facial attributes you want to be returned. This can be the default list of attributes or all attributes. If you don't specify a value for `Attributes` or if you specify `["DEFAULT"]`, the API returns the following subset of facial attributes: `BoundingBox`, `Confidence`, `Pose`, `Quality`, and `Landmarks`. If you provide `["ALL"]`, all facial attributes are returned, but the operation takes longer to complete.

If you provide both, `["ALL", "DEFAULT"]`, the service uses a logical AND operator to determine which attributes to return (in this case, all attributes).

Type: Array of strings

Valid Values: `DEFAULT | ALL`

Required: No

Image (p. 320)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

Response Syntax

```
{
   "FaceDetails": [
      {
         "AgeRange": {
            "High": number,
            "Low": number
         },
         "Beard": {
            "Confidence": number,
            "Value": boolean
         },
         "BoundingBox": {
            "Height": number,
            "Left": number,
            "Top": number,
            "Width": number
         },
         "Confidence": number,
         "Emotions": [
            {
               "Confidence": number,
               "Type": "string"
            }
         ],
         "Eyeglasses": {
            "Confidence": number,
            "Value": boolean
         },
         "EyesOpen": {
            "Confidence": number,
            "Value": boolean
         },
         "Gender": {
            "Confidence": number,
            "Value": "string"
         },
         "Landmarks": [
            {
               "Type": "string",
               "X": number,
               "Y": number
            }
         ],
         "MouthOpen": {
            "Confidence": number,
            "Value": boolean
         },
         "Mustache": {
            "Confidence": number,
            "Value": boolean
         },
         "Pose": {
            "Pitch": number,
            "Roll": number,
```

```
            "Yaw": number
         },
         "Quality": {
            "Brightness": number,
            "Sharpness": number
         },
         "Smile": {
            "Confidence": number,
            "Value": boolean
         },
         "Sunglasses": {
            "Confidence": number,
            "Value": boolean
         }
      }
   ],
   "OrientationCorrection": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceDetails (p. 321)

Details of each face found in the image.

Type: Array of FaceDetail (p. 459) objects

OrientationCorrection (p. 321)

The value of `OrientationCorrection` is always null.

If the input image is in .jpeg format, it might contain exchangeable image file format (Exif) metadata that includes the image's orientation. Amazon Rekognition uses this orientation information to perform image correction. The bounding box coordinates are translated to represent object locations after the orientation information in the Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata.

Amazon Rekognition doesn’t perform image correction for images in .png format and .jpeg images without orientation information in the image Exif metadata. The bounding box coordinates aren't translated and represent the object locations before the image is rotated.

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DetectLabels

Detects instances of real-world entities within an image (JPEG or PNG) provided as input. This includes objects like flower, tree, and table; events like wedding, graduation, and birthday party; and concepts like landscape, evening, and nature.

For an example, see 分析存放在 Amazon S3 儲存貯體中的影像 (p. 24).

> Note
>
> `DetectLabels` does not support the detection of activities. However, activity detection is supported for label detection in videos. For more information, see StartLabelDetection (p. 415).

You pass the input image as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

For each object, scene, and concept the API returns one or more labels. Each label provides the object name, and the level of confidence that the image contains the object. For example, suppose the input image has a lighthouse, the sea, and a rock. The response includes all three labels, one for each object.

```
{Name: lighthouse, Confidence: 98.4629}

{Name: rock,Confidence: 79.2097}

{Name: sea,Confidence: 75.061}
```

In the preceding example, the operation returns one label for each of the three objects. The operation can also return multiple labels for the same object in the image. For example, if the input image shows a flower (for example, a tulip), the operation might return the following three labels.

```
{Name: flower,Confidence: 99.0562}

{Name: plant,Confidence: 99.0562}

{Name: tulip,Confidence: 99.0562}
```

In this example, the detection algorithm more precisely identifies the flower as a tulip.

In response, the API returns an array of labels. In addition, the response also includes the orientation correction. Optionally, you can specify `MinConfidence` to control the confidence threshold for the labels returned. The default is 55%. You can also add the `MaxLabels` parameter to limit the number of labels returned.

> Note
>
> If the object detected is a person, the operation doesn't provide the same facial details that the DetectFaces (p. 320) operation provides.

`DetectLabels` returns bounding boxes for instances of common object labels in an array of Instance (p. 474) objects. An `Instance` object contains a BoundingBox (p. 440) object, for the location of the label on the image. It also includes the confidence by which the bounding box was detected.

`DetectLabels` also returns a hierarchical taxonomy of detected labels. For example, a detected car might be assigned the label car. The label car has two parent labels: Vehicle (its parent) and Transportation (its grandparent). The response returns the entire list of ancestors for a label. Each ancestor is a unique label in the response. In the previous example, Car, Vehicle, and Transportation are returned as unique labels in the response.

This is a stateless API operation. That is, the operation does not persist any data.

This operation requires permissions to perform the `rekognition:DetectLabels` action.

Request Syntax

```
{
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "MaxLabels": number,
   "MinConfidence": number
}
```

Request Parameters

The request accepts the following data in JSON format.

Image (p. 325)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. Images stored in an S3 Bucket do not need to be base64-encoded.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

MaxLabels (p. 325)

Maximum number of labels you want the service to return in the response. The service returns the specified number of highest confidence labels.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

MinConfidence (p. 325)

Specifies the minimum confidence level for the labels to return. Amazon Rekognition doesn't return any labels with confidence lower than this specified value.

If `MinConfidence` is not specified, the operation returns labels with a confidence values greater than or equal to 55 percent.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Response Syntax

```
{
   "LabelModelVersion": "string",
   "Labels": [
      {
         "Confidence": number,
         "Instances": [
            {
               "BoundingBox": {
                  "Height": number,
                  "Left": number,
                  "Top": number,
                  "Width": number
               },
               "Confidence": number
            }
         ],
         "Name": "string",
         "Parents": [
            {
               "Name": "string"
            }
         ]
      }
   ],
   "OrientationCorrection": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

LabelModelVersion (p. 326)

Version number of the label detection model that was used to detect labels.

Type: String

Labels (p. 326)

An array of labels for the real-world objects detected.

Type: Array of Label (p. 477) objects

OrientationCorrection (p. 326)

The value of `OrientationCorrection` is always null.

If the input image is in .jpeg format, it might contain exchangeable image file format (Exif) metadata that includes the image's orientation. Amazon Rekognition uses this orientation information to perform image correction. The bounding box coordinates are translated to represent object locations after the orientation information in the Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata.

Amazon Rekognition doesn’t perform image correction for images in .png format and .jpeg images without orientation information in the image Exif metadata. The bounding box coordinates aren't translated and represent the object locations before the image is rotated.

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go

- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

DetectModerationLabels

Detects unsafe content in a specified JPEG or PNG format image. Use `DetectModerationLabels` to moderate images depending on your requirements. For example, you might want to filter images that contain nudity, but not images containing suggestive content.

To filter images, use the labels returned by `DetectModerationLabels` to determine which types of content are appropriate.

For information about moderation labels, see 偵測不安全的內容 (p. 215).

You pass the input image either as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

Request Syntax

```
{
   "HumanLoopConfig": {
      "DataAttributes": {
         "ContentClassifiers": [ "string" ]
      },
      "FlowDefinitionArn": "string",
      "HumanLoopName": "string"
   },
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "MinConfidence": number
}
```

Request Parameters

The request accepts the following data in JSON format.

HumanLoopConfig (p. 329)

Sets up the configuration for human evaluation, including the FlowDefinition the image will be sent to.

Type: HumanLoopConfig (p. 470) object

Required: No

Image (p. 329)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

MinConfidence (p. 329)

Specifies the minimum confidence level for the labels to return. Amazon Rekognition doesn't return any labels with a confidence level lower than this specified value.

If you don't specify `MinConfidence`, the operation returns labels with confidence values greater than or equal to 50 percent.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Response Syntax

```
{
   "HumanLoopActivationOutput": {
      "HumanLoopActivationConditionsEvaluationResults": "string",
      "HumanLoopActivationReasons": [ "string" ],
      "HumanLoopArn": "string"
   },
   "ModerationLabels": [
      {
         "Confidence": number,
         "Name": "string",
         "ParentName": "string"
      }
   ],
   "ModerationModelVersion": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

HumanLoopActivationOutput (p. 330)

Shows the results of the human in the loop evaluation.

Type: HumanLoopActivationOutput (p. 469) object

ModerationLabels (p. 330)

Array of detected Moderation labels and the time, in milliseconds from the start of the video, they were detected.

Type: Array of ModerationLabel (p. 480) objects

ModerationModelVersion (p. 330)

Version number of the moderation detection model that was used to detect unsafe content.

Type: String

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

HumanLoopQuotaExceededException

The number of in-progress human reviews you have has exceeded the number allowed.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3

- AWS SDK for Python
- AWS SDK for Ruby V3

DetectText

Detects text in the input image and converts it into machine-readable text.

Pass the input image as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, you must pass it as a reference to an image in an Amazon S3 bucket. For the AWS CLI, passing image bytes is not supported. The image must be either a .png or .jpeg formatted file.

The `DetectText` operation returns text in an array of TextDetection (p. 506) elements, `TextDetections`. Each `TextDetection` element provides information about a single word or line of text that was detected in the image.

A word is one or more ISO basic latin script characters that are not separated by spaces. `DetectText` can detect up to 50 words in an image.

A line is a string of equally spaced words. A line isn't necessarily a complete sentence. For example, a driver's license number is detected as a line. A line ends when there is no aligned text after it. Also, a line ends when there is a large gap between words, relative to the length of the words. This means, depending on the gap between words, Amazon Rekognition may detect multiple lines in text aligned in the same direction. Periods don't represent the end of a line. If a sentence spans multiple lines, the `DetectText` operation returns multiple lines.

To determine whether a `TextDetection` element is a line of text or a word, use the `TextDetection` object `Type` field.

To be detected, text must be within +/- 90 degrees orientation of the horizontal axis.

For more information, see 偵測文字 (p. 227).

Request Syntax

```
{
   "Filters": {
      "RegionsOfInterest": [
         {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            }
         }
      ],
      "WordFilter": {
         "MinBoundingBoxHeight": number,
         "MinBoundingBoxWidth": number,
         "MinConfidence": number
      }
   },
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

Filters (p. 333)

Optional parameters that let you set the criteria that the text must meet to be included in your response.

Type: DetectTextFilters (p. 452) object

Required: No

Image (p. 333)

The input image as base64-encoded bytes or an Amazon S3 object. If you use the AWS CLI to call Amazon Rekognition operations, you can't pass image bytes.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

Response Syntax

```
{
   "TextDetections": [
      {
         "Confidence": number,
         "DetectedText": "string",
         "Geometry": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Polygon": [
               {
                  "X": number,
                  "Y": number
               }
            ]
         },
         "Id": number,
         "ParentId": number,
         "Type": "string"
      }
   ],
   "TextModelVersion": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

TextDetections (p. 334)

An array of text that was detected in the input image.

Type: Array of TextDetection (p. 506) objects

TextModelVersion (p. 334)

The model version used to detect text.

Type: String

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface

- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetCelebrityInfo

Gets the name and additional information about a celebrity based on his or her Amazon Rekognition ID. The additional information is returned as an array of URLs. If there is no additional information about the celebrity, this list is empty.

For more information, see 取得名人的相關資訊 (p. 211).

This operation requires permissions to perform the `rekognition:GetCelebrityInfo` action.

Request Syntax

```
{
   "Id": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Id (p. 337)

The ID for the celebrity. You get the celebrity ID from a call to the RecognizeCelebrities (p. 388) operation, which recognizes celebrities in an image.

Type: String

Pattern: `[0-9A-Za-z]*`

Required: Yes

Response Syntax

```
{
   "Name": "string",
   "Urls": [ "string" ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Name (p. 337)

The name of the celebrity.

Type: String

Urls (p. 337)

An array of URLs pointing to additional celebrity information.

Type: Array of strings

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetCelebrityRecognition

Gets the celebrity recognition results for a Amazon Rekognition Video analysis started by StartCelebrityRecognition (p. 400).

Celebrity recognition in a video is an asynchronous operation. Analysis is started by a call to StartCelebrityRecognition (p. 400) which returns a job identifier (`JobId`). When the celebrity recognition operation finishes, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartCelebrityRecognition`. To get the results of the celebrity recognition analysis, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call `GetCelebrityDetection` and pass the job identifier (`JobId`) from the initial call to `StartCelebrityDetection`.

For more information, see 使用儲存的影片來操作 (p. 49).

`GetCelebrityRecognition` returns detected celebrities and the time(s) they are detected in an array (`Celebrities`) of CelebrityRecognition (p. 445) objects. Each `CelebrityRecognition` contains information about the celebrity in a CelebrityDetail (p. 443) object and the time, `Timestamp`, the celebrity was detected.

> Note
>
> `GetCelebrityRecognition` only returns the default facial attributes (`BoundingBox`, `Confidence`, `Landmarks`, `Pose`, and `Quality`). The other facial attributes listed in the `Face` object of the following response syntax are not returned. For more information, see FaceDetail (p. 459).

By default, the `Celebrities` array is sorted by time (milliseconds from the start of the video). You can also sort the array by celebrity by specifying the value `ID` in the `SortBy` input parameter.

The `CelebrityDetail` object includes the celebrity identifer and additional information urls. If you don't store the additional information urls, you can get them later by calling GetCelebrityInfo (p. 337) with the celebrity identifer.

No information is returned for faces not recognized as celebrities.

Use MaxResults parameter to limit the number of labels returned. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetCelebrityDetection` and populate the `NextToken` request parameter with the token value returned from the previous call to `GetCelebrityRecognition`.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string",
   "SortBy": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 339)

Job identifier for the required celebrity recognition analysis. You can get the job identifer from a call to `StartCelebrityRecognition`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 339)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 339)

If the previous response was incomplete (because there is more recognized celebrities to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of celebrities.

Type: String

Length Constraints: Maximum length of 255.

Required: No

SortBy (p. 339)

Sort to use for celebrities returned in `Celebrities` field. Specify `ID` to sort by the celebrity identifier, specify `TIMESTAMP` to sort by the time the celebrity was recognized.

Type: String

Valid Values: `ID | TIMESTAMP`

Required: No

Response Syntax

```
{
   "Celebrities": [
      {
         "Celebrity": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Face": {
               "AgeRange": {
                  "High": number,
                  "Low": number
               },
               "Beard": {
                  "Confidence": number,
```

```
            "Value": boolean
         },
         "BoundingBox": {
            "Height": number,
            "Left": number,
            "Top": number,
            "Width": number
         },
         "Confidence": number,
         "Emotions": [
            {
               "Confidence": number,
               "Type": "string"
            }
         ],
         "Eyeglasses": {
            "Confidence": number,
            "Value": boolean
         },
         "EyesOpen": {
            "Confidence": number,
            "Value": boolean
         },
         "Gender": {
            "Confidence": number,
            "Value": "string"
         },
         "Landmarks": [
            {
               "Type": "string",
               "X": number,
               "Y": number
            }
         ],
         "MouthOpen": {
            "Confidence": number,
            "Value": boolean
         },
         "Mustache": {
            "Confidence": number,
            "Value": boolean
         },
         "Pose": {
            "Pitch": number,
            "Roll": number,
            "Yaw": number
         },
         "Quality": {
            "Brightness": number,
            "Sharpness": number
         },
         "Smile": {
            "Confidence": number,
            "Value": boolean
         },
         "Sunglasses": {
            "Confidence": number,
            "Value": boolean
         }
      },
      "Id": "string",
      "Name": "string",
      "Urls": [ "string" ]
   },
   "Timestamp": number
}
```

```
   ],
   "JobStatus": "string",
   "NextToken": "string",
   "StatusMessage": "string",
   "VideoMetadata": {
      "Codec": "string",
      "DurationMillis": number,
      "Format": "string",
      "FrameHeight": number,
      "FrameRate": number,
      "FrameWidth": number
   }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Celebrities (p. 340)

Array of celebrities recognized in the video.

Type: Array of CelebrityRecognition (p. 445) objects

JobStatus (p. 340)

The current status of the celebrity recognition job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

NextToken (p. 340)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of celebrities.

Type: String

Length Constraints: Maximum length of 255.

StatusMessage (p. 340)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 340)

Information about a video that Amazon Rekognition Video analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition Video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetContentModeration

Gets the unsafe content analysis results for a Amazon Rekognition Video analysis started by StartContentModeration (p. 403).

Unsafe content analysis of a video is an asynchronous operation. You start analysis by calling StartContentModeration (p. 403) which returns a job identifier (`JobId`). When analysis finishes, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartContentModeration`. To get the results of the unsafe content analysis, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call `GetContentModeration` and pass the job identifier (`JobId`) from the initial call to `StartContentModeration`.

For more information, see 使用儲存的影片來操作 (p. 49).

`GetContentModeration` returns detected unsafe content labels, and the time they are detected, in an array, `ModerationLabels`, of ContentModerationDetection (p. 449) objects.

By default, the moderated labels are returned sorted by time, in milliseconds from the start of the video. You can also sort them by moderated label by specifying `NAME` for the `SortBy` input parameter.

Since video analysis can return a large number of results, use the `MaxResults` parameter to limit the number of labels returned in a single call to `GetContentModeration`. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetContentModeration` and populate the `NextToken` request parameter with the value of `NextToken` returned from the previous call to `GetContentModeration`.

For more information, see 偵測不安全的內容 (p. 215).

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string",
   "SortBy": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 344)

The identifier for the unsafe content job. Use `JobId` to identify the job in a subsequent call to `GetContentModeration`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 344)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 344)

If the previous response was incomplete (because there is more data to retrieve), Amazon Rekognition returns a pagination token in the response. You can use this pagination token to retrieve the next set of unsafe content labels.

Type: String

Length Constraints: Maximum length of 255.

Required: No

SortBy (p. 344)

Sort to use for elements in the `ModerationLabelDetections` array. Use `TIMESTAMP` to sort array elements by the time labels are detected. Use `NAME` to alphabetically group elements for a label together. Within each label group, the array element are sorted by detection confidence. The default sort is by `TIMESTAMP`.

Type: String

Valid Values: `NAME | TIMESTAMP`

Required: No

Response Syntax

```
{
   "JobStatus": "string",
   "ModerationLabels": [
      {
         "ModerationLabel": {
            "Confidence": number,
            "Name": "string",
            "ParentName": "string"
         },
         "Timestamp": number
      }
   ],
   "ModerationModelVersion": "string",
   "NextToken": "string",
   "StatusMessage": "string",
   "VideoMetadata": {
      "Codec": "string",
      "DurationMillis": number,
      "Format": "string",
      "FrameHeight": number,
      "FrameRate": number,
      "FrameWidth": number
   }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobStatus (p. 345)

The current status of the unsafe content analysis job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

ModerationLabels (p. 345)

The detected unsafe content labels and the time(s) they were detected.

Type: Array of ContentModerationDetection (p. 449) objects

ModerationModelVersion (p. 345)

Version number of the moderation detection model that was used to detect unsafe content.

Type: String

NextToken (p. 345)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of unsafe content labels.

Type: String

Length Constraints: Maximum length of 255.

StatusMessage (p. 345)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 345)

Information about a video that Amazon Rekognition analyzed. `Videometadata` is returned in every page of paginated responses from `GetContentModeration`.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetFaceDetection

Gets face detection results for a Amazon Rekognition Video analysis started by StartFaceDetection (p. 407).

Face detection with Amazon Rekognition Video is an asynchronous operation. You start face detection by calling StartFaceDetection (p. 407) which returns a job identifier (`JobId`). When the face detection operation finishes, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartFaceDetection`. To get the results of the face detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetFaceDetection (p. 348) and pass the job identifier (`JobId`) from the initial call to `StartFaceDetection`.

`GetFaceDetection` returns an array of detected faces (`Faces`) sorted by the time the faces were detected.

Use MaxResults parameter to limit the number of labels returned. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetFaceDetection` and populate the `NextToken` request parameter with the token value returned from the previous call to `GetFaceDetection`.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 348)

Unique identifier for the face detection job. The `JobId` is returned from `StartFaceDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 348)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 348)

If the previous response was incomplete (because there are more faces to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of faces.

Type: String

Length Constraints: Maximum length of 255.

Required: No

Response Syntax

```
{
   "Faces": [
      {
         "Face": {
            "AgeRange": {
               "High": number,
               "Low": number
            },
            "Beard": {
               "Confidence": number,
               "Value": boolean
            },
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Emotions": [
               {
                  "Confidence": number,
                  "Type": "string"
               }
            ],
            "Eyeglasses": {
               "Confidence": number,
               "Value": boolean
            },
            "EyesOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Gender": {
               "Confidence": number,
               "Value": "string"
            },
            "Landmarks": [
               {
                  "Type": "string",
                  "X": number,
                  "Y": number
               }
            ],
            "MouthOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Mustache": {
```

```
                "Confidence": number,
                "Value": boolean
            },
            "Pose": {
                "Pitch": number,
                "Roll": number,
                "Yaw": number
            },
            "Quality": {
                "Brightness": number,
                "Sharpness": number
            },
            "Smile": {
                "Confidence": number,
                "Value": boolean
            },
            "Sunglasses": {
                "Confidence": number,
                "Value": boolean
            }
        },
        "Timestamp": number
    }
  ],
  "JobStatus": "string",
  "NextToken": "string",
  "StatusMessage": "string",
  "VideoMetadata": {
     "Codec": "string",
     "DurationMillis": number,
     "Format": "string",
     "FrameHeight": number,
     "FrameRate": number,
     "FrameWidth": number
  }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Faces (p. 349)

An array of faces detected in the video. Each element contains a detected face's details and the time, in milliseconds from the start of the video, the face was detected.

Type: Array of FaceDetection (p. 462) objects

JobStatus (p. 349)

The current status of the face detection job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

NextToken (p. 349)

If the response is truncated, Amazon Rekognition returns this token that you can use in the subsequent request to retrieve the next set of faces.

Type: String

Length Constraints: Maximum length of 255.

StatusMessage (p. 349)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 349)

Information about a video that Amazon Rekognition Video analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface

- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetFaceSearch

Gets the face search results for Amazon Rekognition Video face search started by StartFaceSearch (p. 411). The search returns faces in a collection that match the faces of persons detected in a video. It also includes the time(s) that faces are matched in the video.

Face search in a video is an asynchronous operation. You start face search by calling to StartFaceSearch (p. 411) which returns a job identifier (`JobId`). When the search operation finishes, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartFaceSearch`. To get the search results, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call `GetFaceSearch` and pass the job identifier (`JobId`) from the initial call to `StartFaceSearch`.

For more information, see 在集合中搜尋臉部 (p. 147).

The search results are retured in an array, `Persons`, of PersonMatch (p. 488) objects. Each`PersonMatch` element contains details about the matching faces in the input collection, person information (facial attributes, bounding boxes, and person identifer) for the matched person, and the time the person was matched in the video.

> Note
>
> `GetFaceSearch` only returns the default facial attributes (`BoundingBox`, `Confidence`, `Landmarks`, `Pose`, and `Quality`). The other facial attributes listed in the `Face` object of the following response syntax are not returned. For more information, see FaceDetail (p. 459).

By default, the `Persons` array is sorted by the time, in milliseconds from the start of the video, persons are matched. You can also sort by persons by specifying `INDEX` for the `SORTBY` input parameter.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string",
   "SortBy": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 353)

The job identifer for the search request. You get the job identifier from an initial call to `StartFaceSearch`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 353)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 353)

If the previous response was incomplete (because there is more search results to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of search results.

Type: String

Length Constraints: Maximum length of 255.

Required: No

SortBy (p. 353)

Sort to use for grouping faces in the response. Use `TIMESTAMP` to group faces by the time that they are recognized. Use `INDEX` to sort by recognized faces.

Type: String

Valid Values: `INDEX | TIMESTAMP`

Required: No

Response Syntax

```
{
   "JobStatus": "string",
   "NextToken": "string",
   "Persons": [
      {
         "FaceMatches": [
            {
               "Face": {
                  "BoundingBox": {
                     "Height": number,
                     "Left": number,
                     "Top": number,
                     "Width": number
                  },
                  "Confidence": number,
                  "ExternalImageId": "string",
                  "FaceId": "string",
                  "ImageId": "string"
               },
               "Similarity": number
            }
         ],
         "Person": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Face": {
               "AgeRange": {
                  "High": number,
                  "Low": number
```

```
            },
            "Beard": {
               "Confidence": number,
               "Value": boolean
            },
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Emotions": [
               {
                  "Confidence": number,
                  "Type": "string"
               }
            ],
            "Eyeglasses": {
               "Confidence": number,
               "Value": boolean
            },
            "EyesOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Gender": {
               "Confidence": number,
               "Value": "string"
            },
            "Landmarks": [
               {
                  "Type": "string",
                  "X": number,
                  "Y": number
               }
            ],
            "MouthOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Mustache": {
               "Confidence": number,
               "Value": boolean
            },
            "Pose": {
               "Pitch": number,
               "Roll": number,
               "Yaw": number
            },
            "Quality": {
               "Brightness": number,
               "Sharpness": number
            },
            "Smile": {
               "Confidence": number,
               "Value": boolean
            },
            "Sunglasses": {
               "Confidence": number,
               "Value": boolean
            }
         },
         "Index": number
      },
      "Timestamp": number
```

```
      }
   ],
   "StatusMessage": "string",
   "VideoMetadata": {
      "Codec": "string",
      "DurationMillis": number,
      "Format": "string",
      "FrameHeight": number,
      "FrameRate": number,
      "FrameWidth": number
   }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobStatus (p. 354)

The current status of the face search job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

NextToken (p. 354)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of search results.

Type: String

Length Constraints: Maximum length of 255.

Persons (p. 354)

An array of persons, PersonMatch (p. 488), in the video whose face(s) match the face(s) in an Amazon Rekognition collection. It also includes time information for when persons are matched in the video. You specify the input collection in an initial call to `StartFaceSearch`. Each `Persons` element includes a time the person was matched, face match details (`FaceMatches`) for matching faces in the collection, and person information (`Person`) for the matched person.

Type: Array of PersonMatch (p. 488) objects

StatusMessage (p. 354)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 354)

Information about a video that Amazon Rekognition analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition Video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetLabelDetection

Gets the label detection results of a Amazon Rekognition Video analysis started by StartLabelDetection (p. 415).

The label detection operation is started by a call to StartLabelDetection (p. 415) which returns a job identifier (`JobId`). When the label detection operation finishes, Amazon Rekognition publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartlabelDetection`. To get the results of the label detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetLabelDetection (p. 358) and pass the job identifier (`JobId`) from the initial call to `StartLabelDetection`.

`GetLabelDetection` returns an array of detected labels (`Labels`) sorted by the time the labels were detected. You can also sort by the label name by specifying `NAME` for the `SortBy` input parameter.

The labels returned include the label name, the percentage confidence in the accuracy of the detected label, and the time the label was detected in the video.

The returned labels also include bounding box information for common objects, a hierarchical taxonomy of detected labels, and the version of the label model used for detection.

Use MaxResults parameter to limit the number of labels returned. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetlabelDetection` and populate the `NextToken` request parameter with the token value returned from the previous call to `GetLabelDetection`.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string",
   "SortBy": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 358)

Job identifier for the label detection operation for which you want results returned. You get the job identifer from an initial call to `StartlabelDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 358)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 358)

If the previous response was incomplete (because there are more labels to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of labels.

Type: String

Length Constraints: Maximum length of 255.

Required: No

SortBy (p. 358)

Sort to use for elements in the `Labels` array. Use `TIMESTAMP` to sort array elements by the time labels are detected. Use `NAME` to alphabetically group elements for a label together. Within each label group, the array element are sorted by detection confidence. The default sort is by `TIMESTAMP`.

Type: String

Valid Values: `NAME | TIMESTAMP`

Required: No

Response Syntax

```
{
   "JobStatus": "string",
   "LabelModelVersion": "string",
   "Labels": [
      {
         "Label": {
            "Confidence": number,
            "Instances": [
               {
                  "BoundingBox": {
                     "Height": number,
                     "Left": number,
                     "Top": number,
                     "Width": number
                  },
                  "Confidence": number
               }
            ],
            "Name": "string",
            "Parents": [
               {
                  "Name": "string"
               }
            ]
         },
         "Timestamp": number
      }
   ],
   "NextToken": "string",
   "StatusMessage": "string",
   "VideoMetadata": {
```

```
      "Codec": "string",
      "DurationMillis": number,
      "Format": "string",
      "FrameHeight": number,
      "FrameRate": number,
      "FrameWidth": number
   }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobStatus (p. 359)

The current status of the label detection job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

LabelModelVersion (p. 359)

Version number of the label detection model that was used to detect labels.

Type: String

Labels (p. 359)

An array of labels detected in the video. Each element contains the detected label and the time, in milliseconds from the start of the video, that the label was detected.

Type: Array of LabelDetection (p. 478) objects

NextToken (p. 359)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of labels.

Type: String

Length Constraints: Maximum length of 255.

StatusMessage (p. 359)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 359)

Information about a video that Amazon Rekognition Video analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetPersonTracking

Gets the path tracking results of a Amazon Rekognition Video analysis started by StartPersonTracking (p. 419).

The person path tracking operation is started by a call to `StartPersonTracking` which returns a job identifier (`JobId`). When the operation finishes, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartPersonTracking`.

To get the results of the person path tracking operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetPersonTracking (p. 362) and pass the job identifier (`JobId`) from the initial call to `StartPersonTracking`.

`GetPersonTracking` returns an array, `Persons`, of tracked persons and the time(s) their paths were tracked in the video.

> Note
>
> `GetPersonTracking` only returns the default facial attributes (`BoundingBox`, `Confidence`, `Landmarks`, `Pose`, and `Quality`). The other facial attributes listed in the `Face` object of the following response syntax are not returned.
> For more information, see FaceDetail (p. 459).

By default, the array is sorted by the time(s) a person's path is tracked in the video. You can sort by tracked persons by specifying `INDEX` for the `SortBy` input parameter.

Use the `MaxResults` parameter to limit the number of items returned. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetPersonTracking` and populate the `NextToken` request parameter with the token value returned from the previous call to `GetPersonTracking`.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string",
   "SortBy": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 362)

The identifier for a job that tracks persons in a video. You get the `JobId` from a call to `StartPersonTracking`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 362)

Maximum number of results to return per paginated call. The largest value you can specify is 1000. If you specify a value greater than 1000, a maximum of 1000 results is returned. The default value is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 362)

If the previous response was incomplete (because there are more persons to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of persons.

Type: String

Length Constraints: Maximum length of 255.

Required: No

SortBy (p. 362)

Sort to use for elements in the `Persons` array. Use `TIMESTAMP` to sort array elements by the time persons are detected. Use `INDEX` to sort by the tracked persons. If you sort by `INDEX`, the array elements for each person are sorted by detection confidence. The default sort is by `TIMESTAMP`.

Type: String

Valid Values: `INDEX | TIMESTAMP`

Required: No

Response Syntax

```
{
   "JobStatus": "string",
   "NextToken": "string",
   "Persons": [
      {
         "Person": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Face": {
               "AgeRange": {
                  "High": number,
                  "Low": number
               },
               "Beard": {
                  "Confidence": number,
                  "Value": boolean
               },
               "BoundingBox": {
                  "Height": number,
                  "Left": number,
                  "Top": number,
```

```
                "Width": number
             },
             "Confidence": number,
             "Emotions": [
                {
                   "Confidence": number,
                   "Type": "string"
                }
             ],
             "Eyeglasses": {
                "Confidence": number,
                "Value": boolean
             },
             "EyesOpen": {
                "Confidence": number,
                "Value": boolean
             },
             "Gender": {
                "Confidence": number,
                "Value": "string"
             },
             "Landmarks": [
                {
                   "Type": "string",
                   "X": number,
                   "Y": number
                }
             ],
             "MouthOpen": {
                "Confidence": number,
                "Value": boolean
             },
             "Mustache": {
                "Confidence": number,
                "Value": boolean
             },
             "Pose": {
                "Pitch": number,
                "Roll": number,
                "Yaw": number
             },
             "Quality": {
                "Brightness": number,
                "Sharpness": number
             },
             "Smile": {
                "Confidence": number,
                "Value": boolean
             },
             "Sunglasses": {
                "Confidence": number,
                "Value": boolean
             }
          },
          "Index": number
       },
       "Timestamp": number
    }
 ],
 "StatusMessage": "string",
 "VideoMetadata": {
    "Codec": "string",
    "DurationMillis": number,
    "Format": "string",
    "FrameHeight": number,
    "FrameRate": number,
```

```
        "FrameWidth": number
    }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobStatus (p. 363)

The current status of the person tracking job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

NextToken (p. 363)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of persons.

Type: String

Length Constraints: Maximum length of 255.

Persons (p. 363)

An array of the persons detected in the video and the time(s) their path was tracked throughout the video. An array element will exist for each time a person's path is tracked.

Type: Array of PersonDetection (p. 487) objects

StatusMessage (p. 363)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

VideoMetadata (p. 363)

Information about a video that Amazon Rekognition Video analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition Video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

GetTextDetection

Gets the text detection results of a Amazon Rekognition Video analysis started by StartTextDetection (p. 427).

Text detection with Amazon Rekognition Video is an asynchronous operation. You start text detection by calling StartTextDetection (p. 427) which returns a job identifier (`JobId`) When the text detection operation finishes, Amazon Rekognition publishes a completion status to the Amazon Simple Notification Service topic registered in the initial call to `StartTextDetection`. To get the results of the text detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. if so, call `GetTextDetection` and pass the job identifier (`JobId`) from the initial call of `StartLabelDetection`.

`GetTextDetection` returns an array of detected text (`TextDetections`) sorted by the time the text was detected, up to 50 words per frame of video.

Each element of the array includes the detected text, the precentage confidence in the acuracy of the detected text, the time the text was detected, bounding box information for where the text was located, and unique identifiers for words and their lines.

Use MaxResults parameter to limit the number of text detections returned. If there are more results than specified in `MaxResults`, the value of `NextToken` in the operation response contains a pagination token for getting the next set of results. To get the next page of results, call `GetTextDetection` and populate the `NextToken` request parameter with the token value returned from the previous call to `GetTextDetection`.

Request Syntax

```
{
   "JobId": "string",
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

JobId (p. 367)

Job identifier for the label detection operation for which you want results returned. You get the job identifer from an initial call to `StartTextDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: Yes

MaxResults (p. 367)

Maximum number of results to return per paginated call. The largest value you can specify is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 367)

If the previous response was incomplete (because there are more labels to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of text.

Type: String

Length Constraints: Maximum length of 255.

Required: No

Response Syntax

```
{
   "JobStatus": "string",
   "NextToken": "string",
   "StatusMessage": "string",
   "TextDetections": [
      {
         "TextDetection": {
            "Confidence": number,
            "DetectedText": "string",
            "Geometry": {
               "BoundingBox": {
                  "Height": number,
                  "Left": number,
                  "Top": number,
                  "Width": number
               },
               "Polygon": [
                  {
                     "X": number,
                     "Y": number
                  }
               ]
            },
            "Id": number,
            "ParentId": number,
            "Type": "string"
         },
         "Timestamp": number
      }
   ],
   "TextModelVersion": "string",
   "VideoMetadata": {
      "Codec": "string",
      "DurationMillis": number,
      "Format": "string",
      "FrameHeight": number,
      "FrameRate": number,
      "FrameWidth": number
   }
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobStatus (p. 368)

Current status of the text detection job.

Type: String

Valid Values: `IN_PROGRESS | SUCCEEDED | FAILED`

NextToken (p. 368)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of text.

Type: String

Length Constraints: Maximum length of 255.

StatusMessage (p. 368)

If the job fails, `StatusMessage` provides a descriptive error message.

Type: String

TextDetections (p. 368)

An array of text detected in the video. Each element contains the detected text, the time in milliseconds from the start of the video that the text was detected, and where it was detected on the screen.

Type: Array of TextDetectionResult (p. 508) objects

TextModelVersion (p. 368)

Version number of the text detection model that was used to detect text.

Type: String

VideoMetadata (p. 368)

Information about a video that Amazon Rekognition analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition video operation.

Type: VideoMetadata (p. 513) object

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

IndexFaces

Detects faces in the input image and adds them to the specified collection.

Amazon Rekognition doesn't save the actual faces that are detected. Instead, the underlying detection algorithm first detects the faces in the input image. For each face, the algorithm extracts facial features into a feature vector, and stores it in the backend database. Amazon Rekognition uses feature vectors when it performs face match and search operations using the SearchFaces (p. 392) and SearchFacesByImage (p. 395) operations.

For more information, see 新增臉部到集合 (p. 162).

To get the number of faces in a collection, call DescribeCollection (p. 302).

If you're using version 1.0 of the face detection model, `IndexFaces` indexes the 15 largest faces in the input image. Later versions of the face detection model index the 100 largest faces in the input image.

If you're using version 4 or later of the face model, image orientation information is not returned in the `OrientationCorrection` field.

To determine which version of the model you're using, call DescribeCollection (p. 302) and supply the collection ID. You can also get the model version from the value of `FaceModelVersion` in the response from `IndexFaces`

For more information, see 模型版本控制 (p. 9).

If you provide the optional `ExternalImageID` for the input image you provided, Amazon Rekognition associates this ID with all faces that it detects. When you call the ListFaces (p. 382) operation, the response returns the external ID. You can use this external image ID to create a client-side index to associate the faces with each image. You can then use the index to find all faces in an image.

You can specify the maximum number of faces to index with the `MaxFaces` input parameter. This is useful when you want to index the largest faces in an image and don't want to index smaller faces, such as those belonging to people standing in the background.

The `QualityFilter` input parameter allows you to filter out detected faces that don’t meet a required quality bar. The quality bar is based on a variety of common use cases. By default, `IndexFaces` chooses the quality bar that's used to filter faces. You can also explicitly choose the quality bar. Use `QualityFilter`, to set the quality bar by specifying `LOW`, `MEDIUM`, or `HIGH`. If you do not want to filter detected faces, specify `NONE`.

> Note
>
> To use quality filtering, you need a collection associated with version 3 of the face model or higher. To get the version of the face model associated with a collection, call DescribeCollection (p. 302).

Information about faces detected in an image, but not indexed, is returned in an array of UnindexedFace (p. 511) objects, `UnindexedFaces`. Faces aren't indexed for reasons such as:

- The number of faces detected exceeds the value of the `MaxFaces` request parameter.
- The face is too small compared to the image dimensions.
- The face is too blurry.
- The image is too dark.
- The face has an extreme pose.
- The face doesn’t have enough detail to be suitable for face search.

In response, the `IndexFaces` operation returns an array of metadata for all detected faces, `FaceRecords`. This includes:

- The bounding box, `BoundingBox`, of the detected face.
- A confidence value, `Confidence`, which indicates the confidence that the bounding box contains a face.
- A face ID, `FaceId`, assigned by the service for each face that's detected and stored.
- An image ID, `ImageId`, assigned by the service for the input image.

If you request all facial attributes (by using the `detectionAttributes` parameter), Amazon Rekognition returns detailed facial attributes, such as facial landmarks (for example, location of eye and mouth) and other facial attributes. If you provide the same image, specify the same collection, and use the same external ID in the `IndexFaces` operation, Amazon Rekognition doesn't save duplicate face metadata.

The input image is passed either as base64-encoded image bytes, or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes isn't supported. The image must be formatted as a PNG or JPEG file.

This operation requires permissions to perform the `rekognition:IndexFaces` action.

Request Syntax

```
{
   "CollectionId": "string",
   "DetectionAttributes": [ "string" ],
   "ExternalImageId": "string",
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "MaxFaces": number,
   "QualityFilter": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 372)

The ID of an existing collection to which you want to add the faces that are detected in the input images.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

DetectionAttributes (p. 372)

An array of facial attributes that you want to be returned. This can be the default list of attributes or all attributes. If you don't specify a value for `Attributes` or if you specify `["DEFAULT"]`, the API returns the following subset of facial attributes: `BoundingBox`, `Confidence`, `Pose`, `Quality`, and `Landmarks`. If you provide `["ALL"]`, all facial attributes are returned, but the operation takes longer to complete.

If you provide both, `["ALL", "DEFAULT"]`, the service uses a logical AND operator to determine which attributes to return (in this case, all attributes).

Type: Array of strings

Valid Values: `DEFAULT | ALL`

Required: No

ExternalImageId (p. 372)

The ID you want to assign to all the faces detected in the image.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

Image (p. 372)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes isn't supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

MaxFaces (p. 372)

The maximum number of faces to index. The value of `MaxFaces` must be greater than or equal to 1. `IndexFaces` returns no more than 100 detected faces in an image, even if you specify a larger value for `MaxFaces`.

If `IndexFaces` detects more faces than the value of `MaxFaces`, the faces with the lowest quality are filtered out first. If there are still more faces than the value of `MaxFaces`, the faces with the smallest bounding boxes are filtered out (up to the number that's needed to satisfy the value of `MaxFaces`). Information about the unindexed faces is available in the `UnindexedFaces` array.

The faces that are returned by `IndexFaces` are sorted by the largest face bounding box size to the smallest size, in descending order.

`MaxFaces` can be used with a collection associated with any version of the face model.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

QualityFilter (p. 372)

A filter that specifies a quality bar for how much filtering is done to identify faces. Filtered faces aren't indexed. If you specify `AUTO`, Amazon Rekognition chooses the quality bar. If you specify `LOW`, `MEDIUM`, or `HIGH`, filtering removes all faces that don’t meet the chosen quality bar. The default value is `AUTO`. The quality bar is based on a variety of common use cases. Low-quality detections can occur

for a number of reasons. Some examples are an object that's misidentified as a face, a face that's too blurry, or a face with a pose that's too extreme to use. If you specify `NONE`, no filtering is performed.

To use quality filtering, the collection you are using must be associated with version 3 of the face model or higher.

Type: String

Valid Values: `NONE | AUTO | LOW | MEDIUM | HIGH`

Required: No

Response Syntax

```
{
   "FaceModelVersion": "string",
   "FaceRecords": [
      {
         "Face": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "ExternalImageId": "string",
            "FaceId": "string",
            "ImageId": "string"
         },
         "FaceDetail": {
            "AgeRange": {
               "High": number,
               "Low": number
            },
            "Beard": {
               "Confidence": number,
               "Value": boolean
            },
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Emotions": [
               {
                  "Confidence": number,
                  "Type": "string"
               }
            ],
            "Eyeglasses": {
               "Confidence": number,
               "Value": boolean
            },
            "EyesOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Gender": {
               "Confidence": number,
               "Value": "string"
```

```
            },
            "Landmarks": [
               {
                  "Type": "string",
                  "X": number,
                  "Y": number
               }
            ],
            "MouthOpen": {
               "Confidence": number,
               "Value": boolean
            },
            "Mustache": {
               "Confidence": number,
               "Value": boolean
            },
            "Pose": {
               "Pitch": number,
               "Roll": number,
               "Yaw": number
            },
            "Quality": {
               "Brightness": number,
               "Sharpness": number
            },
            "Smile": {
               "Confidence": number,
               "Value": boolean
            },
            "Sunglasses": {
               "Confidence": number,
               "Value": boolean
            }
         }
      }
   ],
   "OrientationCorrection": "string",
   "UnindexedFaces": [
      {
         "FaceDetail": {
            "AgeRange": {
               "High": number,
               "Low": number
            },
            "Beard": {
               "Confidence": number,
               "Value": boolean
            },
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Emotions": [
               {
                  "Confidence": number,
                  "Type": "string"
               }
            ],
            "Eyeglasses": {
               "Confidence": number,
               "Value": boolean
            },
            "EyesOpen": {
```

```
            "Confidence": number,
            "Value": boolean
         },
         "Gender": {
            "Confidence": number,
            "Value": "string"
         },
         "Landmarks": [
            {
               "Type": "string",
               "X": number,
               "Y": number
            }
         ],
         "MouthOpen": {
            "Confidence": number,
            "Value": boolean
         },
         "Mustache": {
            "Confidence": number,
            "Value": boolean
         },
         "Pose": {
            "Pitch": number,
            "Roll": number,
            "Yaw": number
         },
         "Quality": {
            "Brightness": number,
            "Sharpness": number
         },
         "Smile": {
            "Confidence": number,
            "Value": boolean
         },
         "Sunglasses": {
            "Confidence": number,
            "Value": boolean
         }
      },
      "Reasons": [ "string" ]
   }
]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceModelVersion (p. 374)

The version number of the face detection model that's associated with the input collection (`CollectionId`).

Type: String

FaceRecords (p. 374)

An array of faces detected and added to the collection. For more information, see 管理集合中的臉孔 (p. 147).

Type: Array of FaceRecord (p. 464) objects

OrientationCorrection (p. 374)

If your collection is associated with a face detection model that's later than version 3.0, the value of `OrientationCorrection` is always null and no orientation information is returned.

If your collection is associated with a face detection model that's version 3.0 or earlier, the following applies:

- If the input image is in .jpeg format, it might contain exchangeable image file format (Exif) metadata that includes the image's orientation. Amazon Rekognition uses this orientation information to perform image correction - the bounding box coordinates are translated to represent object locations after the orientation information in the Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata. The value of `OrientationCorrection` is null.
- If the image doesn't contain orientation information in its Exif metadata, Amazon Rekognition returns an estimated orientation (ROTATE_0, ROTATE_90, ROTATE_180, ROTATE_270). Amazon Rekognition doesn't perform image correction for images. The bounding box coordinates aren't translated and represent the object locations before the image is rotated.

Bounding box information is returned in the `FaceRecords` array. You can get the version of the face detection model by calling DescribeCollection (p. 302).

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

UnindexedFaces (p. 374)

An array of faces that were detected in the image but weren't indexed. They weren't indexed because the quality filter identified them as low quality, or the `MaxFaces` request parameter filtered them out. To use the quality filter, you specify the `QualityFilter` request parameter.

Type: Array of UnindexedFace (p. 511) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

ListCollections

Returns list of collection IDs in your account. If the result is truncated, the response also provides a `NextToken` that you can use in the subsequent request to fetch the next set of collection IDs.

For an example, see 列出集合 (p. 153).

This operation requires permissions to perform the `rekognition:ListCollections` action.

Request Syntax

```
{
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

MaxResults (p. 379)

Maximum number of collection IDs to return.

Type: Integer

Valid Range: Minimum value of 0. Maximum value of 4096.

Required: No

NextToken (p. 379)

Pagination token from the previous response.

Type: String

Length Constraints: Maximum length of 255.

Required: No

Response Syntax

```
{
   "CollectionIds": [ "string" ],
   "FaceModelVersions": [ "string" ],
   "NextToken": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CollectionIds (p. 379)

An array of collection IDs.

Type: Array of strings

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

FaceModelVersions (p. 379)

Version numbers of the face detection models associated with the collections in the array `CollectionIds`. For example, the value of `FaceModelVersions[2]` is the version number for the face detection model used by the collection in `CollectionId[2]`.

Type: Array of strings

NextToken (p. 379)

If the result is truncated, the response provides a `NextToken` that you can use in the subsequent request to fetch the next set of collection IDs.

Type: String

Length Constraints: Maximum length of 255.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

ListFaces

Returns metadata for faces in the specified collection. This metadata includes information such as the bounding box coordinates, the confidence (that the bounding box contains a face), and face ID. For an example, see 在集合中列出臉部 (p. 172).

This operation requires permissions to perform the `rekognition:ListFaces` action.

Request Syntax

```
{
   "CollectionId": "string",
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 382)

ID of the collection from which to list the faces.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

MaxResults (p. 382)

Maximum number of faces to return.

Type: Integer

Valid Range: Minimum value of 0. Maximum value of 4096.

Required: No

NextToken (p. 382)

If the previous response was incomplete (because there is more data to retrieve), Amazon Rekognition returns a pagination token in the response. You can use this pagination token to retrieve the next set of faces.

Type: String

Length Constraints: Maximum length of 255.

Required: No

Response Syntax

```
{
   "FaceModelVersion": "string",
   "Faces": [
```

```
      {
         "BoundingBox": {
            "Height": number,
            "Left": number,
            "Top": number,
            "Width": number
         },
         "Confidence": number,
         "ExternalImageId": "string",
         "FaceId": "string",
         "ImageId": "string"
      }
   ],
   "NextToken": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceModelVersion (p. 382)

Version number of the face detection model associated with the input collection (`CollectionId`).

Type: String

Faces (p. 382)

An array of `Face` objects.

Type: Array of Face (p. 457) objects

NextToken (p. 382)

If the response is truncated, Amazon Rekognition returns this token that you can use in the subsequent request to retrieve the next set of faces.

Type: String

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

ListStreamProcessors

Gets a list of stream processors that you have created with CreateStreamProcessor (p. 289).

Request Syntax

```
{
   "MaxResults": number,
   "NextToken": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

MaxResults (p. 385)

Maximum number of stream processors you want Amazon Rekognition Video to return in the response. The default is 1000.

Type: Integer

Valid Range: Minimum value of 1.

Required: No

NextToken (p. 385)

If the previous response was incomplete (because there are more stream processors to retrieve), Amazon Rekognition Video returns a pagination token in the response. You can use this pagination token to retrieve the next set of stream processors.

Type: String

Length Constraints: Maximum length of 255.

Required: No

Response Syntax

```
{
   "NextToken": "string",
   "StreamProcessors": [
      {
         "Name": "string",
         "Status": "string"
      }
   ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

NextToken (p. 385)

If the response is truncated, Amazon Rekognition Video returns this token that you can use in the subsequent request to retrieve the next set of stream processors.

Type: String

Length Constraints: Maximum length of 255.

StreamProcessors (p. 385)

List of stream processors that you have created.

Type: Array of StreamProcessor (p. 498) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidPaginationTokenException

Pagination token in the request is not valid.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go

- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

RecognizeCelebrities

Returns an array of celebrities recognized in the input image. For more information, see 辨識名人 (p. 201).

`RecognizeCelebrities` returns the 100 largest faces in the image. It lists recognized celebrities in the `CelebrityFaces` array and unrecognized faces in the `UnrecognizedFaces` array. `RecognizeCelebrities` doesn't return celebrities whose faces aren't among the largest 100 faces in the image.

For each celebrity recognized, `RecognizeCelebrities` returns a `Celebrity` object. The `Celebrity` object contains the celebrity name, ID, URL links to additional information, match confidence, and a `ComparedFace` object that you can use to locate the celebrity's face on the image.

Amazon Rekognition doesn't retain information about which images a celebrity has been recognized in. Your application must store this information and use the `Celebrity` ID property as a unique identifier for the celebrity. If you don't store the celebrity name or additional information URLs returned by `RecognizeCelebrities`, you will need the ID to identify the celebrity in a call to the GetCelebrityInfo (p. 337) operation.

You pass the input image either as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

For an example, see 辨識影像中的名人 (p. 201).

This operation requires permissions to perform the `rekognition:RecognizeCelebrities` operation.

Request Syntax

```
{
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

Image (p. 388)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

Response Syntax

```
{
```

```
   "CelebrityFaces": [
      {
         "Face": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "Landmarks": [
               {
                  "Type": "string",
                  "X": number,
                  "Y": number
               }
            ],
            "Pose": {
               "Pitch": number,
               "Roll": number,
               "Yaw": number
            },
            "Quality": {
               "Brightness": number,
               "Sharpness": number
            }
         },
         "Id": "string",
         "MatchConfidence": number,
         "Name": "string",
         "Urls": [ "string" ]
      }
   ],
   "OrientationCorrection": "string",
   "UnrecognizedFaces": [
      {
         "BoundingBox": {
            "Height": number,
            "Left": number,
            "Top": number,
            "Width": number
         },
         "Confidence": number,
         "Landmarks": [
            {
               "Type": "string",
               "X": number,
               "Y": number
            }
         ],
         "Pose": {
            "Pitch": number,
            "Roll": number,
            "Yaw": number
         },
         "Quality": {
            "Brightness": number,
            "Sharpness": number
         }
      }
   ]
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

CelebrityFaces (p. 388)

Details about each celebrity found in the image. Amazon Rekognition can detect a maximum of 15 celebrities in an image.

Type: Array of Celebrity (p. 442) objects

OrientationCorrection (p. 388)

The orientation of the input image (counterclockwise direction). If your application displays the image, you can use this value to correct the orientation. The bounding box coordinates returned in `CelebrityFaces` and `UnrecognizedFaces` represent face locations before the image orientation is corrected.

Note

If the input image is in .jpeg format, it might contain exchangeable image (Exif) metadata that includes the image's orientation. If so, and the Exif metadata for the input image populates the orientation field, the value of `OrientationCorrection` is null. The `CelebrityFaces` and `UnrecognizedFaces` bounding box coordinates represent face locations after Exif metadata is used to correct the image orientation. Images in .png format don't contain Exif metadata.

Type: String

Valid Values: `ROTATE_0 | ROTATE_90 | ROTATE_180 | ROTATE_270`

UnrecognizedFaces (p. 388)

Details about each unrecognized face in the image.

Type: Array of ComparedFace (p. 446) objects

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

SearchFaces

For a given input face ID, searches for matching faces in the collection the face belongs to. You get a face ID when you add a face to the collection using the IndexFaces (p. 371) operation. The operation compares the features of the input face with faces in the specified collection.

> Note
>
> You can also search faces without indexing faces by using the `SearchFacesByImage` operation.

The operation response returns an array of faces that match, ordered by similarity score with the highest similarity first. More specifically, it is an array of metadata for each face match that is found. Along with the metadata, the response also includes a `confidence` value for each face match, indicating the confidence that the specific face matches the input face.

For an example, see 使用臉部 ID 來搜尋臉孔 (p. 179).

This operation requires permissions to perform the `rekognition:SearchFaces` action.

Request Syntax

```
{
   "CollectionId": "string",
   "FaceId": "string",
   "FaceMatchThreshold": number,
   "MaxFaces": number
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 392)

ID of the collection the face belongs to.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

FaceId (p. 392)

ID of a face to find matches for in the collection.

Type: String

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Required: Yes

FaceMatchThreshold (p. 392)

Optional value specifying the minimum confidence in the face match to return. For example, don't return any matches where confidence in matches is less than 70%. The default value is 80%.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

MaxFaces (p. 392)

Maximum number of faces to return. The operation returns the maximum number of faces with the highest confidence in the match.

Type: Integer

Valid Range: Minimum value of 1. Maximum value of 4096.

Required: No

Response Syntax

```
{
   "FaceMatches": [
      {
         "Face": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "ExternalImageId": "string",
            "FaceId": "string",
            "ImageId": "string"
         },
         "Similarity": number
      }
   ],
   "FaceModelVersion": "string",
   "SearchedFaceId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceMatches (p. 393)

An array of faces that matched the input face, along with the confidence in the match.

Type: Array of FaceMatch (p. 463) objects

FaceModelVersion (p. 393)

Version number of the face detection model associated with the input collection (`CollectionId`).

Type: String

SearchedFaceId (p. 393)

ID of the face that was searched for matches in a collection.

Type: String

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

SearchFacesByImage

For a given input image, first detects the largest face in the image, and then searches the specified collection for matching faces. The operation compares the features of the input face with faces in the specified collection.

> Note
>
> To search for all faces in an input image, you might first call the IndexFaces (p. 371) operation, and then use the face IDs returned in subsequent calls to the SearchFaces (p. 392) operation. You can also call the `DetectFaces` operation and use the bounding boxes in the response to make face crops, which then you can pass in to the `SearchFacesByImage` operation.

You pass the input image either as base64-encoded image bytes or as a reference to an image in an Amazon S3 bucket. If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes is not supported. The image must be either a PNG or JPEG formatted file.

The response returns an array of faces that match, ordered by similarity score with the highest similarity first. More specifically, it is an array of metadata for each face match found. Along with the metadata, the response also includes a `similarity` indicating how similar the face is to the input face. In the response, the operation also returns the bounding box (and a confidence level that the bounding box contains a face) of the face that Amazon Rekognition used for the input image.

For an example, see 使用影像來搜尋臉孔 (p. 183).

The `QualityFilter` input parameter allows you to filter out detected faces that don't meet a required quality bar. The quality bar is based on a variety of common use cases. Use `QualityFilter` to set the quality bar for filtering by specifying `LOW`, `MEDIUM`, or `HIGH`. If you do not want to filter detected faces, specify `NONE`. The default value is `NONE`.

> Note
>
> To use quality filtering, you need a collection associated with version 3 of the face model or higher. To get the version of the face model associated with a collection, call DescribeCollection (p. 302).

This operation requires permissions to perform the `rekognition:SearchFacesByImage` action.

Request Syntax

```
{
   "CollectionId": "string",
   "FaceMatchThreshold": number,
   "Image": {
      "Bytes": blob,
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   },
   "MaxFaces": number,
   "QualityFilter": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

CollectionId (p. 395)

ID of the collection to search.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

FaceMatchThreshold (p. 395)

(Optional) Specifies the minimum confidence in the face match to return. For example, don't return any matches where confidence in matches is less than 70%. The default value is 80%.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Image (p. 395)

The input image as base64-encoded bytes or an S3 object. If you use the AWS CLI to call Amazon Rekognition operations, passing base64-encoded image bytes is not supported.

If you are using an AWS SDK to call Amazon Rekognition, you might not need to base64-encode image bytes passed using the `Bytes` field. For more information, see 影像 (p. 23).

Type: Image (p. 472) object

Required: Yes

MaxFaces (p. 395)

Maximum number of faces to return. The operation returns the maximum number of faces with the highest confidence in the match.

Type: Integer

Valid Range: Minimum value of 1. Maximum value of 4096.

Required: No

QualityFilter (p. 395)

A filter that specifies a quality bar for how much filtering is done to identify faces. Filtered faces aren't searched for in the collection. If you specify `AUTO`, Amazon Rekognition chooses the quality bar. If you specify `LOW`, `MEDIUM`, or `HIGH`, filtering removes all faces that don’t meet the chosen quality bar. The quality bar is based on a variety of common use cases. Low-quality detections can occur for a number of reasons. Some examples are an object that's misidentified as a face, a face that's too blurry, or a face with a pose that's too extreme to use. If you specify `NONE`, no filtering is performed. The default value is `NONE`.

To use quality filtering, the collection you are using must be associated with version 3 of the face model or higher.

Type: String

Valid Values: `NONE | AUTO | LOW | MEDIUM | HIGH`

Required: No

Response Syntax

```
{
   "FaceMatches": [
      {
         "Face": {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            },
            "Confidence": number,
            "ExternalImageId": "string",
            "FaceId": "string",
            "ImageId": "string"
         },
         "Similarity": number
      }
   ],
   "FaceModelVersion": "string",
   "SearchedFaceBoundingBox": {
      "Height": number,
      "Left": number,
      "Top": number,
      "Width": number
   },
   "SearchedFaceConfidence": number
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

FaceMatches (p. 397)

An array of faces that match the input face, along with the confidence in the match.

Type: Array of FaceMatch (p. 463) objects

FaceModelVersion (p. 397)

Version number of the face detection model associated with the input collection (`CollectionId`).

Type: String

SearchedFaceBoundingBox (p. 397)

The bounding box around the face in the input image that Amazon Rekognition used for the search.

Type: BoundingBox (p. 440) object

SearchedFaceConfidence (p. 397)

The level of confidence that the `searchedFaceBoundingBox`, contains a face.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

ImageTooLargeException

The input image size exceeds the allowed limit. For more information, see Amazon Rekognition 中的限制 (p. 515).

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidImageFormatException

The provided image format is not supported.

HTTP Status Code: 400

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartCelebrityRecognition

Starts asynchronous recognition of celebrities in a stored video.

Amazon Rekognition Video can detect celebrities in a video must be stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartCelebrityRecognition` returns a job identifier (`JobId`) which you use to get the results of the analysis. When celebrity recognition analysis is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`. To get the results of the celebrity recognition analysis, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetCelebrityRecognition (p. 339) and pass the job identifier (`JobId`) from the initial call to `StartCelebrityRecognition`.

For more information, see 辨識名人 (p. 201).

Request Syntax

```
{
   "ClientRequestToken": "string",
   "JobTag": "string",
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 400)

Idempotent token used to identify the start request. If you use the same token with multiple `StartCelebrityRecognition` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

JobTag (p. 400)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

NotificationChannel (p. 400)

The Amazon SNS topic ARN that you want Amazon Rekognition Video to publish the completion status of the celebrity recognition analysis to.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 400)

The video in which you want to recognize celebrities. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 401)

The identifier for the celebrity recognition analysis job. Use `JobId` to identify the job in a subsequent call to `GetCelebrityRecognition`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartContentModeration

Starts asynchronous detection of unsafe content in a stored video.

Amazon Rekognition Video can moderate content in a video stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartContentModeration` returns a job identifier (`JobId`) which you use to get the results of the analysis. When unsafe content analysis is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`.

To get the results of the unsafe content analysis, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetContentModeration (p. 344) and pass the job identifier (`JobId`) from the initial call to `StartContentModeration`.

For more information, see 偵測不安全的內容 (p. 215).

Request Syntax

```
{
   "ClientRequestToken": "string",
   "JobTag": "string",
   "MinConfidence": number,
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 403)

Idempotent token used to identify the start request. If you use the same token with multiple `StartContentModeration` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

JobTag (p. 403)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

MinConfidence (p. 403)

Specifies the minimum confidence that Amazon Rekognition must have in order to return a moderated content label. Confidence represents how certain Amazon Rekognition is that the moderated content is correctly identified. 0 is the lowest confidence. 100 is the highest confidence. Amazon Rekognition doesn't return any moderated content labels with a confidence level lower than this specified value. If you don't specify `MinConfidence`, `GetContentModeration` returns labels with confidence values greater than or equal to 50 percent.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

NotificationChannel (p. 403)

The Amazon SNS topic ARN that you want Amazon Rekognition Video to publish the completion status of the unsafe content analysis to.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 403)

The video in which you want to detect unsafe content. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 404)

The identifier for the unsafe content analysis job. Use `JobId` to identify the job in a subsequent call to `GetContentModeration`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartFaceDetection

Starts asynchronous detection of faces in a stored video.

Amazon Rekognition Video can detect faces in a video stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartFaceDetection` returns a job identifier (`JobId`) that you use to get the results of the operation. When face detection is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`. To get the results of the face detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetFaceDetection (p. 348) and pass the job identifier (`JobId`) from the initial call to `StartFaceDetection`.

For more information, see 偵測已存放影片中的臉部 (p. 141).

Request Syntax

```
{
   "ClientRequestToken": "string",
   "FaceAttributes": "string",
   "JobTag": "string",
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 407)

Idempotent token used to identify the start request. If you use the same token with multiple `StartFaceDetection` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

FaceAttributes (p. 407)

The face attributes you want returned.

`DEFAULT` - The following subset of facial attributes are returned: BoundingBox, Confidence, Pose, Quality and Landmarks.

`ALL` - All facial attributes are returned.

Type: String

Valid Values: `DEFAULT | ALL`

Required: No

JobTag (p. 407)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

NotificationChannel (p. 407)

The ARN of the Amazon SNS topic to which you want Amazon Rekognition Video to publish the completion status of the face detection operation.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 407)

The video in which you want to detect faces. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 408)

The identifier for the face detection job. Use `JobId` to identify the job in a subsequent call to `GetFaceDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartFaceSearch

Starts the asynchronous search for faces in a collection that match the faces of persons detected in a stored video.

The video must be stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartFaceSearch` returns a job identifier (`JobId`) which you use to get the search results once the search has completed. When searching is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`. To get the search results, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetFaceSearch (p. 353) and pass the job identifier (`JobId`) from the initial call to `StartFaceSearch`. For more information, see 在儲存的視訊中搜尋臉部 (p. 187).

Request Syntax

```
{
   "ClientRequestToken": "string",
   "CollectionId": "string",
   "FaceMatchThreshold": number,
   "JobTag": "string",
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 411)

Idempotent token used to identify the start request. If you use the same token with multiple `StartFaceSearch` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

CollectionId (p. 411)

ID of the collection that contains the faces you want to search for.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

FaceMatchThreshold (p. 411)

The minimum confidence in the person match to return. For example, don't return any matches where confidence in matches is less than 70%. The default value is 80%.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

JobTag (p. 411)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

NotificationChannel (p. 411)

The ARN of the Amazon SNS topic to which you want Amazon Rekognition Video to publish the completion status of the search.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 411)

The video you want to search. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 412)

The identifier for the search job. Use `JobId` to identify the job in a subsequent call to `GetFaceSearch`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartLabelDetection

Starts asynchronous detection of labels in a stored video.

Amazon Rekognition Video can detect labels in a video. Labels are instances of real-world entities. This includes objects like flower, tree, and table; events like wedding, graduation, and birthday party; concepts like landscape, evening, and nature; and activities like a person getting out of a car or a person skiing.

The video must be stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartLabelDetection` returns a job identifier (`JobId`) which you use to get the results of the operation. When label detection is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`.

To get the results of the label detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetLabelDetection (p. 358) and pass the job identifier (`JobId`) from the initial call to `StartLabelDetection`.

Request Syntax

```
{
   "ClientRequestToken": "string",
   "JobTag": "string",
   "MinConfidence": number,
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 415)

Idempotent token used to identify the start request. If you use the same token with multiple `StartLabelDetection` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

JobTag (p. 415)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

MinConfidence (p. 415)

Specifies the minimum confidence that Amazon Rekognition Video must have in order to return a detected label. Confidence represents how certain Amazon Rekognition is that a label is correctly identified.0 is the lowest confidence. 100 is the highest confidence. Amazon Rekognition Video doesn't return any labels with a confidence level lower than this specified value.

If you don't specify `MinConfidence`, the operation returns labels with confidence values greater than or equal to 50 percent.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

NotificationChannel (p. 415)

The Amazon SNS topic ARN you want Amazon Rekognition Video to publish the completion status of the label detection operation to.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 415)

The video in which you want to detect labels. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 416)

The identifier for the label detection job. Use `JobId` to identify the job in a subsequent call to `GetLabelDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartPersonTracking

Starts the asynchronous tracking of a person's path in a stored video.

Amazon Rekognition Video can track the path of people in a video stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartPersonTracking` returns a job identifier (`JobId`) which you use to get the results of the operation. When label detection is finished, Amazon Rekognition publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`.

To get the results of the person detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. If so, call GetPersonTracking (p. 362) and pass the job identifier (`JobId`) from the initial call to `StartPersonTracking`.

Request Syntax

```
{
   "ClientRequestToken": "string",
   "JobTag": "string",
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 419)

Idempotent token used to identify the start request. If you use the same token with multiple `StartPersonTracking` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidently started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

JobTag (p. 419)

An identifier you specify that's returned in the completion notification that's published to your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

NotificationChannel (p. 419)

The Amazon SNS topic ARN you want Amazon Rekognition Video to publish the completion status of the people detection operation to.

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 419)

The video in which you want to detect people. The video must be stored in an Amazon S3 bucket.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 420)

The identifier for the person detection job. Use `JobId` to identify the job in a subsequent call to `GetPersonTracking`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartProjectVersion

Starts the running of the version of a model. Starting a model takes a while to complete. To check the current state of the model, use DescribeProjectVersions (p. 308).

Once the model is running, you can detect custom labels in new images by calling DetectCustomLabels (p. 316).

> Note
>
> You are charged for the amount of time that the model is running. To stop a running model, call StopProjectVersion (p. 431).

This operation requires permissions to perform the `rekognition:StartProjectVersion` action.

Request Syntax

```
{
   "MinInferenceUnits": number,
   "ProjectVersionArn": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

MinInferenceUnits (p. 422)

The minimum number of inference units to use. A single inference unit represents 1 hour of processing and can support up to 5 Transaction Pers Second (TPS). Use a higher number to increase the TPS throughput of your model. You are charged for the number of inference units that you use.

Type: Integer

Valid Range: Minimum value of 1.

Required: Yes

ProjectVersionArn (p. 422)

The Amazon Resource Name(ARN) of the model version that you want to start.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

Response Syntax

```
{
   "Status": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Status (p. 422)

The current running status of the model.

Type: String

Valid Values: `TRAINING_IN_PROGRESS | TRAINING_COMPLETED | TRAINING_FAILED | STARTING | RUNNING | FAILED | STOPPING | STOPPED | DELETING`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartStreamProcessor

Starts processing a stream processor. You create a stream processor by calling CreateStreamProcessor (p. 289). To tell `StartStreamProcessor` which stream processor to start, use the value of the `Name` field specified in the call to `CreateStreamProcessor`.

Request Syntax

```
{
   "Name": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Name (p. 425)

The name of the stream processor to start processing.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Elements

If the action is successful, the service sends back an HTTP 200 response with an empty HTTP body.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StartTextDetection

Starts asynchronous detection of text in a stored video.

Amazon Rekognition Video can detect text in a video stored in an Amazon S3 bucket. Use Video (p. 512) to specify the bucket name and the filename of the video. `StartTextDetection` returns a job identifier (`JobId`) which you use to get the results of the operation. When text detection is finished, Amazon Rekognition Video publishes a completion status to the Amazon Simple Notification Service topic that you specify in `NotificationChannel`.

To get the results of the text detection operation, first check that the status value published to the Amazon SNS topic is `SUCCEEDED`. if so, call GetTextDetection (p. 367) and pass the job identifier (`JobId`) from the initial call to `StartTextDetection`.

Request Syntax

```
{
   "ClientRequestToken": "string",
   "Filters": {
      "RegionsOfInterest": [
         {
            "BoundingBox": {
               "Height": number,
               "Left": number,
               "Top": number,
               "Width": number
            }
         }
      ],
      "WordFilter": {
         "MinBoundingBoxHeight": number,
         "MinBoundingBoxWidth": number,
         "MinConfidence": number
      }
   },
   "JobTag": "string",
   "NotificationChannel": {
      "RoleArn": "string",
      "SNSTopicArn": "string"
   },
   "Video": {
      "S3Object": {
         "Bucket": "string",
         "Name": "string",
         "Version": "string"
      }
   }
}
```

Request Parameters

The request accepts the following data in JSON format.

ClientRequestToken (p. 427)

Idempotent token used to identify the start request. If you use the same token with multiple `StartTextDetection` requests, the same `JobId` is returned. Use `ClientRequestToken` to prevent the same job from being accidentaly started more than once.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Required: No

Filters (p. 427)

Optional parameters that let you set criteria the text must meet to be included in your response.

Type: StartTextDetectionFilters (p. 497) object

Required: No

JobTag (p. 427)

An identifier returned in the completion status published by your Amazon Simple Notification Service topic. For example, you can use `JobTag` to group related jobs and identify them in the completion notification.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 256.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

NotificationChannel (p. 427)

The Amazon Simple Notification Service topic to which Amazon Rekognition publishes the completion status of a video analysis operation. For more information, see 呼叫 Amazon Rekognition Video 操作 (p. 51).

Type: NotificationChannel (p. 483) object

Required: No

Video (p. 427)

Video file stored in an Amazon S3 bucket. Amazon Rekognition video start operations such as StartLabelDetection (p. 415) use `Video` to specify a video for analysis. The supported file formats are .mp4, .mov and .avi.

Type: Video (p. 512) object

Required: Yes

Response Syntax

```
{
   "JobId": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

JobId (p. 428)

Identifier for the text detection job. Use `JobId` to identify the job in a subsequent call to `GetTextDetection`.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 64.

Pattern: `^[a-zA-Z0-9-_]+$`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

IdempotentParameterMismatchException

A `ClientRequestToken` input parameter was reused with an operation, but at least one of the other input parameters is different from the previous call to the operation.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

InvalidS3ObjectException

Amazon Rekognition is unable to access the S3 object specified in the request.

HTTP Status Code: 400

LimitExceededException

An Amazon Rekognition service limit was exceeded. For example, if you start too many Amazon Rekognition Video jobs concurrently, calls to start operations (`StartLabelDetection`, for example) will raise a `LimitExceededException` exception (HTTP status code: 400) until the number of concurrently running jobs is below the Amazon Rekognition service limit.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

VideoTooLargeException

The file size or duration of the supplied media is too large. The maximum file size is 10GB. The maximum duration is 6 hours.

HTTP Status Code: 400

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StopProjectVersion

Stops a running model. The operation might take a while to complete. To check the current status, call DescribeProjectVersions (p. 308).

Request Syntax

```
{
   "ProjectVersionArn": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

ProjectVersionArn (p. 431)

The Amazon Resource Name (ARN) of the model version that you want to delete.

This operation requires permissions to perform the `rekognition:StopProjectVersion` action.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: Yes

Response Syntax

```
{
   "Status": "string"
}
```

Response Elements

If the action is successful, the service sends back an HTTP 200 response.

The following data is returned in JSON format by the service.

Status (p. 431)

The current status of the stop operation.

Type: String

Valid Values: `TRAINING_IN_PROGRESS | TRAINING_COMPLETED | TRAINING_FAILED | STARTING | RUNNING | FAILED | STOPPING | STOPPED | DELETING`

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

StopStreamProcessor

Stops a running stream processor that was created by CreateStreamProcessor (p. 289).

Request Syntax

```
{
   "Name": "string"
}
```

Request Parameters

The request accepts the following data in JSON format.

Name (p. 433)

The name of a stream processor created by CreateStreamProcessor (p. 289).

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: Yes

Response Elements

If the action is successful, the service sends back an HTTP 200 response with an empty HTTP body.

Errors

AccessDeniedException

You are not authorized to perform the action.

HTTP Status Code: 400

InternalServerError

Amazon Rekognition experienced a service issue. Try your call again.

HTTP Status Code: 500

InvalidParameterException

Input parameter violated a constraint. Validate your parameter before calling the API operation again.

HTTP Status Code: 400

ProvisionedThroughputExceededException

The number of requests exceeded your throughput limit. If you want to increase this limit, contact Amazon Rekognition.

HTTP Status Code: 400

ResourceInUseException

HTTP Status Code: 400

ResourceNotFoundException

The collection specified in the request cannot be found.

HTTP Status Code: 400

ThrottlingException

Amazon Rekognition is temporarily unable to process the request. Try your call again.

HTTP Status Code: 500

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS Command Line Interface
- AWS SDK for .NET
- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for JavaScript
- AWS SDK for PHP V3
- AWS SDK for Python
- AWS SDK for Ruby V3

Data Types

The following data types are supported:

- AgeRange (p. 437)
- Asset (p. 438)
- Beard (p. 439)
- BoundingBox (p. 440)
- Celebrity (p. 442)
- CelebrityDetail (p. 443)
- CelebrityRecognition (p. 445)
- ComparedFace (p. 446)
- ComparedSourceImageFace (p. 447)
- CompareFacesMatch (p. 448)
- ContentModerationDetection (p. 449)
- CustomLabel (p. 450)
- DetectionFilter (p. 451)
- DetectTextFilters (p. 452)
- Emotion (p. 453)
- EvaluationResult (p. 454)
- Eyeglasses (p. 455)
- EyeOpen (p. 456)

- Face (p. 457)
- FaceDetail (p. 459)
- FaceDetection (p. 462)
- FaceMatch (p. 463)
- FaceRecord (p. 464)
- FaceSearchSettings (p. 465)
- Gender (p. 466)
- Geometry (p. 467)
- GroundTruthManifest (p. 468)
- HumanLoopActivationOutput (p. 469)
- HumanLoopConfig (p. 470)
- HumanLoopDataAttributes (p. 471)
- Image (p. 472)
- ImageQuality (p. 473)
- Instance (p. 474)
- KinesisDataStream (p. 475)
- KinesisVideoStream (p. 476)
- Label (p. 477)
- LabelDetection (p. 478)
- Landmark (p. 479)
- ModerationLabel (p. 480)
- MouthOpen (p. 481)
- Mustache (p. 482)
- NotificationChannel (p. 483)
- OutputConfig (p. 484)
- Parent (p. 485)
- PersonDetail (p. 486)
- PersonDetection (p. 487)
- PersonMatch (p. 488)
- Point (p. 489)
- Pose (p. 490)
- ProjectDescription (p. 491)
- ProjectVersionDescription (p. 492)
- RegionOfInterest (p. 494)
- S3Object (p. 495)
- Smile (p. 496)
- StartTextDetectionFilters (p. 497)
- StreamProcessor (p. 498)
- StreamProcessorInput (p. 499)
- StreamProcessorOutput (p. 500)
- StreamProcessorSettings (p. 501)
- Summary (p. 502)
- Sunglasses (p. 503)
- TestingData (p. 504)
- TestingDataResult (p. 505)
- TextDetection (p. 506)

- TextDetectionResult (p. 508)
- TrainingData (p. 509)
- TrainingDataResult (p. 510)
- UnindexedFace (p. 511)
- Video (p. 512)
- VideoMetadata (p. 513)

AgeRange

Structure containing the estimated age range, in years, for a face.

Amazon Rekognition estimates an age range for faces detected in the input image. Estimated age ranges can overlap. A face of a 5-year-old might have an estimated range of 4-6, while the face of a 6-year-old might have an estimated range of 4-8.

Contents

High

The highest estimated age.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

Low

The lowest estimated age.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Asset

Assets are the images that you use to train and evaluate a model version. Assets are referenced by Sagemaker GroundTruth manifest files.

Contents

GroundTruthManifest

The S3 bucket that contains the Ground Truth manifest file.

Type: GroundTruthManifest (p. 468) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Beard

Indicates whether or not the face has a beard, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the face has beard or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

BoundingBox

Identifies the bounding box around the label, face, or text. The `left` (x-coordinate) and `top` (y-coordinate) are coordinates representing the top and left sides of the bounding box. Note that the upper-left corner of the image is the origin (0,0).

The `top` and `left` values returned are ratios of the overall image size. For example, if the input image is 700x200 pixels, and the top-left coordinate of the bounding box is 350x50 pixels, the API returns a `left` value of 0.5 (350/700) and a `top` value of 0.25 (50/200).

The `width` and `height` values represent the dimensions of the bounding box as a ratio of the overall image dimension. For example, if the input image is 700x200 pixels, and the bounding box width is 70 pixels, the width returned is 0.1.

> Note
>
> The bounding box coordinates can have negative values. For example, if Amazon Rekognition is able to detect a face that is at the image edge and is only partially visible, the service can return coordinates that are outside the image bounds and, depending on the image edge, you might get negative values or values greater than 1 for the `left` or `top` values.

Contents

Height

Height of the bounding box as a ratio of the overall image height.

Type: Float

Required: No

Left

Left coordinate of the bounding box as a ratio of overall image width.

Type: Float

Required: No

Top

Top coordinate of the bounding box as a ratio of overall image height.

Type: Float

Required: No

Width

Width of the bounding box as a ratio of the overall image width.

Type: Float

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++

- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Celebrity

Provides information about a celebrity recognized by the RecognizeCelebrities (p. 388) operation.

Contents

Face

Provides information about the celebrity's face, such as its location on the image.

Type: ComparedFace (p. 446) object

Required: No

Id

A unique identifier for the celebrity.

Type: String

Pattern: `[0-9A-Za-z]*`

Required: No

MatchConfidence

The confidence, in percentage, that Amazon Rekognition has that the recognized face is the celebrity.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Name

The name of the celebrity.

Type: String

Required: No

Urls

An array of URLs pointing to additional information about the celebrity. If there is no additional information about the celebrity, this list is empty.

Type: Array of strings

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

CelebrityDetail

Information about a recognized celebrity.

Contents

BoundingBox

Bounding box around the body of a celebrity.

Type: BoundingBox (p. 440) object

Required: No

Confidence

The confidence, in percentage, that Amazon Rekognition has that the recognized face is the celebrity.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Face

Face details for the recognized celebrity.

Type: FaceDetail (p. 459) object

Required: No

Id

The unique identifier for the celebrity.

Type: String

Pattern: `[0-9A-Za-z]*`

Required: No

Name

The name of the celebrity.

Type: String

Required: No

Urls

An array of URLs pointing to additional celebrity information.

Type: Array of strings

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++

- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

CelebrityRecognition

Information about a detected celebrity and the time the celebrity was detected in a stored video. For more information, see GetCelebrityRecognition (p. 339).

Contents

Celebrity

Information about a recognized celebrity.

Type: CelebrityDetail (p. 443) object

Required: No

Timestamp

The time, in milliseconds from the start of the video, that the celebrity was recognized.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ComparedFace

Provides face metadata for target image faces that are analyzed by `CompareFaces` and `RecognizeCelebrities`.

Contents

BoundingBox

Bounding box of the face.

Type: BoundingBox (p. 440) object

Required: No

Confidence

Level of confidence that what the bounding box contains is a face.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Landmarks

An array of facial landmarks.

Type: Array of Landmark (p. 479) objects

Required: No

Pose

Indicates the pose of the face as determined by its pitch, roll, and yaw.

Type: Pose (p. 490) object

Required: No

Quality

Identifies face image brightness and sharpness.

Type: ImageQuality (p. 473) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ComparedSourceImageFace

Type that describes the face Amazon Rekognition chose to compare with the faces in the target. This contains a bounding box for the selected face and confidence level that the bounding box contains a face. Note that Amazon Rekognition selects the largest face in the source image for this comparison.

Contents

BoundingBox

Bounding box of the face.

Type: BoundingBox (p. 440) object

Required: No

Confidence

Confidence level that the selected bounding box contains a face.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

CompareFacesMatch

Provides information about a face in a target image that matches the source image face analyzed by `CompareFaces`. The `Face` property contains the bounding box of the face in the target image. The `Similarity` property is the confidence that the source image face matches the face in the bounding box.

Contents

Face

Provides face metadata (bounding box and confidence that the bounding box actually contains a face).

Type: ComparedFace (p. 446) object

Required: No

Similarity

Level of confidence that the faces match.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ContentModerationDetection

Information about an unsafe content label detection in a stored video.

Contents

ModerationLabel

The unsafe content label detected by in the stored video.

Type: ModerationLabel (p. 480) object

Required: No

Timestamp

Time, in milliseconds from the beginning of the video, that the unsafe content label was detected.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

CustomLabel

A custom label detected in an image by a call to DetectCustomLabels (p. 316).

Contents

Confidence

The confidence that the model has in the detection of the custom label. The range is 0-100. A higher value indicates a higher confidence.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Geometry

The location of the detected object on the image that corresponds to the custom label. Includes an axis aligned coarse bounding box surrounding the object and a finer grain polygon for more accurate spatial information.

Type: Geometry (p. 467) object

Required: No

Name

The name of the custom label.

Type: String

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

DetectionFilter

A set of parameters that allow you to filter out certain results from your returned results.

Contents

MinBoundingBoxHeight

Sets the minimum height of the word bounding box. Words with bounding box heights lesser than this value will be excluded from the result. Value is relative to the video frame height.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 1.

Required: No

MinBoundingBoxWidth

Sets the minimum width of the word bounding box. Words with bounding boxes widths lesser than this value will be excluded from the result. Value is relative to the video frame width.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 1.

Required: No

MinConfidence

Sets confidence of word detection. Words with detection confidence below this will be excluded from the result. Values should be between 0.5 and 1 as Text in Video will not return any result below 0.5.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

DetectTextFilters

A set of optional parameters that you can use to set the criteria that the text must meet to be included in your response. `WordFilter` looks at a word's height, width, and minimum confidence. `RegionOfInterest` lets you set a specific region of the image to look for text in.

Contents

RegionsOfInterest

A Filter focusing on a certain area of the image. Uses a `BoundingBox` object to set the region of the image.

Type: Array of RegionOfInterest (p. 494) objects

Array Members: Minimum number of 0 items. Maximum number of 10 items.

Required: No

WordFilter

A set of parameters that allow you to filter out certain results from your returned results.

Type: DetectionFilter (p. 451) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Emotion

The emotions that appear to be expressed on the face, and the confidence level in the determination. The API is only making a determination of the physical appearance of a person's face. It is not a determination of the person’s internal emotional state and should not be used in such a way. For example, a person pretending to have a sad face might not be sad emotionally.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Type

Type of emotion detected.

Type: String

Valid Values: `HAPPY | SAD | ANGRY | CONFUSED | DISGUSTED | SURPRISED | CALM | UNKNOWN | FEAR`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

EvaluationResult

The evaluation results for the training of a model.

Contents

F1Score

The F1 score for the evaluation of all labels. The F1 score metric evaluates the overall precision and recall performance of the model as a single value. A higher value indicates better precision and recall performance. A lower score indicates that precision, recall, or both are performing poorly.

Type: Float

Required: No

Summary

The S3 bucket that contains the training summary.

Type: Summary (p. 502) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Eyeglasses

Indicates whether or not the face is wearing eye glasses, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the face is wearing eye glasses or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

EyeOpen

Indicates whether or not the eyes on the face are open, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the eyes on the face are open.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Face

Describes the face properties such as the bounding box, face ID, image ID of the input image, and external image ID that you assigned.

Contents

BoundingBox

Bounding box of the face.

Type: BoundingBox (p. 440) object

Required: No

Confidence

Confidence level that the bounding box contains a face (and not a different object such as a tree).

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

ExternalImageId

Identifier that you assign to all the faces in the input image.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-:]+`

Required: No

FaceId

Unique identifier that Amazon Rekognition assigns to the face.

Type: String

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Required: No

ImageId

Unique identifier that Amazon Rekognition assigns to the input image.

Type: String

Pattern: `[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++

- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

FaceDetail

Structure containing attributes of the face that the algorithm detected.

A `FaceDetail` object contains either the default facial attributes or all facial attributes. The default attributes are `BoundingBox`, `Confidence`, `Landmarks`, `Pose`, and `Quality`.

GetFaceDetection (p. 348) is the only Amazon Rekognition Video stored video operation that can return a `FaceDetail` object with all attributes. To specify which attributes to return, use the `FaceAttributes` input parameter for StartFaceDetection (p. 407). The following Amazon Rekognition Video operations return only the default attributes. The corresponding Start operations don't have a `FaceAttributes` input parameter.

- GetCelebrityRecognition
- GetPersonTracking
- GetFaceSearch

The Amazon Rekognition Image DetectFaces (p. 320) and IndexFaces (p. 371) operations can return all facial attributes. To specify which attributes to return, use the `Attributes` input parameter for `DetectFaces`. For `IndexFaces`, use the `DetectAttributes` input parameter.

Contents

AgeRange

The estimated age range, in years, for the face. Low represents the lowest estimated age and High represents the highest estimated age.

Type: AgeRange (p. 437) object

Required: No

Beard

Indicates whether or not the face has a beard, and the confidence level in the determination.

Type: Beard (p. 439) object

Required: No

BoundingBox

Bounding box of the face. Default attribute.

Type: BoundingBox (p. 440) object

Required: No

Confidence

Confidence level that the bounding box contains a face (and not a different object such as a tree). Default attribute.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Emotions

The emotions that appear to be expressed on the face, and the confidence level in the determination. The API is only making a determination of the physical appearance of a person's face. It is not a determination of the person's internal emotional state and should not be used in such a way. For example, a person pretending to have a sad face might not be sad emotionally.

Type: Array of Emotion (p. 453) objects

Required: No

Eyeglasses

Indicates whether or not the face is wearing eye glasses, and the confidence level in the determination.

Type: Eyeglasses (p. 455) object

Required: No

EyesOpen

Indicates whether or not the eyes on the face are open, and the confidence level in the determination.

Type: EyeOpen (p. 456) object

Required: No

Gender

The predicted gender of a detected face.

Type: Gender (p. 466) object

Required: No

Landmarks

Indicates the location of landmarks on the face. Default attribute.

Type: Array of Landmark (p. 479) objects

Required: No

MouthOpen

Indicates whether or not the mouth on the face is open, and the confidence level in the determination.

Type: MouthOpen (p. 481) object

Required: No

Mustache

Indicates whether or not the face has a mustache, and the confidence level in the determination.

Type: Mustache (p. 482) object

Required: No

Pose

Indicates the pose of the face as determined by its pitch, roll, and yaw. Default attribute.

Type: Pose (p. 490) object

Required: No

Quality

Identifies image brightness and sharpness. Default attribute.

Type: ImageQuality (p. 473) object

Required: No

Smile

Indicates whether or not the face is smiling, and the confidence level in the determination.

Type: Smile (p. 496) object

Required: No

Sunglasses

Indicates whether or not the face is wearing sunglasses, and the confidence level in the determination.

Type: Sunglasses (p. 503) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

FaceDetection

Information about a face detected in a video analysis request and the time the face was detected in the video.

Contents

Face

The face properties for the detected face.

Type: FaceDetail (p. 459) object

Required: No

Timestamp

Time, in milliseconds from the start of the video, that the face was detected.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

FaceMatch

Provides face metadata. In addition, it also provides the confidence in the match of this face with the input face.

Contents

Face

Describes the face properties such as the bounding box, face ID, image ID of the source image, and external image ID that you assigned.

Type: Face (p. 457) object

Required: No

Similarity

Confidence in the match of this face with the input face.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

FaceRecord

Object containing both the face metadata (stored in the backend database), and facial attributes that are detected but aren't stored in the database.

Contents

Face

Describes the face properties such as the bounding box, face ID, image ID of the input image, and external image ID that you assigned.

Type: Face (p. 457) object

Required: No

FaceDetail

Structure containing attributes of the face that the algorithm detected.

Type: FaceDetail (p. 459) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

FaceSearchSettings

Input face recognition parameters for an Amazon Rekognition stream processor. `FaceRecognitionSettings` is a request parameter for CreateStreamProcessor (p. 289).

Contents

CollectionId

The ID of a collection that contains faces that you want to search for.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 255.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: No

FaceMatchThreshold

Minimum face match confidence score that must be met to return a result for a recognized face. Default is 80. 0 is the lowest confidence. 100 is the highest confidence.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Gender

The predicted gender of a detected face.

Amazon Rekognition makes gender binary (male/female) predictions based on the physical appearance of a face in a particular image. This kind of prediction is not designed to categorize a person's gender identity, and you shouldn't use Amazon Rekognition to make such a determination. For example, a male actor wearing a long-haired wig and earrings for a role might be predicted as female.

Using Amazon Rekognition to make gender binary predictions is best suited for use cases where aggregate gender distribution statistics need to be analyzed without identifying specific users. For example, the percentage of female users compared to male users on a social media platform.

We don't recommend using gender binary predictions to make decisions that impact an individual's rights, privacy, or access to services.

Contents

Confidence

Level of confidence in the prediction.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

The predicted gender of the face.

Type: String

Valid Values: `Male | Female`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Geometry

Information about where an object (DetectCustomLabels (p. 316)) or text (DetectText (p. 333)) is located on an image.

Contents

BoundingBox

An axis-aligned coarse representation of the detected item's location on the image.

Type: BoundingBox (p. 440) object

Required: No

Polygon

Within the bounding box, a fine-grained polygon around the detected item.

Type: Array of Point (p. 489) objects

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

GroundTruthManifest

The S3 bucket that contains the Ground Truth manifest file.

Contents

S3Object

Provides the S3 bucket name and object name.

The region for the S3 bucket containing the S3 object must match the region you use for Amazon Rekognition operations.

For Amazon Rekognition to process an S3 object, the user must have permission to access the S3 object. For more information, see Amazon Rekognition 資源型政策 (p. 250).

Type: S3Object (p. 495) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

HumanLoopActivationOutput

Shows the results of the human in the loop evaluation. If there is no HumanLoopArn, the input did not trigger human review.

Contents

HumanLoopActivationConditionsEvaluationResults

Shows the result of condition evaluations, including those conditions which activated a human review.

Type: String

Length Constraints: Maximum length of 10240.

Required: No

HumanLoopActivationReasons

Shows if and why human review was needed.

Type: Array of strings

Array Members: Minimum number of 1 item.

Required: No

HumanLoopArn

The Amazon Resource Name (ARN) of the HumanLoop created.

Type: String

Length Constraints: Maximum length of 256.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

HumanLoopConfig

Sets up the flow definition the image will be sent to if one of the conditions is met. You can also set certain attributes of the image before review.

Contents

DataAttributes

Sets attributes of the input data.

Type: HumanLoopDataAttributes (p. 471) object

Required: No

FlowDefinitionArn

The Amazon Resource Name (ARN) of the flow definition.

Type: String

Length Constraints: Maximum length of 256.

Required: Yes

HumanLoopName

The name of the human review used for this image. This should be kept unique within a region.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 63.

Pattern: `^[a-z0-9](-*[a-z0-9])*`

Required: Yes

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

HumanLoopDataAttributes

Allows you to set attributes of the image. Currently, you can declare an image as free of personally identifiable information.

Contents

ContentClassifiers

Sets whether the input image is free of personally identifiable information.

Type: Array of strings

Array Members: Maximum number of 256 items.

Valid Values: `FreeOfPersonallyIdentifiableInformation | FreeOfAdultContent`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Image

Provides the input image either as bytes or an S3 object.

You pass image bytes to an Amazon Rekognition API operation by using the `Bytes` property. For example, you would use the `Bytes` property to pass an image loaded from a local file system. Image bytes passed by using the `Bytes` property must be base64-encoded. Your code may not need to encode image bytes if you are using an AWS SDK to call Amazon Rekognition API operations.

For more information, see 分析從本機檔案系統載入的影像 (p. 29).

You pass images stored in an S3 bucket to an Amazon Rekognition API operation by using the `S3Object` property. Images stored in an S3 bucket do not need to be base64-encoded.

The region for the S3 bucket containing the S3 object must match the region you use for Amazon Rekognition operations.

If you use the AWS CLI to call Amazon Rekognition operations, passing image bytes using the Bytes property is not supported. You must first upload the image to an Amazon S3 bucket and then call the operation using the S3Object property.

For Amazon Rekognition to process an S3 object, the user must have permission to access the S3 object. For more information, see Amazon Rekognition 資源型政策 (p. 250).

Contents

Bytes

Blob of image bytes up to 5 MBs.

Type: Base64-encoded binary data object

Length Constraints: Minimum length of 1. Maximum length of 5242880.

Required: No

S3Object

Identifies an S3 object as the image source.

Type: S3Object (p. 495) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ImageQuality

Identifies face image brightness and sharpness.

Contents

Brightness

Value representing brightness of the face. The service returns a value between 0 and 100 (inclusive). A higher value indicates a brighter face image.

Type: Float

Required: No

Sharpness

Value representing sharpness of the face. The service returns a value between 0 and 100 (inclusive). A higher value indicates a sharper face image.

Type: Float

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Instance

An instance of a label returned by Amazon Rekognition Image (DetectLabels (p. 324)) or by Amazon Rekognition Video (GetLabelDetection (p. 358)).

Contents

BoundingBox

The position of the label instance on the image.

Type: BoundingBox (p. 440) object

Required: No

Confidence

The confidence that Amazon Rekognition has in the accuracy of the bounding box.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

KinesisDataStream

The Kinesis data stream Amazon Rekognition to which the analysis results of a Amazon Rekognition stream processor are streamed. For more information, see CreateStreamProcessor (p. 289).

Contents

Arn

ARN of the output Amazon Kinesis Data Streams stream.

Type: String

Pattern: `(^arn:([a-z\d-]+):kinesis:([a-z\d-]+):\d{12}:.+$)`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

KinesisVideoStream

Kinesis video stream stream that provides the source streaming video for a Amazon Rekognition Video stream processor. For more information, see CreateStreamProcessor (p. 289).

Contents

Arn

ARN of the Kinesis video stream stream that streams the source video.

Type: String

Pattern: `(^arn:([a-z\d-]+):kinesisvideo:([a-z\d-]+):\d{12}:.+$)`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Label

Structure containing details about the detected label, including the name, detected instances, parent labels, and level of confidence.

Contents

Confidence

Level of confidence.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Instances

If `Label` represents an object, `Instances` contains the bounding boxes for each instance of the detected object. Bounding boxes are returned for common object labels such as people, cars, furniture, apparel or pets.

Type: Array of Instance (p. 474) objects

Required: No

Name

The name (label) of the object or scene.

Type: String

Required: No

Parents

The parent labels for a label. The response includes all ancestor labels.

Type: Array of Parent (p. 485) objects

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

LabelDetection

Information about a label detected in a video analysis request and the time the label was detected in the video.

Contents

Label

Details about the detected label.

Type: Label (p. 477) object

Required: No

Timestamp

Time, in milliseconds from the start of the video, that the label was detected.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Landmark

Indicates the location of the landmark on the face.

Contents

Type

Type of landmark.

Type: String

Valid Values: `eyeLeft | eyeRight | nose | mouthLeft | mouthRight | leftEyeBrowLeft | leftEyeBrowRight | leftEyeBrowUp | rightEyeBrowLeft | rightEyeBrowRight | rightEyeBrowUp | leftEyeLeft | leftEyeRight | leftEyeUp | leftEyeDown | rightEyeLeft | rightEyeRight | rightEyeUp | rightEyeDown | noseLeft | noseRight | mouthUp | mouthDown | leftPupil | rightPupil | upperJawlineLeft | midJawlineLeft | chinBottom | midJawlineRight | upperJawlineRight`

Required: No

X

The x-coordinate from the top left of the landmark expressed as the ratio of the width of the image. For example, if the image is 700 x 200 and the x-coordinate of the landmark is at 350 pixels, this value is 0.5.

Type: Float

Required: No

Y

The y-coordinate from the top left of the landmark expressed as the ratio of the height of the image. For example, if the image is 700 x 200 and the y-coordinate of the landmark is at 100 pixels, this value is 0.5.

Type: Float

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ModerationLabel

Provides information about a single type of unsafe content found in an image or video. Each type of moderated content has a label within a hierarchical taxonomy. For more information, see 偵測不安全的內容 (p. 215).

Contents

Confidence

Specifies the confidence that Amazon Rekognition has that the label has been correctly identified.

If you don't specify the `MinConfidence` parameter in the call to `DetectModerationLabels`, the operation returns labels with a confidence value greater than or equal to 50 percent.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Name

The label name for the type of unsafe content detected in the image.

Type: String

Required: No

ParentName

The name for the parent label. Labels at the top level of the hierarchy have the parent label "".

Type: String

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

MouthOpen

Indicates whether or not the mouth on the face is open, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the mouth on the face is open or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Mustache

Indicates whether or not the face has a mustache, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the face has mustache or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

NotificationChannel

The Amazon Simple Notification Service topic to which Amazon Rekognition publishes the completion status of a video analysis operation. For more information, see 呼叫 Amazon Rekognition Video 操作 (p. 51).

Contents

RoleArn

The ARN of an IAM role that gives Amazon Rekognition publishing permissions to the Amazon SNS topic.

Type: String

Pattern: `arn:aws:iam::\d{12}:role/?[a-zA-Z_0-9+=,.@\-_/]+`

Required: Yes

SNSTopicArn

The Amazon SNS topic to which Amazon Rekognition to posts the completion status.

Type: String

Pattern: `(^arn:aws:sns:.*:\w{12}:.+$)`

Required: Yes

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

OutputConfig

The S3 bucket and folder location where training output is placed.

Contents

S3Bucket

The S3 bucket where training output is placed.

Type: String

Length Constraints: Minimum length of 3. Maximum length of 255.

Pattern: `[0-9A-Za-z\.\-_]*`

Required: No

S3KeyPrefix

The prefix applied to the training output files.

Type: String

Length Constraints: Maximum length of 1024.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Parent

A parent label for a label. A label can have 0, 1, or more parents.

Contents

Name

The name of the parent label.

Type: String

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

PersonDetail

Details about a person detected in a video analysis request.

Contents

BoundingBox

Bounding box around the detected person.

Type: BoundingBox (p. 440) object

Required: No

Face

Face details for the detected person.

Type: FaceDetail (p. 459) object

Required: No

Index

Identifier for the person detected person within a video. Use to keep track of the person throughout the video. The identifier is not stored by Amazon Rekognition.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

PersonDetection

Details and path tracking information for a single time a person's path is tracked in a video. Amazon Rekognition operations that track people's paths return an array of `PersonDetection` objects with elements for each time a person's path is tracked in a video.

For more information, see GetPersonTracking (p. 362).

Contents

Person

Details about a person whose path was tracked in a video.

Type: PersonDetail (p. 486) object

Required: No

Timestamp

The time, in milliseconds from the start of the video, that the person's path was tracked.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

PersonMatch

Information about a person whose face matches a face(s) in an Amazon Rekognition collection. Includes information about the faces in the Amazon Rekognition collection (FaceMatch (p. 463)), information about the person (PersonDetail (p. 486)), and the time stamp for when the person was detected in a video. An array of `PersonMatch` objects is returned by GetFaceSearch (p. 353).

Contents

FaceMatches

Information about the faces in the input collection that match the face of a person in the video.

Type: Array of FaceMatch (p. 463) objects

Required: No

Person

Information about the matched person.

Type: PersonDetail (p. 486) object

Required: No

Timestamp

The time, in milliseconds from the beginning of the video, that the person was matched in the video.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Point

The X and Y coordinates of a point on an image. The X and Y values returned are ratios of the overall image size. For example, if the input image is 700x200 and the operation returns X=0.5 and Y=0.25, then the point is at the (350,50) pixel coordinate on the image.

An array of `Point` objects, `Polygon`, is returned by DetectText (p. 333) and by DetectCustomLabels (p. 316). `Polygon` represents a fine-grained polygon around a detected item. For more information, see Geometry (p. 467).

Contents

X

The value of the X coordinate for a point on a `Polygon`.

Type: Float

Required: No

Y

The value of the Y coordinate for a point on a `Polygon`.

Type: Float

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Pose

Indicates the pose of the face as determined by its pitch, roll, and yaw.

Contents

Pitch

Value representing the face rotation on the pitch axis.

Type: Float

Valid Range: Minimum value of -180. Maximum value of 180.

Required: No

Roll

Value representing the face rotation on the roll axis.

Type: Float

Valid Range: Minimum value of -180. Maximum value of 180.

Required: No

Yaw

Value representing the face rotation on the yaw axis.

Type: Float

Valid Range: Minimum value of -180. Maximum value of 180.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ProjectDescription

A description of a Amazon Rekognition Custom Labels project.

Contents

CreationTimestamp

The Unix timestamp for the date and time that the project was created.

Type: Timestamp

Required: No

ProjectArn

The Amazon Resource Name (ARN) of the project.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: No

Status

The current status of the project.

Type: String

Valid Values: `CREATING | CREATED | DELETING`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

ProjectVersionDescription

The description of a version of a model.

Contents

BillableTrainingTimeInSeconds

The duration, in seconds, that the model version has been billed for training. This value is only returned if the model version has been successfully trained.

Type: Long

Valid Range: Minimum value of 0.

Required: No

CreationTimestamp

The Unix datetime for the date and time that training started.

Type: Timestamp

Required: No

EvaluationResult

The training results. `EvaluationResult` is only returned if training is successful.

Type: EvaluationResult (p. 454) object

Required: No

MinInferenceUnits

The minimum number of inference units used by the model. For more information, see StartProjectVersion (p. 422).

Type: Integer

Valid Range: Minimum value of 1.

Required: No

OutputConfig

The location where training results are saved.

Type: OutputConfig (p. 484) object

Required: No

ProjectVersionArn

The Amazon Resource Name (ARN) of the model version.

Type: String

Length Constraints: Minimum length of 20. Maximum length of 2048.

Pattern: `(^arn:[a-z\d-]+:rekognition:[a-z\d-]+:\d{12}:project\/[a-zA-Z0-9_.\-]{1,255}\/version\/[a-zA-Z0-9_.\-]{1,255}\/[0-9]+$)`

Required: No

Status

The current status of the model version.

Type: String

Valid Values: `TRAINING_IN_PROGRESS | TRAINING_COMPLETED | TRAINING_FAILED | STARTING | RUNNING | FAILED | STOPPING | STOPPED | DELETING`

Required: No

StatusMessage

A descriptive message for an error or warning that occurred.

Type: String

Required: No

TestingDataResult

The manifest file that represents the testing results.

Type: TestingDataResult (p. 505) object

Required: No

TrainingDataResult

The manifest file that represents the training results.

Type: TrainingDataResult (p. 510) object

Required: No

TrainingEndTimestamp

The Unix date and time that training of the model ended.

Type: Timestamp

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

RegionOfInterest

Specifies a location within the frame that Rekognition checks for text. Uses a `BoundingBox` object to set a region of the screen.

A word is included in the region if the word is more than half in that region. If there is more than one region, the word will be compared with all regions of the screen. Any word more than half in a region is kept in the results.

Contents

BoundingBox

The box representing a region of interest on screen.

Type: BoundingBox (p. 440) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

S3Object

Provides the S3 bucket name and object name.

The region for the S3 bucket containing the S3 object must match the region you use for Amazon Rekognition operations.

For Amazon Rekognition to process an S3 object, the user must have permission to access the S3 object. For more information, see Amazon Rekognition 資源型政策 (p. 250).

Contents

Bucket

Name of the S3 bucket.

Type: String

Length Constraints: Minimum length of 3. Maximum length of 255.

Pattern: `[0-9A-Za-z\.\-_]*`

Required: No

Name

S3 object key name.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 1024.

Required: No

Version

If the bucket is versioning enabled, you can specify the object version.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 1024.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Smile

Indicates whether or not the face is smiling, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the face is smiling or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

StartTextDetectionFilters

Set of optional parameters that let you set the criteria text must meet to be included in your response. `WordFilter` looks at a word's height, width and minimum confidence. `RegionOfInterest` lets you set a specific region of the screen to look for text in.

Contents

RegionsOfInterest

Filter focusing on a certain area of the frame. Uses a `BoundingBox` object to set the region of the screen.

Type: Array of RegionOfInterest (p. 494) objects

Array Members: Minimum number of 0 items. Maximum number of 10 items.

Required: No

WordFilter

Filters focusing on qualities of the text, such as confidence or size.

Type: DetectionFilter (p. 451) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

StreamProcessor

An object that recognizes faces in a streaming video. An Amazon Rekognition stream processor is created by a call to CreateStreamProcessor (p. 289). The request parameters for `CreateStreamProcessor` describe the Kinesis video stream source for the streaming video, face recognition parameters, and where to stream the analysis resullts.

Contents

Name

Name of the Amazon Rekognition stream processor.

Type: String

Length Constraints: Minimum length of 1. Maximum length of 128.

Pattern: `[a-zA-Z0-9_.\-]+`

Required: No

Status

Current status of the Amazon Rekognition stream processor.

Type: String

Valid Values: `STOPPED | STARTING | RUNNING | FAILED | STOPPING`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

StreamProcessorInput

Information about the source streaming video.

Contents

KinesisVideoStream

The Kinesis video stream input stream for the source streaming video.

Type: KinesisVideoStream (p. 476) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

StreamProcessorOutput

Information about the Amazon Kinesis Data Streams stream to which a Amazon Rekognition Video stream processor streams the results of a video analysis. For more information, see CreateStreamProcessor (p. 289).

Contents

KinesisDataStream

The Amazon Kinesis Data Streams stream to which the Amazon Rekognition stream processor streams the analysis results.

Type: KinesisDataStream (p. 475) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

StreamProcessorSettings

Input parameters used to recognize faces in a streaming video analyzed by a Amazon Rekognition stream processor.

Contents

FaceSearch

Face search settings to use on a streaming video.

Type: FaceSearchSettings (p. 465) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Summary

The S3 bucket that contains the training summary. The training summary includes aggregated evaluation metrics for the entire testing dataset and metrics for each individual label.

You get the training summary S3 bucket location by calling DescribeProjectVersions (p. 308).

Contents

S3Object

Provides the S3 bucket name and object name.

The region for the S3 bucket containing the S3 object must match the region you use for Amazon Rekognition operations.

For Amazon Rekognition to process an S3 object, the user must have permission to access the S3 object. For more information, see Amazon Rekognition 資源型政策 (p. 250).

Type: S3Object (p. 495) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Sunglasses

Indicates whether or not the face is wearing sunglasses, and the confidence level in the determination.

Contents

Confidence

Level of confidence in the determination.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

Value

Boolean value that indicates whether the face is wearing sunglasses or not.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TestingData

The dataset used for testing. Optionally, if `AutoCreate` is set, Amazon Rekognition Custom Labels creates a testing dataset using an 80/20 split of the training dataset.

Contents

Assets

The assets used for testing.

Type: Array of Asset (p. 438) objects

Required: No

AutoCreate

If specified, Amazon Rekognition Custom Labels creates a testing dataset with an 80/20 split of the training dataset.

Type: Boolean

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TestingDataResult

A Sagemaker Groundtruth format manifest file representing the dataset used for testing.

Contents

Input

The testing dataset that was supplied for training.

Type: TestingData (p. 504) object

Required: No

Output

The subset of the dataset that was actually tested. Some images (assets) might not be tested due to file formatting and other issues.

Type: TestingData (p. 504) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TextDetection

Information about a word or line of text detected by DetectText (p. 333).

The `DetectedText` field contains the text that Amazon Rekognition detected in the image.

Every word and line has an identifier (`Id`). Each word belongs to a line and has a parent identifier (`ParentId`) that identifies the line of text in which the word appears. The word `Id` is also an index for the word within a line of words.

For more information, see 偵測文字 (p. 227).

Contents

Confidence

The confidence that Amazon Rekognition has in the accuracy of the detected text and the accuracy of the geometry points around the detected text.

Type: Float

Valid Range: Minimum value of 0. Maximum value of 100.

Required: No

DetectedText

The word or line of text recognized by Amazon Rekognition.

Type: String

Required: No

Geometry

The location of the detected text on the image. Includes an axis aligned coarse bounding box surrounding the text and a finer grain polygon for more accurate spatial information.

Type: Geometry (p. 467) object

Required: No

Id

The identifier for the detected text. The identifier is only unique for a single call to `DetectText`.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

ParentId

The Parent identifier for the detected text identified by the value of `ID`. If the type of detected text is `LINE`, the value of `ParentId` is `Null`.

Type: Integer

Valid Range: Minimum value of 0.

Required: No

Type

The type of text that was detected.

Type: String

Valid Values: `LINE | WORD`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TextDetectionResult

Information about text detected in a video. Incudes the detected text, the time in milliseconds from the start of the video that the text was detected, and where it was detected on the screen.

Contents

TextDetection

Details about text detected in a video.

Type: TextDetection (p. 506) object

Required: No

Timestamp

The time, in milliseconds from the start of the video, that the text was detected.

Type: Long

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TrainingData

The dataset used for training.

Contents

Assets

A Sagemaker GroundTruth manifest file that contains the training images (assets).

Type: Array of Asset (p. 438) objects

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

TrainingDataResult

A Sagemaker Groundtruth format manifest file that represents the dataset used for training.

Contents

Input

The training assets that you supplied for training.

Type: TrainingData (p. 509) object

Required: No

Output

The images (assets) that were actually trained by Amazon Rekognition Custom Labels.

Type: TrainingData (p. 509) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

UnindexedFace

A face that IndexFaces (p. 371) detected, but didn't index. Use the `Reasons` response attribute to determine why a face wasn't indexed.

Contents

FaceDetail

The structure that contains attributes of a face that `IndexFaces`detected, but didn't index.

Type: FaceDetail (p. 459) object

Required: No

Reasons

An array of reasons that specify why a face wasn't indexed.

- EXTREME_POSE - The face is at a pose that can't be detected. For example, the head is turned too far away from the camera.
- EXCEEDS_MAX_FACES - The number of faces detected is already higher than that specified by the `MaxFaces` input parameter for `IndexFaces`.
- LOW_BRIGHTNESS - The image is too dark.
- LOW_SHARPNESS - The image is too blurry.
- LOW_CONFIDENCE - The face was detected with a low confidence.
- SMALL_BOUNDING_BOX - The bounding box around the face is too small.

Type: Array of strings

Valid Values: `EXCEEDS_MAX_FACES | EXTREME_POSE | LOW_BRIGHTNESS | LOW_SHARPNESS | LOW_CONFIDENCE | SMALL_BOUNDING_BOX | LOW_FACE_QUALITY`

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Video

Video file stored in an Amazon S3 bucket. Amazon Rekognition video start operations such as StartLabelDetection (p. 415) use `Video` to specify a video for analysis. The supported file formats are .mp4, .mov and .avi.

Contents

S3Object

The Amazon S3 bucket name and file name for the video.

Type: S3Object (p. 495) object

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

VideoMetadata

Information about a video that Amazon Rekognition analyzed. `Videometadata` is returned in every page of paginated responses from a Amazon Rekognition video operation.

Contents

Codec

Type of compression used in the analyzed video.

Type: String

Required: No

DurationMillis

Length of the video in milliseconds.

Type: Long

Valid Range: Minimum value of 0.

Required: No

Format

Format of the analyzed video. Possible values are MP4, MOV and AVI.

Type: String

Required: No

FrameHeight

Vertical pixel dimension of the video.

Type: Long

Valid Range: Minimum value of 0.

Required: No

FrameRate

Number of frames per second in the video.

Type: Float

Required: No

FrameWidth

Horizontal pixel dimension of the video.

Type: Long

Valid Range: Minimum value of 0.

Required: No

See Also

For more information about using this API in one of the language-specific AWS SDKs, see the following:

- AWS SDK for C++
- AWS SDK for Go
- AWS SDK for Java
- AWS SDK for Ruby V3

Amazon Rekognition 中的限制

以下是 Amazon Rekognition 中的限制的清單。如需可變更限制的詳細資訊，請參閱 AWS 服務配額。若要變更限制，請參閱建立案例。

如需每秒交易 (TPS) 限制的詳細資訊，請參閱 AWS 服務配額。

如需 Amazon Rekognition 自訂標籤限制，請參閱 Amazon Rekognition 自訂標籤中的限制。

Amazon Rekognition Image

- 存放為 Amazon S3 物件的影像大小上限為 15 MB。
- 高度與寬度的影像大小最低為 80 像素。
- 在 1920X1080 像素的影像中，臉部必須大於或等於 40x40 像素才能偵測得到。大小高於 1920X1080 像素的影像需要按比例放大最小臉部大小。
- 以原始位元組傳入 API 做為參數的最大影像大小為 5 MB。
- Amazon Rekognition 支援 PNG 與 JPEG 影像格式。也就是說，您提供做為各種 API 操作 (例如 `DetectLabels` 與 `IndexFaces`) 輸入的影像必須使用其中一種支援的格式。
- 您在單一臉部集合中最多可存放 2,000 萬張臉。
- 搜尋 API 最多傳回 4096 張相符的臉部。
- DetectText 最多可在一個影像中偵測 50 個單字。

如需影像與臉部比較的最佳實務資訊，請參閱感應器、輸入影像和影片的最佳實務 (p. 103)。

Amazon Rekognition Video 已存放影片

- Amazon Rekognition Video 可分析的已存放影片最大為 10GB。
- Amazon Rekognition Video可分析長度最多 6 小時的已存放影片。
- Amazon Rekognition Video 支援每個帳戶最多 20 個並行工作。
- 已存放影片必須使用 H.264 轉碼器來編碼。支援的檔案格式為 MPEG-4 與 MOV。
- 適用於分頁符記的轉碼器 (TTL) 期間為 24 小時。分頁符記位於 Get 操作 (例如 `GetLabeldetection`) 傳回的 `NextToken` 欄位中。

Amazon Rekognition Video 串流視訊

- Kinesis Video 輸入串流最多可與 1 個 Amazon Rekognition Video 串流處理器連接。
- Kinesis Data 輸出串流最多可與 1 個 Amazon Rekognition Video 串流處理器連接。
- 與 Amazon Rekognition Video 串流處理器相關的 Kinesis Video 輸入串流與 Kinesis Data 輸出串流無法供多個處理器共用。

Amazon Rekognition 的文件歷史記錄

下表說明 Amazon Rekognition 開發人員指南每個版本的重要變更。如需有關此文件更新的通知，您可以訂閱 RSS 摘要。

- 文件最新更新時間：2020 年 3 月 3 日

update-history-change	update-history-description	update-history-date
Amazon Rekognition 現在支援 Amazon VPC 端點政策 (p. 516)	藉由指定政策，您可以限制存取 Amazon Rekognition Amazon VPC 端點。	March 3, 2020
Amazon Rekognition 現在支援 Augmented AI (預覽) 和 Amazon Rekognition 自訂標籤 (p. 516)	透過 Amazon Rekognition 自訂標籤，您可以藉由建立自己的機器學習模型，來偵測影像中的特殊物件、場景和概念。DetectModerationLabels 現在支援 Amazon Augmented AI (預覽)。	December 3, 2019
Amazon Rekognition 臉部篩選 (p. 516)	Amazon Rekognition 為 IndexFaces API 操作新增了進階臉部篩選支援，並為 CompareFaces 和 SearchFacesByImage API 操作引進了面部篩選功能。	September 12, 2019
Amazon Rekognition 現在支援 AWS PrivateLink (p. 516)	您可以使用 AWS PrivateLink，在 VPC 和 Amazon Rekognition 之間建立私有連線。	September 12, 2019
Amazon Rekognition Video 範例已更新 (p. 516)	Amazon Rekognition Video 範例程式碼已更新，以建立並設定 Amazon SNS 主題和 Amazon SQS 佇列。	September 5, 2019
Ruby 和 Node.js 範例已新增 (p. 516)	Amazon Rekognition Image針對同步標籤和臉部偵測新增的 Ruby 和 Node.js 範例。	August 19, 2019
不安全內容偵測已更新 (p. 516)	Amazon Rekognition 不安全內容偵測現在可以偵測暴力內容。	August 9, 2019
GetContentModeration 操作已更新 (p. 516)	GetContentModeration 現在會傳回用於偵測不安全內容的管制偵測模型版本。	February 13, 2019
GetLabelDetection 和 DetectModerationLabels 操作已更新 (p. 516)	GetLabelDetection 現在會傳回常見物件的週框方塊資訊以及所偵測到標籤的階層式分類法。現在會傳回用於偵測標籤的模型版本。DetectModerationLabels 現在會傳回用於偵測不安全內容的模型版本。	January 17, 2019

DetectFaces 和 IndexFaces 操作已更新 (p. 516)	此版本更新了 DetectFaces 和 IndexFaces 操作。當 Attributes 輸入參數設為 ALL 時，臉部位置特徵點包含 5 個新的特徵點：upperJawlineLeft、midJawlineLeft、chinBottom、midJawlineRight、upperJawlineRi	November 19, 2018
DetectLabels 操作已更新 (p. 516)	現在，對於特定物件會傳回週框方塊。階層式分類現在可用於標籤。您現在可以取得用於偵測的偵測模型版本。	November 1, 2018
IndexFaces 操作已更新 (p. 516)	運用 IndexFaces，您現在可以使用 QualityFilter 輸入參數來篩選排除偵測品質過低的臉孔。您也可以使用 MaxFaces 輸入參數，根據臉部偵測品質過低及偵測臉部尺寸等條件，減少傳回的臉孔數目。	September 18, 2018
已新增 DescribeCollection 操作 (p. 516)	您現在可以透過呼叫 DescribeCollection 操作來取得現有集合的相關資訊。	August 22, 2018
新的 Python 範例 (p. 516)	Python 範例已新增到 Amazon Rekognition 影片內容以及一些內容重組。	June 26, 2018
更新內容版面配置 (p. 516)	Amazon Rekognition 影像內容已重新整理過，並新增了新的 Python 與 C# 範例。	May 29, 2018
Amazon Rekognition支援AWS CloudTrail (p. 516)	Amazon Rekognition 已與 AWS CloudTrail 整合，這種服務提供由使用者、角色或 Amazon Rekognition 中 AWS 服務所採取之動作的記錄。如需詳細資訊，請參閱使用 AWS CloudTrail 記錄 Amazon Rekognition API 呼叫。	April 6, 2018
分析已存放與串流的影片。新目錄 (p. 516)	如需分析已儲存影片的相關資訊，請參閱使用儲存的影片。如需分析串流影片的相關資訊，請參閱使用串流影片。Amazon Rekognition 文件的目錄已經過重新編排，以納入影像與影片操作的說明。	November 29, 2017
影像與臉部偵測模型中的文字 (p. 516)	Amazon Rekognition 現在可以偵測影像中的文字。如需詳細資訊，請參閱偵測文字。Amazon Rekognition 會介紹臉部偵測深度學習模型的版本控制。如需詳細資訊，請參閱模型版本控制。	November 21, 2017
名人辨識 (p. 516)	Amazon Rekognition 現在可以分析名人的影像。如需詳細資訊，請參閱辨識名人。	June 8, 2017

影像管制 (p. 516)	Amazon Rekognition 現在可以判斷影像是否含有露骨或暗示性的成人內容。如需詳細資訊，請參閱刪除不安全的內容。	April 19, 2017
偵測到之臉部的年齡範圍。彙總的 Rekognition 指標窗格 (p. 516)	Amazon Rekognition 現在會傳回為 Rekognition API 偵測到之臉部所估計的年齡範圍 (以歲為單位)。如需詳細資訊，請參閱 AgeRange。Rekognition 主控台現在具有指標窗格，其中顯示 Rekognition 在一段特定時間內的 Amazon CloudWatch 指標彙總活動圖表。如需詳細資訊，請參閱練習 4：請參閱彙總指標 (主控台)。	February 9, 2017
新的服務與指南 (p. 516)	這是影像分析服務 Amazon Rekognition 及 Amazon Rekognition 開發人員指南的初始版本。	November 30, 2016

AWS Glossary

For the latest AWS terminology, see the AWS Glossary in the AWS General Reference.